

BEDROCK ARCHITECTURE

Programmer's Reference Manual

CONTENTS

I	Architectural Foundations	1
1	Overview	2
1.1	Contract Position	2
1.2	Manual Scope	2
1.3	Architectural Profile	3
2	Terminology and Compatibility	4
2.1	Address Values	4
2.2	Memory Behavior	4
2.3	Segmentation and Paging	4
2.4	Instruction Encoding	5
2.5	Control Flow	5
2.6	Tracing Terms	5
2.7	Architectural State	6
2.8	Floating-Point and Architectural Exceptions	6
2.9	Compatibility	6
2.10	Compatibility Contract	8
2.11	Reserved Fields	8
2.12	Reserved Encodings	8
2.13	CPUID Compatibility	9
3	Conformance	10
3.1	Conformance Claim	10
3.2	Normative Material and Specificity	10
3.3	Implementation-Defined Disclosures	11
4	Programming Model	12
4.1	Register Model	12
4.2	FLAGS and STATUS Registers	12
4.3	Floating-Point Register Model	14
4.4	FFLAGS and FSTATUS Registers	14
4.5	Segment Registers	14
4.6	Segment Register Operand Class	15

4.7	Control Registers	16
5	Data Formats	20
5.1	Integer Data Sizes	20
5.2	Little-Endian Memory Organization	21
5.3	Floating-Point Data Formats	22
II	Encoding, Addressing, and Execution	23
6	Instruction Encoding	24
6.1	Instruction Stream and Record Boundary	24
6.2	Instruction Header Formats	24
6.3	Instruction Payload Ordering	27
6.4	Representative Encodings	28
7	Effective-Address Profiles	30
7.1	Compact Profile Encodings	31
7.1.1	Floating-Point FEA Immediate Forms	36
7.1.2	Vector VEA Lane Addressing	37
7.2	Shared Extended Descriptor	38
7.3	Extended EA Addressing Modes	40
8	Memory Address Translation	48
8.1	Segment Pre-Translation	48
8.2	Paging Stage	49
8.3	LA48 Four-Level Paging	51
8.4	LA57 Five-Level Paging	52
8.5	Page-Table Entry Format	53
8.6	Translation-Cache Identity and Invalidation	54
8.7	Leaf Byte Addresses and Access Ranges	56
8.8	Physical Class and MMIO Accesses	56
8.9	Multi-Page Accesses	56
9	Instruction Execution Model	57
9.1	Architectural Result Priority	57
9.2	Implicit Stack Operations	57
9.3	Operand Evaluation and EA Defaults	58

9.4	Shared Side-Effect Rules	58
10	Flags and Condition Codes	60
10.1	Condition Encoding	60
10.2	Flag Meanings	61
10.3	Conditional Consumers	61
10.4	Floating-Point Interactions	61
11	Memory Model	62
11.1	Baseline Ordering	62
11.2	Ordinary Load Values and Preserved Order	62
11.3	PTE Cache Policies	62
11.4	Access Atomicity and Alignment	63
11.5	Cache-Maintenance Blocks	64
11.6	Atomic Memory-Order Selectors	64
11.7	Fence Instructions	65
11.8	Global Sequentially Consistent Order	65
12	Repeated Scalar Execution	66
III	System Programming	67
13	Privileged Execution Model	68
13.1	Privilege and Event State	68
13.2	User Context Bank	68
13.3	Initial Event Stacks and Payloads	68
13.4	Event Entry and Failure	68
13.5	ERET Routing and Atomicity	69
14	Architectural Event Processing Model	70
14.1	Event Code and Sources	70
14.2	Event Priority and Debug Trace	72
14.3	Entry Admission and Event Depth	73
14.4	Supervisor Event Stacks and Nesting	73
14.5	Event Entry State	73
14.6	Architectural Event Frame	75
14.7	Event Payloads	76

14.7.1 Address-Context Error Code	77
14.7.2 Other Exception Error Codes	78
14.7.3 Auxiliary Payloads	79
14.8 Machine-Check and Bus-Error Continuation	81
14.9 Delivery Failure and Shutdown	82
14.10 Event Reset and Context State	82
15 CPUID Feature Discovery	84
15.1 Overview	84
15.2 Query Model	84
15.3 Discovery Classes	85
15.4 Leaf Directory	85
15.5 Base Identity	86
15.6 Optional-Extension Directory	87
15.7 Vector-Parameter Discovery	88
15.8 FPTRANSA Accuracy Discovery	88
15.9 Implementation-Class Directory	90
15.10 Cache-Topology Discovery	90
15.11 Performance-Counter Discovery	91
15.12 Address-Width Discovery	92
15.13 SAVE-Area Layout Discovery	92
16 Processor-State Save and Restore	95
16.1 Save-Area Layout	95
16.2 Fixed Architectural State	95
16.3 Extension Components and Clean State	95
16.4 SAVE Responsibilities	96
16.5 RESTORE Responsibilities	96
IV Instruction Set Reference	97
Reading an Instruction Description	98
17 General Instructions	100
17.1 Summary	100
18 Floating-Point Instructions	335

18.1	Common Floating-Point Semantics	335
18.1.1	Input, NaN, Rounding, and Result Order	335
18.1.2	Floating-Point Exception Causes and Commit	336
18.1.3	Comparison, Minimum, and Maximum	336
18.1.4	Conversions	337
18.2	Summary	337
19	Approximate Floating-Point Transcendental Instructions	424
19.1	FPTRANSA Common Model	424
19.2	Summary	425
20	Scalable-Vector Instructions	464
20.1	Element Types and Assembly Spelling	464
20.2	Predicates and Destination Images	464
20.3	Lane Mapping and Scalar Bridges	465
20.4	Vector Memory Operations	465
20.4.1	One-Lane Gather and Scatter Steps	466
20.5	Floating-Point Exceptions, Conversions, and Reductions	466
20.6	Execution and Saved State	467
20.7	Summary	467
A	Reference Indexes	673
A.1	Canonical Field Names	673
A.2	Architectural State Index	673
A.3	Architectural Event Index	675
A.4	CPUID Feature Index	675

LIST OF TABLES

2-1	Reserved Field Defaults	8
3-1	Implementation-Defined Disclosure Register	11
4-1	FLAGS Fields	13
4-2	STATUS Fields	13
4-3	Floating-Point State Fields	14
4-4	Segment Register Fields	15
4-5	Segment Register Operand Encoding	15
4-6	Control-Register Selectors	16
4-7	WRCR Selector Rules	16
4-8	PTCR Fields	17
4-9	UINFO Fields	18
4-10	ASCR Fields	18
4-11	ECR Fields	18
4-12	UCTL Fields	19
4-13	PMC Fields	19
5-1	Integer Data Sizes	20
5-2	Floating-Point Scalar Sizes	22
6-1	Encoded Instruction Length Truth Table	26
6-2	Encoding Class Summary	27
6-3	Immediate Operand Interpretation	28
7-1	Compact Effective-Address Profile Encoding	36
7-2	EA Payload Names	39
8-1	Segment Pre-Translation Modes	49
8-2	Leaf Descriptor Fields (P=1, T=0)	53
8-3	Leaf AM Encodings	53
8-4	Table-Pointer Descriptor Fields (P=1, T=1)	53
8-5	Translation-Cache Entry Identity	54
8-6	Local Translation-Cache Transitions	55
8-7	Remote Translation Shootdown Protocol	55
9-1	Ordinary Implicit Stack Operations	58
10-1	Condition Code Encoding	60
10-2	Integer Condition-Code Bits	61
11-1	Normal-Memory Cache Policies	63
11-2	Atomic Memory-Order Selectors	64

11-3	Fence Ordered-Before Pairs	65
14-1	Architectural Event Classes	70
14-2	Assigned Base Exception IDs	70
14-3	Architectural Event Boundaries and Priority	71
14-4	Architectural Event Producers and Delivery State	71
14-5	Supervisor Stack Roles	73
14-6	Architectural Event Frame Fields	75
14-7	FRAME_CONTROL Fields	75
14-8	Architectural Event Frame Types and Sizes	76
14-9	Event-to-Frame-Type Assignment	76
14-10	Address-Context ERROR_CODE Fields	77
14-11	PAGE_FAULT Causes	77
14-12	ACCESS_FAULT Causes	77
14-13	ILLEGAL_INSTRUCTION Causes	78
14-14	INVALID_CONTROL_STATE Causes	78
14-15	VECTOR_RANGE_ERROR Causes	78
14-16	DOUBLE_FAULT Delivery Stages	79
14-17	MACHINE_CHECK ERROR_CODE Fields	80
14-18	MACHINE_CHECK Valid Continuation Combinations	80
14-19	BUS_ERROR ERROR_CODE Fields	81
14-20	Warm RESET Contract	82
14-21	Architectural Reset State	82
15-1	CPUID Query Selector	84
15-2	CPUID Common Leaf Header	85
15-3	CPUID Class and Leaf Directory	86
15-4	Base Identity Indexes	87
15-5	Optional-Extension Feature Bits	88
15-6	FPTRANSA Accuracy Result	89
15-7	FPTRANSA Accuracy Contracts	89
15-8	Cache-Maintenance Properties	91
15-9	Cache-Topology Descriptor	91
15-10	SAVE-AREA-LAYOUT Indexes	92
15-11	SAVE Component Descriptor A	93
15-12	SAVE Component Descriptor B	93
15-13	Floating-Point SAVE Component	93

15-14	Scalable-Vector SAVE Component (B = VLEN bytes)	94
16-1	SAVE/RESTORE Fixed Base Block	95
17-1	General Instructions Summary (Informative)	100
18-1	Floating-Point Instructions Summary (Informative)	337
18-2	FMOVCR Constant IDs (IEEE-754 binary64)	391
19-1	Approximate Floating-Point Transcendental Instructions Summary (Informative)	425
20-1	Scalable-Vector Instructions Summary (Informative)	467
A-1	Canonical Field Names	673
A-2	Architectural State Index	673
A-3	Architectural Event Index	675
A-4	CPUID Feature Index	675

LIST OF FIGURES

4-1	Architectural Register Model	12
4-2	FLAGS Register Format	13
4-3	STATUS Register Format	13
4-4	FFLAGS Register Format	14
4-5	FSTATUS Register Format	14
4-6	Segment Register Format	15
5-1	Rn Register Operand Subfields	21
5-2	Little-Endian Byte Order for a 64-Bit Value	21
5-3	Instruction Start Bytes	21
5-4	Floating-Point Register Encodings	22
6-1	Instruction Start Byte Formats	25
6-2	Opcode-space Allocation Namespaces	26
6-3	SETF Flag Bitmap	28
6-4	Short Encoding Layout for MOV.X(z:L/Q) Rn(s), Rn(d)	28
6-5	Medium Encoding Layout for MOV.X(z:B/W) Rn(s), <ea>(e)	28
6-6	Long Encoding Layout for ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)	29
7-1	Compact Effective-Address Field	38
7-2	EXT1 Descriptor Forms	38
7-3	EXT2 Descriptor Byte Layouts	39
8-1	Address Translation Pipeline	48
8-2	Linear-Address Paging Fields	50
8-3	LA48 Four-Level Page Walk	51
8-4	LA57 Five-Level Page Walk	52
14-1	Architectural Event Code	70
14-2	Architectural Event Frame	75
14-3	FRAME_CONTROL Format	75
14-4	PAGE_FAULT and ACCESS_FAULT ERROR_CODE Format	77
14-5	Simple Exception ERROR_CODE Formats	79
14-6	DOUBLE_FAULT Auxiliary Formats	79
14-7	MACHINE_CHECK ERROR_CODE Format	80
14-8	BUS_ERROR ERROR_CODE Format	81
15-1	CPUID Query Selector Format	84
15-2	CPUID Common Leaf Header Formats	85
15-3	CPUID Base Identity Result Formats	87

15-4	Optional-Extension Directory Result	88
15-5	FPTRANSA Accuracy Result Format	89
15-6	Cache-Maintenance Properties Result	91
15-7	Cache-Topology Descriptor Format	91
15-8	Performance-Counter Presence Result	92
15-9	SAVE-Area Layout CPUID Result Formats	93
16-1	SAVE/RESTORE Base Header	95

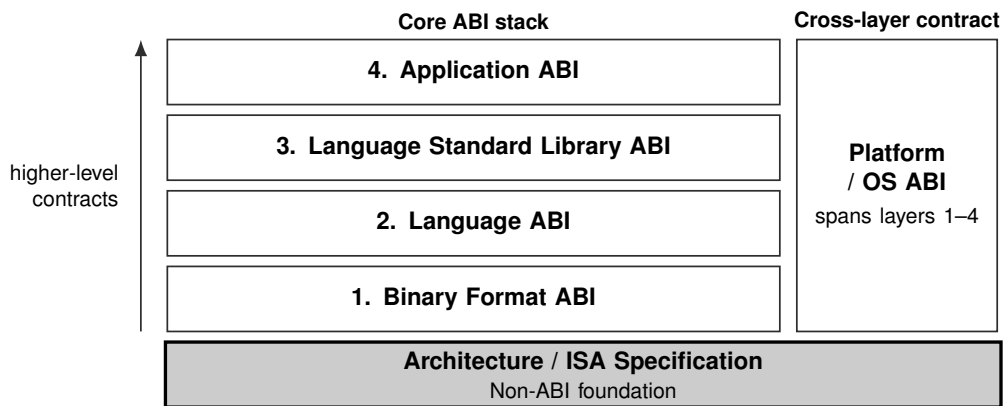
PART I

BEDROCK ARCHITECTURE

Architectural Foundations

1 OVERVIEW

1.1 Contract Position



Current document: Bedrock Programmer's Reference Manual. The shaded block identifies its primary contract position. The Platform / OS ABI spans and constrains the four core ABI layers; the ISA specification is their non-ABI architectural foundation.

1.2 Manual Scope

This manual defines programmer-visible architectural state, operand encoding, execution semantics, and instruction behavior for the Bedrock instruction set architecture.

1.3 Architectural Profile

Bedrock defines sixteen 64-bit general-purpose registers, R0 through R15. SP and PC are special architectural registers outside the Rn namespace, with SP-relative addressing tied to SS and PC-relative addressing tied to CS.

- The PC names a byte in the instruction stream; sequential execution advances it by the encoded instruction length determined during decoding.
- Encoded instruction length is explicit and bounded; the base profile defines encoded instruction lengths from 1 to 18 bytes.
- Four integer operation sizes: B, W, L, and Q.
- The optional scalable-vector extension provides 32 VLEN-bit vector registers and 16 packed predicate registers; VLEN is a reset-stable power of two from 128 through 2048 bits.
- The effective-address model covers register, memory, immediate, absolute, segment-qualified, indexed, and auto-update operands.
- Segment pre-translation precedes optional page-table translation.
- User and supervisor modes share the instruction set, with privileged instructions and checked user-access moves defining supervisor-only behavior.

2 TERMINOLOGY AND COMPATIBILITY

2.1 Address Values

An *effective address (EA)* is the address value or operand designator produced by an addressing mode before segment pre-translation. A memory EA produces an address, while register and immediate EAs name non-memory operands.

A *pre-segment virtual address* is a virtual-address value before segment pre-translation. Segment pre-translation either converts it to a linear address or reports the architecturally defined fault.

A *linear address* is the result of segment pre-translation. The page-table walker consumes it when paging is enabled; otherwise the memory system uses it directly.

A *physical address* is the byte address produced by page-table translation. With paging disabled, the linear address is used directly after the implementation PABITS check.

A *memory-system address* is the byte address presented to the memory subsystem after every active architectural translation stage.

Normal memory is the coherent memory-location model. *Device memory* is the platform-owned physical class that requires MMIO execution rules. A leaf AM selects Normal or MMIO access semantics, while the orthogonal CP field selects one of four cache policies.

Address generation is the instruction-local calculation that combines registers, displacement, index, scale, and immediate fields into an effective address. It does not by itself read memory.

2.2 Memory Behavior

A *memory access* is an individual architectural instruction-fetch, read, or write attempt. A *memory operation* is an instruction-level semantic operation that may contain one or more memory accesses or memory events. A *memory event* is a read or write unit that participates in coherence or memory ordering. A *memory effect* is a result that becomes observable to another logical processor or an external system.

The term *ordinary* classifies an operation, such as an ordinary load or store as distinct from an atomic or cache-maintenance operation. It does not identify a memory type.

2.3 Segmentation and Paging

Segment pre-translation is the stage between effective-address calculation and paging. A disabled segment passes the EA through. A translated window checks an offset and adds its base, while a bounds-only window checks an already-linear address without adding the base.

A *segment register image* is the 64-bit architectural value containing the segment base page, size exponent, size mantissa, and bounds-only enable bit.

A *disabled segment* has a zero size mantissa. It performs no segment-window check or base addition and passes the effective address through as the linear address.

A *translated segment* is enabled with bounds-only mode clear. It checks the effective address as an offset within the segment span and then adds the segment base.

A *bounds-only segment* is enabled with bounds-only mode set. It treats the effective address as already linear and checks only that it lies inside the segment window.

Page-table translation is the optional PTCR.PE-controlled stage that maps a canonical linear address to a physical frame and applies effective permissions, access class, cache policy, and A/D state. Bit 7 is A in both descriptor layouts.

A *PFN* is a physical frame number. A leaf PTE at level L combines its naturally aligned PFN base with the low $12 + 9(L - 1)$ linear-address bits to form a byte address.

2.4 Instruction Encoding

An *opcode* is the value that selects an instruction operation. The *opcode space* is the contiguous leading region of an encoded instruction, from byte 0 through the byte immediately before the first payload. It includes the header and may contain operand or control fields in addition to the opcode.

The *header* is the leading byte range within the opcode space that determines the encoded instruction length and opcode-space selection. It is the first byte for extrashort and short instructions and the first two bytes for medium and longer instructions.

A *payload* is an independently determined unit after the opcode space that supplies additional information and increases the required instruction length. Adjacent units are separate payloads when their presence, length, or meaning is determined independently. A *field* is a defined bit region within the opcode space or a particular payload. An undivided payload may carry its value without defining a separate field.

The *required instruction length* is the opcode-space length plus the length of every payload. It excludes padding. The *encoded instruction length* is the required instruction length plus any trailing padding in the encoded record. Padding carries no information and is not a payload.

2.5 Control Flow

A *near control transfer* is a CALL, RET, JMP, or conditional branch that changes only PC within the current code-segment context.

A *segment-changing control transfer* updates CS and PC in one architectural commit. Long jumps, long calls, and long returns belong to this category.

Long Call, *Long Jump*, and *Long Return* name the corresponding instruction families. A *long return frame* is the two-value return context created by Long Call and consumed by Long Return.

2.6 Tracing Terms

The *TRACE instruction* records its immediate marker and current instruction boundary in the implementation trace stream when that stream is enabled. A *trace unit* is the execution-completion unit sampled by STATUS.TF. *DEBUG_TRACE* is the architectural event delivered after successful completion of a TF-selected trace unit. These names identify the marker instruction, execution boundary, and event, respectively.

2.7 Architectural State

A *logical processor* is an architecturally independent execution context and owns its architectural state. A software *thread* may execute on a logical processor but is not the architectural execution subject.

The *general-purpose registers* are the sixteen 64-bit registers R0 through R15. An *Rn register* is one of those registers selected through the four-bit Rn operand namespace. It may hold integers, addresses, counters, selectors, or temporary data according to the instruction encoding.

A *vector register* is one of the 32 VLEN-bit registers V0 through V31. A *predicate register* is one of P0 through P15 and contains one governing bit for each byte of a vector register. A *logical vector lane* is one selected-width element; predicate bit iE governs lane i for an E -byte element. VLEN is the reset-stable implementation-selected vector register width in bits.

The *stack pointer register* is SP. It lies outside the four-bit Rn namespace and uses SS as the fixed segment for SP-relative addressing.

A *segment register operand* is a 64-bit segment register exposed through the SREG operand class. The SREG selector table maps its eight encodings to DS, SS, and GS0 through GS5. RDSEG CS and PUSH CS read the current CS image through their fixed CS encodings.

A *state register* is named architectural state such as FLAGS or STATUS. RDFLAGS, WRFLAGS, RDSTATUS, and WRSTATUS access these registers.

A *control register* is a privileged register exposed through the CR operand class, such as PTCR, ASCR, ECR, EPC, UCTL, and UINFO.

Architectural state is programmer-visible current state. A *register image* is the complete bit representation read from or written to one register. A *processor-state image* is an aggregate representation of multiple architectural-state components, such as a SAVE area or reset-state table. *Processor state* names the complete state set and the SAVE/RESTORE facility. A working copy used during operand evaluation is *temporary operand-evaluation state*.

The *program counter*, or *PC register*, is the architectural state object that selects instruction execution. An *instruction address* is the numeric address of an instruction. A *continuation*, or *continuation PC*, is a saved or selected resumption point.

2.8 Floating-Point and Architectural Exceptions

An *architectural exception* is a synchronous architectural event delivered as a fault or trap. A *floating-point exception condition*, or *floating-point exception cause*, is an IEEE-754 condition such as NV, DZ, OF, UF, or NX. A *floating-point exception flag* is the corresponding accrued FFLAGS bit. An enabled floating-point exception condition raises the FLOATING_POINT_FAULT architectural event before the instruction commits the effects suppressed by that event.

2.9 Compatibility

Reserved means that software must not depend on the field's value or behavior. *Must be zero* requires software to write zero, and *read-as-zero* means reads return zero while the field remains reserved. *Ignored* means hardware accepts the field without using it for the current operation. *Preserved* requires software to retain the previous value when rewriting the containing object.

Software-defined state is reserved for software use; hardware interpretation is limited to behavior explicitly defined by the containing structure. *Implementation-defined* behavior is selected by the implementation and published through CUID, platform documentation, or firmware. *Illegal* use of an opcode, effective-address form, selector, or operation raises the exception named by the defining rule.

2.10 Compatibility Contract

This section defines the default compatibility contract for reserved fields and optional features. A later section may narrow one of these defaults for a specific structure, but it must say so explicitly. This manual treats reserved and unallocated encodings as one architectural category: software must not depend on their value, decode result, or behavior.

2.11 Reserved Fields

Architected registers and architectural memory structures use these defaults unless their defining section gives a narrower rule.

Software-defined fields are not reserved fields. Software may store values in them, and hardware ignores them except where the containing structure explicitly says otherwise.

Table 2-1. Reserved Field Defaults

Field Class	Read Rule	Write or Use Rule	Fault
Architected register reserved bits	zero	must be zero	—
Control-register reserved bits	zero	must be zero	INVALID_CONTROL_STATE
Reserved selector values	—	invalid selector	INVALID_CONTROL_STATE
Consumed reserved PTE bits	—	invalid translation input	PAGE_FAULT
Reserved event-code classes or IDs	—	must not be generated by the base profile	—
Reserved architectural event-frame bits	—	must be zero	INVALID_CONTROL_STATE on ERET

2.12 Reserved Encodings

Reserved instruction opcodes, reserved extension opcodes, reserved effective-address forms, and optional instruction-group encodings whose CPUID feature bit is clear raise `ILLEGAL_INSTRUCTION` during decode.

Noncanonical encodings are invalid unless an instruction description explicitly defines an alias or priority rule. Assemblers emit canonical encodings by default; disassemblers prefer canonical spellings.

2.13 CPUID Compatibility

CPUID is the compatibility boundary for optional architectural behavior. Software must not use an optional instruction group, optional register state, or optional state image unless the corresponding CPUID bit or leaf reports support.

Unknown selectors return zero. Software ignores reserved CPUID result bits. CPUID is unprivileged. For a logical processor, CPUID results remain stable until that logical processor is reset.

3 CONFORMANCE

3.1 Conformance Claim

A Bedrock implementation conformance claim identifies the architecture revision reported by CPUID and the CPUID feature set for the logical processor to which the claim applies. For that revision and discovered feature set, the implementation accepts and executes every architecturally valid encoded instruction according to its encoding, state, architectural-event, memory, and instruction-specific rules. The same claim covers the architected discovery results, processor-state images, event frames, and reset state defined for that logical processor.

3.2 Normative Material and Specificity

Architecture prose, ordered steps, architecture tables, instruction encoding diagrams, and the Operation and Detailed Semantics fields of instruction entries are normative unless a section explicitly identifies material as a summary, example, or validation status.

Rule set or provider	Covered material
Instruction definitions	Concrete instruction encodings, operand and effective-address grammar, extension wiring, and document ordering.
Instruction Execution Model common rules and instruction-entry Operation and Detailed Semantics	Executable instruction behavior, architectural state transitions, fault and commit ordering, repeat and architectural-event behavior, and single-logical-processor memory-access sequencing.
Formal memory model	Permitted cross-logical-processor ordering and visibility.
ABI specifications	ELF, C ABI, and calling-convention contracts.
Compiler-interface specifications	Source-language and compiler-facing target-interface contracts.
External numerical floating-point providers	Numerical results and certificates only; input validation and the resulting architectural trap or commit behavior follow the applicable Instruction Execution Model common rules and instruction entry's Operation and Detailed Semantics.

Presented material remains normative where declared and follows the applicable rule set. For executable behavior, readers apply the Instruction Execution Model common rules together with the applicable instruction entry's Operation and Detailed Semantics. The following specificity rules govern narrowing among applicable rules:

1. Opcode-space length, required instruction length, encoded instruction length, field values, and decoding validity follow the selected instruction encoding, its field constraints, and the Instruction Encoding and Effective Addressing chapters.
2. Execution of a decoded instruction follows its Operation and Detailed Semantics. An instruction-specific rule narrows the common execution, operand, flag, memory, or event rule that it names.

3. Common architectural behavior follows the chapter that defines that behavior. A structure-specific rule narrows the defaults in Terminology and Compatibility.
4. Summary tables and examples provide navigation or explanatory context and do not replace a normative rule.

Two normative statements at the same specificity are required to agree. A discrepancy at that level is a specification defect and is resolved by correcting the normative sources and their derived tables together.

3.3 Implementation-Defined Disclosures

An implementation-defined selection is stable for the implementation revision named by CPUID and is published through the channel named by its defining rule. The publication identifies the selected value or behavior and the processor revisions to which it applies.

Table 3-1. Implementation-Defined Disclosure Register

Item	Defining rule	Publication
<code>cpuid_higher_classes</code>	CPUID Feature Discovery, Base Identity and Leaf Directory	CPUID class and leaf directories plus implementation documentation for the class payload
<code>performance_counter_ids</code>	CPUID Performance-Counter Discovery and RDPMC	CPUID PERFORMANCE_COUNTERS presence results plus implementation documentation naming each counter event
<code>event_aux_syndrome</code>	Machine-check EVENT_AUX payload	platform documentation or firmware interface documentation for the event source
<code>fptransa_approximation</code>	FPTRANSA Common Model and CPUID accuracy contracts	CPUID accuracy-contract discovery plus implementation documentation for the deterministic approximation

4 PROGRAMMING MODEL

4.1 Register Model

The general-purpose register set contains sixteen 64-bit registers, R0 through R15. Instruction encodings use these registers for integer values, addresses, counters, selectors, and temporary data.

SP and PC are special architectural registers outside the four-bit Rn namespace. SP-relative memory forms use SS by default; PC-relative memory forms use CS by default.

CS, DS, SS, and GS0 through GS5 are 64-bit architectural segment registers. Segment-qualified EA forms select them explicitly, while SP and PC forms omit the segment field because their segment association is fixed.

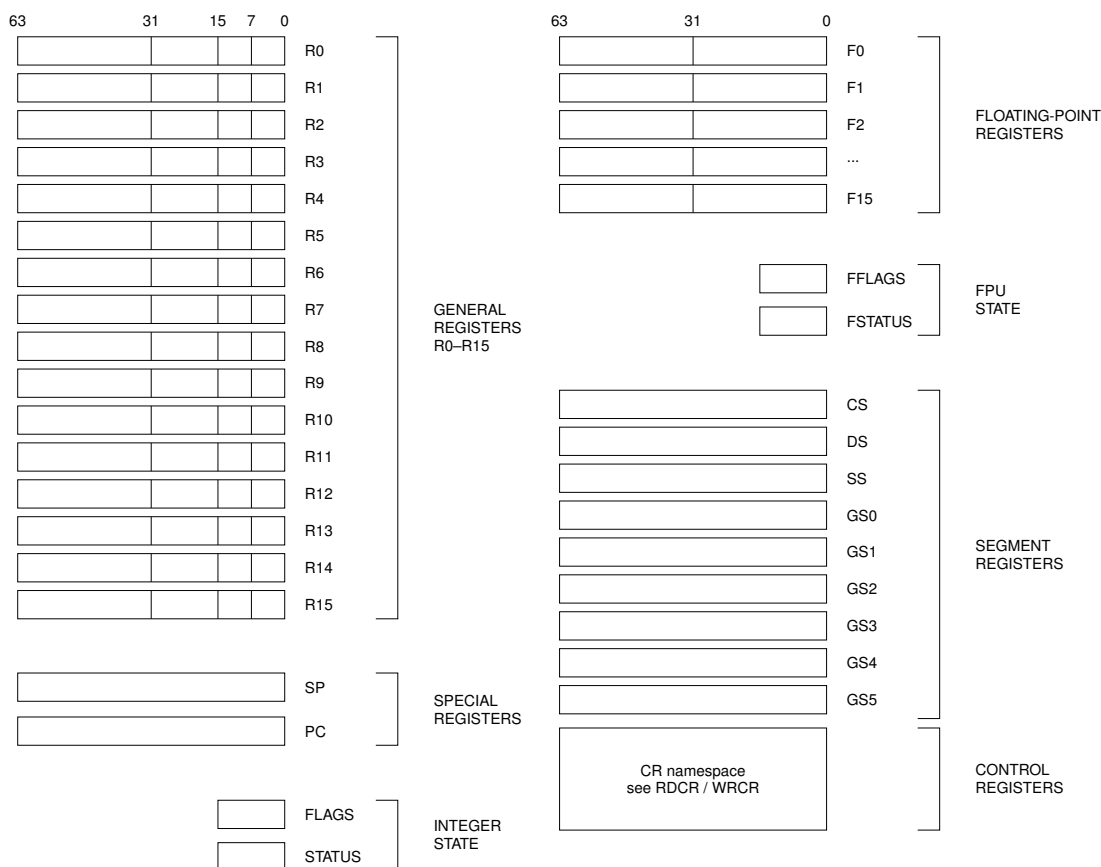


Figure 4-1. Architectural Register Model

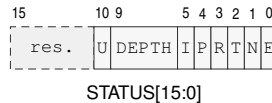
4.2 FLAGS and STATUS Registers

FLAGS and STATUS are accessed through dedicated read/write instructions.

**Figure 4-2. FLAGS Register Format****Table 4-1. FLAGS Fields**

Field	Bits	Meaning
reserved	15..4	reads as zero and must be zero when written
Z	3	result value is zero
N	2	most significant result bit at the operand size
C	1	unsigned carry out for addition or unsigned borrow for subtraction
V	0	signed overflow or an explicitly reported exceptional condition

RDFLAGS and WRFLAGS are unprivileged. RDFLAGS reads and zero-extends the 16-bit image; WRFLAGS writes the low 16 bits and requires every reserved bit to be zero. Instruction-specific flag production and conditional use are defined in Flags and Condition Codes.

**Figure 4-3. STATUS Register Format**

In the diagram, U, DEPTH, I, P, R, T, N, and E abbreviate U0, EDEPTH, IE, PM, RF, TF, NI, and EA.

Table 4-2. STATUS Fields

Field	Bits	Meaning
reserved	15..11	reads as zero and must be zero when written
U0	10	outer active event originated in user mode
EDEPTH	9..6	current architectural event depth
IE	5	permits maskable-interrupt admission when the other admission conditions hold
PM	4	selects user mode when zero and supervisor mode when one
RF	3	one-shot suppression of DEBUG_TRACE
TF	2	requests post-success DEBUG_TRACE for a trace unit when RF is clear
NI	1	inhibits NMI admission while preserving the NMI pending
EA	0	hardware-managed architectural event-active state

In user mode, PM, EA, EDEPTH, and U0 are always observed as zero.

RDSTATUS is unprivileged; WRSTATUS is supervisor-only. WRSTATUS atomically updates IE, NI, TF, and RF; reserved bits must be zero, and supplied PM, EA, EDEPTH, and U0 must equal their current values. Privilege transitions are defined in the Privileged Execution Model; event admission, tracing, and event-active transitions are defined in the Architectural Event Processing Model.

4.3 Floating-Point Register Model

The floating-point extension exposes F0 through F15 as 64-bit registers when the floating-point instruction group is implemented.

4.4 FFLAGS and FSTATUS Registers

FFLAGS holds accrued IEEE-754 floating-point exception flags. FSTATUS holds the matching exception-condition enables, rounding mode, and optional non-default floating-point modes. Both registers are present only with the floating-point extension and reset to zero.

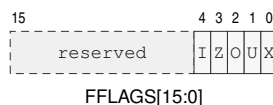


Figure 4-4. FFLAGS Register Format

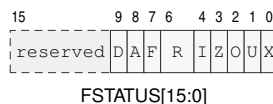


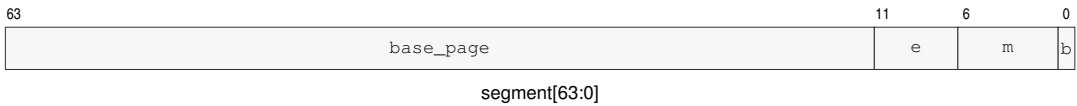
Figure 4-5. FSTATUS Register Format

In the diagrams, I, Z, O, U, and X denote the NV, DZ, OF, UF, and NX flag or enable bits. In FSTATUS, D, A, F, and R denote DN, DAZ, FTZ, and RM.

Table 4-3. Floating-Point State Fields

Field	Bits	Meaning
NV . . NX	4 . . 0	accrued invalid, divide-by-zero, overflow, underflow, and inexact flags
NV_EN . . NX_EN	4 . . 0	floating-point exception-condition enables corresponding to FFLAGS.NV through FFLAGS.NX
RM	6 . . 5	0 nearest-even; 1 toward zero; 2 toward negative; 3 toward positive
FTZ	7	flush tiny results to signed zero
DAZ	8	treat subnormal inputs as signed zero
DN	9	produce the architectural default NaN
reserved	15 . . 10	must be zero

4.5 Segment Registers

**Figure 4-6. Segment Register Format****Table 4-4. Segment Register Fields**

Field	Bits	Meaning
base_page	63..12	unsigned 4-KiB page number from which <i>base</i> is derived
e	11..7	unsigned exponent used to derive <i>span</i>
m	6..1	unsigned mantissa used to derive <i>span</i> ; zero selects the disabled image
b	0	for an enabled image, zero selects translated-window mode and one selects bounds-only mode

The following quantities are evaluated mathematically without 64-bit truncation:

$$base = base_page \times 4096,$$

$$span = m \times 2^e \times 4096,$$

$$limit = base + span.$$

A segment image with *m*=0 is a valid disabled image. An enabled image is valid only when $limit \leq 2^{64}$. The disabled, translated-window, and bounds-only address checks and their fault rules are defined in Segment Pre-Translation.

4.6 Segment Register Operand Class

Instructions that take an explicit segment-register operand use the three-bit SREG encoding below.

SP-relative EA forms do not carry this field and use SS. PC-relative EA forms do not carry this field and use CS.

SREG selects DS, SS, and GS0 through GS5. Instruction fetch and fixed PC-relative addressing select CS implicitly. RDSEG CS and PUSH CS read the current CS image. Segment-changing control transfers update CS and PC together.

Table 4-5. Segment Register Operand Encoding

Segment	Bits	Role	Use
DS	000	data	default data segment
SS	001	stack	stack segment; fixed for SP-relative forms
GS0	010	general	general segment register 0
GS1	011	general	general segment register 1
GS2	100	general	general segment register 2
GS3	101	general	general segment register 3
GS4	110	general	general segment register 4
GS5	111	general	general segment register 5

4.7 Control Registers

RDCR and WRDCR use a 16-bit selector and transfer a 64-bit register image. Both instructions are supervisor-only. Selectors not listed below raise `INVALID_CONTROL_STATE`. Reserved register bits read as zero, and a write with a nonzero reserved bit raises `INVALID_CONTROL_STATE` without changing the register. The entry, stack, and user-return registers are per-logical-processor architectural state.

Table 4-6. Control-Register Selectors

Selector	Register	Use
0x0000	PTCR	page-table root and translation control
0x0001	ASCR	address-space identifier control
0x0002	ECR	architectural event-delivery control
0x0108	UPC	user-return program counter
0x0109	USP	user-return stack pointer
0x010a	UCS	user-return code-segment image
0x010b	UDS	user-return data-segment image
0x010c	USS	user-return stack-segment image
0x010d	UCTL	user-return FLAGS, STATUS, and valid state
0x010e	UINFO	user-origin outer-event code
0x0110	EPC	common architectural event-entry PC
0x0111	ECS	architectural event-entry code-segment image
0x0112	EDS	architectural event-entry data-segment image
0x0200	SSS	user-origin SYSTEM_CALL initial stack-segment image
0x0201	SSP	user-origin SYSTEM_CALL initial stack top
0x0210	ISS	maskable-interrupt stack-segment image
0x0211	ISP	maskable-interrupt stack top
0x0220	FSS	fault/exception stack-segment image
0x0221	FSP	fault/exception stack top
0x0230	DSS	double-fault stack-segment image
0x0231	DSP	double-fault stack top
0x1000	BOOTPC	warm-reset target supplied by the platform at cold reset
0x1001	BOOTCFG	opaque platform boot configuration preserved by warm reset
0x1100	PMC	performance-monitor enable

Table 4-7. WRDCR Selector Rules

Register	Writable Image	Validation	Commit and Side Effect
PTCR	root_page, LA57, and PE; reserved bits must be zero	the root frame fits the implementation PABITS value reported by CPUID ADDRESS_WIDTHS	apply the translation-cache transition selected by the changed fields
ASCR	ASID and AE; reserved bits must be zero	the complete ASCR image is validated before commit	apply the translation-cache transition selected by the changed fields
ECR	MAX_EDEPTH and V; NMI_P is hardware managed and the supplied value is ignored	setting V validates EPC, ECS, EDS, and all configured event stack pairs	replace the complete writable image atomically

Table 4-7 (continued)

Register	Writable Image	Validation	Commit and Side Effect
EPC	complete 64-bit entry address	16-byte aligned, canonical, and executable under the matching code-segment image	replace the selected register atomically
ECS, EDS, SSS, ISS, FSS, DSS	complete 64-bit segment image	apply the architectural segment-image validity rule	replace the selected register atomically
SSP, ISP, FSP, DSP	complete 64-bit configured stack top	64-byte aligned	replace the selected register atomically
UPC, USP, UCS, UDS, USS	complete 64-bit saved user-return value	defer complete-bank validation until UCTL.V is set	replace the selected register atomically
UINFO	EVENT_CODE in bits 31 through 0; bits 63 through 32 must be zero	reject a nonzero reserved bit; validate the event-code-dependent bank when UCTL.V is set	replace UINFO atomically
UCTL	V, STATUS, and FLAGS; reserved bits must be zero	setting V validates the complete user-return bank and requires saved PM, EA, EDEPTH, and UO to be zero	replace UCTL atomically after complete-bank validation
BOOTPC	complete 64-bit warm-reset target	the value is used as the exact PC after warm RESET	replace BOOTPC atomically
BOOTCFG	complete opaque 64-bit platform boot configuration	no image transformation	replace BOOTCFG atomically
PMC	EN; reserved bits must be zero	EN is Boolean	replace PMC atomically



PTCR[63:0]

PTCR Format

In the diagram, L and P abbreviate LA57 and PE.

Table 4-8. PTCR Fields

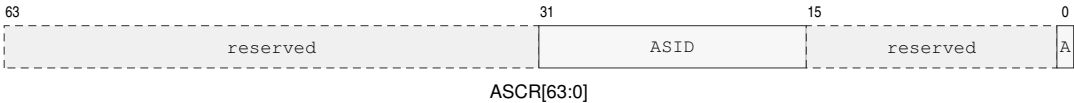
Field	Bits	Meaning
reserved	63..56	must be zero in this page-table format
root_page	55..12	physical frame number of the page-table root; must fit implementation PABITS
reserved	11..8	must be zero
LA57	7	zero selects four levels; one selects five levels
reserved	6..1	must be zero
PE	0	page-table translation enable



UINFO Format

Table 4-9. UINFO Fields

Field	Bits	Meaning
reserved	63 . . 32	must be zero
EVENT_CODE	31 . . 0	code for the outer user-origin event and its optional payload shape



ASCR Format

A abbreviates AE in the diagram.

Table 4-10. ASCR Fields

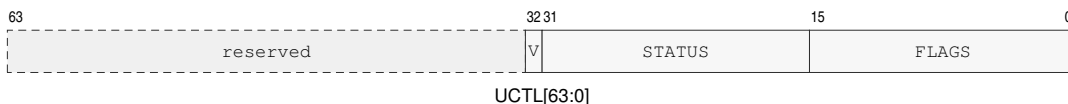
Field	Bits	Meaning
reserved	63 . . 32	must be zero
ASID	31 . . 16	current address-space identifier
reserved	15 . . 1	must be zero
AE	0	address-space identifier matching enable



ECR Format

Table 4-11. ECR Fields

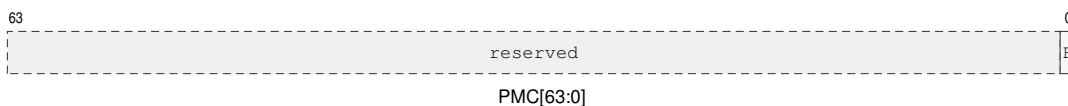
Field	Bits	Meaning
reserved	63 . . 12	must be zero
MAX_EDEPTH	11 . . 8	maximum depth at which a maskable interrupt may be admitted
NMI_P	7	hardware-managed coalesced pending-NMI state while entry is invalid or NI is set; WRCR ignores the supplied bit
reserved	6 . . 1	must be zero
V	0	architectural event-entry state is valid



UCTL Format

Table 4-12. UCTL Fields

Field	Bits	Meaning
reserved	63 . . 33	must be zero
V	32	complete user-return bank contains a valid return context
STATUS	31 . . 16	saved user STATUS image
FLAGS	15 . . 0	saved user FLAGS image



PMC Format

E abbreviates EN in the diagram.

Table 4-13. PMC Fields

Field	Bits	Meaning
reserved	63 . . 1	must be zero
EN	0	enable performance-counter increments when set

EPC is the exact common event-entry address, must be 16-byte aligned, and must be canonical and executable under ECS. SSP, ISP, FSP, and DSP are 64-byte-aligned configured stack tops and are not modified by entry or return. ECS, EDS, SSS, ISS, FSS, and DSS contain validated segment images. Selectors 0x0100–0x0102 are reserved. A write that establishes UCTL.V validates the complete user bank and requires saved PM, EA, EDEPTH, and UO to be zero. ERET revalidates the selected return source before committing. BOOTPC and BOOTCFG are initialized by the platform during cold reset and preserved by RESET.

5 DATA FORMATS

5.1 Integer Data Sizes

Integer operands use size suffixes to select the low-order subfield of an R_n register and the number of bytes transferred by memory operands.

Byte, word, and long operations use the low-order subfield of the selected register. Q-sized operations use the full 64-bit register.

Every R_n destination write defines all 64 bits. A Q-sized write uses the complete result. For a B-, W-, or L-sized write, ABS, DIVS, MODS, DIVMODS, MINS, MAXS, SAR, and the explicit EXTS operations sign-extend the selected-width result to 64 bits. Every other integer operation, including MOV, neutral modular arithmetic, logical and bit operations, count and Boolean results, address results, atomic return values, and state-register reads, zero-extends the selected-width result to 64 bits. This result extension does not change FLAGS calculation, which continues to use the selected operation size.

Memory destination writes transfer only the selected number of bytes. Extension-store instructions instead write the explicitly named W, L, or Q destination width. SP has only Q-sized writable forms and is not subject to the R_n result-extension rule.

Table 5-1. Integer Data Sizes

Code	Suffix	Bits	Bytes	Name
B	.B	8	1	Byte
W	.W	16	2	Word
L	.L	32	4	Long
Q	.Q	64	8	Quad

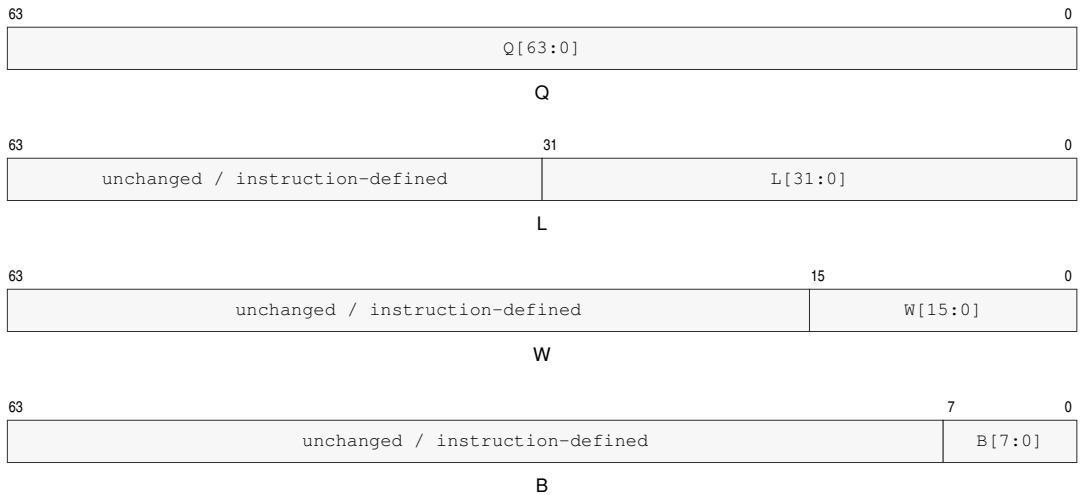


Figure 5-1. Rn Register Operand Subfields

5.2 Little-Endian Memory Organization

Integer data, immediates, displacements, and absolute address payloads are little-endian. Multi-byte numeric payloads are stored least significant byte first. The decoder consumes instruction-header fields byte by byte in instruction-stream order.

For an N-byte integer value stored at byte address A, byte A contains bits 7..0, byte A+1 contains bits 15..8, and so on.

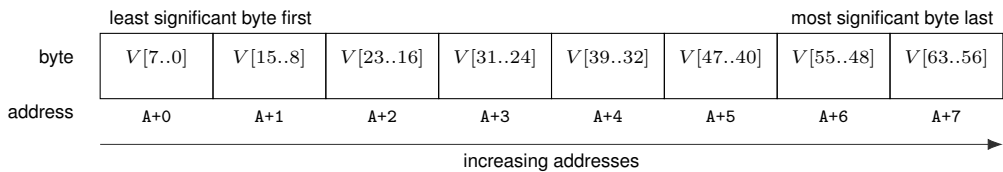


Figure 5-2. Little-Endian Byte Order for a 64-Bit Value

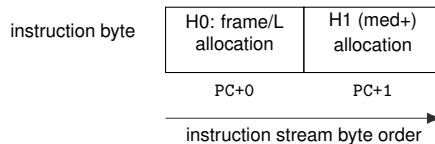


Figure 5-3. Instruction Start Bytes

H0 and H1 denote header bytes 0 and 1; H1 belongs to the header only for medium and longer instructions.

5.3 Floating-Point Data Formats

Table 5-2. Floating-Point Scalar Sizes

Code	Suffix	Bits	Bytes	Name
S	.S	32	4	Single
D	.D	64	8	Double

Floating-point scalar formats are little-endian in memory as well. S is IEEE-754 binary32 with 24 bits of significand precision, minimum normal exponent -126 , and minimum subnormal quantum 2^{-149} . D is IEEE-754 binary64 with 53 bits of significand precision, minimum normal exponent -1022 , and minimum subnormal quantum 2^{-1074} .

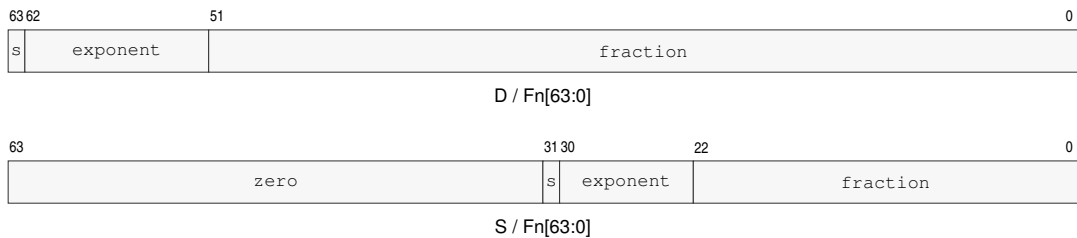


Figure 5-4. Floating-Point Register Encodings

For a NaN, the most significant fraction bit is the quiet bit: bit 22 for S and bit 51 for D. *s* denotes the sign bit. An S result occupies Fn bits 31..0 and writes zero to Fn bits 63..32.

The bit-level floating-point encoding, NaN policy, exception flags, and rounding behavior are defined by the floating-point instruction and state descriptions; their memory byte order follows the same least-significant-byte-first rule as integer data.

PART II

BEDROCK ARCHITECTURE

Encoding, Addressing, and Execution

6 INSTRUCTION ENCODING

6.1 Instruction Stream and Record Boundary

The byte addressed by PC is byte 0 of the current instruction. Instruction decoding determines the complete encoded instruction length before operand evaluation begins.

The opcode space begins at byte 0, includes the header, and ends immediately before the first payload. Any appended EA descriptor, displacement, immediate, bitmap, or instruction-specific value is a payload. Independently selected appended units are separate payloads. Bytes after the required instruction length are padding, not payloads.

The header is byte 0 for extrashort and short instructions and the first two bytes for medium and longer instructions. It determines the encoded instruction length and selects the opcode space before operand evaluation begins. An encoded instruction length that cannot contain the selected encoding raises `ILLEGAL_INSTRUCTION` before any architectural state is changed.

The required instruction length is the opcode-space length plus every required payload length. The encoded instruction length must be at least that large. For an extended instruction, every trailing byte after the required instruction length is uninterpreted padding; every byte value is valid. Padding never extends an operand, payload, descriptor, or opcode space.

Before decode validation or operand evaluation, instruction fetch and permission checks cover the complete encoded record, including padding. An inaccessible padding byte therefore reports the applicable instruction-fetch fault. An undersized record raises `ILLEGAL_INSTRUCTION.INSUFFICIENT_LENGTH` before architectural state changes.

6.2 Instruction Header Formats

Instruction framing is byte-oriented. Byte 0 alone identifies extrashort and short instructions; extended instructions use byte 0 and byte 1 to determine both encoded instruction length and opcode-space selection.

Extended byte0[5:2] is L , and the encoded instruction length is $3+L$ bytes. The opcode-space allocation bits begin at byte0[1:0], continue through byte1, and then through later opcode-space bytes as required by the encoding class.

An extended instruction is invalid if its encoded instruction length is shorter than the opcode-space length: 3 bytes for medium, 4 bytes for long, 5 bytes for extralong, and 6 bytes for xxlong.

The first eight allocation bits select the extended opcode space. The value `11111111` selects the xxlong class, whose six-byte opcode space provides 42 allocation bits. The value `11111110` is reserved for custom extensions; it retains extralong framing and is unallocated by the standard ISA. All other existing selectors retain their class and allocation.

`LEN n, <instruction>` requests an encoded extended instruction length of n bytes, where $3 \leq n \leq 18$. The requested length must cover the required instruction length. Without `LEN`, the assembler emits that required length. It fills requested trailing padding with zero. A disassembler emits `LEN n`, when the encoded instruction length exceeds the required instruction length so reassembly preserves the length; padding byte values are not preserved. Extrashort and short encodings retain their fixed lengths.

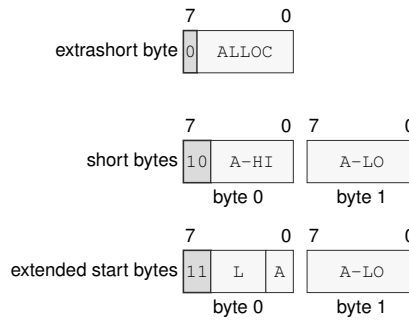


Figure 6-1. Instruction Start Byte Formats

Figure 6-1 annotations are read from high to low bit. In byte 0, bit 7 distinguishes extrashort framing; bits 7..6 select short (10) or extended (11) framing. For short instructions, bits 5..2 and 1..0 continue the opcode-space allocation bits. For extended instructions, bits 5..2 are L, bits 1..0 begin the opcode-space allocation bits, and byte 1 continues them. Thus L encodes the encoded instruction length as $3 + L$ bytes directly in Figure 6-1.

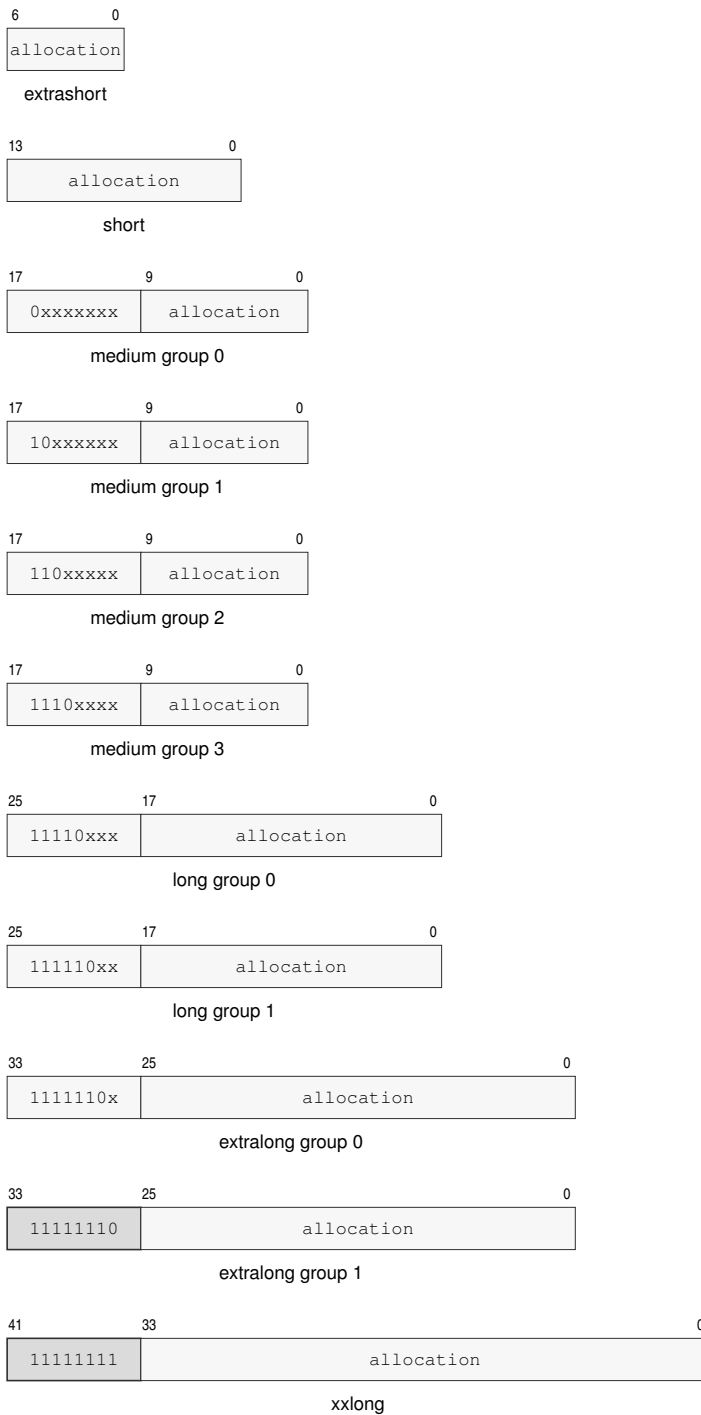


Figure 6-2. Opcode-space Allocation Namespaces

Table 6-1. Encoded Instruction Length Truth Table

Byte 0 Pattern	Byte 1 Pattern	Bytes
0xxxxxxx	—	1
10xxxxxx	xxxxxxx	2
110000oo	bbbbxxxx	3
110001oo	bbbbxxxx	4
110010oo	bbbbxxxx	5
110011oo	bbbbxxxx	6
110100oo	bbbbxxxx	7
110101oo	bbbbxxxx	8
110110oo	bbbbxxxx	9
110111oo	bbbbxxxx	10
111000oo	bbbbxxxx	11
111001oo	bbbbxxxx	12
111010oo	bbbbxxxx	13
111011oo	bbbbxxxx	14
111100oo	bbbbxxxx	15
111101oo	bbbbxxxx	16
111110oo	bbbbxxxx	17
111111oo	bbbbxxxx	18

Table 6-2. Encoding Class Summary

Class	Allocation Bits	Framing or Opcode Selector	Opcode-space Bytes	Validity
extrashort	7	0xxxxxxx	1	exactly 1 byte
short	14	10xxxxxx	2	exactly 2 bytes
medium	18	0xxxxxxx, 10xxxxxx, 110xxxxx, 1110xxxx	3	any extended encoded instruction length
long	26	11110xxx, 111110xx	4	encoded instruction length at least 4 bytes
extralong	34	1111110x, 11111110	5	encoded instruction length at least 5 bytes
xxlong	42	11111111	6	encoded instruction length at least 6 bytes

6.3 Instruction Payload Ordering

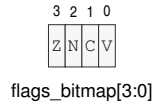
The instruction header is byte 0 for extrashort and short instructions and the first two bytes for medium and longer instructions. The opcode space may continue after the header. If a descriptor, immediate, displacement, or bitmap follows the opcode space, each independently determined unit is a separate payload, and the payloads occupy increasing addresses in decode order. An extended EA descriptor and a following displacement are therefore separate payloads.

Signed displacement and immediate payloads use two's-complement representation at their declared width before any sign extension performed by the consuming instruction or effective-address encoding.

Table 6-3. Immediate Operand Interpretation

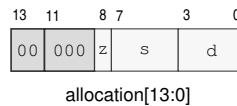
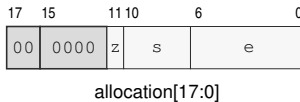
Operand	Bits	Value	Range	Extension	Applies When
PAIR _n	3	identifier	0..7	none	PUSHP/POPP canonical register-pair selector
pt_level	3	enumeration	1..5	none	PTQUERY page-table level selector
flags_bitmap	4	bitmap	0..15	none	SETF integer FLAGS image
imm6	6	unsigned	0..63	zero-extend	integer encodings with an explicit B/W/L/Q operation size
imm7	7	unsigned	0..127	zero-extend	EXTRACT register-pair encodings

PUSHP and POPP interpret the 3-bit `pair_id` as a canonical register-pair index, so every encoding from 0..7 is valid. PTQUERY uses a 3-bit `pt_level`; assigned levels are 1..5, while encodings 0, 6, and 7 are reserved. SETF interprets the 4-bit `flags_bitmap` range 0..15 as a complete FLAGS image: bit 3 writes Z, bit 2 writes N, bit 1 writes C, and bit 0 writes V. Each one writes the corresponding flag to one, and each zero writes it to zero.

**Figure 6-3. SETF Flag Bitmap**

A 6-bit `imm6` value in 0..63 is zero-extended to the selected operation size before use when that size is wider than 6 bits. EXTRACT uses the 7-bit `imm7` range 0..127 as the offset into its concatenated register pair.

6.4 Representative Encodings

**Figure 6-4. Short Encoding Layout for MOV.X(z:L/Q) Rn(s), Rn(d)****Figure 6-5. Medium Encoding Layout for MOV.X(z:B/W) Rn(s), <ea>(e)**

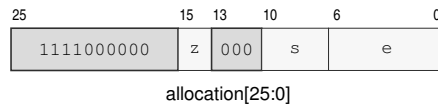


Figure 6-6. Long Encoding Layout for ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)

7 EFFECTIVE-ADDRESS PROFILES

EA, FEA, and VEA are seven-bit compact effective-address profiles selected by the consuming instruction's operand type. They share the memory address calculation rules and the EXT1 and EXT2 descriptor grammar in this section; the compact comparison table identifies the profile-specific encodings. An effective-address field is an operand descriptor with an instruction-defined role and interpretation width. A value role reads or writes the selected register, immediate, or memory value. An address role uses a register or immediate as an address value and uses a memory-form EA as the calculated address without an initial data read. A control-target role uses a register or immediate value directly and reads an interpretation-width target from a memory form. Memory forms continue into segment pre-translation and paging.

Each compact field is embedded in the instruction's opcode space, and all compact fields precede any appended effective-address payload bytes. Any displacement, absolute address, immediate, or extended descriptor bytes are appended as one operand's payload sequence. Sequences appear in declared instruction operand order. For two payload-bearing operands, the encoded stream is: opcode space containing operand 0 and operand 1 → complete operand 0 payload sequence → complete operand 1 payload sequence → any trailing padding.

Multi-byte payloads, such as displacements, absolute addresses, and immediates, are little-endian. Payload names ending in *s* are signed and are sign-extended before use; payload names without *s* are unsigned unless the consuming instruction explicitly says otherwise. Extended descriptor bytes appear in stream order, byte 0 first. Each descriptor byte is independently numbered from bit 7 through bit 0.

Indexed scale is the EA interpretation width declared by the consuming instruction encoding. B, W, L, and Q widths select scales 1, 2, 4, and 8. An encoding whose width is `operation_size` uses its selected instruction size. LEA and SEGLEA use their suffix; size-less instructions with address or control-target roles use Q.

For every compact or extended-descriptor PC-based EA, the PC term is the architectural PC naming byte 0 of the current instruction. The PC term remains that instruction-start value throughout operand evaluation.

7.1 Compact Profile Encodings

The scalar EA profile encodes common Rn/SP/PC memory operands, absolute memory operands, integer immediate operands, and the EXT1 and EXT2 escapes. FEA retains the scalar memory and EXT encodings except that 0x58, 0x5B, and 0x5C are reserved and 0x5D and 0x5E select floating-point immediates. VEA retains the scalar memory and EXT encodings except that 0x58 and 0x5B–0x5E are reserved. These values do not inherit the scalar EA forms at the same compact selectors.

Compact memory forms that do not name SP or PC use the operation default data segment. SP memory forms are fixed to SS, and PC memory forms are fixed to CS.

Rn Memory

[Rn(r)]

[Rn(r) + disp8s]

[Rn(r) + disp16s]

[Rn(r) + disp32s]

[Rn(r) + disp64]

EA encoding: 000rrrr has no displacement; 001rrrr, 010rrrr, 011rrrr, and 100rrrr select disp8s, disp16s, disp32s, and disp64.

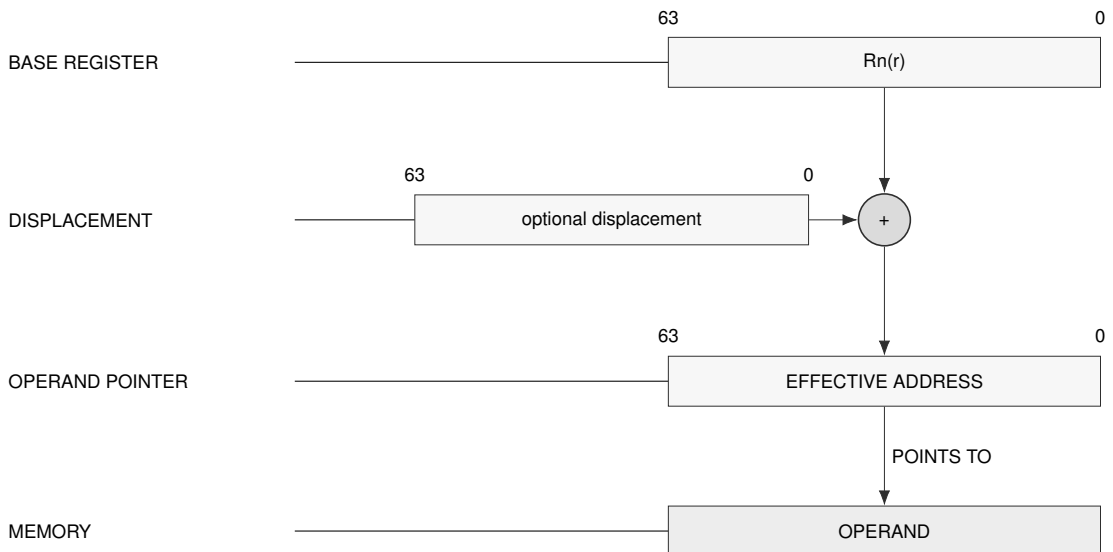
EA type: memory operand.

Generation: offset = temporary Rn(r) + displacement. The no-displacement form uses displacement 0.

Segment: Uses the operation default data segment unless the instruction defines a different default.

Payload: Only displacement forms append payload bytes. Displacement payload is little-endian and has the size named by the EA form.

Update: No auto-update in compact Rn memory forms.



Schema — Rn memory address generation

SP Memory

- [SP]
- [SP + disp8s]
- [SP + disp16s]
- [SP + disp32s]
- [SP + disp64]

EA encoding: 1011000 has no displacement; 1010000, 1010001, 1010010, and 1010011 select disp8s, disp16s, disp32s, and disp64.

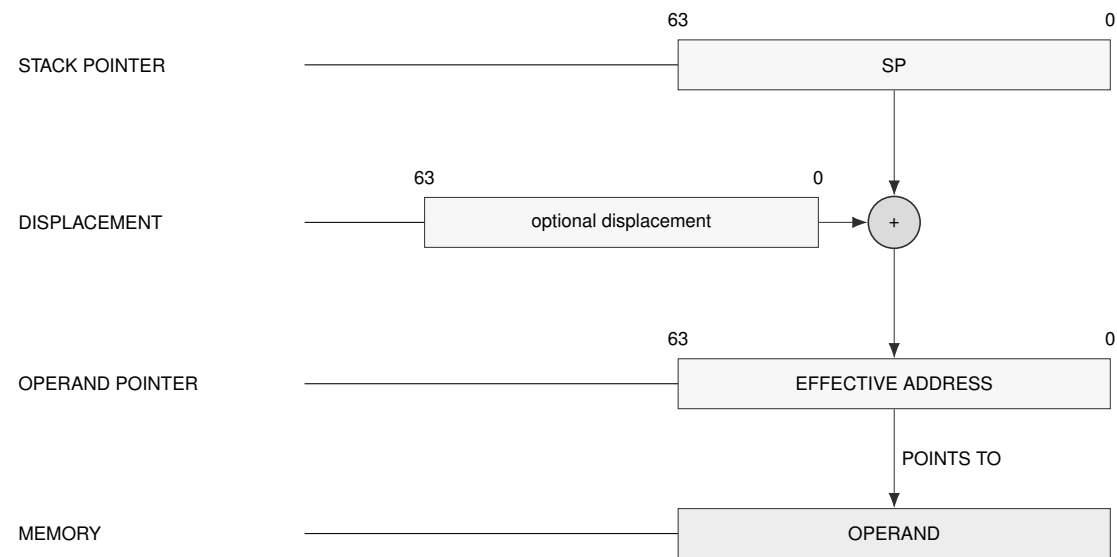
EA type: memory operand.

Generation: offset = temporary SP + displacement. The no-displacement form uses displacement 0.

Segment: Fixed to SS; no segment field is encoded.

Payload: Only displacement forms append payload bytes. Displacement payload is little-endian and has the size named by the EA form.

Update: No auto-update in compact SP memory forms.



Schema — SP memory address generation

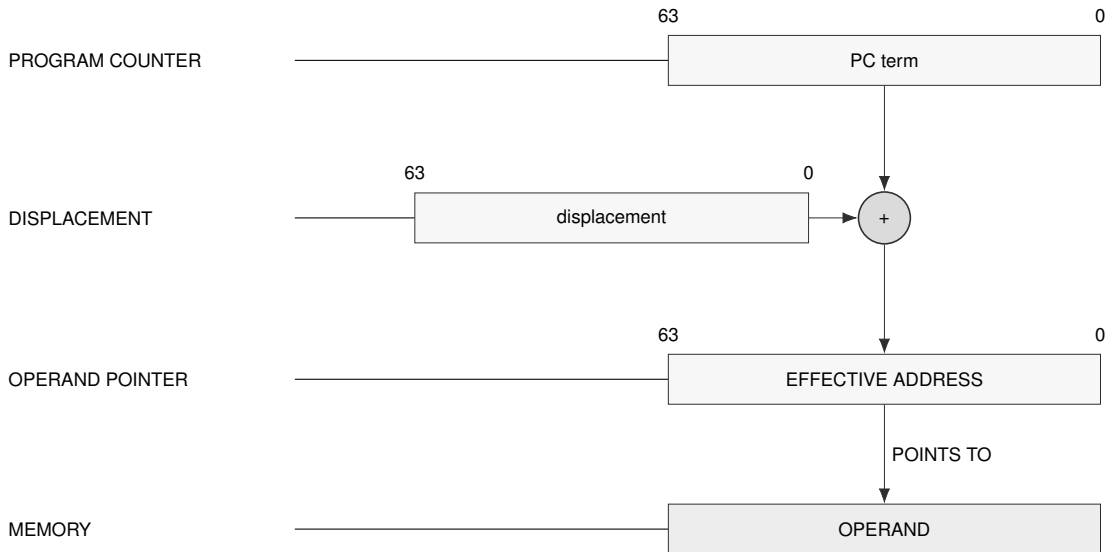
PC Memory

[PC + disp8s]

[PC + disp16s]

[PC + disp32s]

[PC + disp64]

EA encoding: 1010100, 1010101, 1010110, and 1010111 select disp8s, disp16s, disp32s, and disp64.**EA type:** memory operand.**Generation:** offset = instruction-start PC + displacement. PC names byte 0 of the current instruction.**Segment:** Fixed to CS; no segment field is encoded.**Payload:** A displacement payload is always present. Payload is little-endian and has the size named by the EA form.**Update:** No auto-update for PC forms.**Schema — PC-relative address generation**

Absolute Memory
[abs32s]
[abs64]

EA encoding: 1011001 selects abs32s; 1011010 selects abs64.

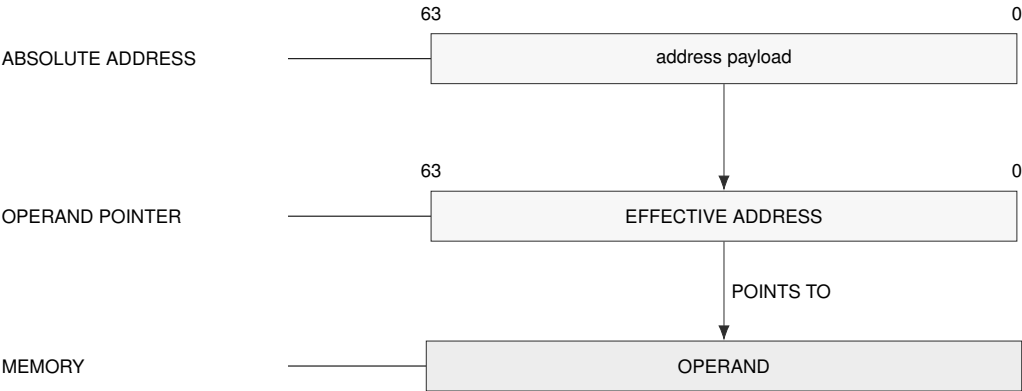
EA type: memory operand.

Generation: offset = sign-extended abs32s or unsigned abs64.

Segment: Uses the operation default data segment unless the instruction defines a different default.

Payload: The absolute address payload is little-endian and has the size named by the EA form.

Update: No auto-update.



Schema — Absolute memory address generation

Immediate
imm8s
imm16s
imm32s
imm64

EA encoding: 1011011, 1011100, 1011101, and 1011110 select imm8s, imm16s, imm32s, and imm64.

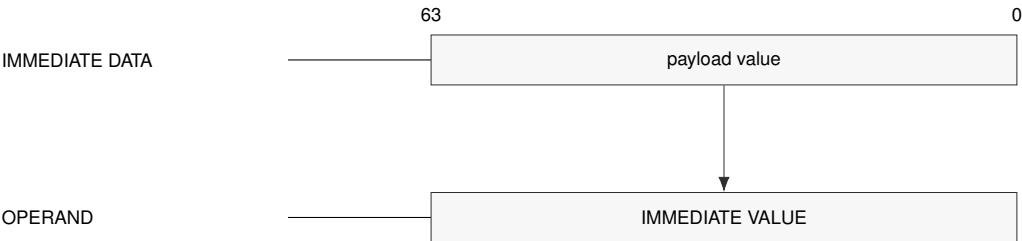
EA type: immediate operand, not a memory reference.

Generation: The operand value is decoded from the immediate payload. Signed payloads are sign-extended according to the consuming instruction's operand rule.

Segment: No segment is selected.

Payload: The immediate payload is little-endian and has the size named by the EA form.

Update: No auto-update. Immediate forms are not valid destinations unless an instruction explicitly defines such a form.



Schema — Immediate operand generation

EXT1 Escape**EXT1**

EXT1/disp8s

EXT1/disp16s

EXT1/disp32s

EXT1/disp64

EA encoding: 1100011 selects no displacement; 1011111, 1100000, 1100001, and 1100010 select disp8s, disp16s, disp32s, and disp64.

EA type: memory operand described by the appended EXT1 descriptor.

Generation: The one-byte EXT1 descriptor selects segment, base, and auto-update behavior. The compact EA escape selects the descriptor family and displacement size.

Segment: Segment-qualified forms encode SREG SEG(s); default base-update forms use the operation default data segment.

Payload: The one descriptor byte comes first, followed by the optional displacement payload selected by the compact EA escape.

Update: Only forms with ++ or -- update the temporary operand-evaluation state.

EXT2 Escape**EXT2**

EXT2/disp8s

EXT2/disp16s

EXT2/disp32s

EXT2/disp64

EA encoding: 1101000 selects no displacement; 1100100, 1100101, 1100110, and 1100111 select disp8s, disp16s, disp32s, and disp64.

EA type: memory operand described by the appended EXT2 descriptor.

Generation: The two-byte EXT2 descriptor selects segment, base, index, and auto-update behavior. The compact EA escape selects the descriptor family and displacement size.

Segment: Segment-qualified forms encode SREG SEG(s); SP and PC indexed forms use SS and CS.

Payload: The two descriptor bytes come first, followed by the optional displacement payload selected by the compact EA escape.

Update: Only forms with ++ or -- update the temporary operand-evaluation state.

Table 7-1. Compact Effective-Address Profile Encoding

Encoding	EA form	FEA form	VEA form
000rrrr (0x00--0x0F)	[Rn(r)]	[Rn(r)]	[Rn(r)]
001rrrr (0x10--0x1F)	[Rn(r) + disp8s]	[Rn(r) + disp8s]	[Rn(r) + disp8s]
010rrrr (0x20--0x2F)	[Rn(r) + disp16s]	[Rn(r) + disp16s]	[Rn(r) + disp16s]
011rrrr (0x30--0x3F)	[Rn(r) + disp32s]	[Rn(r) + disp32s]	[Rn(r) + disp32s]
100rrrr (0x40--0x4F)	[Rn(r) + disp64]	[Rn(r) + disp64]	[Rn(r) + disp64]
1010000 (0x50)	[SP + disp8s]	[SP + disp8s]	[SP + disp8s]
1010001 (0x51)	[SP + disp16s]	[SP + disp16s]	[SP + disp16s]
1010010 (0x52)	[SP + disp32s]	[SP + disp32s]	[SP + disp32s]
1010011 (0x53)	[SP + disp64]	[SP + disp64]	[SP + disp64]
1010100 (0x54)	[PC + disp8s]	[PC + disp8s]	[PC + disp8s]
1010101 (0x55)	[PC + disp16s]	[PC + disp16s]	[PC + disp16s]
1010110 (0x56)	[PC + disp32s]	[PC + disp32s]	[PC + disp32s]
1010111 (0x57)	[PC + disp64]	[PC + disp64]	[PC + disp64]
1011000 (0x58)	[SP]	reserved	reserved
1011001 (0x59)	[abs32s]	[abs32s]	[abs32s]
1011010 (0x5A)	[abs64]	[abs64]	[abs64]
1011011 (0x5B)	imm8s	reserved	reserved
1011100 (0x5C)	imm16s	reserved	reserved
1011101 (0x5D)	imm32s	immsf	reserved
1011110 (0x5E)	imm64	immdf	reserved
1011111 (0x5F)	EXT1/disp8s	EXT1/disp8s	EXT1/disp8s
1100000 (0x60)	EXT1/disp16s	EXT1/disp16s	EXT1/disp16s
1100001 (0x61)	EXT1/disp32s	EXT1/disp32s	EXT1/disp32s
1100010 (0x62)	EXT1/disp64	EXT1/disp64	EXT1/disp64
1100011 (0x63)	EXT1	EXT1	EXT1
1100100 (0x64)	EXT2/disp8s	EXT2/disp8s	EXT2/disp8s
1100101 (0x65)	EXT2/disp16s	EXT2/disp16s	EXT2/disp16s
1100110 (0x66)	EXT2/disp32s	EXT2/disp32s	EXT2/disp32s
1100111 (0x67)	EXT2/disp64	EXT2/disp64	EXT2/disp64
1101000 (0x68)	EXT2	EXT2	EXT2
0x69--0x7F	reserved	reserved	reserved

7.1.1 Floating-Point FEA Immediate Forms

`immsf` carries the exact IEEE 754 binary32 interchange encoding and `immdf` carries the exact binary64 interchange encoding. Either source form is valid for either an S or D instruction. The payload format defines the source format and the instruction suffix defines the operation format. When they differ, operand evaluation converts the immediate to the operation format under the architectural IEEE 754 environment before the instruction operation. The conversion uses `FSTATUS.RM`; its floating-point exception causes are combined with the operation's causes and are tested and committed atomically under the ordinary precise floating-point fault rules. An FEA floating-point immediate is never a valid destination.

7.1.2 Vector VEA Lane Addressing

For an ordinary compact or EXT VEA memory form, address calculation first produces one scalar anchor using the shared base, scalar Rn index, scale, displacement, segment, and auto-update grammar. Lane n accesses $\text{anchor} + n * \text{element_bytes}$. Scatter/gather addressing is not a VEA descriptor form and is reserved for separate vector instructions.

7.2 Shared Extended Descriptor

A compact EXT1 or EXT2 value in any profile selects both an exact descriptor length and the displacement width. The descriptor bytes then select segment, base, index, zero-base, SP/PC indexed, and auto-update behavior within that family.

Each extended descriptor selects its segment from the encoded SREG field SEG(s), the fixed SS segment for SP, the fixed CS segment for PC, or the operation's default data segment.

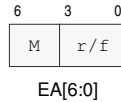


Figure 7-1. Compact Effective-Address Field

M abbreviates mode, and r/f abbreviates register/form.

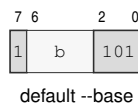
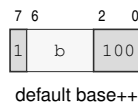
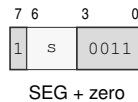
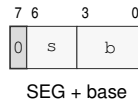
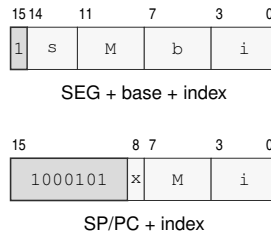


Figure 7-2. EXT1 Descriptor Forms

**Figure 7-3. EXT2 Descriptor Byte Layouts**

M abbreviates mode.

Table 7-2. EA Payload Names

Payload	Bytes	Value	Use
disp8s	1	signed	displacement added to base/index expression
disp16s	2	signed	displacement added to base/index expression
disp32s	4	signed	displacement added to base/index expression
disp64	8	unsigned	displacement added to base/index expression
abs32s	4	signed	absolute address, sign-extended before use
abs64	8	unsigned	absolute address payload
imm8s	1	signed	immediate operand payload
imm16s	2	signed	immediate operand payload
imm32s	4	signed	immediate operand payload
imm64	8	unsigned	immediate operand payload
immsf	4	unsigned	exact IEEE 754 binary32 floating-point payload
immdf	8	unsigned	exact IEEE 754 binary64 floating-point payload
EXT1 descriptor	1	encoded	one-byte extended EA descriptor; present only for EXT1 escapes
EXT2 descriptor	2	encoded	two-byte extended EA descriptor; present only for EXT2 escapes

For a compact displacement, absolute address, or immediate, the selected payload follows the compact field at that operand's payload position. Extended-descriptor construction begins with the compact escape value in the opcode space and exactly one EXT1 or two EXT2 descriptor bytes; a displaced form places its selected displacement after those descriptor bytes. When an instruction has more than one payload-bearing EA operand, the decoder consumes each complete appended EA payload sequence in declared instruction operand order. Each complete sequence, including any extended descriptor bytes and selected displacement, ends before the next payload-bearing EA operand's sequence begins.

7.3 Extended EA Addressing Modes

EXT1 and EXT2 are shared unchanged by EA, FEA, and VEA when a compact form cannot express the operand: explicit segment selection, indexed addressing, zero-base addressing, SP/PC indexed addressing, or auto-update addressing. EXT1 selects exactly one descriptor byte; EXT2 selects exactly two descriptor bytes.

Each extended-descriptor form selects its segment from the encoded SREG field SEG(s), the fixed SS segment for SP, the fixed CS segment for PC, or the operation’s default data segment.

EXT1 Explicit Segment Base

[SEG(s):Rn(b) + displacement]

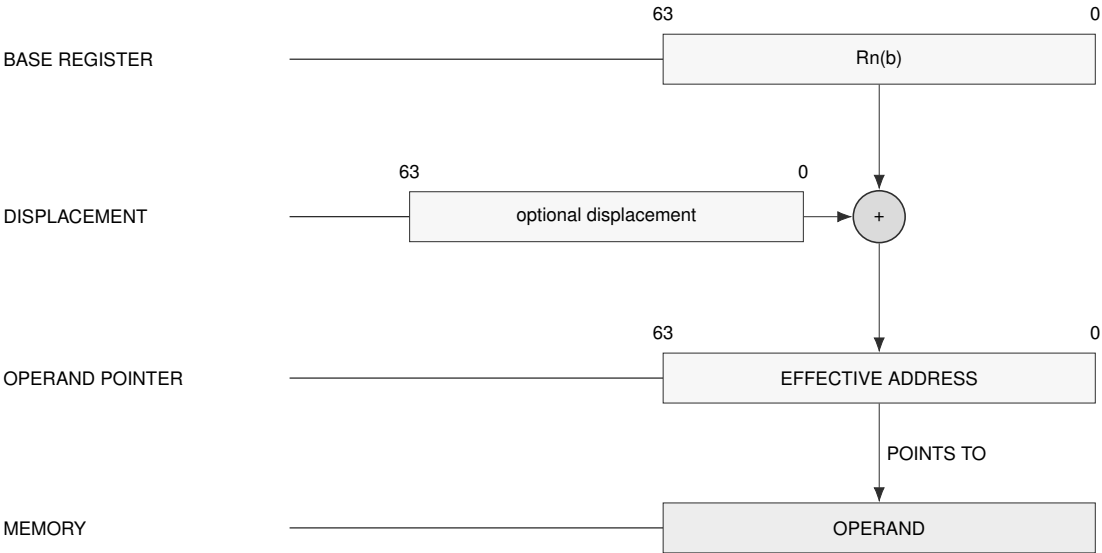
Descriptor: 0sssbbbb, one byte.

Generation: offset = temporary Rn(b) + displacement.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: One descriptor byte plus the optional displacement selected by the compact EXT1 escape.

Update: No auto-update.



Schema — EXT1 explicit segment base

EXT2 Explicit Segment Indexed

$$[\text{SEG}(s):\text{Rn}(b) + \text{Rn}(i) * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):\text{Rn}(b) + \text{Rn}(i)++ * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):\text{Rn}(b) + --\text{Rn}(i) * \text{scale} + \text{displacement}]$$

Descriptor: Byte 0 is 1sss0010, 1sss0000, or 1sss0001 for no update, postincrement, or predecrement.

Byte 1 is bbbbiiii.

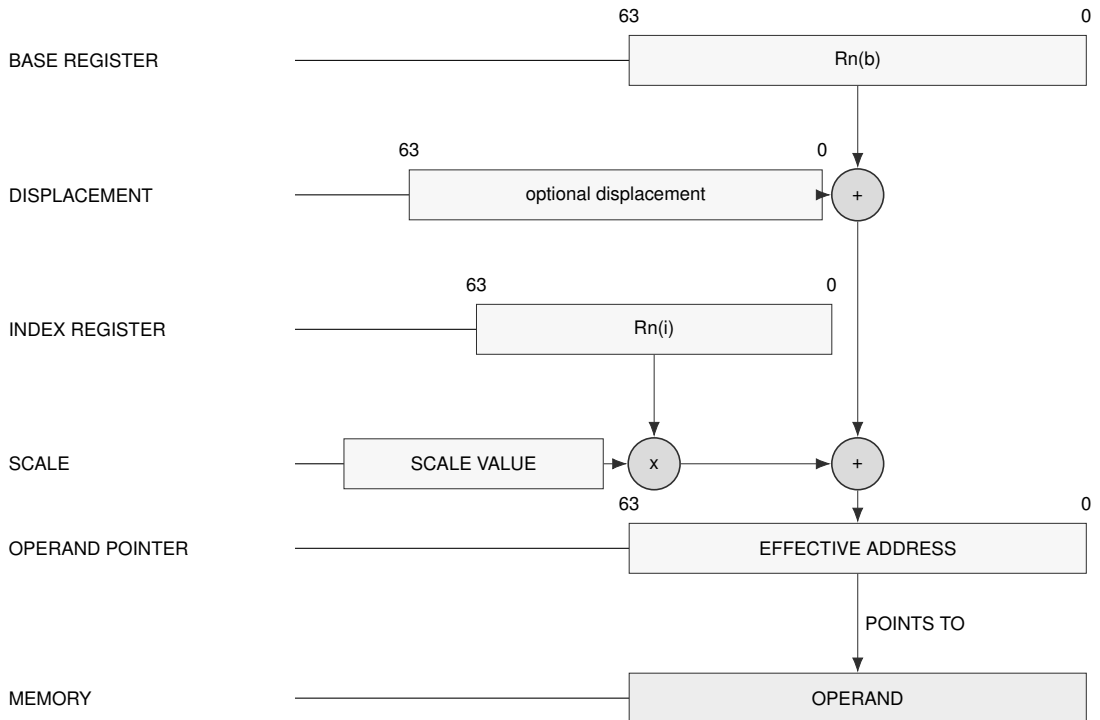
Generation: offset = temporary $\text{Rn}(b) + \text{index_term} * \text{scale} + \text{displacement}$.

Segment: $\text{SEG}(s)$ selects DS, SS, or GS0..GS5.

Scale: Scale is implicit in the consuming instruction. B/W/L/Q normally imply 1/2/4/8.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT2 escape.

Update: Index postincrement uses the current temporary $\text{Rn}(i)$, then increments it by one element. Index predecrement decrements temporary $\text{Rn}(i)$ by one element before use.



Schema — EXT2 explicit segment indexed

EXT1 Explicit Segment Zero Base

$[\text{SEG}(s):0 + \text{displacement}]$

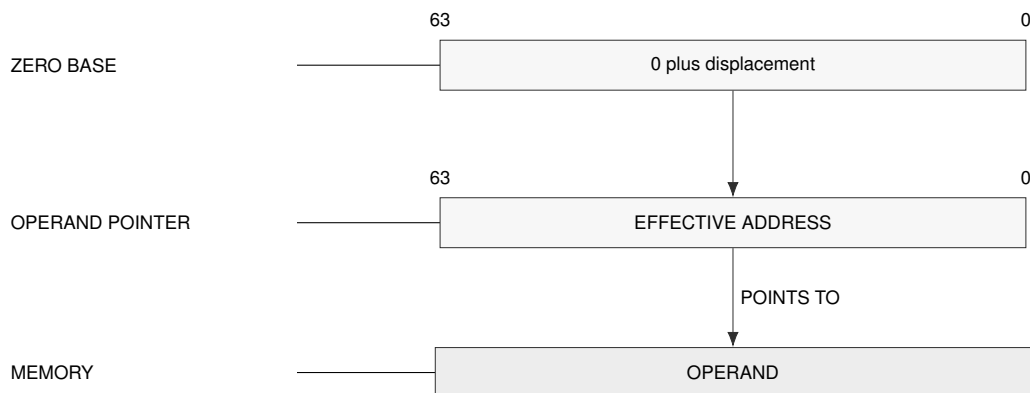
Descriptor: 1sss0011, one byte.

Generation: offset = displacement. Without a displacement payload, offset is 0.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: One descriptor byte plus the optional displacement selected by the compact EXT1 escape.

Update: No auto-update.



Schema — EXT1 zero-base address generation

EXT2 Explicit Segment Base Auto-Update

[SEG(s):Rn(b)++ + displacement]

[SEG(s):--Rn(b) + displacement]

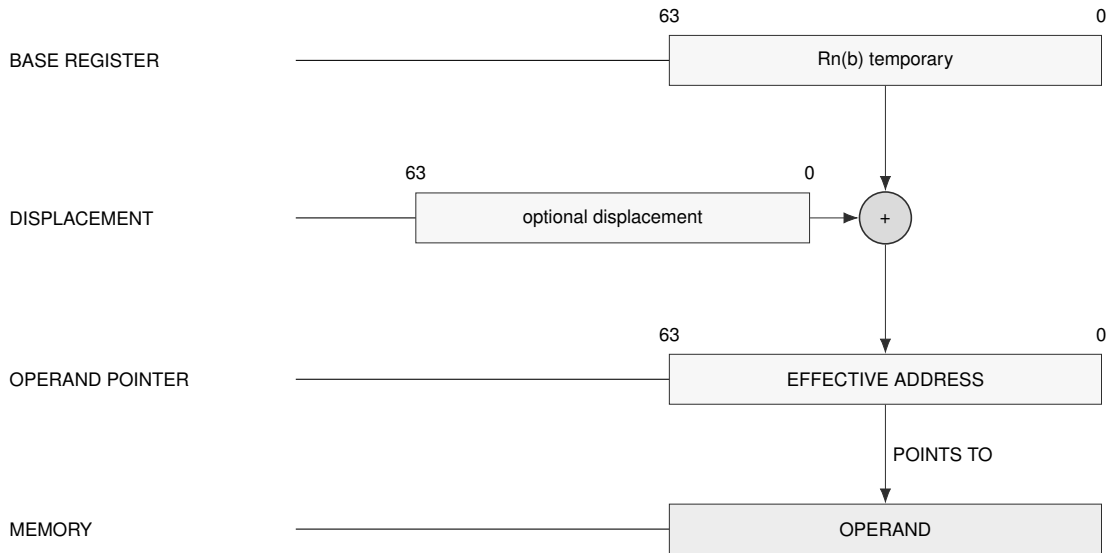
Descriptor: Byte 0 is 1sss1000. Byte 1 is bbbb0000 for base postincrement or bbbb0001 for base predecrement.

Generation: offset = base_term + displacement.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT2 escape.

Update: Base postincrement uses the current temporary Rn(b), then increments it by the memory access size. Base predecrement decrements temporary Rn(b) by the memory access size before use.



Schema — EXT2 base auto-update

EXT2 Explicit Segment Zero-Base Indexed

[SEG(s):0 + Rn(i) * scale + displacement]

[SEG(s):0 + Rn(i)++ * scale + displacement]

[SEG(s):0 + --Rn(i) * scale + displacement]

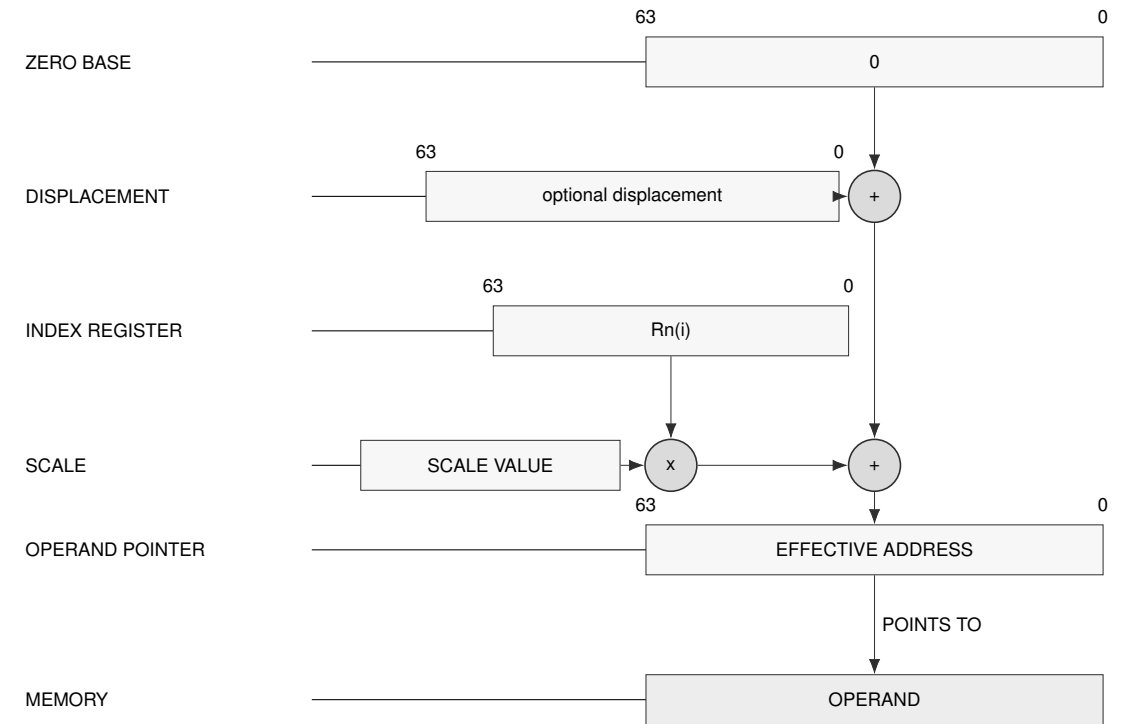
Descriptor: Byte 0 is 1sss1001. Byte 1 is 0010iiii, 0000iiii, or 0001iiii for no update, postincrement, or predecrement.

Generation: offset = index_term * scale + displacement.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT2 escape.

Update: Index update follows the same one-element temporary-state rule as other indexed update forms.



Schema — EXT2 zero-base indexed

EXT2 SP/PC Indexed

$[SP + Rn(i) * scale + displacement]$
 $[SP + Rn(i)++ * scale + displacement]$
 $[SP + --Rn(i) * scale + displacement]$
 $[PC + Rn(i) * scale + displacement]$
 $[PC + Rn(i)++ * scale + displacement]$
 $[PC + --Rn(i) * scale + displacement]$

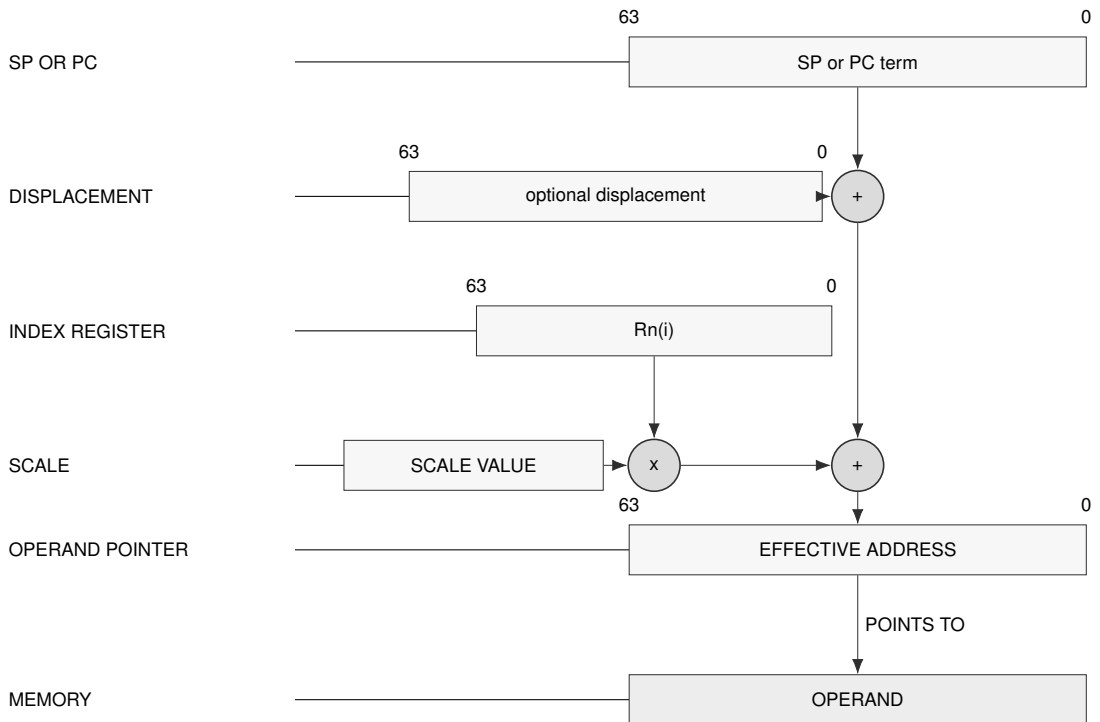
Descriptor: Byte 0 is 10001010 for SP or 10001011 for PC. Byte 1 is 0010iiii, 0000iiii, or 0001iiii for no update, postincrement, or predecrement.

Generation: offset = temporary SP or instruction-start PC + index_term * scale + displacement. A PC term names byte 0 of the current instruction.

Segment: SP forms are fixed to SS. PC forms are fixed to CS.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT2 escape.

Update: Auto-update applies to the Rn(i) index register.

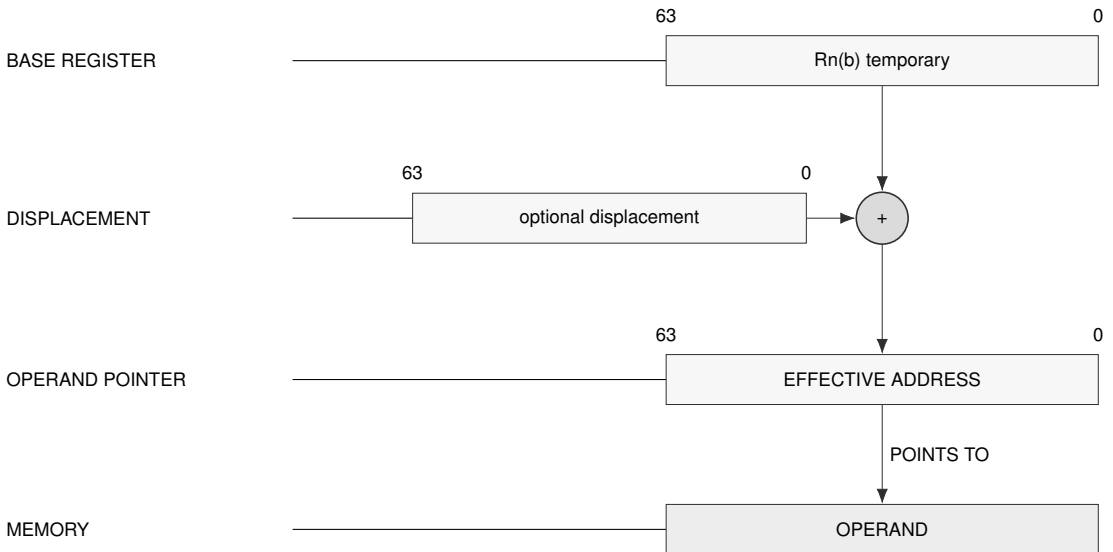


Schema — EXT2 SP/PC indexed

EXT1 Default-Segment Base Auto-Update

[Rn(b)++ + displacement]

[--Rn(b) + displacement]

Descriptor: 1bbbb100 for base postincrement; 1bbbb101 for base predecrement. One byte.**Generation:** offset = base_term + displacement.**Segment:** Uses the operation default data segment.**Payload:** One descriptor byte plus the optional displacement selected by the compact EXT1 escape.**Update:** Postincrement uses the current temporary Rn(b), then increments it by the memory access size.
Predecrement decrements temporary Rn(b) by the memory access size before use.**Schema — EXT1 default-segment base auto-update**

EA evaluation begins by copying the architectural registers into temporary operand-evaluation state. Operands are then processed in instruction order. Within one EA, the base term is evaluated before the index term and then the displacement. Auto-update changes only the temporary operand-evaluation state while operands are being evaluated. Memory reads and writes are collected as instruction side effects, and neither those effects nor the register updates become architecturally visible until the instruction commits. A fault before commit therefore leaves both register and memory state unchanged apart from the exception-entry transition.

Postincrement forms use the current temporary value and update it afterward. Predecrement forms update the temporary value first and then use the result. A base register changes by the EA interpretation width in bytes; an index register changes by one element before scaling. Each REP iteration applies and commits these rules independently.

When a future vector instruction consumes VEA, a base-register update changes the base by $\pm$ VLEN bytes. A scalar Rn index update changes the index by $\pm$ the instruction's number of elements before the existing element-size scale is applied. Predicate activity does not change either update quantity. These vector quantities replace the scalar per-access quantities for that VEA evaluation; the descriptor encodings and update ordering are otherwise unchanged.

Size-less address-role instructions INV PAGE, FL SHDCACHE, INVDCACHE, INVICACHE, WR-BKDCACHE, SYNCCACHE, PREFETCH, PREFETCHNT, PTQUERY, SAVE, and RESTORE have Q interpretation width. Size-less control-target instructions DJcc, IJcc, LCALL, and LJMP also have Q interpretation width. Their extended-descriptor index scale is eight and their base pre-decrement or postincrement amount is eight. Cache range loops advance their explicit address by the CPUID maintenance granule, independently of this per-EA update amount.

When one register supplies both terms in $[R1 + R1++ * scale]$, the base and index terms use the same current temporary R1 before the index update. In $[R1 + --R1 * scale]$, the base uses the current value; the index term then decrements temporary R1 and uses the updated value.

8 MEMORY ADDRESS TRANSLATION

Memory address translation is a separate pipeline after effective-address calculation. Effective-address evaluation produces an address value or operand designator; segmentation optionally turns that value into a linear address, and paging optionally turns the linear address into a memory-system address.

Instruction fetches use CS, stack accesses use SS, and ordinary data accesses use the operation default data segment unless an effective-address form explicitly selects another segment. SP and PC forms use their fixed segments.

SEGLEA exposes the linear address produced by segment pre-translation when software explicitly needs it. Its instruction description defines the point check, FLAGS result, destination suppression, and auto-update commit behavior. SEGLEA applies segment pre-translation to the computed starting point.

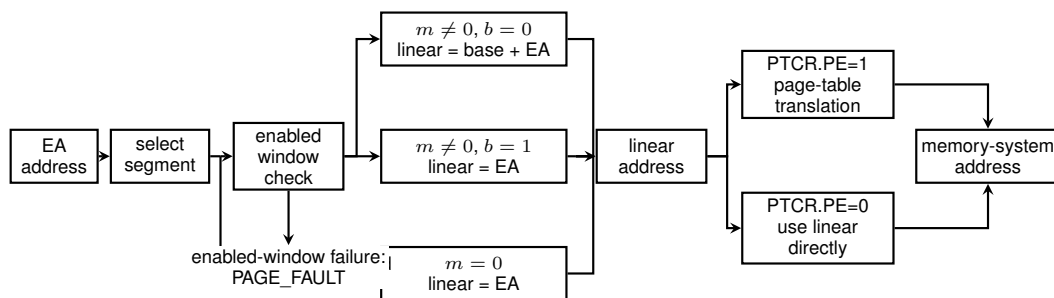


Figure 8-1. Address Translation Pipeline

8.1 Segment Pre-Translation

Segment pre-translation interprets the selected 64-bit segment image before paging. The segment fields, derived quantities, and image-validity condition are defined in Segment Registers. A disabled segment passes the effective address through. An enabled segment either checks the effective address as an offset and adds the base, or checks an already-linear address in bounds-only mode.

Table 8-1. Segment Pre-Translation Modes

Mode	Segment State	Check	Linear Address
disabled	$m = 0$	no segment-window check	linear = EA
translated window	$m \neq 0, b = 0$	complete byte range lies within $0 \leq EA < \text{span}$	linear = base + EA
bounds-only window	$m \neq 0, b = 1$	complete byte range lies within $\text{base} \leq EA < \text{limit}$	linear = EA

Address validation begins by forming the starting EA and its segment-pretranslated linear value. The starting point must be within the selected segment. With paging enabled, the complete byte range is translated in increasing linear-address order after canonicity checks. With paging disabled, the same byte-range rules apply directly.

Complete range additions are mathematical: 64-bit wraparound never makes an out-of-range access valid. A range failure raises `PAGE_FAULT` before the instruction's target memory access begins.

The segment result is the linear address. With `PTCR.PE` set, the page-table walker consumes that address and applies the selected PTE frame and attributes. With paging disabled, the linear address is the physical byte address and must fit the implementation `PABITS` value. An out-of-range direct address raises `ACCESS_FAULT.PHYSICAL_ADDRESS`. An invalid segment image raises `INVALID_CONTROL_STATE` when written; a failed segment bound or canonical-address check raises `PAGE_FAULT` during access.

8.2 Paging Stage

Paging is enabled by `PTCR.PE` and translates the linear address produced by segment pre-translation. Before the walk can produce a translation, the linear address must be canonical for the mode selected by `PTCR.LA57`. A noncanonical address raises `PAGE_FAULT`. With paging disabled, the memory system uses the linear address directly.

Each page-table level uses a nine-bit address field to select one of 512 64-bit entries in a 4-KiB table page. For a table-page base address (B) and index (i), the selected entry is at $(B + 8i)$. The walker starts at the physical table page named by `PTCR.root_page`. At each active level, a present selected entry with $T=1$ points to the next table page; this form is valid only above L1. A selected entry with $T=0$ terminates the walk. A terminating entry is a byte-addressed leaf at any active level. If its level is L , its naturally aligned physical base is combined with the low $12 + 9(L - 1)$ linear-address bits:

$$\text{byte_address} = (\text{leaf_PFN} \ll 12) + \text{linear}[12 + 9(L - 1) - 1 : 0].$$

The resulting L1 through L5 leaf sizes are 4 KiB, 2 MiB, 1 GiB, 512 GiB, and 256 TiB. Bit 7 is the `A` field in both descriptor layouts and does not participate in address formation.

The walker intersects table-pointer `R/W/X/U` fields while descending. The terminating leaf supplies decoded `AM` permissions, `U`, the final frame number, `G`, `CP`, and the `Normal/MMIO` class. The detailed entry-validity, permission, accessed, and dirty rules are specified under `Page-Table Entry Format`. At each consumed level, result priority is not-present first, structural validity second, and effective permission third; the walker resolves those results before consuming a lower level.

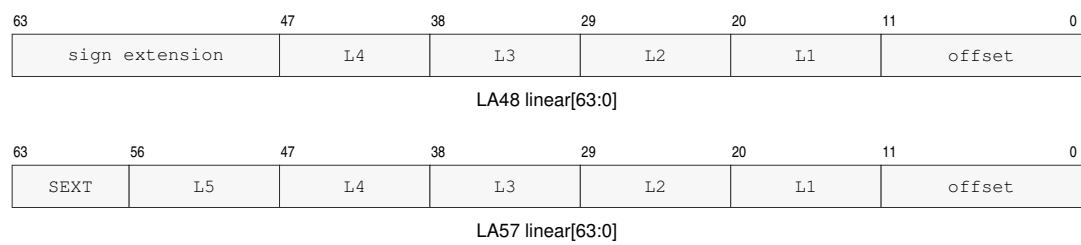
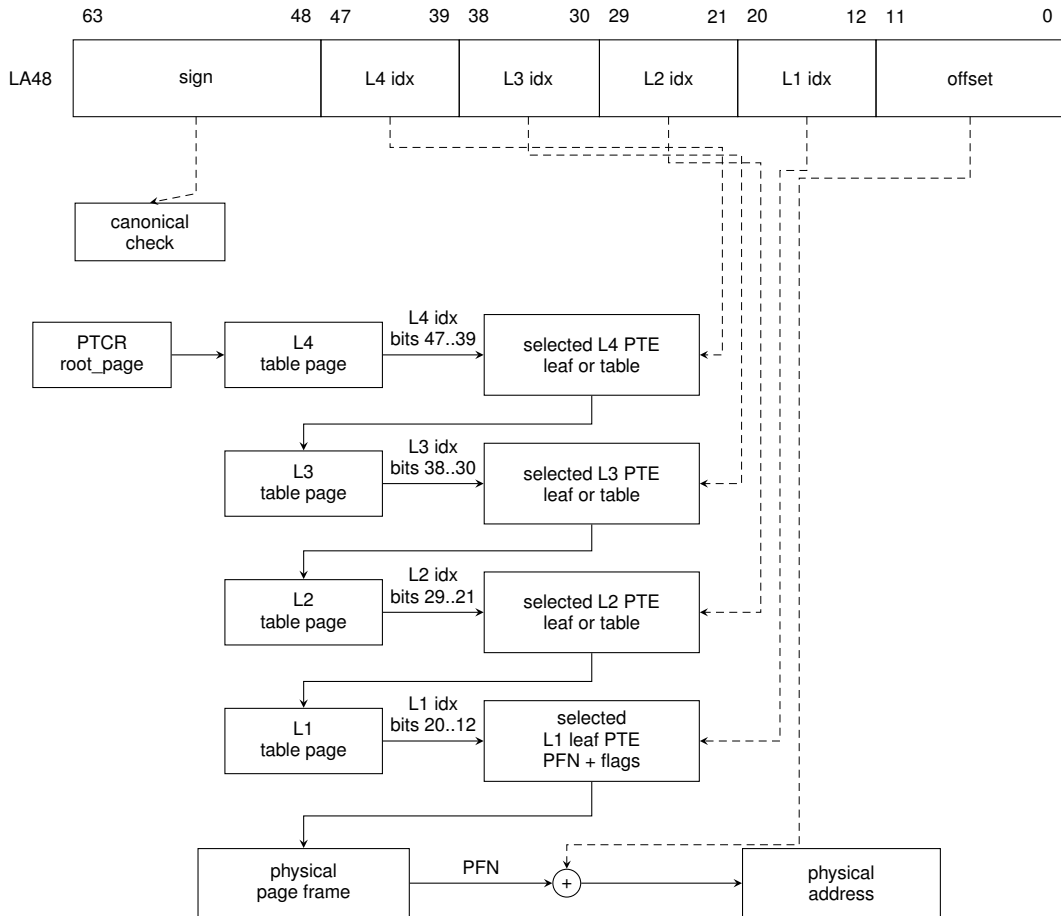


Figure 8-2. Linear-Address Paging Fields

8.3 LA48 Four-Level Paging

When `PTCR.LA57` is zero, bits 63..48 of the linear address must equal the sign extension of bit 47. Bits 47..39, 38..30, 29..21, and 20..12 select the L4, L3, L2, and L1 entries respectively. A byte-addressed leaf may terminate the walk at any of those levels; all bits below the terminating level's index form its offset. The L4 table is the root table named by `PTCR.root_page`.



Maximum-depth walk shown. At L4–L2, T=1 continues and T=0 terminates at an aligned byte leaf; the terminating level selects the offset width. L1 terminates as a byte leaf.

Figure 8-3. LA48 Four-Level Page Walk

8.4 LA57 Five-Level Paging

When `PTCR.LA57` is one, bits 63..57 of the linear address must equal the sign extension of bit 56. Bits 56..48 select an additional L5 entry. A table pointer continues into the same L4-through-L1 hierarchy used by LA48, while a byte-addressed leaf at any selected level terminates the walk and uses all lower linear-address bits as its offset. The L5 table is the root table named by `PTCR.root_page`.

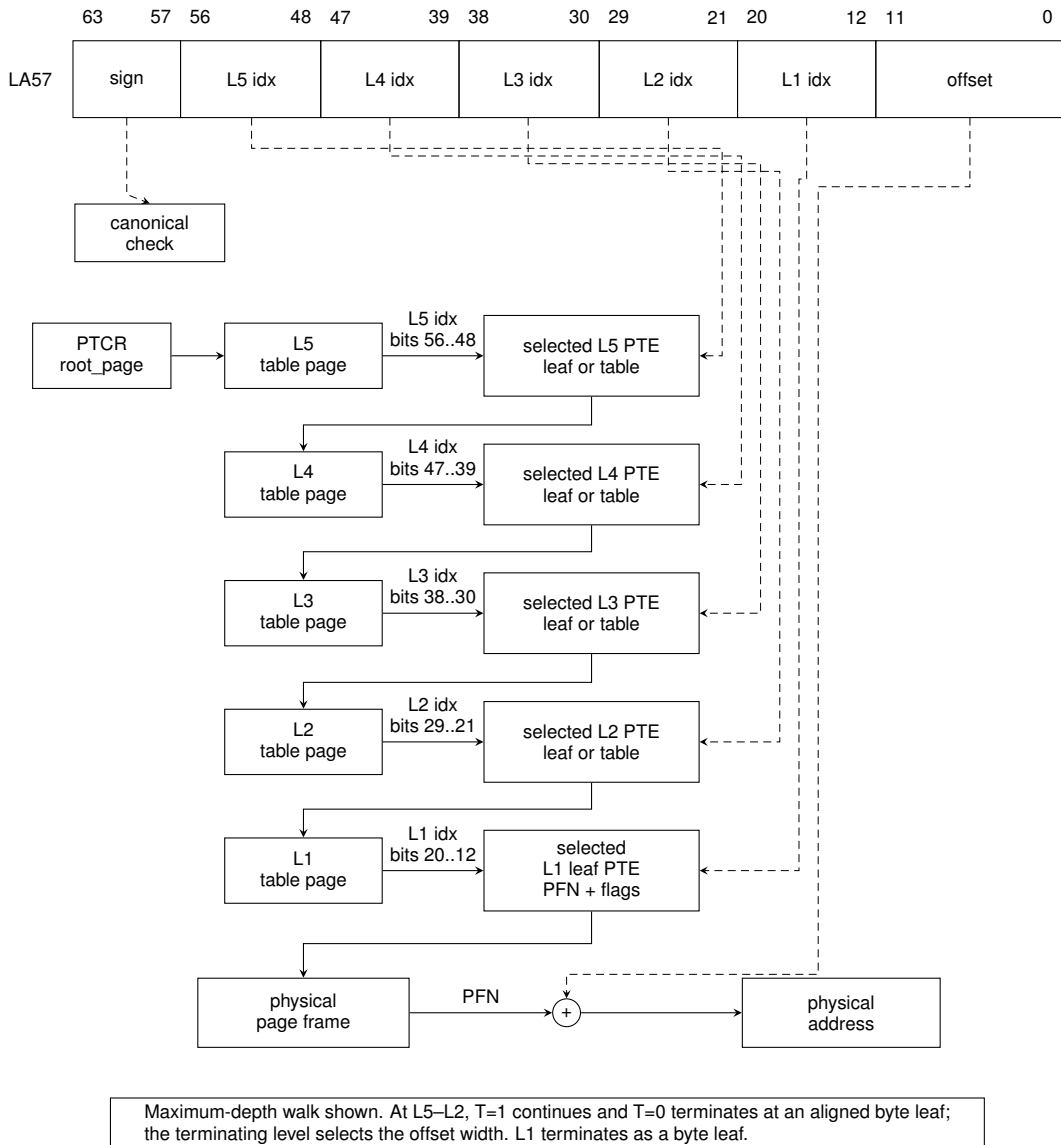


Figure 8-4. LA57 Five-Level Page Walk

8.5 Page-Table Entry Format

Every page-table entry is one naturally aligned 64-bit descriptor. Bits 55..12 are a fixed physical frame number field, so this format has a 56-bit architectural physical-address ceiling. An implementation reports its exact PABITS value, from 32 through 56, through CPUID ADDRESS_WIDTHS. In a present descriptor, PFN bits at or above that value must be zero. Bits 57..56 are SW[1:0] and are ignored by hardware; bits 63..58 are reserved and must be zero.

P in bit 0 is common. When P=0, the remaining bits are ignored and the entry is not present. When P=1, T in bit 1 selects the leaf or table-pointer layout. A nonzero reserved bit in a present descriptor raises PAGE_FAULT.MALFORMED_PTE.

Table 8-2. Leaf Descriptor Fields (P=1, T=0)

Field	Bits	Meaning
P	0	present, one
T	1	leaf selector, zero
AM	4..2	permission and Normal/MMIO access class
U	5	user-domain permission
G	6	global translation
A	7	accessed state
D	8	exact dirty state
CP	10..9	cache policy
reserved	11	must be zero
PFN	55..12	mapped physical frame number
SW[1:0]	57..56	software-defined, ignored by hardware
reserved	63..58	must be zero

Table 8-3. Leaf AM Encodings

AM	Permission	Class	Name
000	read	Normal	R
001	write	Normal	W
010	execute	Normal	X
011	read/write	Normal	RW
100	read/execute	Normal	RX
101	read	MMIO	MMIO_R
110	write	MMIO	MMIO_W
111	read/write	MMIO	MMIO_RW

No leaf encoding grants both write and execute, and no MMIO leaf grants execute. This W^X property applies to one selected translation. The architecture does not compare aliases: distinct descriptors for the same physical page may have different permissions, AM classes, or CP values.

Table 8-4. Table-Pointer Descriptor Fields (P=1, T=1)

Field	Bits	Meaning
P	0	present, one
T	1	table-pointer selector, one
R/W/X	2/3/4	subtree maximum permissions
U	5	subtree maximum user-domain permission
reserved	6	must be zero
A	7	accessed state
reserved	11..8	must be zero
PFN	55..12	next-table physical frame number
SW[1:0]	57..56	software-defined, ignored by hardware
reserved	63..58	must be zero

A table pointer with $R=W=X=0$, or a table pointer at L1, is malformed. The walker begins with R, W, X, and U enabled, intersects each table bitmap and U bit, then intersects the terminating leaf's decoded AM permissions and U bit. The leaf alone supplies G, CP, and Normal/MMIO class. A leaf at level L must have a physical base aligned to $2^{12+9(L-1)}$ bytes.

The CP field is orthogonal to AM: all 32 AM/CP combinations are structurally valid. CP values 0 through 3 mean cacheable write-back, coherent uncacheable, coherent write-through, and coherent write-combining respectively. CP never weakens an MMIO execution rule.

Hardware sets A on each structurally valid descriptor consumed by a completed walk and may also set A during a speculative walk whose instruction later faults or is squashed. D is set only when the corresponding store or atomic update commits, and every required D update succeeds before its memory update becomes visible. Hardware A/D changes are conditional atomic updates of the complete 64-bit PTE and preserve every other field. A concurrent change causes a re-read and complete revalidation; stale fields are never written back. PTQUERY and VTOP do not modify A or D.

8.6 Translation-Cache Identity and Invalidation

A translation-cache entry is identified by the components below, including the leaf-aligned linear range that it maps. PTCR `root_page` is not a hardware tag. While ASCR.AE is one, system software binds one ASID to one PTCR root and translation geometry; it completes the required shutdown before reusing that ASID for a different PTCR image. While AE is zero, the current untagged context is invalidated when PTCR changes.

Table 8-5. Translation-Cache Entry Identity

Identity Component	Applies To
leaf-aligned linear mapping range	every entry
LA57	every entry
ASID	non-global entry while ASCR.AE is one

Table 8-6. Local Translation-Cache Transitions

Operation	Non-Global Entries	Global Entries
WRCR PTCR with changed root_page	invalidate all current untagged entries when AE is zero; when AE is one software must invalidate the ASID before rebinding it	preserve only mappings whose complete leaf attributes are identical in every context
WRCR PTCR with changed LA57	invalidate entries using the old translation geometry	invalidate entries using the old translation geometry
WRCR ASCR changing ASID while AE remains one	select the new ASID-tagged entries	preserve
WRCR ASCR changing AE	invalidate the old untagged context or select the new tagged context as applicable	preserve
SWPT	install PTCR and invalidate the current untagged context	preserve only identical global mappings
SWPTA	install PTCR and ASCR and select the new ASID-tagged context	preserve only identical global mappings
INVPAGE	invalidate every mapping containing the addressed linear value for the current ASID, or the current untagged context when AE is zero	invalidate every addressed global mapping containing that linear value
INVASID	invalidate every entry tagged with the selected ASID	preserve
INVTLB	invalidate every local entry	invalidate every local entry

Table 8-7. Remote Translation Shutdown Protocol

Step	Required Action
1	PTE store
2	AFENCE
3	local invalidate
4	release shutdown request
5	target acquire and local invalidate
6	target release acknowledgement
7	updater acquire of every acknowledgement
8	AFENCE
9	reclaim or reuse

A global leaf ignores ASID matching. Its PFN, effective permissions, AM, and CP attributes must be identical in every context in which that global entry can be used. Changing any of those attributes requires invalidating the global entry on every target logical processor.

The page-table walker reads page tables as CP0 coherent normal memory. Each 64-bit PTE read observes one coherence version; it does not combine fields from different writes. An in-progress walk may complete with the old or new coherence version, so the updater does not reclaim the old mapping until the complete shutdown protocol finishes. The local invalidation instructions affect only the executing logical processor; the request and acknowledgement steps are ordinary shared-memory communication ordered by the stated release, acquire, and AFENCE operations.

8.7 Leaf Byte Addresses and Access Ranges

Every leaf at level L forms a byte address with $o = 12 + 9(L - 1)$:

$$byte_address = (PFN \ll 12) | linear[o - 1 : 0].$$

A width- n access covers the half-open byte interval $[a, a + n)$. Bit 7 records A and may be set by an ordinary walk. Atomics require natural byte alignment; a misaligned operand raises `ATOMIC_ALIGNMENT` before the memory read or write. `PTQUERY` returns the raw selected descriptor, while `VTOP` reports the translated byte address; neither instruction modifies A or D.

8.8 Physical Class and MMIO Accesses

The platform assigns each physical address an authoritative Normal or Device class. A Normal leaf selecting Device physical memory raises `PAGE_FAULT.MEMORY_TYPE`. A MMIO leaf may select either physical class and imposes MMIO rules; paging-disabled Device accesses also use those rules. Paging-disabled Normal addresses are Normal accesses.

A permitted MMIO operation is one naturally aligned scalar load or store of 1, 2, 4, or 8 bytes. All complete-range and translation checks finish before one architectural MMIO event is produced, and that event is not split. A permitted scalar that is misaligned raises `ACCESS_FAULT.MMIO_ALIGNMENT`. Atomic and read-modify-write, vector, gather/scatter, and repeat memory operations raise `ACCESS_FAULT.MMIO_OPERATION` before alignment is considered and before any target effect. A prefetch to MMIO is a no-op. Instruction fetch is never permitted from MMIO.

Within one memory operand, failures are selected in this order: segment range and wrap, canonicity, complete translation of all covered mappings in increasing address order, physical memory class, MMIO operation class, MMIO or atomic alignment, then the target access and bus fault. Thus a later mapping's translation fault precedes an MMIO condition already visible in an earlier mapping.

8.9 Multi-Page Accesses

For a byte range that intersects more than one distinct leaf mapping, translation and accumulated permission checks are resolved once per mapping in increasing linear-address order. Each mapping's walk reaches its existing not-present, structural-validity, and permission result before the next mapping is consumed. The first mapping in that order that fails supplies the architectural fault and walk level.

Every covered leaf mapping of a store is validated before any byte of the store becomes visible. A later-mapping translation, permission or synchronous data-access fault leaves every destination byte and every leaf D bit unchanged. A updates completed while consuming an earlier valid entry retain the speculative-A rule and may remain set when a later mapping faults.

Instruction-record fetch uses the same increasing-address rule for all bytes required by the encoded record. A fault in a later mapping completes as an instruction-fetch fault before decode or operand evaluation, while earlier instruction-cache and PTE A effects retain their ordinary rules.

9 INSTRUCTION EXECUTION MODEL

This section defines the common execution contract used by the instruction descriptions. An individual instruction may add stricter rules, but it does not silently replace the rules below.

9.1 Architectural Result Priority

When one instruction could encounter more than one exceptional condition, the architecturally reported result is the first applicable condition in the following order.

1. Instruction fetch and fetch translation.
2. Instruction framing and acquisition of the complete instruction record.
3. Opcode recognition.
4. Availability of every extension required by the decoded operation.
5. Fixed-field, reserved-field, and effective-address-form legality.
6. Privilege checks.
7. Selector, control-image, and other architectural-state validation.
8. Predicate evaluation for a conditional operation.
9. Every non-suppressed operand access in architectural access order.
10. Execution-stage integer faults and enabled floating-point exception conditions that raise `FLOATING_POINT_FAULT`.

A false predicate suppresses the conditional operation's target-memory and implicit stack accesses. It does not suppress instruction framing, opcode and extension checks, encoding legality, privilege checks, or control-state validation. Explicit operands are accessed in instruction operand order unless an instruction states another order. For a memory-to-memory operation, the complete source read precedes access to the destination, so a source-access failure has priority over a destination-access failure.

Implicit accesses use the order stated by the instruction. In particular, `PUSH` captures its source before accessing the destination stack slot; `POP` reads the stack slot before validating or writing its destination; `PUSHP` uses listed register order and `POPP` uses reverse listed order; `CALL` validates its target before accessing the return stack; `RET` reads the return stack before validating its target; and Long Call, Long Return, Event Return, and supervisor-entry instructions use the step order in their instruction descriptions.

9.2 Implicit Stack Operations

Ordinary implicit stack operations capture the current `SS` and `SP` before their first stack access. Every stack slot is 64 bits and is addressed through the captured `SS`. The ranges below are expressed in terms of the captured `old_sp`; the applicable segment, translation, and complete-range checks finish before the operation commits.

Table 9-1. Ordinary Implicit Stack Operations

Operation	Stack range	Committed SP	Slot layout or value order
PUSH, CALL	[old_sp - 8, old_sp)	old_sp-8	one source value or the following instruction's continuation PC at old_sp-8
POP, RET	[old_sp, old_sp + 8)	old_sp+8	one destination value or continuation PC read at old_sp
PUSHP, FPUSHP	[old_sp - 16, old_sp)	old_sp-16	listed pair's second register at old_sp-16 and first register at old_sp-8
POPP, FPOPP	[old_sp, old_sp + 16)	old_sp+16	listed pair's second register from old_sp, then first register from old_sp+8
LCALL	[old_sp - 16, old_sp)	old_sp-16	continuation PC at old_sp-16 and return CS image at old_sp-8
LRET	[old_sp, old_sp + 16)	old_sp+16	continuation PC at old_sp and return CS image at old_sp+8

The complete range and all source values are established before a push operation changes memory or SP. A pop operation reads its complete range and completes any applicable destination validation before changing a destination or SP. A successful operation commits all listed stack and register effects together; a preceding fault exposes none of them.

Architectural event entry and ERET use the return source selected by the Architectural Event Processing Model. SYSCALL is the SYSTEM_CALL event; user-origin entry keeps its continuation in the user context bank described by the Privileged Execution Model and creates no memory frame.

9.3 Operand Evaluation and EA Defaults

Operands are decoded in instruction order. Source reads complete before the final destination write unless an atomic instruction says otherwise. Effective-address calculation may produce an address without reading memory.

Auto-update EA forms update temporary operand-evaluation state. The architectural register update becomes visible only when the instruction commits.

Ordinary instructions accept at most one memory operand. The explicitly encoded memory-memory exceptions are CMP, EXTSL, EXTSEQ, EXTSTW, EXTZL, EXTZQ, EXTZW, MOV, MOVCU, MOVUC, MOVUU, FBNDII, FBNDIX, FBNDXI, FBNDXX. Repeat behavior is available only through the repeat instructions and body-eligibility rules defined in Repeated Scalar Execution.

An instruction either replaces the complete Z, N, C, and V FLAGS image or preserves the complete image; partial FLAGS writes do not exist. FFLAGS remains an IEEE-754 accrued exception-cause bank, so FFLAGS fields that an instruction does not mention remain unchanged. Operand evaluation follows instruction operand order, and an instruction has one architectural commit point unless its description explicitly defines partial completion.

9.4 Shared Side-Effect Rules

An instruction that faults before commit leaves destination registers, destination memory, and auto-update register effects unchanged unless the instruction explicitly defines partial completion.

Atomic read-modify-write instructions have a single architectural commit point for the memory update and any associated register result. Cache, TLB, and system-state instructions describe their own completion and visibility rules.

Ordinary integer ALU instructions use the common one-memory-operand rule. Compare and test instructions may read two operands and update FLAGS without storing their temporary result. EXTS and EXTZ additionally provide explicitly encoded register-memory and memory-memory conversion stores; EXTSQ also provides register-register and memory-register sign extension. These operations preserve FLAGS unless a repeat rule observes a derived value. Data movement, LEA, and segment-address operations also preserve FLAGS unless their descriptions say otherwise.

Control transfers make PC and CS changes at a single control-transfer commit point. Atomic operations require a naturally aligned addressable memory operand and commit the read-modify-write as one architectural update. TLB, page-table context, and privileged control operations require supervisor privilege. Cache-maintenance operations preserve FLAGS and define their own visibility contract. Approximate transcendental floating-point operations are available only when CPUID reports FPTRANSA and marks the instruction's accuracy-contract ID present.

10 FLAGS AND CONDITION CODES

10.1 Condition Encoding

Conditional instructions encode a four-bit condition code. The zero-valued condition is T, which is also used for canonical aliases such as JMP for Jcc.T.

The condition-code bits are the low four meaningful bits of FLAGS, as shown in the Programming Model section.

Table 10-1. Condition Code Encoding

Bits	Name	Aliases	Expression
0000	T	-	true
0001	F	-	false
0010	EQ	Z	Z == 1
0011	NE	NZ	Z == 0
0100	ULT	C	C == 1
0101	UGE	NC	C == 0
0110	MI	N	N == 1
0111	PL	NN	N == 0
1000	VS	V	V == 1
1001	VC	NV	V == 0
1010	ULE	-	C == 1 Z == 1
1011	UGT	-	C == 0 && Z == 0
1100	LT	-	N != V
1101	GE	-	N == V
1110	LE	-	Z == 1 N != V
1111	GT	-	Z == 0 && N == V

10.2 Flag Meanings

Integer condition codes are held in FLAGS. Integer instructions leave FLAGS unchanged unless their instruction description explicitly defines a FLAGS write. Every FLAGS-writing instruction replaces the complete Z, N, C, and V image; partial FLAGS writes do not exist. An instruction that does not write FLAGS preserves the complete image.

For an operand width of n bits, ordinary integer ALU results are evaluated modulo 2^n unless the instruction explicitly defines a wider intermediate.

Explicit FLAGS-writing integer instructions include CMP, TEST, ADC, SBB, INCF, DECF, BTEST, BSET, BCLR, BCHG, SETF, WRFLAGS, CMPXCHG, SEGLEA, and the bounds-check instructions. CMPJcc and TESTJcc compute temporary condition flags for their branch decision and do not update architectural FLAGS. ADD, SUB, logic operations, INC, DEC, shifts, rotates, moves, extensions, min/max, and multiply/divide instructions leave FLAGS unchanged.

Table 10-2. Integer Condition-Code Bits

Bit	Name	Meaning
Z	zero	Set when the result value is zero.
N	negative	Set from the most significant result bit at the operand size.
C	carry/borrow	Set on unsigned carry out for addition or unsigned borrow for subtraction.
V	overflow	Set on signed overflow or an explicitly reported exceptional condition.

10.3 Conditional Consumers

Conditional branches, calls, traps, and sets consume the condition-code table without recomputing FLAGS. An ordinary conditional consumer observes the FLAGS image produced by the last committed FLAGS-writing instruction. REPcc instead tests a temporary flag image derived from the body instruction's repeat-observed value, as defined in Repeated Scalar Execution, and does not write architectural FLAGS merely by performing that test.

A false condition suppresses the conditional action unless the instruction description explicitly defines a different commit rule, such as a repeat instruction's terminating iteration rule.

10.4 Floating-Point Interactions

Floating-point comparison instructions may update integer condition codes only when their instruction definition says so. FFLAGS holds floating-point exception flags, and FSTATUS holds the corresponding exception-condition enables and rounding state. IEEE-754 FFLAGS causes accrue independently and are not subject to the complete-image FLAGS rule.

Integer conditional consumers do not read FFLAGS directly; floating-point instructions that want integer control-flow consumers must materialize the selected condition into FLAGS or an Rn value.

11 MEMORY MODEL

The memory model defines instruction fetches, loads, stores, atomic read-modify-write operations, cache maintenance, and translation-cache maintenance after EA calculation and address translation.

11.1 Baseline Ordering

Normal memory is coherent. Every logical processor observes writes to a given byte in one common coherence order, and writes by one logical processor to that byte enter the order in program order. A naturally aligned tear-free scalar store enters that order as one memory write event.

Normal-memory writes are multi-copy atomic. When a memory write event becomes observable by a logical processor other than the writer, it becomes observable to all such logical processors at the same architectural point. An implementation may still forward a logical processor's own pending store to its later loads before that store is globally observable.

The baseline ordering is weak. Except for same-byte coherence or ordering imposed by an atomic memory-order selector or fence, loads and stores to different locations may be issued, completed, and observed in an order different from program order. Coherence and multi-copy atomicity do not by themselves order accesses to different locations.

11.2 Ordinary Load Values and Preserved Order

Each byte returned by a load comes from the initial value of that physical byte or from an actual store or successful atomic read-modify-write to that byte. A load does not obtain a value solely from a cyclic chain in which the load value is needed to produce the write that would supply it.

Among writes already globally observable when a load takes its value, the load observes the latest write in the byte's coherence order. A load may instead forward a byte from an earlier store by the same logical processor before that store is globally observable. A store later than the load in program order cannot supply the load.

Program order is preserved from an earlier read or write to a later write when their byte ranges overlap. Two loads of the same byte, with no intervening write by the same logical processor to that byte, do not observe the byte's coherence order moving backward. These rules apply independently to every byte of an access, including each visible portion of an unaligned access.

A load is ordered before a later memory access whose effective address depends on the loaded value. A load is also ordered before a later store when the stored value depends on the load or when execution of that store is control-dependent on the load. Control dependence alone does not order a later load; software uses an appropriate fence when that ordering is required.

11.3 PTE Cache Policies

The two-bit PTE CP field selects a cache policy for byte-addressed normal memory. Every policy uses the same coherence order, multi-copy atomicity, tearing rules, atomic operations, and fence semantics defined in this chapter. The policy changes allocation, retained cache state, and store combination; it does not create a separate value or ordering domain.

Table 11-1. Normal-Memory Cache Policies

CP	Policy	Reads and Fetches	Stores	Retained State
0	CACHEABLE_WRITE_BACK	may allocate a coherent cache copy	may retire into a coherent dirty cache copy	clean or dirty coherent cache copies
1	COHERENT_UNCACHEABLE	enter coherence without retaining a cache copy	enter coherence before retirement without retaining a dirty copy	no retained cache copy
2	COHERENT_WRITE_THROUGH	may allocate a coherent clean cache copy	enter coherence before retirement without retaining a dirty copy	clean coherent cache copies only
3	COHERENT_WRITE_COMBINING	enter coherence without retaining a cache copy	may combine adjacent byte writes before entering coherence	pending combined stores, but no retained cache copy

Aliases of one physical byte with different CP values participate in the same coherence order. A load, instruction fetch, or atomic operation through any alias observes the same physical value subject to ordinary ordering, and atomic operations serialize at the physical location. CP2 stores enter coherence before retirement and leave no dirty cache copy. CP3 combined stores preserve the byte writes and their per-location program order.

WFENCE and AFENCE complete earlier pending CP3 stores. WRBKDCACHE, FLSHDCACHE, and SYNCCACHE complete pending CP3 stores for their selected block before performing their named cache action. The data-write, cache-maintenance, rendezvous, and local instruction-cache sequence below applies without regard to the CP value used by the modified code mapping.

11.4 Access Atomicity and Alignment

Unaligned ordinary normal-memory accesses are architecturally permitted. A completed unaligned load may observe bytes from multiple memory events in coherence order, and a completed unaligned store may enter coherence as separately visible portions. Each visible store portion is coherent and multi-copy atomic.

An unaligned store is nevertheless fault-atomic for synchronous faults. If any byte of the complete store range would raise a synchronous fault, no byte of the store becomes architecturally visible.

Atomic read-modify-write instructions require natural byte alignment. A misaligned atomic operand raises PAGE_FAULT with the ATOMIC_ALIGNMENT cause before any memory update or architectural result commits; it must not complete as a non-atomic update.

For every scalar size, a naturally aligned normal-memory load or store is tear-free. Unaligned accesses retain the successful-access and synchronous-fault rules above.

Natural alignment equals the operand width in bytes: B is one byte and may use any byte address; W, L, and Q are two, four, and eight bytes and require addresses divisible by two, four, and eight respectively.

Stores to memory that is later executed as instructions are ordinary data stores until an explicit synchronization sequence makes the modified bytes visible to instruction fetch.

11.5 Cache-Maintenance Blocks

CPUID `CACHE_TOPOLOGY` reports a cache-maintenance granule G . A cache-maintenance instruction computes and translates its address-role EA without reading an operand value, then selects the physical byte interval $[\lfloor A/G \rfloor G, \lfloor A/G \rfloor G + G)$ containing the translated physical address A . One instruction operates on exactly that block. Because G is at most 4096 bytes, the selected physical block does not cross a 4 KiB page boundary.

FLSHDCACHE, INVDCACHE, and WRBKDCACHE apply to every data or unified cache copy of the selected block in its normal-memory coherence domain. FLSHDCACHE writes dirty bytes into normal-memory coherence and invalidates the copies. INVDCACHE discards the copies without writeback. WRBKDCACHE writes dirty bytes into normal-memory coherence and retains clean copies. Each operation completes the selected block before retirement.

INVICACHE invalidates the selected block in the executing logical processor's instruction cache, decoded instruction state, and instruction-prefetch state. SYNCCACHE first completes the data writeback for the block throughout the coherence domain, then performs the same local instruction-side invalidation, and finally serializes later instruction fetch so that it observes the synchronized bytes. To publish modified code to another logical processor, software completes SYNCCACHE on the publisher, performs a release/acquire rendezvous, and executes INVICACHE for the block on every receiving logical processor before branching to the modified bytes.

Address translation and permission checks precede a block's cache effects. A fault leaves that instruction's selected block unchanged.

11.6 Atomic Memory-Order Selectors

Table 11-2. Atomic Memory-Order Selectors

Selector	Code	Architectural Ordering Effect
RELAXED	0	Atomicity only. No ordering is imposed on unrelated memory operations.
ACQUIRE	1	Later memory operations by the same logical processor are ordered after the atomic operation.
RELEASE	2	Earlier memory operations by the same logical processor are ordered before the atomic operation.
ACQREL	3	Applies both acquire and release ordering.
SEQCST	4	Applies acquire-release ordering and participates in the single global sequentially consistent order.

CMPXCHG, FETCHADD, FETCHSUB, FETCHAND, FETCHOR, and FETCHXOR are the atomic read-modify-write instructions. Each performs its memory read, successful memory write, and architectural register result as one indivisible operation in the coherence order of the addressed physical location.

A successful release or stronger atomic write heads a release sequence. The sequence continues through immediately following successful atomic read-modify-write operations to the same location in coherence order and ends before any other write. An acquire or stronger atomic operation that takes its value from any member of the sequence observes the effects ordered before its head before effects ordered after the acquiring operation.

One encoded selector governs the complete `CMPXCHG` operation on both success and failure. A failed `CMPXCHG` contributes a memory read event. Its acquire component orders later memory operations after that read, and its release component orders earlier memory operations before that read. On success, the memory write event carries the selected release effect to observers of that write; on failure, the read is the operation's memory event. A failed `CMPXCHG` contributes no write and therefore creates no release sequence. A failed `CMPXCHG.SEQCST` additionally participates in the global sequentially consistent order as an SC load.

11.7 Fence Instructions

The table gives every ordered-before pair imposed by a fence on memory operations issued by the same logical processor. An entry marked ordered places every earlier operation of the row kind before every later operation of the column kind.

Table 11-3. Fence Ordered-Before Pairs

Fence	read to read	read to write	write to read	write to write
RFENCE	ordered	baseline	baseline	baseline
WFENCE	baseline	baseline	baseline	ordered
AFENCE	ordered	ordered	ordered	ordered

A baseline entry retains the weak-ordering rules of this chapter. `AFENCE` is cumulative: memory effects observed before the fence are propagated before effects ordered after it, including through another logical processor. The linked instruction entries define the matching completion behavior.

11.8 Global Sequentially Consistent Order

All `AFENCE` operations and all `SEQCST` atomic operations participate in one global total order that is consistent with each logical processor's ordered-before relation and with per-location coherence. A failed `CMPXCHG.SEQCST` participates in that order as an SC load even though it performs no write.

A naturally aligned tear-free scalar load or store also participates as an SC operation when it is the only memory access between its immediately surrounding `AFENCE` operations in memory-event order. Thus `AFENCE; access; AFENCE` is the architectural sequence for an SC scalar load or store. The access and both fences occupy their required positions in the same global order.

Acquire and release operations may be strengthened with `AFENCE`; the resulting instruction sequence has the union of the ordering constraints imposed by its operations. Leaf AM and CP are orthogonal. An MMIO access retains its non-speculative, operation-class, alignment, and ordering rules under every CP value. MMIO accesses from one logical processor are observed in program order relative to other MMIO accesses from that logical processor. Normal-memory and MMIO accesses retain the baseline weak ordering between classes; `AFENCE` orders earlier accesses before later accesses across that boundary.

12 REPEATED SCALAR EXECUTION

REP Rn, (<instruction>) and REPcc Rn, (<instruction>) repeat one scalar instruction. The body is decoded and checked for repeat eligibility before a zero-count annul is applied, then remains fixed for that repeat context. A body may not be a repeat operation, a repeat-control instruction, or a control transfer.

Entering repeat state captures the selected Rn value as an unsigned remaining count. A valid zero count performs no body side effects. Otherwise each iteration evaluates operands and commits FLAGS, FFLAGS, memory, and auto-update effects according to the body's ordinary scalar rules. A body read of the selected counter register observes the iteration-start architectural value; a body write to that register is ignored.

Each successfully committed iteration decrements the captured count before an event may be admitted at the next iteration boundary. A condition-false REPcc iteration still commits its body and decrements the count before repeat state is cleared. If the body faults, that iteration commits nothing and does not decrement the count; earlier iterations remain committed. Event entry then saves the repeat-prefix address in SAVED_PC and clears hidden repeat state, so ERET returns to the prefix and re-executes it normally with the unchanged counter.

The REPcc observation attribute identifies the value tested after the body executes. A result:operand or source:operand observation names an operand; computed names the derived value defined by that instruction. The temporary condition image sets Z exactly when the observed value is zero, sets N from its most significant bit at the observation width, and clears C and V. It does not sample or overwrite architectural FLAGS, although the body's own FLAGS effects commit normally. REP is the unconditional T spelling; condition F is reserved.

While repeat state is active, architectural PC identifies the body instruction. After a successful body commit, the counter decrement completes before an event may be admitted between iterations. Both that event path and the fault path above save the repeat-prefix address in SAVED_PC and clear hidden repeat state. ERET restores the ordinary saved architectural state and begins execution at that prefix address; it does not restore hidden repeat continuation state. The prefix is therefore decoded and executed normally, using the already-decremented architectural Rn value. TF and RF apply once to each committed body iteration; a zero-count annul is also one trace unit.

PART III

BEDROCK ARCHITECTURE

System Programming

13 PRIVILEGED EXECUTION MODEL

All exceptions, interrupts, NMI, and explicit synchronous traps enter supervisor execution through the architectural event mechanism. `SYSCALL` is the explicit `SYSTEM_CALL` event (`EXC:0x20`); it uses the common `EPC/ECS/EDS` entry and returns with `ERET`.

13.1 Privilege and Event State

`STATUS.PM` is zero in user mode and one in supervisor mode. `STATUS.EA` indicates active event processing. `STATUS.EDEPTH` in bits 9..6 is the active event depth, and `STATUS.UO` in bit 10 records that the outer event originated in user mode. `PM`, `EA`, `EDEPTH`, and `UO` are hardware managed; `WRSTATUS` must preserve them. User execution always observes `PM=EA=EDEPTH=UO=0`.

`IE` enables maskable interrupts, `TF` requests single-step tracing, `RF` suppresses one trace boundary, and `NI` inhibits nested NMI admission. `RDCR`, `WRCR`, translation-state changes, event-control changes, and page-table context switches require supervisor privilege unless an instruction explicitly states otherwise.

13.2 User Context Bank

The user context bank consists of `UPC`, `USP`, `UCS`, `UDS`, `USS`, `UCTL`, and `UINFO`. `UCTL` contains saved `FLAGS` in bits 15..0, saved `STATUS` in bits 31..16, and `V` in bit 32. `UINFO` contains the 32-bit event code; its upper half is zero. A saved user `STATUS` image has `PM`, `EA`, `EDEPTH`, and `UO` clear.

Software stages a context by writing `UPC`, `USP`, `UCS`, `UDS`, `USS`, and `UINFO` before writing `UCTL.V=1`. That final write validates the complete bank. The bank belongs to the interrupted user thread and must be saved with its supervisor context across preemption or migration.

13.3 Initial Event Stacks and Payloads

A first user-origin `SYSTEM_CALL` selects `SSS:SSP`, a maskable interrupt selects `ISS:ISP`, and an exception or NMI selects `FSS:FSP`. BASIC events, including `SYSTEM_CALL`, have no payload and perform no stack-memory access. `ERROR` events store 16 bytes; `PAGE_FAULT` and `AUXILIARY` events store 32 bytes. The event code in `UINFO` defines the payload shape.

A supervisor-origin event and every nested event push a complete event frame on the current `SS:SP`. They do not promote to another configured event stack. The saved `SS:SP` in the frame identifies the interrupted stack.

13.4 Event Entry and Failure

User-origin entry atomically saves the user context bank, optionally stores the payload, loads `EPC/ECS/EDS` and the selected initial stack, sets `PM` and `EA`, sets `EDEPTH` to one and `UO` to one, and clears `IE`, `TF`, and `RF`. Supervisor and nested entry save the prior context in the full frame and increment `EDEPTH` while preserving `UO`.

Delivery failure selects DSS:DSP and attempts `DOUBLE_FAULT`. The hidden current-DFA bit is set when that entry commits. Failure while current DFA is set, or failure of the double-fault attempt, enters shutdown. A full frame saves the prior DFA state in `FRAME_CONTROL.SAVED_DFA`.

13.5 ERET Routing and Atomicity

ERET classifies its source before any stack access. `PM=1`, `EA=0`, `EDEPTH=0`, `UO=0`, and `UCTL.V=1` performs direct initial user entry from the bank. `PM=1`, `EA=1`, `EDEPTH=1`, `UO=1`, and `UCTL.V=1` returns the outer user event from the bank. Other supervisor states with `EA=1` and nonzero `EDEPTH` restore a full frame from the current `SS:SP`. Every other combination raises `INVALID_CONTROL_STATE`.

A U-bank return validates `UCTL`, `UINFO`, the user segment images, `USP`, and the executable `UPC` before committing. It restores user execution, clears `UCTL.V`, `EDEPTH`, `UO`, `EA`, and current DFA, and never reads the stack. A stacked return validates the entire frame, requires saved `STATUS.EDEPTH+1` to equal the current depth and saved `UO` to match the current chain, then restores `STATUS` and `SAVED_DFA` atomically. A failed validation or target probe leaves the current context and return source unchanged.

Normative instruction-local behavior is defined by `SYSCALL` and `ERET`.

14 ARCHITECTURAL EVENT PROCESSING MODEL

An *architectural event* is a synchronous or asynchronous condition delivered through the common event-entry mechanism. Synchronous exceptions are faults or traps. Asynchronous events are maskable interrupts or NMIs. A fault and a trap are exception subtypes, not peers of exception. SYSCALL produces the explicit synchronous SYSTEM_CALL exception (EXC:0x20) and uses this same mechanism.

Every event transfers to the exact program counter in EPC after loading ECS and EDS. Hardware supplies an event code and either a user-bank payload or a full supervisor frame whose layout is fixed by the event source. Software begins at one common entry point and dispatches by the event code.

Related normative instruction entries are BKPT, SYSCALL, ERET, and RESET.

14.1 Event Code and Sources

Every delivered event carries a 32-bit code. The high byte is the event class and the low 24 bits are an ID in that class's namespace.

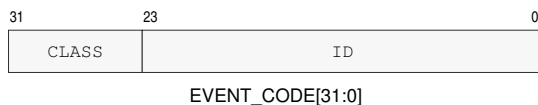


Figure 14-1. Architectural Event Code

Table 14-1. Architectural Event Classes

Value	Class	ID interpretation
0x00	EXCEPTION	architecture-defined exception ID
0x01	INTERRUPT	opaque platform-assigned global interrupt ID
0x02	NMI	zero or an architecturally/platform-defined NMI source ID
0x03..0xFF	reserved	—

Table 14-2. Assigned Base Exception IDs

ID	Name	Subtype or source
0x00	DEBUG_TRACE	debug trace trap
0x02	BREAKPOINT	breakpoint trap
0x03	ILLEGAL_INSTRUCTION	illegal-instruction fault
0x08	PRIVILEGE_FAULT	privilege fault
0x09	PAGE_FAULT	address-translation, access, or atomic-alignment fault
0x0A	DIVIDE_ERROR	integer divide fault
0x0D	INVALID_CONTROL_STATE	invalid control-state fault
0x0E	FLOATING_POINT_FAULT	enabled floating-point exception condition
0x18	DOUBLE_FAULT	event-delivery failure
0x19	MACHINE_CHECK	corrected trap or recoverable/fatal machine-check exception

Table 14-2 (continued)

ID	Name	Subtype or source
0x1A	BUS_ERROR	synchronous precise bus-error fault

Table 14-3. Architectural Event Boundaries and Priority

Code	Event	Kind	Saved PC	Priority
EXC:00	DEBUG_TRACE	trap	next trace unit	5
EXC:02	BREAKPOINT	explicit trap	following instruction	4
EXC:03	ILLEGAL_INSTRUCTION	fault	faulting instruction	3
EXC:08	PRIVILEGE_FAULT	fault	faulting instruction	3
EXC:09	PAGE_FAULT	fault	faulting instruction	3
EXC:0a	DIVIDE_ERROR	fault	faulting instruction	3
EXC:0b	VECTOR_RANGE_ERROR	fault	faulting instruction	3
EXC:0c	ACCESS_FAULT	fault	faulting instruction	3
EXC:0d	INVALID_CONTROL_STATE	fault	faulting instruction	3
EXC:0e	FLOATING_POINT_FAULT	fault	faulting instruction	3
EXC:18	DOUBLE_FAULT	delivery failure	interrupted event boundary	1
EXC:19	MACHINE_CHECK	machine check	selected by PRECISE and corrected status	2
EXC:1a	BUS_ERROR	fault	faulting instruction	3
EXC:20	SYSTEM_CALL	explicit trap	following instruction	4
NMI:source	NMI	asynchronous	next instruction or repeat-prefix address	6
INTERRUPT:platform	INTERRUPT	asynchronous	next instruction or repeat-prefix address	7

Table 14-4. Architectural Event Producers and Delivery State

Event	Producer	Committed State	Frame	Payload
DEBUG_TRACE	completion of a TF-selected trace unit	completed trace unit	BASIC	none
BREAKPOINT	BKPT	BKPT completed	BASIC	none
ILLEGAL_INSTRUCTION	decode or instruction-validity check	no effects of the faulting instruction	ERROR	error code
PRIVILEGE_FAULT	privilege check	no effects of the faulting instruction	BASIC	none
PAGE_FAULT	segment, translation, memory-type, or atomic-alignment check	no effects of the faulting instruction	PAGE_FAULT	error code, fault EA, and linear address
DIVIDE_ERROR	integer divide operation	no effects of the faulting instruction	ERROR	error code
VECTOR_RANGE_ERROR	PLOOP offset or vector lane-index validation	no effects of the faulting instruction	ERROR	error code
ACCESS_FAULT	physical-address-width or MMIO access-rule check	no effects of the faulting instruction	PAGE_FAULT	error code, fault EA, and linear address
INVALID_CONTROL_STATE	control-state validation	no effects of the faulting instruction	ERROR	error code
FLOATING_POINT_FAULT	enabled floating-point exception condition	no destination or accrued-flag effects of the faulting instruction	ERROR	error code

Table 14-4 (continued)

Event	Producer	Committed State	Frame	Payload
DOUBLE_FAULT	architectural event-delivery failure	no partial first delivery state	AUXILIARY	delivery-failure auxiliary state
MACHINE_CHECK	processor or memory-system machine-check source	selected by PRECISE and RETRY_SAFE	AUXILIARY	machine-check auxiliary state
BUS_ERROR	synchronous acknowledged bus failure	no architectural-state effects of the faulting instruction	AUXILIARY	bus-error auxiliary state
SYSTEM_CALL	SYSCALL	SYSCALL completed	BASIC	none
NMI	admitted non-maskable interrupt source	all earlier committed state	BASIC	none
INTERRUPT	admitted maskable interrupt source	all earlier committed state	BASIC	none

Unassigned base-profile exception IDs are reserved. External interrupt IDs do not share a namespace with exception IDs. Interrupt-controller routing, native-ID mapping, ownership, acknowledgement, trigger mode, masking, and end-of-interrupt signaling belong to the platform ABI. An interrupt ID unknown to software still reaches the common entry; the platform dispatcher applies its unhandled or spurious-interrupt policy.

A restartable fault saves the faulting instruction or its architecturally defined restart point. A trap saves the following instruction. An interrupt or NMI saves the next instruction that would otherwise execute. A faulting repeat iteration rolls back without decrementing its counter, then saves the repeat-prefix address. After a successful repeat iteration, the counter decrement commits before an event may be admitted between iterations; that event also saves the repeat-prefix address. Event entry clears hidden repeat state, and ERET executes the saved prefix normally without restoring repeat continuation.

14.2 Event Priority and Debug Trace

When events compete at one architectural boundary, the lower priority number in the event table wins. This orders delivery failure before machine check, current synchronous faults, explicit traps, DEBUG_TRACE, NMI, and maskable interrupt. A lower-priority asynchronous event remains pending after a higher-priority event is selected.

A trace unit is one ordinary committed instruction or one committed REP or REPcc body iteration. Completion of a zero-count REP or REPcc is also one trace unit. TF is sampled at the start of the unit. If sampled TF is one and sampled RF is zero, successful completion commits the unit and then delivers DEBUG_TRACE with the following trace-unit boundary as its saved PC.

The TRACE marker instruction is an ordinary instruction for this trace-unit rule and separately records the marker described by its instruction entry. The tracing terminology distinguishes the instruction, trace unit, and event.

RF is a one-shot DEBUG_TRACE suppressor. If sampled RF is one, successful trace-unit completion clears RF and does not produce DEBUG_TRACE. A synchronous fault or asynchronous event before that completion preserves RF. WRSTATUS, ERET changes to TF or RF apply to the next trace unit. Event entry saves the old STATUS image and then clears live TF and RF. RF does not suppress an explicit trap such as BKPT.

14.3 Entry Admission and Event Depth

The event-entry state is usable only while ECR.V is set. A maskable interrupt is admitted only when ECR.V, STATUS.IE, and the depth limit all permit it: the current event depth must be less than ECR.MAX_EDEPTH. Otherwise the interrupt remains pending at the interrupt controller. A synchronous exception that requires delivery while ECR.V is clear cannot be deferred and enters SHUTDOWN. An NMI observed while ECR.V is clear sets the hardware-managed ECR.NMI_P latch and is not admitted. After ECR.V becomes one, hardware admits that pending NMI before the next instruction boundary when STATUS.NI is clear. While STATUS.NI remains set, the NMI remains pending. Multiple NMI observations coalesce in ECR.NMI_P.

Event depth is zero outside event handling. A successful entry saves STATUS in the selected return context and increments STATUS.EDEPTH. ECR.MAX_EDEPTH equal to zero prevents maskable interrupt admission; values 1 through 15 limit only maskable interrupts and do not suppress synchronous exceptions or NMI. Depth 15 is representable but terminal: any further event requirement enters SHUTDOWN. At depth 14 or less, a delivery failure can still create a DOUBLE_FAULT frame; at depth 15 there is no representable child frame.

14.4 Supervisor Event Stacks and Nesting

The configured supervisor stack pairs select only the first user-origin entry and delivery-failure recovery.

Table 14-5. Supervisor Stack Roles

Pair	Role	Use
SSS:SSP	system call	first user-origin SYSTEM_CALL
ISS:ISP	interrupt	first user-origin maskable interrupt
FSS:FSP	fault	first user-origin exception or NMI
DSS:DSP	recovery	DOUBLE_FAULT after delivery failure

A first user-origin entry loads the applicable configured pair. Supervisor-origin and all nested entries retain the current SS:SP and push a full frame there; event class never promotes an active handler to another configured stack. Delivery failure uses DSS:DSP when the hidden current-DFA bit is clear. Failure with current DFA set enters shutdown.

For user-origin entry, hardware rounds only the event payload to 16 bytes and subtracts it from the configured top. BASIC events have zero payload and perform no stack access. For supervisor or nested entry, hardware subtracts the complete full-frame size from current SP. Each nonzero range is validated and stored atomically before the new SP commits; configured tops are never modified.

DOUBLE_FAULT and event-delivery failure use DSS:DSP. Supervisor-origin and nested events retain the current SS:SP; ERET validates and restores the saved STATUS depth and origin chain.

14.5 Event Entry State

Before making an event visible, hardware validates EPC, ECS, EDS, the selected stack state, and the complete frame storage range. It then stores the frame, loads the entry segments and PC, sets STATUS.PM and STATUS.EA, and clears STATUS.IE, STATUS.TF, and STATUS.RF as one

architectural commit. R0 through R15 and GS0 through GS5 are unchanged; the common entry code preserves whatever its software ABI requires.

NMI entry additionally sets STATUS.NI. Other event classes preserve NI. A further NMI observed while NI is set is recorded by setting ECR.NMI_P. ECR.NMI_P is hardware managed, so WRCCR ignores the supplied value for that bit. When entry of an NMI admitted from ECR.NMI_P commits, hardware clears ECR.NMI_P as part of the same architectural transition. A newly observed NMI may set the latch again. An entry failure does not clear the latch before committing the specified delivery-failure result. When ERET successfully restores NI to zero, a pending NMI is delivered before the next instruction boundary.

STATUS.EA, STATUS.EDEPTH, STATUS.UO, and STATUS.PM are hardware-managed transition state. WRSTATUS must preserve their current values. An attempted change raises INVALID_CONTROL_STATE without modifying STATUS. WRSTATUS may change IE, NI, TF, and RF. Clearing NI while ECR.NMI_P is one admits the pending NMI at the next instruction boundary. ERET classifies direct U-bank, outer U-bank, and stacked returns before any stack access; an invalid combination raises INVALID_CONTROL_STATE without consuming either return source.

14.6 Architectural Event Frame

Every event frame begins with exactly eight 64-bit slots. `EVENT_INFO` contains the event code in bits 31 through 0; its upper half is zero. `FRAME_CONTROL` contains only frame shape, saved nesting state, saved `FLAGS`, and saved `STATUS`. Event classification is carried exclusively by `EVENT_INFO`.

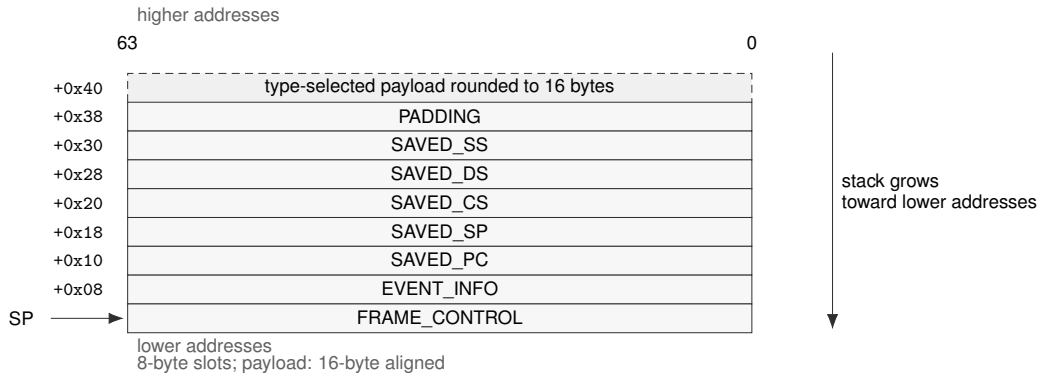


Figure 14-2. Architectural Event Frame

Table 14-6. Architectural Event Frame Fields

Offset	Slot	Meaning
+0x00	FRAME_CONTROL	frame shape, nesting metadata, saved <code>FLAGS</code> , and saved <code>STATUS</code>
+0x08	EVENT_INFO	event code in bits 31..0; bits 63..32 are zero
+0x10	SAVED_PC	saved program counter
+0x18	SAVED_SP	saved stack pointer
+0x20	SAVED_CS	saved code-segment image
+0x28	SAVED_DS	saved data-segment image
+0x30	SAVED_SS	saved stack-segment image
+0x38	PADDING	ignored by <code>ERET</code> ; software may write any value
+0x40	ERROR_CODE	exception-specific error code; unassigned bits are zero
+0x48	FAULT_EA	numeric effective address before segment pre-translation
+0x50	FAULT_LINEAR	address after segment pre-translation
+0x58	EVENT_AUX	double-fault, machine-check, or bus-error auxiliary information

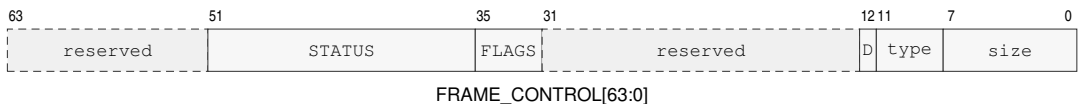


Figure 14-3. FRAME_CONTROL Format

In the diagram, `D` abbreviates `SAVED_DFA`.

Table 14-7. FRAME_CONTROL Fields

Field	Bits	Meaning
frame_size	0..7	total allocated frame size in 8-byte units
frame_type	8..11	optional payload layout code; it does not classify the event
saved_dfa	12	prior hidden double-fault-active state
reserved	13..31	reserved; must be zero
flags	32..35	saved FLAGS image
status	36..51	saved STATUS image, including EDEPTH and UO
reserved	52..63	reserved; must be zero

ERET first validates the frame shape and nesting metadata. BASIC requires frame_size 8, ERROR requires 10, and PAGE_FAULT and AUXILIARY require 12. It requires saved STATUS.PM=1, saved STATUS.EDEPTH plus one to equal the current depth, and saved STATUS.UO to match the active outer-origin chain.

Only after those checks does ERET validate the saved control state, segment images, SP, and PC. On success it restores FLAGS, STATUS, CS, DS, SS, SP, PC, and current DFA as one commit. It ignores the fixed-header padding slot at +0x38 and restores no hidden repeat state. EVENT_INFO and the event-specific payload remain available to software.

14.7 Event Payloads

Payload starts at offset +0x40. Hardware allocates only the defined payload prefix, rounds it up to a 16-byte boundary, and zeros the padding. The frame-size unit remains one 8-byte slot.

Table 14-8. Architectural Event Frame Types and Sizes

Code	Type	Payload	Allocated	Total	Size	Last slot
0x0	BASIC	0	0	64	8	none
0x1	ERROR	8	16	80	10	ERROR_CODE
0x2	PAGE_FAULT	24	32	96	12	FAULT_LINEAR
0x3	AUXILIARY	32	32	96	12	EVENT_AUX

Table 14-9. Event-to-Frame-Type Assignment

Delivered event	Frame type
DEBUG_TRACE, BREAKPOINT, PRIVILEGE_FAULT, SYSTEM_CALL	BASIC
ILLEGAL_INSTRUCTION, DIVIDE_ERROR, VECTOR_RANGE_ERROR, INVALID_CONTROL_STATE, FLOATING_POINT_FAULT	ERROR
PAGE_FAULT, ACCESS_FAULT	PAGE_FAULT
DOUBLE_FAULT, MACHINE_CHECK, BUS_ERROR	AUXILIARY
interrupt and NMI	BASIC

The delivered event determines the frame type. ERROR_CODE is defined separately by each exception and has zero in every unassigned bit. PAGE_FAULT and ACCESS_FAULT extend that slot with effective-address and linear-address context. AUXILIARY includes all four named slots and permits EVENT_AUX to describe double-fault, machine-check, or bus-error state.

A misaligned atomic memory operand uses the PAGE_FAULT frame with ERROR_CODE sub-type ATOMIC_ALIGNMENT. FAULT_EA identifies the misaligned effective-address operand and FAULT_LINEAR contains its segment-pretranslated starting address. The fault is reported before the atomic instruction performs a memory read, memory write, or architectural result update.

14.7.1 Address-Context Error Code

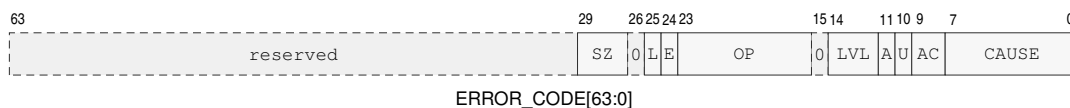


Figure 14-4. PAGE_FAULT and ACCESS_FAULT ERROR_CODE Format

SZ, L, E, OP, LVL, and AC denote ACCESS_SIZE, FAULT_LINEAR_VALID, FAULT_EA_VALID, OPERAND, WALK_LEVEL, and ACCESS; A and U denote ATOMIC and USER_DOMAIN.

Table 14-10. Address-Context ERROR_CODE Fields

Bits	Field	Meaning
7..0	CAUSE	cause in the selected exception's cause space
9..8	ACCESS	0 none, 1 read, 2 write, 3 execute
10	USER_DOMAIN	access selected the user-domain permission path
11	ATOMIC	access was an atomic read-modify-write
14..12	WALK_LEVEL	0 before or without a walk; 1 through 5 identify the PTE level
15	reserved	zero
23..16	OPERAND	zero-based explicit operand ordinal; 0xff for implicit fetch or stack access
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	reserved	zero
29..27	ACCESS_SIZE	0 unavailable or inapplicable; 1 B; 2 W; 3 L; 4 Q
63..30	reserved	zero

Table 14-11. PAGE_FAULT Causes

Value	Cause	Detection class
0	NOT_PRESENT	required PTE is not present
1	PERMISSION	accumulated permission excludes the access
2	MALFORMED_PTE	structurally invalid PTE
3	NONCANONICAL	noncanonical address
4	SEGMENT_BOUNDS	segment bounds failure
5	ATOMIC_ALIGNMENT	misaligned atomic operand
6	MEMORY_TYPE	physical Device memory is selected by a Normal leaf AM

Table 14-12. ACCESS_FAULT Causes

Value	Cause	Meaning
0	PHYSICAL_ADDRESS	paging-disabled physical address exceeds implementation PABITS
1	MMIO_ALIGNMENT	permitted scalar MMIO operand is not naturally aligned
2	MMIO_OPERATION	operation class is prohibited for MMIO

Saved STATUS.PM records the originating privilege. USER_DOMAIN distinguishes the ordinary user permission path from the checked user-access path used by supervisor instructions such as

MOVUC, MOVCU, and MOVUU. An atomic read-modify-write reports ACCESS=write. NOT_PRESENT and MALFORMED_PTE identify the actual PTE level. PERMISSION identifies the first level, in root-to-leaf walk order, whose accumulated permissions exclude the requested access. A cause detected before a walk reports level zero. Every ACCESS_FAULT reports WALK_LEVEL=0; translation has already completed for MMIO causes, while PHYSICAL_ADDRESS applies with paging disabled.

ACCESS_SIZE records the architectural transfer width when one of the B, W, L, or Q widths applies. It is zero when no such width is available or applicable.

FAULT_EA is the numeric effective address before segment pre-translation. FAULT_LINEAR is the result after segment pre-translation. A value not produced is zero and has its corresponding validity bit clear.

14.7.2 Other Exception Error Codes

DIVIDE_ERROR.ERROR_CODE[7:0] is 0 for ZERO_DIVISOR and 1 for SIGNED_OVERFLOW; bits 63..8 are zero.

Table 14-13. ILLEGAL_INSTRUCTION Causes

Value	Cause	Meaning
0	INVALID_OPCODE	opcode is not assigned
1	INVALID_EA_FORM	effective-address form is not legal for the instruction
2	RESERVED_ENCODING	assigned instruction uses a reserved field value
3	UNAVAILABLE_EXTENSION	required extension is unavailable
4	EXPLICIT_ILLEGAL	architected ILLEGAL instruction
5	INSUFFICIENT_LENGTH	encoded instruction length is shorter than the selected encoding requires
6	INVALID_OPERAND_RELATION	writable operands use a prohibited overlap relation

Table 14-14. INVALID_CONTROL_STATE Causes

Value	Cause	Meaning
0	INVALID_SELECTOR	selector names no available resource
1	RESERVED_BITS	reserved bits are nonzero or changed illegally
2	INVALID_IMAGE	supplied register or processor-state image is invalid
3	INVALID_TRANSITION	requested state transition is illegal

Table 14-15. VECTOR_RANGE_ERROR Causes

Value	Cause	Meaning
0	LOOP_OFFSET	PLOOP receives an invalid unsigned logical-lane Roffset/index
1	LANE_INDEX	a scalar lane index is outside the selected vector element count

An unavailable RDPMC ID reports INVALID_SELECTOR; an illegal reserved-bit update reports RESERVED_BITS; an invalid segment, control-register, or SAVE/RESTORE image reports INVALID_IMAGE; and an illegal ERET or privilege-state transition reports INVALID_TRANSITION.

FLOATING_POINT_FAULT.ERROR_CODE[4:0] is a cause bitmap: bits 4 through 0 are NV, DZ, OF, UF, and NX. All causes generated by the operation are reported together; bits 63..5 are zero.

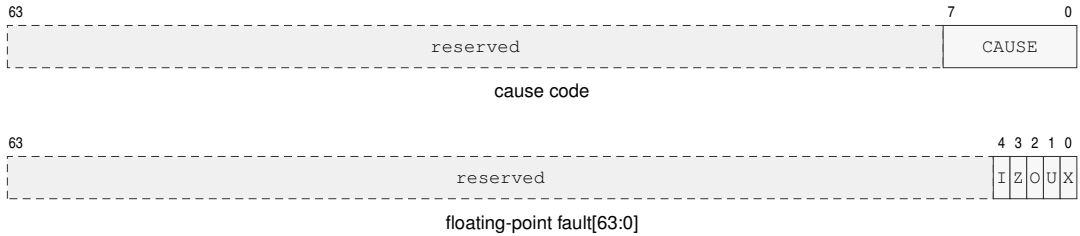


Figure 14-5. Simple Exception ERROR_CODE Formats

The cause-code row applies to DIVIDE_ERROR, ILLEGAL_INSTRUCTION, VECTOR_RANGE_ERROR, and INVALID_CONTROL_STATE. In the floating-point row, I, Z, O, U, and X abbreviate NV, DZ, OF, UF, and NX.

The bounds instructions report failure through FLAGS.V. Segment bounds failures use PAGE_FAULT. BKPT raises BREAKPOINT; DEBUG_TRACE is generated by the architectural trace mechanism.

14.7.3 Auxiliary Payloads

For an AUXILIARY frame, ERROR_CODE bit 26 is EVENT_AUX_VALID. Bits 24 and 25 are FAULT_EA_VALID and FAULT_LINEAR_VALID when the event can supply those addresses. Every unassigned bit is zero, and an auxiliary or address value not produced is zero with its validity bit clear.

For DOUBLE_FAULT, ERROR_CODE[7:0] identifies the delivery stage:

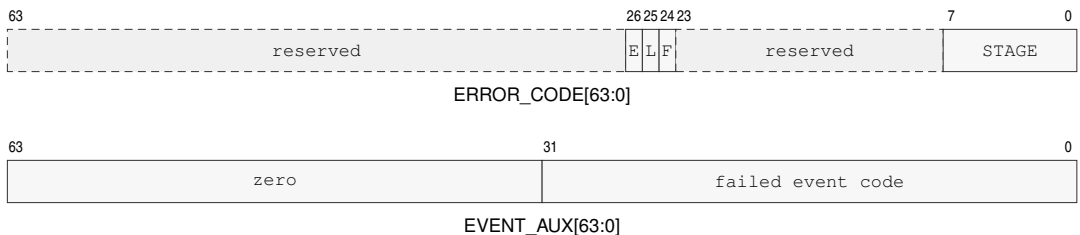


Figure 14-6. DOUBLE_FAULT Auxiliary Formats

E, L, and F are EVENT_AUX_VALID, FAULT_LINEAR_VALID, and FAULT_EA_VALID.

Table 14-16. DOUBLE_FAULT Delivery Stages

Value	Stage	Failure
0	ENTRY_STATE	entry code or segment state validation
1	STACK_STATE	selected event-stack state
2	FRAME_ADDRESS	complete frame-address formation or translation
3	FRAME_STORE	complete frame storage

When valid, `EVENT_AUX[31:0]` contains the event code whose delivery failed and bits 63..32 are zero. A `FRAME_ADDRESS` or `FRAME_STORE` failure supplies `FAULT_EA` and `FAULT_LINEAR` when those values were produced.

For `MACHINE_CHECK`, the error code has this layout:

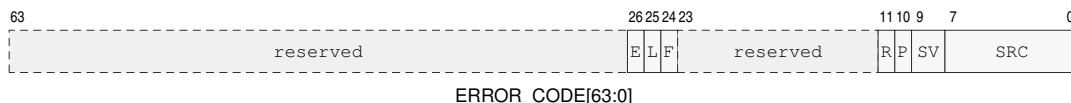


Figure 14-7. MACHINE_CHECK ERROR_CODE Format

E, L, and F are `EVENT_AUX_VALID`, `FAULT_LINEAR_VALID`, and `FAULT_EA_VALID`; P, R, SV, and SRC denote `PRECISE`, `RETRY_SAFE`, `SEVERITY`, and `SOURCE`.

Table 14-17. MACHINE_CHECK ERROR_CODE Fields

Bits	Field	Meaning
7..0	SOURCE	0 unknown; 1 core; 2 cache; 3 translation; 4 memory; 5 interconnect
9..8	SEVERITY	0 corrected; 1 recoverable; 2 fatal; 3 reserved
10	PRECISE	saved PC identifies the responsible instruction and saved architectural state precedes it
11	RETRY_SAFE	retry at that precise boundary cannot duplicate an external effect
23..12	reserved	zero
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	EVENT_AUX_VALID	EVENT_AUX was produced
63..27	reserved	zero

When valid, `EVENT_AUX` contains an implementation-defined syndrome. Associated effective and linear addresses are supplied when available.

`RETRY_SAFE=1` requires `PRECISE=1`. The valid severity and continuation combinations are:

Table 14-18. MACHINE_CHECK Valid Continuation Combinations

Severity	PRECISE	RETRY_SAFE	Saved PC and continuation meaning
corrected	0	0	Trap; saved PC is the instruction following the corrected operation.
recoverable	0	0	Saved PC is the next instruction that would execute at the delivery boundary; saved architectural state is the state at that boundary. It does not identify the responsible operation.
recoverable	1	0	Saved PC identifies the responsible instruction and architectural state precedes it; retry may duplicate an external effect.

Table 14-18 (continued)

Severity	PRECISE	RETRY_SAFE	Saved PC and continuation meaning
recoverable	1	1	Saved PC identifies the responsible instruction and architectural state precedes it; retry cannot duplicate an external effect.
fatal	0	0	Saved PC is the next instruction at the delivery boundary and is diagnostic only.
fatal	1	0	Saved PC identifies the responsible instruction and pre-instruction architectural state for diagnosis only.

All other combinations are invalid and are not generated. In particular, corrected machine checks never set either bit, PRECISE=0, RETRY_SAFE=1 is invalid for every severity, fatal machine checks never set RETRY_SAFE, and severity 3 is reserved. A fatal event provides no reliable continuation even when its diagnostic state is precise.

For BUS_ERROR, the error code has this layout:

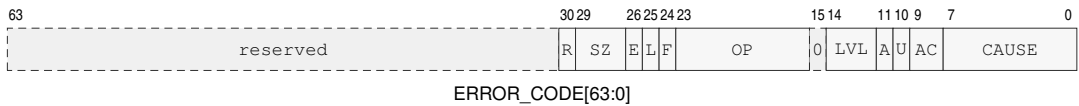


Figure 14-8. BUS_ERROR ERROR_CODE Format

E, L, and F are EVENT_AUX_VALID, FAULT_LINEAR_VALID, and FAULT_EA_VALID; SZ, OP, LVL, and AC denote ACCESS_SIZE, OPERAND, WALK_LEVEL, and ACCESS; A, U, and R denote ATOMIC, USER_DOMAIN, and RETRY_SAFE.

Table 14-19. BUS_ERROR ERROR_CODE Fields

Bits	Field	Meaning
7..0	CAUSE	0 no responder; 1 access denied; 2 timeout; 3 data error; 4 other
9..8	ACCESS	0 none, 1 read, 2 write, 3 execute
10	USER_DOMAIN	access selected the user-domain permission path
11	ATOMIC	access was an atomic read-modify-write
14..12	WALK_LEVEL	0 outside a walk; 1 through 5 identify the PTE level
15	reserved	zero
23..16	OPERAND	zero-based explicit operand ordinal; 0xff for implicit fetch or stack access
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	EVENT_AUX_VALID	EVENT_AUX was produced
29..27	ACCESS_SIZE	0 unavailable or inapplicable; 1 B; 2 W; 3 L; 4 Q
30	RETRY_SAFE	retry cannot duplicate an external effect
63..31	reserved	zero

A bus error during a page walk reports the level whose PTE access failed. When valid, EVENT_AUX contains the final byte address available for the failed access.

14.8 Machine-Check and Bus-Error Continuation

A corrected MACHINE_CHECK is a trap, has both PRECISE and RETRY_SAFE clear, and saves the following instruction. A recoverable machine check uses the valid combinations in the auxiliary-payload table. When PRECISE is one, the saved PC identifies the responsible instruction boundary

and logical-processor architectural state is the state from before that instruction. When it is zero, the saved PC is the next instruction that would execute at the delivery boundary and architectural state corresponds to that boundary; neither identifies the responsible operation. `RETRY_SAFE` may be one only with `PRECISE=1` and additionally guarantees that returning to the responsible boundary cannot duplicate an external effect. A fatal machine check always clears `RETRY_SAFE`; `PRECISE` then describes diagnostic location quality and never guarantees reliable continuation.

`BUS_ERROR` is a synchronous precise fault. Logical-processor architectural state remains uncommitted and the saved PC identifies the responsible instruction boundary. Its separate `RETRY_SAFE` bit states whether an external effect might already have occurred. Page-walk and ordinary memory bus errors are retry-safe.

Software interprets the machine-check continuation fields before choosing a return or recovery action.

14.9 Delivery Failure and Shutdown

Failure to validate the common entry context, establish the selected stack, or store a complete event frame is delivered as `DOUBLE_FAULT` on `DSS:DSP`. Failure while `DOUBLE_FAULT` is already being delivered enters `SHUTDOWN`. `SHUTDOWN` retires no further instructions and can be exited only by a platform reset. The frame save and all architectural entry state remain atomic, so a failed attempt never exposes a partially stored frame or partially changed execution context.

14.10 Event Reset and Context State

Table 14-20. Warm RESET Contract

Property	Architectural Rule
Scope	executing logical processor
Required privilege	supervisor
Serialization	complete prior memory effects and pending write-combining stores before reset state commits
Restart	running at <code>BOOTPC</code> after instruction-fetch synchronization
Other logical processors	unchanged

Table 14-21. Architectural Reset State

State	Reset Value
R0, R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15, SP	0
FLAGS	0
STATUS	PM=1; all other STATUS bits 0
CS, DS, SS, GS0, GS1, GS2, GS3, GS4, GS5	0
PC	BOOTPC
PTCR, ASCR, ECR	0
EPC, ECS, EDS	0
UPC, USP, UCS, UDS, USS, UCTL, UINFO	0
SSS, SSP, ISS, ISP, FSS, FSP, DSS, DSP	0

Table 14-21 (continued)

State	Reset Value
PMC, CYCLE, INSTRET, PTWALK	0
F0–F15, FSTATUS, FFLAGS, extension_ state	0
V0_V31, P0_P15	0
hidden_repeat_state, hidden_load_ reservation	invalid
hidden current DFA	0
ECR.NMI_P	0
local_translation_cache, local_page_ walk_cache, local_prefetch_state	invalid
coherent_data_and_instruction_cache_ contents	retained as coherent contents
execution_state	running at BOOTPC
BOOTPC, BOOTCFG	platform supplied on cold reset; preserved by warm RESET

Warm RESET applies the processor-state image shown in the preceding table to the executing logical processor only. Cold reset initializes BOOTPC and BOOTCFG before applying the same processor-state image to each logical processor selected by the platform reset event. Software initializes entry segments, exact entry PCs, and configured stack tops before setting ECR.V or entering user mode.

ECR, entry and stack registers, STATUS.EDEPTH/UO, hidden current DFA, and the user context bank are per-logical-processor state. System software includes the applicable state when switching supervisor contexts. Payloads and full event frames reside in byte-addressed normal memory owned by the interrupted context.

15 CPUID FEATURE DISCOVERY

15.1 Overview

CPUID is the architectural discovery interface for optional instruction groups, processor-state save areas, implementation properties, and feature-specific parameters.

The CPUID instruction uses a selected Rn register as both query selector and result register. Software writes the selector before execution and reads the returned 64-bit value after execution.

CPUID exposes program-visible architectural information.

The normative instruction entry is CPUID.

15.2 Query Model

A query selector identifies a discovery class, leaf, and optional result index. The selector is a Q-sized value held in the selected Rn register.

CPUID is unprivileged under the base compatibility rules. For a logical processor, CPUID results remain stable until that logical processor is reset. A discovery result applies to execution on the logical processor that returned it. Software associates cached discovery state with that logical processor and uses results obtained on each logical processor before selecting its optional architectural behavior.

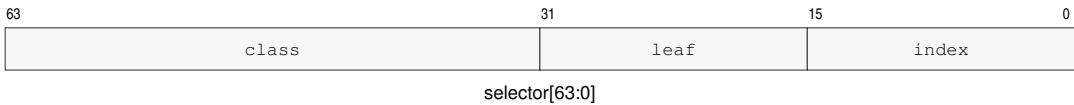
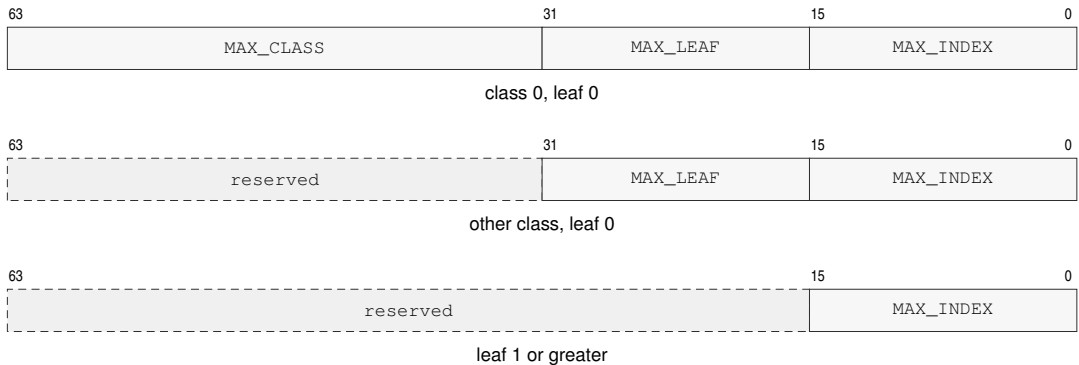


Figure 15-1. CPUID Query Selector Format

Table 15-1. CPUID Query Selector

Bits	Field	Meaning
0..15	index	result index within the selected leaf
16..31	leaf	information leaf within the selected class
32..63	class	CPUID information class

Index zero of every defined leaf is a common discovery header. Leaf-specific payload begins at index one. An unknown class, leaf, or index returns zero.

**Figure 15-2. CPUID Common Leaf Header Formats****Table 15-2. CPUID Common Leaf Header**

Query	Bits 63..32	Bits 31..16	Bits 15..0
class 0, leaf 0, index 0	MAX_CLASS	MAX_LEAF	MAX_INDEX
other class, leaf 0, index 0	reserved, zero	MAX_LEAF	MAX_INDEX
any class, leaf 1 or greater, index 0	reserved, zero	reserved, zero	MAX_INDEX

MAX_CLASS is the greatest class recognized by the implementation, MAX_LEAF is the greatest leaf recognized within the selected class, and MAX_INDEX is the greatest index recognized within the selected leaf. Each field is a maximum selector value, not a count. A leaf may be sparse below MAX_INDEX. Software ignores reserved bits in every returned value, so a later revision can assign them without invalidating existing discovery code.

CPUID bit fields are feature predicates unless the leaf description says they encode a numeric value. Reserved result bits are ignored by software.

A set feature bit only permits software to use the corresponding architectural behavior; the instruction's normal privilege, operand, and compatibility checks still apply.

The FP, FPTRANSA, and VECTOR predicates report the base floating-point, approximate transcendental floating-point, and scalable-vector groups. Numeric fields use names that describe their unit. SAVE_AREA_SIZE_BYTES, OFFSET_BYTES, MAX_SIZE_BYTES, and ALIGNMENT_BYTES are byte counts.

15.3 Discovery Classes

Discovery classes partition architectural information by compatibility purpose. Software first checks the class and leaf directory before consuming optional feature bits.

The base class describes the mandatory scalar ISA and implementation identity. Optional groups are reported in the optional-extension directory.

15.4 Leaf Directory

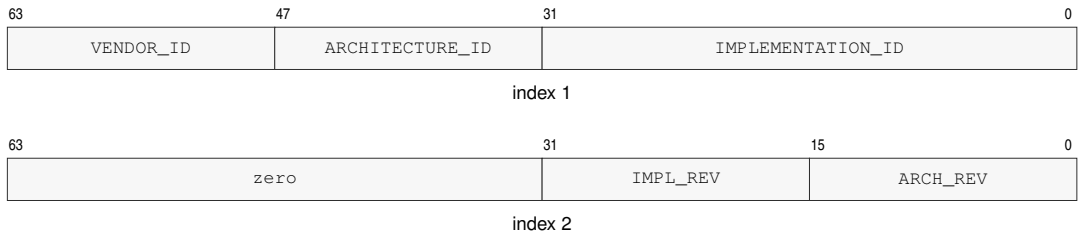
Each discovery class may expose a leaf directory. A directory leaf reports which subordinate leaves are defined and how indexed leaves should be enumerated.

Table 15-3. CPUID Class and Leaf Directory

Class	Leaf	Name	Indexes	Summary
0x00000000	—	<i>Bedrock base ISA</i>	—	base identity, architecture and implementation revisions, vendor name, and processor name
0x00000000	0x0000	BASE_IDENTITY	0..18	common header, identity, revisions, vendor name, and processor name
0x00000001	—	<i>Optional extensions</i>	—	optional floating-point, transcendental, and scalable-vector groups
0x00000001	0x0000	EXTENSION_DIRECTORY	0..1	common header and optional architectural-group bits
0x00000001	0x0001	FPTRANSA_ACCURACY	0..0x0044	common header and sparse per-function accuracy contracts
0x00000001	0x0002	VECTOR_PARAMETERS	0..1	common header and the implementation-selected VLEN logarithm
0x00000002	—	<i>Implementation properties</i>	—	cache topology, performance counters, address widths, and SAVE/RESTORE area layout
0x00000002	0x0000	IMPLEMENTATION_DIRECTORY	0..0	common implementation-class header
0x00000002	0x0001	CACHE_TOPOLOGY	0..n	common header, maintenance granule, and cache-sharing descriptors
0x00000002	0x0002	PERFORMANCE_COUNTERS	0..3	common header and standard RDPMC-counter presence
0x00000002	0x0003	ADDRESS_WIDTHS	0..1	common header and the implementation physical-address width
0x00000002	0x0004	SAVE_AREA_LAYOUT	0..n	common header, SAVE/RESTORE sizes, bitmap, and component descriptors

15.5 Base Identity

Class 0x00000000, leaf 0x0000 reports MAX_LEAF=0 and MAX_INDEX=18 in its index-zero header. An implementation recognizing only the classes defined by this specification reports MAX_CLASS=2; an implementation that recognizes a higher implementation-defined class reports the highest such selector. The base-identity payload is:

**Figure 15-3. CPUID Base Identity Result Formats****Table 15-4. Base Identity Indexes**

Index	Contents
1	bits 31..0 IMPLEMENTATION_ID; bits 47..32 ARCHITECTURE_ID; bits 63..48 VENDOR_ID
2	bits 15..0 ARCHITECTURE_REVISION; bits 31..16 IMPLEMENTATION_REVISION; bits 63..32 zero
3–10	64-byte vendor name
11–18	64-byte processor name

VENDOR_ID selects the vendor namespace in which IMPLEMENTATION_ID is interpreted. IMPLEMENTATION_ID is assigned within that VENDOR_ID namespace. ARCHITECTURE_ID selects the base architectural contract, so the (VENDOR_ID, IMPLEMENTATION_ID) pair identifies an implementation and ARCHITECTURE_ID identifies the ISA contract it implements.

ARCHITECTURE_REVISION is a monotonically increasing unsigned 16-bit compatibility level within one ARCHITECTURE_ID. A processor reporting revision *N* implements the base architectural behavior of every defined revision less than or equal to *N*. Optional instruction groups, state, and parameterized contracts are selected from their CPUID leaves and feature bits. IMPLEMENTATION_REVISION is an unsigned revision assigned by the vendor and is compared only when both VENDOR_ID and IMPLEMENTATION_ID match.

The name fields are UTF-8 byte strings in increasing index order. Within each 64-bit result, the first byte occupies bits 7..0 and subsequent bytes occupy successively higher bits. A name shorter than 64 bytes is terminated by a zero byte and all remaining bytes are zero. A 64-byte name has no terminator. Producers must not split a multi-byte UTF-8 code point at the field boundary; consumers replace an invalid sequence rather than reading beyond the field.

15.6 Optional-Extension Directory

Class 0x00000001, leaf 0x0000, index one reports optional architectural groups. The index-zero header reports MAX_LEAF=2 and MAX_INDEX=1.



Figure 15-4. Optional-Extension Directory Result

Table 15-5. Optional-Extension Feature Bits

Class	Leaf	Index	Bit	Feature	Extension
0x00000001	0x0000	1	0	FP	fpu
0x00000001	0x0000	1	1	FPTRANSA	fpu.transcendental_approx
0x00000001	0x0000	1	2	VECTOR	vector

Unlisted bits are reserved and read as zero. FPTRANSA requires FP and is set when at least one approximate transcendental contract is present.

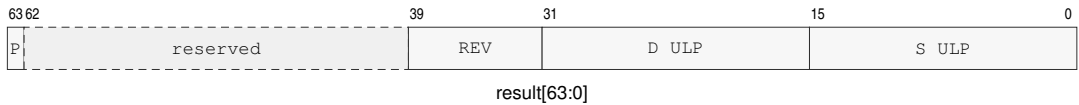
15.7 Vector-Parameter Discovery

Class 0x00000001, leaf 0x0002 is VECTOR_PARAMETERS. Index zero reports MAX_INDEX=1. Index one returns VLEN_LOG2_BITS in bits 3 through 0 and zero in bits 63 through 4. The value is the base-2 logarithm of VLEN in bits and is in the range 7 through 11. Consequently VLEN is an implementation-selected power of two from 128 through 2048 bits. The selected value is stable until reset.

15.8 FPTRANSA Accuracy Discovery

Class 0x00000001, leaf 0x0001 uses indexes 1 through 0x0044 as a sparse space of stable 16-bit accuracy-contract IDs. Index zero is the common header and reports MAX_INDEX=0x0044. A contract ID identifies the reference function, input domain, special-value behavior, error metric, exact anchors, and mathematical properties; it is independent of instruction encoding placement. An incompatible change receives a new ID, and a retired ID is not reused. The contracts defined here use CONTRACT_REVISION=1. Weakening a guarantee, changing the domain, or changing the error metric requires a new ID rather than a revision increment. An implementation that advertises a smaller certified ULP bound strengthens the existing contract and retains the same ID and revision.

The sparse ID space reserves 0x0001..0x000F for reduced-domain circular trigonometric functions, 0x0010..0x001F for inverse trigonometric functions, 0x0020..0x002F for hyperbolic and inverse hyperbolic functions, 0x0030..0x003F for exponential functions, and 0x0040..0x004F for logarithmic functions. The low-nibble-zero selector of each family is unassigned, and actual contracts in each family begin at low nibble one.

**Figure 15-5. FPTRANSA Accuracy Result Format**

P is PRESENT; REV is CONTRACT_REVISION; the D and S ULP fields use unsigned Q8.8.

Table 15-6. FPTRANSA Accuracy Result

Bits	Field	Meaning
0..15	S_MAX_ULP_Q8_8	certified worst-case S-format ULP bound, rounded upward to unsigned Q8.8
16..31	D_MAX_ULP_Q8_8	certified worst-case D-format ULP bound, rounded upward to unsigned Q8.8
32..39	CONTRACT_REVISION	value 1 for the contracts defined here
40..62	reserved	zero
63	PRESENT	the instruction and both S and D encodings are available

For a certified bound K , the field value is $\lceil 256K \rceil$ and the advertised bound is the field divided by 256. Thus 0.5, 1, 1.5, 2, and 4 ULP encode as 0x0080, 0x0100, 0x0180, 0x0200, and 0x0400. A present contract has both fields in 0x0001..0x0400. Software that does not recognize the returned revision uses its fallback path.

Software first checks FP and FPTRANSA in EXTENSION_DIRECTORY, then queries the intended contract ID in FPTRANSA_ACCURACY. It uses the instruction only when PRESENT is set, the returned CONTRACT_REVISION is 1, and the S or D bound required by the call site is no larger than its quality threshold; otherwise it uses a software fallback. For example, a D-format FLOG2A path requiring at most 2 ULP accepts selector 0x0000000100010043 only when the returned D field is at most 0x0200.

The selector is $(0x00000001 \ll 32) | (0x0001 \ll 16) | \text{contract-id}$. For example, FSINA uses selector 0x0000000100010001, and FLOG2A uses 0x0000000100010043. An implementation that certifies FSINA at 1.5 ULP for S and 2 ULP for D returns 0x8000000102000180.

PRESENT=1 guarantees that the instruction and both S and D encodings are architecturally usable under the returned contract. The S and D bounds are worst-case bounds over every input in the contract domain and apply to every execution on the logical processor that returned the CPUID result until reset. Executing the instruction with PRESENT=0 raises ILLEGAL_INSTRUCTION before operand or floating-point-state access.

Table 15-7. FPTRANSA Accuracy Contracts

Contract ID	Mnemonic	Reference	Domain	ISA maximum
0x0001	FSINA	$\sin(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0002	FCOSA	$\cos(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0003	FTANA	$\tan(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP

Table 15-7 (continued)

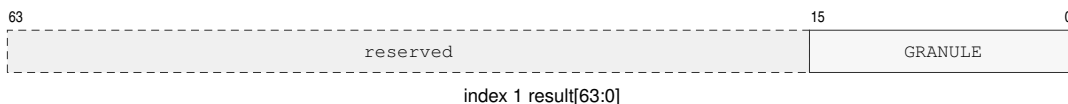
Contract ID	Mnemonic	Reference	Domain	ISA maximum
0x0004	FSINCOSA	paired $\sin(x)$ and $\cos(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0011	FASINA	$\text{asin}(x)$ in radians	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0012	FACOSA	$\text{acos}(x)$ in radians	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0013	FATANA	$\text{atan}(x)$ in radians	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0021	FSINHA	$\sinh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0022	FCOSHA	$\cosh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0023	FTANHA	$\tanh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0024	FATANHA	$\text{atanh}(x)$	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0031	FETOXA	e^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0032	FETOXM1A	$e^x - 1$ without an intermediate rounded exponential	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0033	FTWOTOXA	2^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0034	FTENTOXAX	10^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0041	FLOGNA	$\ln(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0042	FLOGNP1A	$\ln(1 + x)$ without an intermediate rounded addition	finite $x \geq -1$ and positive infinity after DAZ; -1 is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0043	FLOG2A	$\log_2(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0044	FLOG10A	$\log_{10}(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP

15.9 Implementation-Class Directory

Class 0x00000002, leaf 0x0000 consists of the common header. It reports `MAX_LEAF=4` and `MAX_INDEX=0`.

15.10 Cache-Topology Discovery

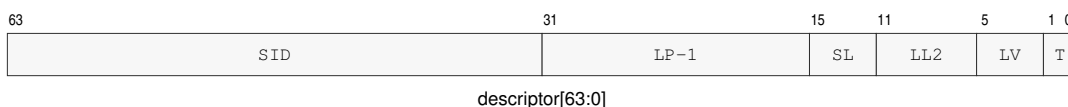
Class 0x00000002, leaf 0x0001 reports the cache-maintenance granule and the cache instances visible to the current logical processor. Index zero is the common header. Index one is the maintenance-properties result, and indexes 2 through `MAX_INDEX` are a dense array of cache descriptors. With N descriptors, `MAX_INDEX=1+N`.

**Figure 15-6. Cache-Maintenance Properties Result**

GRANULE is MAINTENANCE_GRANULE_BYTES.

Table 15-8. Cache-Maintenance Properties

Index	Bits	Field	Meaning
1	15..0	MAINTENANCE_GRANULE_BYTES	one-block cache-maintenance granule in bytes
1	63..16	reserved	zero

**Figure 15-7. Cache-Topology Descriptor Format**

SID, LP-1, SL, LL2, LV, and T denote SHARING_ID, SHARING_LP_COUNT_MINUS1, SHARING_LEVEL, LINE_LOG2, LEVEL, and TYPE.

Table 15-9. Cache-Topology Descriptor

Bits	Field	Meaning
1..0	TYPE	1 data; 2 instruction; 3 unified
5..2	LEVEL	cache level, in the range 1 through 15
11..6	LINE_LOG2	cache-line bytes are 1 shifted left by this value
15..12	SHARING_LEVEL	nested cache-sharing scope; zero is one logical processor
31..16	SHARING_LP_COUNT_MINUS1	logical-processor count in the sharing scope, minus one
63..32	SHARING_ID	identifier of the processor set at the reported sharing level

MAINTENANCE_GRANULE_BYTES is a nonzero power of two no greater than 4096 and has the same value on every logical processor in one normal-memory coherence domain. Every reported cache-line size divides the maintenance granule. A descriptor has TYPE 1, 2, or 3 and a nonzero LEVEL. Descriptors are ordered by increasing cache level, cache type, sharing level, and sharing ID.

SHARING_LEVEL values describe strictly nested processor sets. The pair (SHARING_LEVEL, SHARING_ID) is equal exactly when two descriptors name the same logical-processor set. At sharing level zero, SHARING_LP_COUNT_MINUS1 is zero. At a greater sharing level, the count field equals the number of logical processors in that set minus one. The tuple of cache type, cache level, sharing level, and sharing ID is unique within the leaf.

15.11 Performance-Counter Discovery

Class 0x00000002, leaf 0x0002 uses indexes 1 through 65535 as RDPMC counter IDs. Index zero is the common header. Result bit 0 is PRESENT; bits 63..1 are reserved and read as zero. CYCLE,

INSTRET, and PTWALK IDs 1, 2, and 3 are mandatory and therefore always report present; the index-zero header reports MAX_INDEX=3 when no implementation-defined counter has a greater ID. An implementation-defined ID may be used only when its query reports present.



Figure 15-8. Performance-Counter Presence Result

P is PRESENT.

15.12 Address-Width Discovery

Class 0x00000002, leaf 0x0003 is ADDRESS_WIDTHS. Index zero reports MAX_INDEX=1. At index one, bits 5..0 contain the exact implementation PABITS value in the inclusive range 32 through 56; bits 63..6 are zero. PABITS is stable until reset, applies to page-table roots, every present PTE PFN, translated results, and paging-disabled physical addresses, and is not a page-table geometry or TLB-identity selector.

15.13 SAVE-Area Layout Discovery

Class 0x00000002, leaf 0x0004 describes the memory image used by SAVE and RESTORE. Index zero is the common header. Indexes one and two describe the complete image and its fixed block. Each available extension component contributes descriptor A followed by descriptor B, so for *N* components MAX_INDEX=2+2*N*. Component descriptors are ordered by increasing COMPONENT_ID.

Table 15-10. SAVE-AREA-LAYOUT Indexes

Index	Contents
0	common CPUID leaf header
1	SAVE_AREA_SIZE_BYTES in bits 63..0
2	FIXED_SIZE_BYTES[15:0], COMPONENT_COUNT[31:16], BITMAP_WORDS[47:32], FMT[51:48]; bits 63..52 are zero
3+2 <i>i</i>	component descriptor A for zero-based component ordinal <i>i</i>
4+2 <i>i</i>	component descriptor B for zero-based component ordinal <i>i</i>

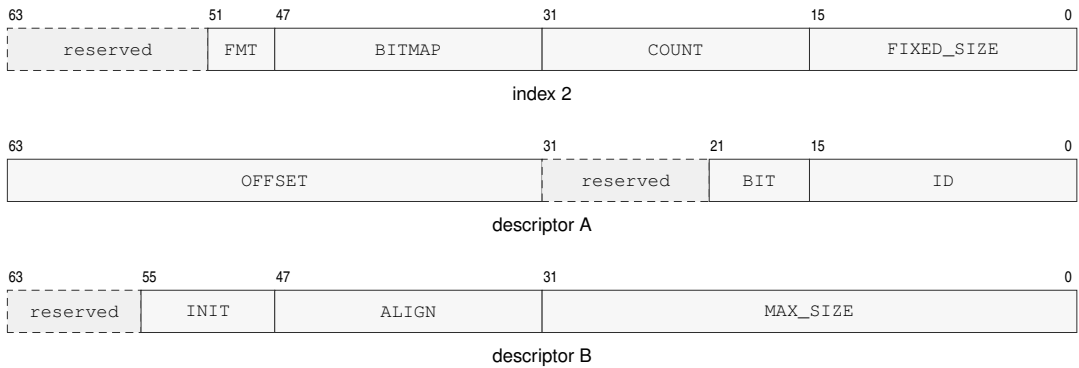
**Figure 15-9. SAVE-Area Layout CPUID Result Formats**

Diagram labels abbreviate the field names in the descriptor tables below; BITMAP, COUNT, and FIXED_SIZE are BITMAP_WORDS, COMPONENT_COUNT, and FIXED_SIZE_BYTES.

Table 15-11. SAVE Component Descriptor A

Bits	Field	Meaning
15..0	COMPONENT_ID	stable component identifier
21..16	BITMAP_BIT	bit selecting this component
31..22	reserved	zero
63..32	OFFSET_BYTES	offset from the save-area base

Table 15-12. SAVE Component Descriptor B

Bits	Field	Meaning
31..0	MAX_SIZE_BYTES	allocated component size
47..32	ALIGNMENT_BYTES	required component alignment
55..48	INIT_POLICY	1 restores the component-defined initial state when its bitmap bit is clear
63..56	reserved	zero

Table 15-13. Floating-Point SAVE Component

Component Offset	State	Contents
0x000..0x07f	F0_F15	F0 through F15 in ascending register order
0x080..0x087	FFLAGS	FFLAGS in bits 15..0; remaining bits zero
0x088..0x08f	FSTATUS	FSTATUS in bits 15..0; remaining bits zero

Table 15-14. Scalable-Vector SAVE Component (B = VLEN bytes)

Component Offset	State	Contents
$n * B$	V_n	V0 through V31; each register occupies B bytes
$32 * B + n * B/8$	P_n	P0 through P15; each packed predicate image occupies B/8 bytes
$34 * B \dots \text{align_up}(34 * B, 64) - 1$	padding	zero on SAVE and ignored on RESTORE

SAVE_AREA_SIZE_BYTES is the complete byte range validated and accessed by SAVE or RESTORE. It is at least FIXED_SIZE_BYTES, covers the end of every component, and is a multiple of 64 bytes. FIXED_SIZE_BYTES is 0x00c0, BITMAP_WORDS is 1, and FMT is zero for the save-image format defined by this specification.

Each descriptor names one available extension component. Component IDs and bitmap bits are unique. Component regions begin at or after the fixed block, meet their reported alignment, and do not overlap. A set state-block bitmap bit selects the descriptor carrying the same BITMAP_BIT. INIT_POLICY=1 means that RESTORE reconstructs the component-defined initial state when that component's bitmap bit is clear.

When the FP extension is present, component ID 1 uses bitmap bit 0, offset 0x00c0, size 0x00c0, and alignment 64. Its initial state is F0 through F15 equal to positive zero, FFLAGS equal to zero, and FSTATUS equal to zero.

16 PROCESSOR-STATE SAVE AND RESTORE

SAVE and RESTORE use a CPUID-defined memory image whose base is 4 KiB aligned.

16.1 Save-Area Layout

Software obtains every extension component's offset and maximum size from CPUID SAVE_AREA_LAYOUT. Each component starts on a 64-byte boundary.

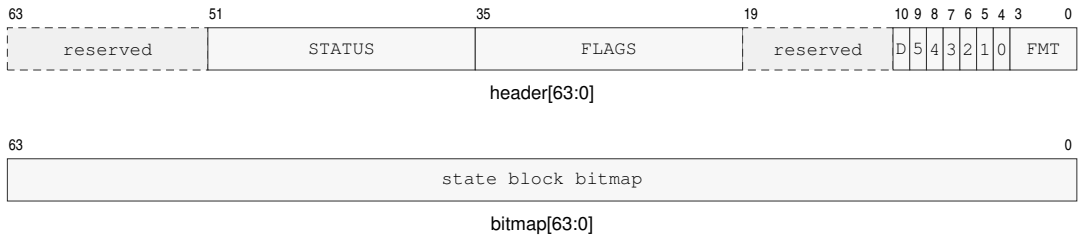


Figure 16-1. SAVE/RESTORE Base Header

The labels 5 through 0 are the six GS_VALID bits. SAVE writes GS_VALID=0x3f and FMT=0. The label D denotes DFA, which records the hidden current double-fault-active bit.

Table 16-1. SAVE/RESTORE Fixed Base Block

offset from the 4 KiB-aligned save-area base					offsets increase downward			
0x000–0x00f	BASE HEADER 0x000 · 8 bytes				STATE BLOCK BITMAP 0x008 · 8 bytes			
0x010–0x04f	R0	R1	R2	R3	R4	R5	R6	R7
0x050–0x08f	R8	R9	R10	R11	R12	R13	R14	R15
0x090–0x0bf	GS0	GS1	GS2	GS3	GS4	GS5	RESTORE uses slots selected by GS_VALID	
0x0c0+	EXTENSION STATE COMPONENTS CPUID-defined slots · each component starts on a 64-byte boundary							
Each R _n and GS _n cell is one 8-byte slot.					R _n =0x010+8n; GS _n =0x090+8n.			

16.2 Fixed Architectural State

The fixed block contains the base header, state-block bitmap, R0 through R15, and GS0 through GS5 images.

The header carries FLAGS, STATUS, the hidden current-DFA bit, format, and GS-validity state. GS_VALID selects which GS images are meaningful.

16.3 Extension Components and Clean State

An extension component is architecturally clean when its state is known to equal its component-defined initial state. Reset and RESTORE of a clear component bit make that component clean.

A committed write that can make a component differ from its initial state makes it modified; RESTORE of a non-initial component image also makes it modified. An implementation may mark a component clean whenever it establishes that the complete component again equals its initial state.

When floating-point state is enabled, its component contains F0 through F15, FFLAGS, and FSTATUS in the layout reported in the CPUID chapter. The floating-point component's initial state is positive zero in F0 through F15 and zero in FFLAGS and FSTATUS.

When VECTOR is present, component ID 2 uses bitmap bit 1 and contains the complete V0 through V31 and packed P0 through P15 images. Let B be VLEN in bytes. V_n begins at component offset nB , and P_n begins at $32B + nB/8$. The reported component size is $\text{align_up}(34B, 64)$. Its initial state is all zero. A committed vector or predicate register write marks the component modified. SAVE writes component padding as zero; RESTORE ignores that padding.

16.4 SAVE Responsibilities

SAVE writes FMT=0, sets GS_VALID=0x3f, records the complete STATUS image and current DFA, and records GS0 through GS5. SAVE sets bitmap bits only for components described by its CPUID SAVE_AREA_LAYOUT leaf.

SAVE may omit an architecturally clean extension component by clearing its bitmap bit. SAVE may clear a bitmap bit only for a clean component. SAVE does not change clean or modified state.

16.5 RESTORE Responsibilities

RESTORE replaces each GS n whose corresponding GS_VALID bit is set and preserves each GS n whose bit is clear. It validates every selected segment image before changing architectural state.

RESTORE uses the state-block bitmap as authoritative when reconstructing extension state. It accepts only the defined format and only bitmap bits that name CPUID-described components for that format. RESTORE raises INVALID_CONTROL_STATE before changing architectural state when the format is unrecognized or a set bitmap bit does not name a CPUID-described component for that format. RESTORE of a clear, described component bit installs that component's initial state.

User-mode RESTORE ignores saved supervisor-only STATUS and DFA state. Supervisor RESTORE is an architectural context transition: it restores EDEPTH, UO, and current DFA together with STATUS after validating the complete event-state invariants. It requires saved PM=1; UO requires a valid live U bank, DFA requires an active event, and inactive state requires EDEPTH=0, UO=0, and DFA=0. Ordinary WRSTATUS remains unable to change these fields.

The normative instruction entries are SAVE and RESTORE.

PART IV

BEDROCK ARCHITECTURE

Instruction Set Reference

READING AN INSTRUCTION DESCRIPTION

Entry Organization

Each instruction entry presents a brief Operation summary and its Assembler Syntax before applicability, repeat, event, and status information relevant to that instruction. Detailed Semantics contains the complete explanation and places illustrative diagrams directly after the explanation, before the concrete encodings. The same brief summaries appear in the instruction-set summary tables. Each mnemonic begins on a new page so its encoding diagrams, field explanations, and side-effect text stay together as one reference unit.

Mnemonics and Suffixes

The B, W, L, and Q suffixes select 8-, 16-, 32-, and 64-bit integer sizes; S and D select 32- and 64-bit floating-point scalar sizes where defined.

When an instruction encoding includes a size selector *z*, its definition maps *z* to the suffix set shown for that encoding. A mnemonic without a size suffix has a single architecturally defined size or no data-size operand. Conditional mnemonic variants use the condition names listed in the Flags and Condition Codes chapter.

CMPJcc, DJcc, IJcc, Jcc, MOVcc, SETcc, TESTJcc, and FMOVcc place the condition suffix in the mnemonic. An encoding with only one architectural size omits the size suffix.

Operand and Status Notation

R_n names an R_n register selected from the general-purpose registers R0 through R15; SP and PC are special registers outside that encoding namespace. An <ea> operand names a compact effective-address field and any appended EA payload.

FLAGS, STATUS, FFLAGS, and FSTATUS are architectural state registers. Instruction descriptions state whether an encoding updates, preserves, reads, or ignores each status register. A FLAGS update always names and replaces the complete ZNCV image; FFLAGS retains its accrued per-cause behavior.

In a transposed flag-effects table, - marks an unchanged flag and 0 or 1 marks a constant result. * marks the single nontrivial effect defined by that table's local legend. When a table contains multiple nontrivial effects, letters a through z distinguish their local legend entries.

In instruction pseudocode, Operand Read applies the addressing mode and operation size. Operand Write stores the selected-size result; an R_n write defines all 64 bits and extends a sub-Q result according to the common integer result-extension rule, while a memory write changes only the selected bytes. EA Address computes an address without reading memory. A control-target memory operand reads its declared-width target, while a control-target register or immediate operand supplies the target value directly. A selector may name an R_n register, a segment register, or an encoded immediate. FLAGS ZNCV refers to the integer Z, N, C, and V bits, while FFLAGS names the accrued floating-point exception flags.

Encodings

In an encoding diagram, fixed 0 and 1 fields identify framing, opcode-selection, or reserved values, and the notes below decode lettered fields. An <ea> field is the compact effective-address field and may select one or more independently determined EA payloads in operand order.

In each instruction entry, Encodings presents the encoding variants, operand and effective-address grammar, and size selectors.

For an extended encoding, the four-bit L field selects a 3+L-byte encoded instruction length. The required instruction length is the opcode-space length plus the selected operand payload lengths. Every larger encoded instruction length through 18 bytes is valid, and the trailing bytes are uninterpreted padding. The `LENn`, spelling records an explicit encoded instruction length.

17 GENERAL INSTRUCTIONS

17.1 Summary

Table 17-1. General Instructions Summary (Informative)

Mnemonic	Brief description
ABS	Performs the absolute value operation.
ADC	Performs the add with carry operation.
ADD	Adds the source operand to the destination operand.
AFENCE	Order prior memory operations before later memory operations.
AND	Computes the bitwise AND of the source and destination operands.
BCHG	Performs the bit change operation.
BCLR	Performs the bit clear operation.
BKPT	Raise the breakpoint exception.
BNDSII	Checks signed value membership in the inclusive-inclusive interval [lo, hi].
BNDSIX	Checks signed value membership in the inclusive-exclusive interval [lo, hi).
BNDSXI	Checks signed value membership in the exclusive-inclusive interval (lo, hi].
BNDSXX	Checks signed value membership in the exclusive-exclusive interval (lo, hi).
BNDUII	Checks unsigned value membership in the inclusive-inclusive interval [lo, hi].
BNDUIX	Checks unsigned value membership in the inclusive-exclusive interval [lo, hi).
BNDUXI	Checks unsigned value membership in the exclusive-inclusive interval (lo, hi].
BNDUXX	Checks unsigned value membership in the exclusive-exclusive interval (lo, hi).
BSET	Performs the bit set operation.
BTEST	Performs the bit test operation.
CALL	Performs the call operation.
CALLcc	Performs a call when the condition is true.
CLMUL	Performs the carryless multiply operation.
CLMULH	Performs the carryless multiply high operation.
CLR	Performs the clear operation.
CLS	Performs the count leading set bits operation.
CLZ	Performs the count leading zeros operation.
CMP	Computes a comparison result and updates condition codes without storing the temporary value.
CMPJcc	Compares two Rn registers and branches on the selected condition without writing FLAGS.
CMPXCHG	Performs the compare and exchange operation.
CPUID	Queries architectural discovery information through a selected Rn register.
CTS	Performs the count trailing set bits operation.
CTZ	Performs the count trailing zeros operation.
DEC	Performs the decrement operation.
DECF	Decrements the destination operand and updates integer FLAGS.
DIVMODS	Performs the signed divide with remainder operation.
DIVMODU	Performs the unsigned divide with remainder operation.
DIVS	Performs the signed divide operation.
DIVU	Performs the unsigned divide operation.
DJcc	Performs the decrement and jump conditional operation.
ERET	Return through a staged user context or from an architectural event.

Table 17-1 (continued)

Mnemonic	Brief description
EXTRACT	Extracts a selected-width value from a concatenated pair of Rn registers.
EXTSL	Performs the sign extend to long operation.
EXTSQ	Performs the sign extend to quad operation.
EXTSW	Performs the sign extend to word operation.
EXTZL	Performs the zero extend to long operation.
EXTZQ	Performs the zero extend to quad operation.
EXTZW	Performs the zero extend to word operation.
FETCHADD	Performs the atomic fetch add operation.
FETCHAND	Performs the atomic fetch and operation.
FETCHOR	Performs the atomic fetch or operation.
FETCHSUB	Performs the atomic fetch subtract operation.
FETCHXOR	Performs the atomic fetch xor operation.
FLSHDCACHE	Write back and invalidate one cache-maintenance block.
HALT	Halt the executing logical processor until an asynchronous architectural event is admitted.
IJcc	Performs the increment and jump conditional operation.
ILLEGAL	Raise illegal-instruction exception.
INC	Performs the increment operation.
INCF	Increments the destination operand and updates integer FLAGS.
INVASID	Invalidate TLB entries for ASID.
INVDACACHE	Invalidate one data-cache maintenance block.
INVICACHE	Invalidate one local instruction-cache maintenance block.
INVPAGE	Invalidate TLB mapping containing a linear address.
INVTLB	Invalidate all TLB entries.
Jcc	Transfers control when the encoded condition is true.
JMP	Transfers control to the target address.
LCALL	Pushes a long return frame and atomically transfers control to a new CS:PC pair.
LEA	Performs the load effective address operation.
LJMP	Atomically transfers control to a new CS:PC pair.
LRET	Pops a long return frame and atomically restores CS:PC.
MAXS	Performs the signed maximum operation.
MAXU	Performs the unsigned maximum operation.
MINS	Performs the signed minimum operation.
MINU	Performs the unsigned minimum operation.
MODS	Performs the signed modulo operation.
MODU	Performs the unsigned modulo operation.
MOV	Copies the source operand to the destination operand.
MOVcc	Performs the move conditional operation.
MOVCU	Copies from a current-domain source to user-domain memory.
MOVNT	Stores an Rn register value to memory with a non-temporal access hint.
MOVUC	Copies from user-domain memory to a current-domain destination.
MOVUU	Copies from user-domain memory to user-domain memory.
MUL	Multiplies the destination by the source and keeps the low half.
MULHS	Performs the signed multiply high operation.
MULHSU	Performs the signed by unsigned multiply high operation.
MULHU	Performs the unsigned multiply high operation.

Table 17-1 (continued)

Mnemonic	Brief description
NEG	Performs the negate operation.
NOP	No architectural state change.
NOT	Performs the bitwise not operation.
OR	Computes the bitwise OR of the source and destination operands.
PARITY	Computes operand parity and writes the predicate to an Rn register.
POP	Pops one 64-bit stack value into an Rn or SREG destination.
POPCNT	Performs the population count operation.
POPP	Pops one canonical Rn register pair selected by an inline pair index.
PREFETCH	Hints that EA will be read soon; faults only under the configured prefetch-fault policy.
PREFETCHNT	Hint that EA will be read with non-temporal locality.
PTQUERY	Non-destructively reads the selected raw page-table descriptor.
PUSH	Pushes one Rn register value, SREG image, or the current CS image onto the stack.
PUSHP	Pushes one canonical Rn register pair selected by an inline pair index.
RDCR	Read a privileged control register.
RDFLAGS	Read the integer condition-code flags into an Rn register.
RDPMC	Read the 64-bit performance counter selected by a constant counter ID.
RDSEG	Read an SREG or the current CS image into an Rn register.
RDSTATUS	Read the processor STATUS register into an Rn register.
REPcc	Repeats the next instruction while the counter and condition remain active.
RESET	Perform warm reset; preserve BOOTPC and BOOTCFG.
RESTORE	Restore processor state from a CPUID-defined save area.
RET	Performs the return operation.
REVBYTE	Performs the reverse bytes operation.
RFENCE	Order prior reads before later reads.
ROL	Performs the rotate left operation.
ROR	Performs the rotate right operation.
SAR	Performs the shift arithmetic right operation.
SAVE	Saves processor state to a CPUID-defined save area.
SBB	Performs the subtract with borrow operation.
SEGLEA	Performs the segment load effective address operation.
SET	Writes integer one to the destination operand.
SETcc	Performs the set conditional operation.
SETF	Replaces integer FLAGS from an immediate bitmap.
SHL	Performs the shift left operation.
SHR	Performs the shift right operation.
SUB	Subtracts the source operand from the destination operand.
SWPT	Performs the switch page table operation.
SWPTA	Switches the page table and enables address-space context matching.
SYNCCACHE	Synchronize one cache-maintenance block for local instruction fetch.
SYSCALL	Raise the SYSTEM_CALL architectural event.
TEST	Computes a bitwise conjunction and updates condition codes without storing the temporary value.
TESTJcc	Tests two Rn registers and branches on the selected condition without writing FLAGS.
TRACE	Records a marker and instruction boundary in the implementation trace stream.
VTOP	Non-destructively queries the current translation of a linear address.

Table 17-1 (continued)

Mnemonic	Brief description
WAIT	Permits a finite implementation-selected delay, including zero.
WFENCE	Order prior writes before later writes.
WRBKDCACHE	Write back one data-cache maintenance block.
WRCR	Write a privileged control register from an Rn register.
WRFLAGS	Write the integer condition-code flags from an Rn register.
WRSEG	Write an SREG segment image from an Rn register.
WRSTATUS	Write the processor STATUS register from an Rn register.
XCHG	Performs the exchange operation.
XOR	Computes the bitwise XOR of the source and destination operands.
YIELD	Hints that the current logical processor may be deprioritized.

ABS

Absolute Value

ABS

Operation:	Performs the absolute value operation.
Assembler Syntax:	ABS.LQ(z) Rn(r) ABS.BWLQ(z) <ea>(e)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

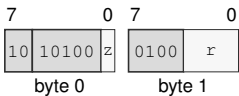
Detailed Semantics

Replaces the selected-width destination value with its two's-complement absolute value. The most-negative value wraps to itself and does not trap. A sub-Q Rn result is sign-extended to 64 bits.

Encodings:

ABS.LQ(z) Rn(r)

Instruction Format:



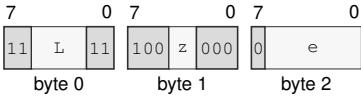
Format — Instruction format for ABS.LQ(z) Rn(r)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field r—Selects the operand.

ABS.BWLQ(z) <ea>(e)

Instruction Format:



Format — Instruction format for ABS.BWLQ(z) <ea>(e)

Instruction Fields:

- Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z—Selects B/W/L/Q.
- Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADC**Add With Carry****ADC**

Operation: Performs the add with carry operation.

Assembler Syntax: `ADC.BWLQ(z) Rn(s), Rn(d)`
`ADC.BWLQ(z) Rn(s), <ea>(e)`
`ADC.BWLQ(z) <ea>(e), Rn(s)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Adds the selected-width source value and the incoming C flag to the destination, writes the truncated sum, and updates Z, N, C, and V from the complete addition. REPcc observes the written destination result rather than architectural FLAGS.

Let n be the selected width, d and s the unsigned operand bit patterns, and c the incoming C flag. The written result is

$$r = (d + s + c) \bmod 2^n.$$

Z is set when $r = 0$, and N receives the most-significant bit of r . C reports an unsigned carry out of either addition. V reports signed overflow for the complete $d + s + c$ operation.

FLAGS Effects

Z	N	C	V
a	b	c	d

a: `result == 0`.

b: `most_significant_bit(result)`.

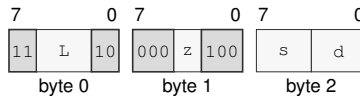
c: unsigned carry out.

d: signed overflow.

Encodings:

ADC.BWLQ(z) Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for ADC.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

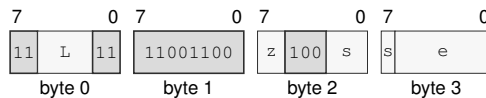
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

ADC.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for ADC.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

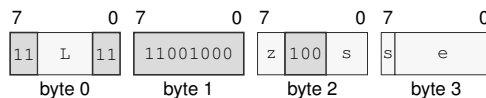
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADC.BWLQ(z) <ea>(e), Rn(s)

Instruction Format:



Format — Instruction format for ADC.BWLQ(z) <ea>(e), Rn(s)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the destination operand.

Effective Address field e—Specifies the source operand.

ADD**Add****ADD**

Operation: Adds the source operand to the destination operand.

Assembler Syntax:

```

ADD.Q 8, SP
ADD.LQ(z) Rn(s), Rn(d)
ADD.Q imm8(i), SP
ADD.Q <imm16s>, SP
ADD.Q <imm32s>, SP
ADD.BWLQ(z) Rn(s), <ea>(e)
ADD.BWLQ(z) <ea>(e), Rn(d)
ADD.BWLQ(z) <imm8s>, <ea>(e)
ADD.WLQ(z) <imm16s>, <ea>(e)
ADD.LQ(z) <imm32s>, <ea>(e)
ADD.Q <imm64>, <ea>(e)

```

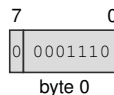
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

ADD reads the source and destination operands at the selected integer width, adds their values modulo that width, and writes the low result bits to the destination. The common integer result-extension rule defines the upper bits of an Rn destination. The operation leaves FLAGS unchanged.

Encodings:

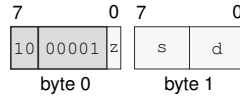
ADD.Q 8, SP

Instruction Format:

Format — Instruction format for ADD.Q 8, SP

ADD.LQ(z) Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for ADD.LQ(z) Rn(s), Rn(d)

Instruction Fields:

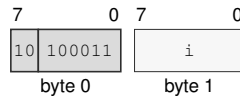
Size field z—Selects L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

ADD.Q imm8(i), SP

Instruction Format:



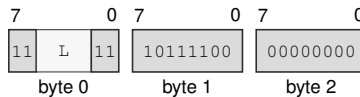
Format — Instruction format for ADD.Q imm8(i), SP

Instruction Fields:

Immediate field i—Encodes the immediate value.

ADD.Q <imm16s>, SP

Instruction Format:



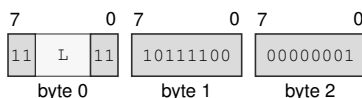
Format — Instruction format for ADD.Q <imm16s>, SP

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

ADD.Q <imm32s>, SP

Instruction Format:



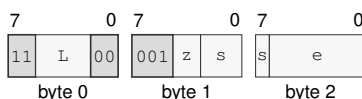
Format — Instruction format for ADD.Q <imm32s>, SP

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

ADD.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for ADD.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

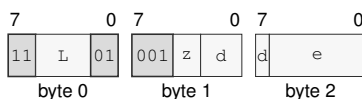
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADD.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for ADD.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

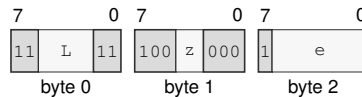
Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

ADD.BWLQ(z) <imm8s>, <ea>(e)

Instruction Format:



Format — Instruction format for ADD.BWLQ(z) <imm8s>, <ea>(e)

Instruction Fields:

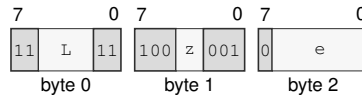
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADD.WLQ(z) <imm16s>, <ea>(e)

Instruction Format:



Format — Instruction format for ADD.WLQ(z) <imm16s>, <ea>(e)

Instruction Fields:

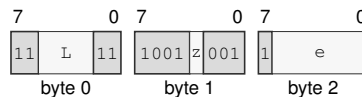
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADD.LQ(z) <imm32s>, <ea>(e)

Instruction Format:



Format — Instruction format for ADD.LQ(z) <imm32s>, <ea>(e)

Instruction Fields:

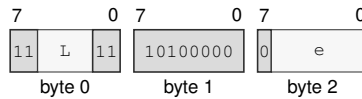
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ADD.Q <imm64>, <ea>(e)

Instruction Format:



Format — Instruction format for ADD.Q <imm64>, <ea>(e)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

AFENCE

Address Fence

AFENCE

Operation: Order prior memory operations before later memory operations.

Assembler Syntax: AFENCE

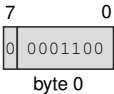
Detailed Semantics

Acts as a full cumulative fence. Every earlier memory read and write is ordered before every later memory read and write on the same logical processor, and memory effects observed before the fence are propagated before effects ordered after it. Every AFENCE participates in the single global sequentially consistent order. It performs no memory access and changes no register or flag.

Encodings:

AFENCE

Instruction Format:



Format — Instruction format for AFENCE

AND**Bitwise And****AND**

Operation: Computes the bitwise AND of the source and destination operands.

Assembler Syntax: `AND.LQ(z) Rn(s), Rn(d)`
`AND.BWLQ(z) Rn(s), <ea>(e)`
`AND.BWLQ(z) <ea>(e), Rn(d)`
`AND.BWLQ(z) <imm8s>, <ea>(e)`
`AND.WLQ(z) <imm16s>, <ea>(e)`
`AND.LQ(z) <imm32s>, <ea>(e)`
`AND.Q <imm64>, <ea>(e)`

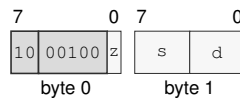
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Writes the selected-width bitwise conjunction of the source and destination values to the destination.

Encodings:

`AND.LQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for AND.LQ(z) Rn(s), Rn(d)

Instruction Fields:

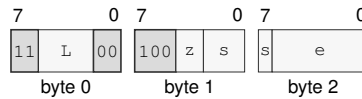
Size field *z*—Selects L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

AND.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for AND.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

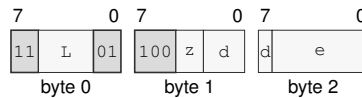
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

AND.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for AND.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

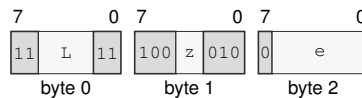
Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

AND.BWLQ(z) <imm8s>, <ea>(e)

Instruction Format:



Format — Instruction format for AND.BWLQ(z) <imm8s>, <ea>(e)

Instruction Fields:

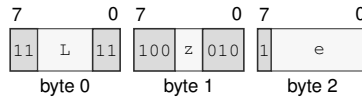
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

AND.WLQ(z) <imm16s>, <ea>(e)

Instruction Format:



Format — Instruction format for AND.WLQ(z) <imm16s>, <ea>(e)

Instruction Fields:

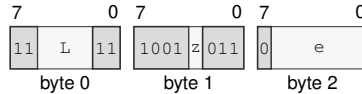
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

AND.LQ(z) <imm32s>, <ea>(e)

Instruction Format:



Format — Instruction format for AND.LQ(z) <imm32s>, <ea>(e)

Instruction Fields:

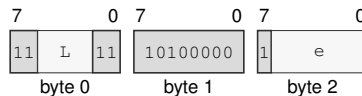
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

AND.Q <imm64>, <ea>(e)

Instruction Format:



Format — Instruction format for AND.Q <imm64>, <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BCHG

Bit Change

BCHG

- Operation: Performs the bit change operation.
- Assembler Syntax: BCHG Rn(b) , Rn(e)
BCHG Rn(b) , <ea>(e)
BCHG imm6(i) , <ea>(e)
BCHG imm6(i) , Rn(e)
- Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Complements the destination bit selected by the low six bits of the bit index, writes Z from whether that bit was previously clear, and clears N, C, and V. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store. REPcc observes the old tested bit as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

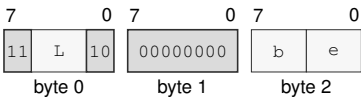
Z	N	C	V
*	0	0	0

- *: old_bit == 0.
- 0: writes zero.

Encodings:

BCHG Rn(b) , Rn(e)

Instruction Format:

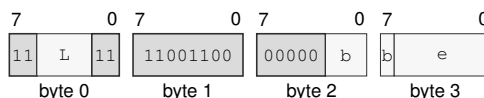


Format — Instruction format for BCHG Rn(b), Rn(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** b—Selects the source operand.
- Register field** e—Selects the destination operand.

BCHG Rn(b), <ea>(e)

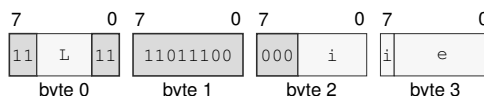
Instruction Format:**Format — Instruction format for BCHG Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BCHG imm6(i), <ea>(e)

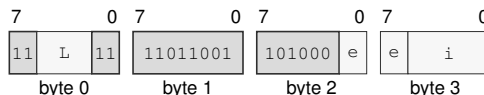
Instruction Format:**Format — Instruction format for BCHG imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BCHG imm6(i), Rn(e)

Instruction Format:**Format — Instruction format for BCHG imm6(i), Rn(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field e—Selects the destination operand.

Immediate field i—Encodes the immediate value.

BCLR

Bit Clear

BCLR

Operation:

Performs the bit clear operation.

Assembler Syntax:

BCLR Rn(b) , Rn(e)
BCLR Rn(b) , <ea>(e)
BCLR imm6(i) , <ea>(e)
BCLR imm6(i) , Rn(e)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Clears the destination bit selected by the low six bits of the bit index, writes Z from whether that bit was previously clear, and clears N, C, and V. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store. REPcc observes the old tested bit as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

Z	N	C	V
*	0	0	0

*: old_bit == 0.
0: writes zero.

Encodings:

BCLR Rn(b) , Rn(e)

Instruction Format:

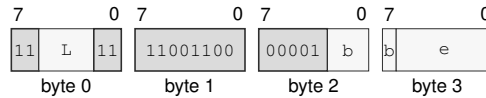


Format — Instruction format for BCLR Rn(b), Rn(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** b—Selects the source operand.
- Register field** e—Selects the destination operand.

BCLR Rn(b), <ea>(e)

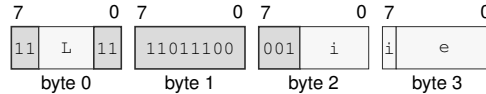
Instruction Format:**Format — Instruction format for BCLR Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BCLR imm6(i), <ea>(e)

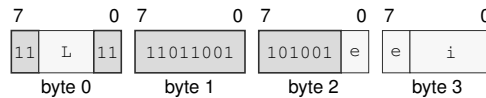
Instruction Format:**Format — Instruction format for BCLR imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BCLR imm6(i), Rn(e)

Instruction Format:**Format — Instruction format for BCLR imm6(i), Rn(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field e—Selects the destination operand.

Immediate field i—Encodes the immediate value.

BKPT

Breakpoint

BKPT

- Operation: Raise the breakpoint exception.
- Assembler Syntax: BKPT
- Exceptions: BREAKPOINT: when the instruction executes

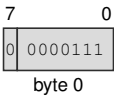
Detailed Semantics

Raises BREAKPOINT and saves the next instruction address without executing that instruction before event entry commits.

Encodings:

BKPT

Instruction Format:



Format — Instruction format for BKPT

BND\$II**Signed Bounds Check Inclusive-Inclusive****BND\$II**

Operation: Checks signed value membership in the inclusive-inclusive interval [lo, hi].

Assembler Syntax: `BND$II.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BND$II.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BND\$II treats both bounds as inclusive. It sets V when signed(value) is outside [signed(lo), signed(hi)] and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

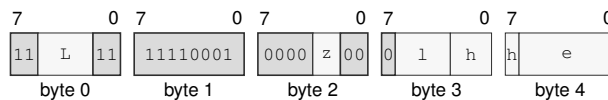
Z	N	C	V
0	0	0	*

0: writes zero.

*: signed(value) is outside the selected interval.

Encodings:

`BND$II.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BND\$II.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

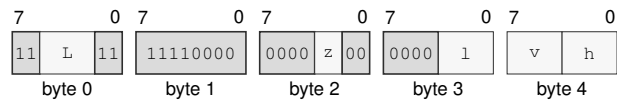
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDSII.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDSII.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field l**—Selects operand 1.
- Register field v**—Selects operand 2.
- Register field h**—Selects operand 3.

BNDSEX**Signed Bounds Check Inclusive-Exclusive****BNDSEX**

Operation: Checks signed value membership in the inclusive-exclusive interval [lo, hi).

Assembler Syntax: `BNDSEX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BNDSEX.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDSEX treats the low bound as inclusive and the high bound as exclusive. It sets V when `signed(value)` is outside `[signed(lo), signed(hi))` and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

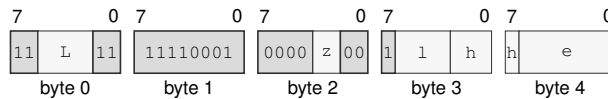
Z	N	C	V
0	0	0	*

0: writes zero.

*: `signed(value)` is outside the selected interval.

Encodings:

`BNDSEX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BNDSEX.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

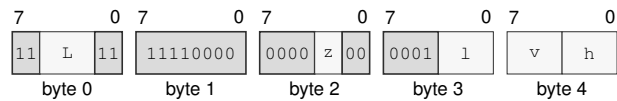
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDSIX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDSIX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects operand 1.

Register field v—Selects operand 2.

Register field h—Selects operand 3.

BNDXSI**Signed Bounds Check Exclusive-Inclusive****BNDXSI**

Operation: Checks signed value membership in the exclusive-inclusive interval (lo, hi].

Assembler Syntax: `BNDXSI.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BNDXSI.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDXSI treats the low bound as exclusive and the high bound as inclusive. It sets V when `signed(value)` is outside `(signed(lo), signed(hi)]` and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

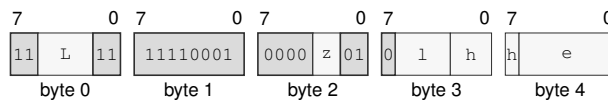
Z	N	C	V
0	0	0	*

0: writes zero.

*: `signed(value)` is outside the selected interval.

Encodings:

`BNDXSI.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BNDXSI.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

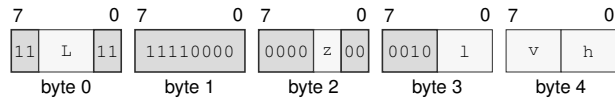
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

`BNDSXI.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Instruction Format:



Format — Instruction format for BNDSXI.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *l*—Selects operand 1.

Register field *v*—Selects operand 2.

Register field *h*—Selects operand 3.

BNDXXX**Signed Bounds Check Exclusive-Exclusive****BNDXXX**

Operation: Checks signed value membership in the exclusive-exclusive interval (lo, hi).

Assembler Syntax: `BNDXXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BNDXXX.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDXXX treats both bounds as exclusive. It sets V when signed(value) is outside (signed(lo), signed(hi)) and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

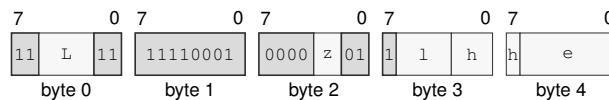
Z	N	C	V
0	0	0	*

0: writes zero.

*: signed(value) is outside the selected interval.

Encodings:

`BNDXXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for `BNDXXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

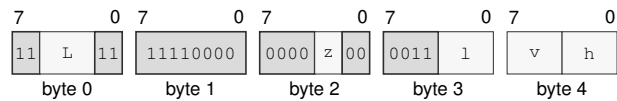
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDSXX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDSXX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field l**—Selects operand 1.
- Register field v**—Selects operand 2.
- Register field h**—Selects operand 3.

BNDUII**Unsigned Bounds Check Inclusive-Inclusive****BNDUII**

Operation:	Checks unsigned value membership in the inclusive-inclusive interval [lo, hi].
Assembler Syntax:	<code>BNDUII.BWLQ(z) Rn(l), <ea>(e), Rn(h)</code> <code>BNDUII.BWLQ(z) Rn(l), Rn(v), Rn(h)</code>
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDUII treats both bounds as inclusive. It sets V when `unsigned(value)` is outside `[unsigned(lo), unsigned(hi)]` and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

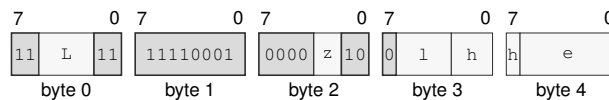
Z	N	C	V
0	0	0	*

0: writes zero.

*: `unsigned(value)` is outside the selected interval.

Encodings:

`BNDUII.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for `BNDUII.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

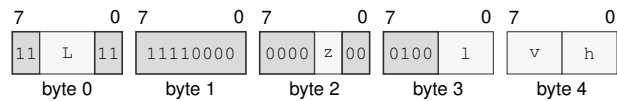
Register field *l*—Selects operand 1.

Register field *h*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

BNDUII.BWLQ(*z*) Rn(*l*), Rn(*v*), Rn(*h*)

Instruction Format:



Format — Instruction format for BNDUII.BWLQ(*z*) Rn(*l*), Rn(*v*), Rn(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *l*—Selects operand 1.

Register field *v*—Selects operand 2.

Register field *h*—Selects operand 3.

BNDUIX**Unsigned Bounds Check Inclusive-Exclusive****BNDUIX**

Operation: Checks unsigned value membership in the inclusive-exclusive interval [lo, hi).

Assembler Syntax: `BNDUIX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BNDUIX.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDUIX treats the low bound as inclusive and the high bound as exclusive. It sets V when `unsigned(value)` is outside `[unsigned(lo), unsigned(hi))` and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

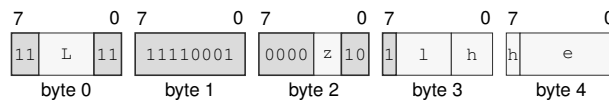
Z	N	C	V
0	0	0	*

0: writes zero.

*: `unsigned(value)` is outside the selected interval.

Encodings:

`BNDUIX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BNDUIX.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

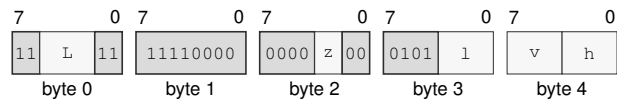
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDUIX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDUIX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects operand 1.

Register field v—Selects operand 2.

Register field h—Selects operand 3.

BNDUXI**Unsigned Bounds Check Exclusive-Inclusive****BNDUXI**

Operation: Checks unsigned value membership in the exclusive-inclusive interval [lo, hi].

Assembler Syntax: `BNDUXI.BWLQ(z) Rn(l), <ea>(e), Rn(h)`
`BNDUXI.BWLQ(z) Rn(l), Rn(v), Rn(h)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDUXI treats the low bound as exclusive and the high bound as inclusive. It sets V when `unsigned(value)` is outside `[unsigned(lo), unsigned(hi)]` and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

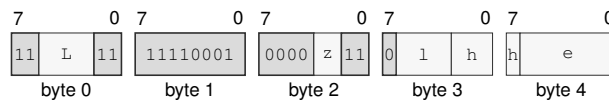
Z	N	C	V
0	0	0	*

0: writes zero.

*: `unsigned(value)` is outside the selected interval.

Encodings:

`BNDUXI.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BNDUXI.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

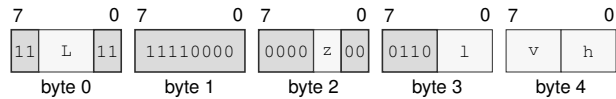
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDUXI.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDUXI.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field l**—Selects operand 1.
- Register field v**—Selects operand 2.
- Register field h**—Selects operand 3.

BNDUXX**Unsigned Bounds Check Exclusive-Exclusive****BNDUXX**

Operation:	Checks unsigned value membership in the exclusive-exclusive interval (lo, hi).
Assembler Syntax:	<code>BNDUXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)</code> <code>BNDUXX.BWLQ(z) Rn(l), Rn(v), Rn(h)</code>
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

BNDUXX treats both bounds as exclusive. It sets V when unsigned(value) is outside (unsigned(lo), unsigned(hi)) and clears Z, N, and C. REPcc observes the violation result as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

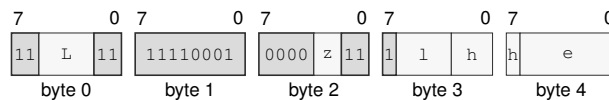
Z	N	C	V
0	0	0	*

0: writes zero.

*: unsigned(value) is outside the selected interval.

Encodings:

`BNDUXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)`

Instruction Format:

Format — Instruction format for BNDUXX.BWLQ(z) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

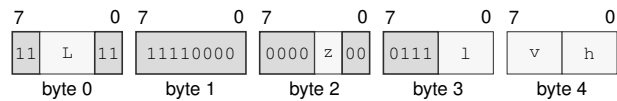
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field e—Specifies the source operand.

BNDUXX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Format:



Format — Instruction format for BNDUXX.BWLQ(z) Rn(l), Rn(v), Rn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects operand 1.

Register field v—Selects operand 2.

Register field h—Selects operand 3.

BSET**Bit Set****BSET**

Operation: Performs the bit set operation.

Assembler Syntax: BSET Rn(b), Rn(e)
 BSET Rn(b), <ea>(e)
 BSET imm6(i), <ea>(e)
 BSET imm6(i), Rn(e)

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Sets the destination bit selected by the low six bits of the bit index, writes Z from whether that bit was previously clear, and clears N, C, and V. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store. REPcc observes the old tested bit as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

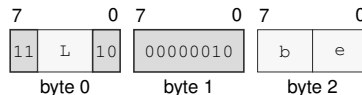
Z	N	C	V
*	0	0	0

*: old_bit == 0.

0: writes zero.

Encodings:

BSET Rn(b), Rn(e)

Instruction Format:

Format — Instruction format for BSET Rn(b), Rn(e)

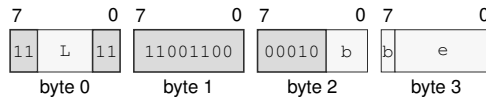
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Register field e—Selects the destination operand.

BSET Rn(b), <ea>(e)

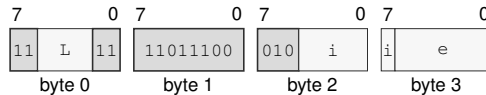
Instruction Format:**Format — Instruction format for BSET Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BSET imm6(i), <ea>(e)

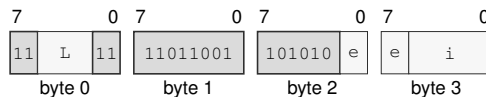
Instruction Format:**Format — Instruction format for BSET imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

BSET imm6(i), Rn(e)

Instruction Format:**Format — Instruction format for BSET imm6(i), Rn(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field e—Selects the destination operand.

Immediate field i—Encodes the immediate value.

BTEST**Bit Test****BTEST**

Operation: Performs the bit test operation.

Assembler Syntax: BTEST Rn(b), Rn(e)
 BTEST Rn(b), <ea>(e)
 BTEST imm6(i), <ea>(e)
 BTEST imm6(i), Rn(e)

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Tests the destination bit selected by the low six bits of the bit index and writes Z from whether that bit was clear while clearing N, C, and V, without changing the destination. REPcc observes the old tested bit as a B-sized 0 or 1 rather than architectural FLAGS.

FLAGS Effects

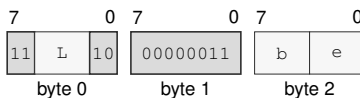
Z	N	C	V
*	0	0	0

*: old_bit == 0.

0: writes zero.

Encodings:

BTEST Rn(b), Rn(e)

Instruction Format:

Format — Instruction format for BTEST Rn(b), Rn(e)

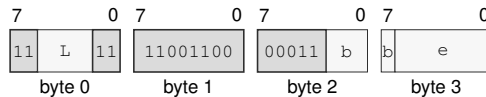
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Register field e—Selects the destination operand.

BTEST Rn(b), <ea>(e)

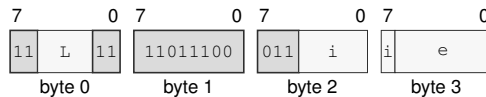
Instruction Format:**Format — Instruction format for BTEST Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the source operand.

BTEST imm6(i), <ea>(e)

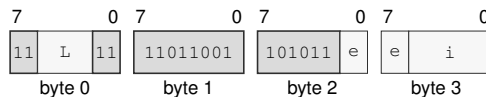
Instruction Format:**Format — Instruction format for BTEST imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the source operand.

BTEST imm6(i), Rn(e)

Instruction Format:**Format — Instruction format for BTEST imm6(i), Rn(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field e—Selects the destination operand.

Immediate field i—Encodes the immediate value.

CALL**Call****CALL**

Operation: Performs the call operation.

Assembler Syntax: `CALL <imm16s>`
`CALL <imm32s>`
`CALL <ea>(e)`
`CALL Rn(r)`

Detailed Semantics

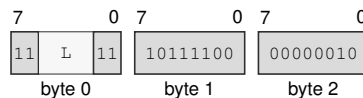
Pushes the following instruction's continuation PC as one 64-bit stack value and transfers control to the target. An EA memory operand reads a 64-bit target; an EA register operand supplies its 64-bit value directly. The stack update and control transfer use the ordinary SS:SP and target-validation rules.

1. Evaluate the target operand in the old architectural context and validate the target under the current CS.
2. Probe the 8-byte return-address store through the old SS:SP context.
3. Commit the return-address store, decremented SP, and new PC as one successful control-transfer operation.

For CALLcc, a false condition is evaluated before target-memory or stack access and suppresses both. Encoding, extension, privilege, and control-state validation still occur. A fault before the final commit changes neither the stack nor PC.

Encodings:

CALL <imm16s>

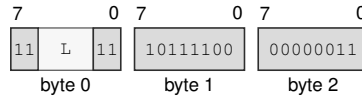
Instruction Format:

Format — Instruction format for CALL <imm16s>

Instruction Fields:

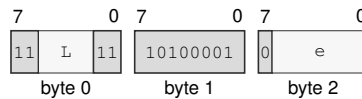
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

CALL <imm32s>

Instruction Format:**Format — Instruction format for CALL <imm32s>****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

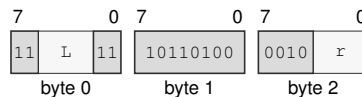
CALL <ea>(e)

Instruction Format:**Format — Instruction format for CALL <ea>(e)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the control target. Immediate addressing is unavailable in this form.

CALL Rn(r)

Instruction Format:**Format — Instruction format for CALL Rn(r)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field *r*—Selects the operand.

CALLcc**Call Conditional****CALLcc**

Operation: Performs a call when the condition is true.

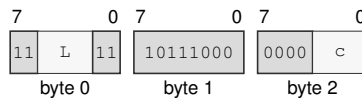
Assembler Syntax: `CALLcc <imm16s>`
`CALLcc <imm32s>`
`CALLcc <ea>(e)`
`CALLcc Rn(r)`

Detailed Semantics

When the encoded condition is true, pushes the following instruction's continuation PC as one 64-bit stack value and transfers control to the target; otherwise execution continues without changing the stack.

Encodings:

`CALLcc <imm16s>`

Instruction Format:

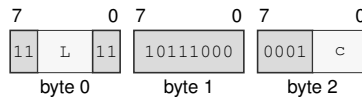
Format — Instruction format for CALLcc <imm16s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

`CALLcc <imm32s>`

Instruction Format:

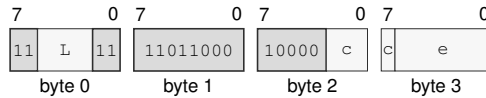
Format — Instruction format for CALLcc <imm32s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

CALLcc <ea>(e)

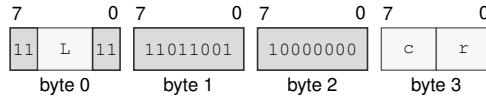
Instruction Format:**Format — Instruction format for CALLcc <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Effective Address field e—Specifies the source operand. Immediate addressing is unavailable in this form.

CALLcc Rn(r)

Instruction Format:**Format — Instruction format for CALLcc Rn(r)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the operand.

CLMUL**Carryless Multiply****CLMUL**

Operation: Performs the carryless multiply operation.

Assembler Syntax: `CLMUL.BWLQ(z) Rn(s), Rn(d)`
`CLMUL.BWLQ(z) <ea>(e), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Forms the carryless polynomial product of the selected-width source and destination values and writes its low half to the destination.

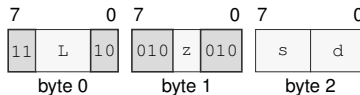
Treat each selected-width operand as a polynomial over $GF(2)$, with operand bit i as the coefficient of x^i . Their double-width product is

$$P = \bigoplus_{i=0}^{n-1} s_i(d \ll i).$$

Addition in this expression is exclusive-or, so no carry propagates between bit positions. CLMUL writes the low n bits of P to the destination.

Encodings:

`CLMUL.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for CLMUL.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

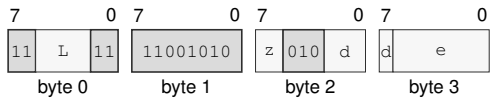
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

CLMUL.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for CLMUL.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field d**—Selects the destination operand.
- Effective Address field e**—Specifies the source operand.

CLMULH**Carryless Multiply High****CLMULH**

Operation: Performs the carryless multiply high operation.

Assembler Syntax: `CLMULH.Q Rn(s), Rn(d)`
`CLMULH.Q <ea>(e), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Forms the carryless polynomial product of the selected-width source and destination values and writes its high half to the destination.

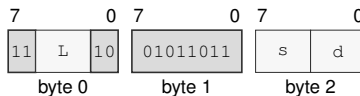
Treat each selected-width operand as a polynomial over $GF(2)$, with operand bit i as the coefficient of x^i . Their double-width product is

$$P = \bigoplus_{i=0}^{n-1} s_i(d \ll i).$$

Addition in this expression is exclusive-or, so no carry propagates between bit positions. CLMULH writes the high n bits of P to the destination.

Encodings:

`CLMULH.Q Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for CLMULH.Q Rn(s), Rn(d)

Instruction Fields:

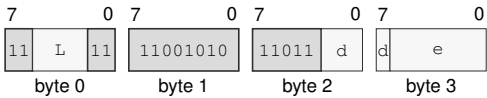
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

CLMULH.Q <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for CLMULH.Q <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.

CLR**Clear****CLR**

Operation: Performs the clear operation.

Assembler Syntax: CLR.Q Rn(r)
CLR.L Rn(r)
CLR.BWLQ(z) <ea>(e)

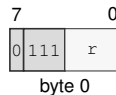
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Writes zero to the selected-width destination.

Encodings:

CLR.Q Rn(r)

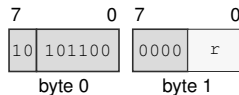
Instruction Format:

Format — Instruction format for CLR.Q Rn(r)

Instruction Fields:

Register field r—Selects the operand.

CLR.L Rn(r)

Instruction Format:

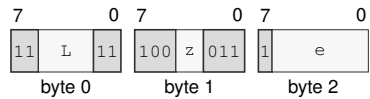
Format — Instruction format for CLR.L Rn(r)

Instruction Fields:

Register field r—Selects the operand.

CLR.BWLQ(z) <ea>(e)

Instruction Format:



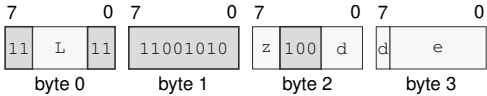
Format — Instruction format for CLR.BWLQ(z) <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Effective Address field e**—Specifies the destination operand. Immediate addressing is unavailable in this form.

CLS.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for CLS.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.

CLZ**Count Leading Zeros****CLZ**

Operation: Performs the count leading zeros operation.

Assembler Syntax: `CLZ.BWLQ(z) Rn(s), Rn(d)`
`CLZ.BWLQ(z) <ea>(e), Rn(d)`

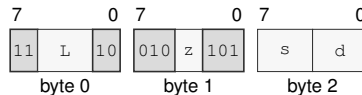
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Counts the leading zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Encodings:

`CLZ.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for CLZ.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

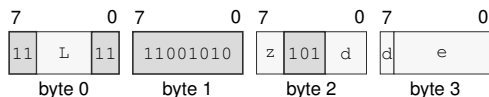
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

CLZ.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for CLZ.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects B/W/L/Q.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

CMP**Compare****CMP**

Operation: Computes a comparison result and updates condition codes without storing the temporary value.

Assembler Syntax: `CMP.LQ(z) Rn(s), Rn(d)`
`CMP.BWLQ(z) Rn(s), <ea>(e)`
`CMP.BWLQ(z) <ea>(e), Rn(d)`
`CMP.BWLQ(z) <ea>(s), <ea>(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Subtracts the source from the destination only to derive Z, N, C, and V; the temporary selected-width difference is not written to either operand. REPcc observes that temporary difference rather than architectural FLAGS.

FLAGS Effects

Z	N	C	V
a	b	c	d

a: result == 0.

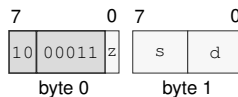
b: most_significant_bit(result).

c: unsigned borrow.

d: signed overflow.

Encodings:

`CMP.LQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for CMP.LQ(z) Rn(s), Rn(d)

Instruction Fields:

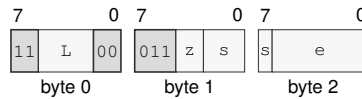
Size field z—Selects L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

`CMP.BWLQ(z) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `CMP.BWLQ(z) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

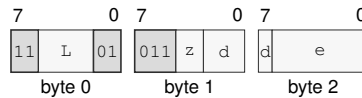
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the source operand.

`CMP.BWLQ(z) <ea>(e), Rn(d)`

Instruction Format:



Format — Instruction format for `CMP.BWLQ(z) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

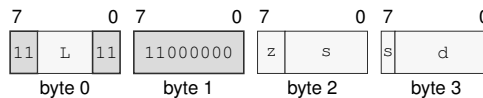
Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

`CMP.BWLQ(z) <ea>(s), <ea>(d)`

Instruction Format:



Format — Instruction format for `CMP.BWLQ(z) <ea>(s), <ea>(d)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Effective Address field *s*—Specifies the source operand.

Effective Address field *d*—Specifies the source operand.

CMPJcc**Compare And Jump Conditional****CMPJcc**

Operation: Compares two Rn registers and branches on the selected condition without writing FLAGS.

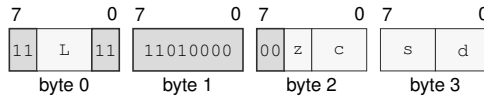
Assembler Syntax: `CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>`
`CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>`

Detailed Semantics

CMPJcc computes the same temporary compare result as CMP for the selected operand size, evaluates the encoded condition from that temporary result, and branches to the relative target when the condition is true. The computed condition is consumed only by the branch decision; architectural FLAGS are not read or written. Conditions T and F are reserved and are not valid encodings.

Encodings:

`CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>`

Instruction Format:

Format — Instruction format for CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

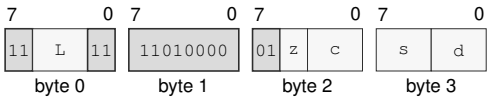
Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field s—Selects operand 1.

Register field d—Selects operand 2.

CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>

Instruction Format:



Format — Instruction format for CMPJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field s—Selects operand 1.

Register field d—Selects operand 2.

CMPXCHG**Compare And Exchange****CMPXCHG**

Operation: Performs the compare and exchange operation.

Assembler Syntax: `CMPXCHG.BWLQ(z)/order(o) Rn(x), Rn(d), <ea>(e)`

Detailed Semantics

Compares memory with the expected Rn register and conditionally stores the desired Rn register. After the operation, the expected register contains the observed old memory value. Z=1 means the store happened; Z=0 means memory was left unchanged.

The memory read, optional memory write, expected-register update, and flag update form one atomic read-modify-write operation. The encoded memory-order selector governs the complete operation on both comparison outcomes. On failure, the operation contributes the observed memory read as its ordering event: acquire orders later memory operations after that read, release orders earlier memory operations before that read, and sequential consistency places that read in the global SC order. On success, the memory write carries the same selected order to observers of that write. A failed comparison contributes no memory write and creates no release sequence.

- If memory equals the original expected value, the desired value is stored and Z is set.
- Otherwise memory is unchanged and Z is cleared.
- In both cases the expected register receives the value originally read from memory, and N, C, and V are cleared.

A memory-access fault exposes none of these updates.

FLAGS Effects

Z	N	C	V
*	0	0	0

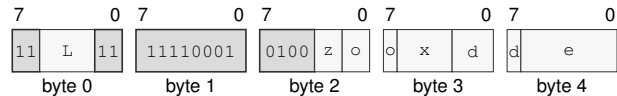
*: store succeeded.

0: writes zero.

Encodings:

CMPXCHG.BWLQ(z)/order(o) Rn(x), Rn(d), <ea>(e)

Instruction Format:



Format — Instruction format for CMPXCHG.BWLQ(z)/order(o) Rn(x), Rn(d), <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Memory-order field o**—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.
- Register field x**—Selects operand 1.
- Register field d**—Selects operand 2.
- Effective Address field e**—Specifies the read/write operand. Immediate addressing is unavailable in this form.

CPUID**CPU Identification****CPUID**

Operation: Queries architectural discovery information through a selected Rn register.

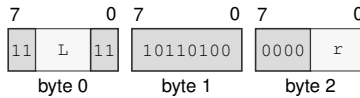
Assembler Syntax: CPUID Rn(r)

Detailed Semantics

Queries architectural discovery information. The selected Rn register contains the CPUID query selector before execution and receives the selected 64-bit result after execution.

Encodings:

CPUID Rn(r)

Instruction Format:

Format — Instruction format for CPUID Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field r—Selects the operand.

CTS

Count Trailing Set Bits

CTS

Operation:

Performs the count trailing set bits operation.

Assembler Syntax:

CTS.BWLQ(z) Rn(s), Rn(d)
CTS.BWLQ(z) <ea>(e), Rn(d)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

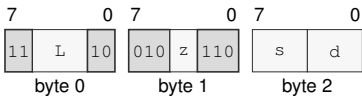
Detailed Semantics

Counts the trailing one bits within the selected operand size and writes the count to the destination register. A zero operand returns 0; an operand whose selected bits are all one returns the operand width in bits.

Encodings:

CTS.BWLQ(z) Rn(s), Rn(d)

Instruction Format:



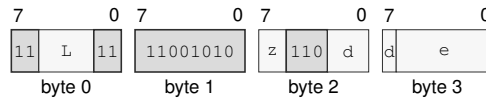
Format — Instruction format for CTS.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

CTS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for CTS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

CTZ

Count Trailing Zeros

CTZ

- Operation: Performs the count trailing zeros operation.
- Assembler Syntax: CTZ.BWLQ(z) Rn(s), Rn(d)
CTZ.BWLQ(z) <ea>(e), Rn(d)
- Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

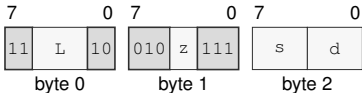
Detailed Semantics

Counts the trailing zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Encodings:

CTZ.BWLQ(z) Rn(s), Rn(d)

Instruction Format:



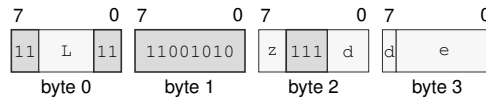
Format — Instruction format for CTZ.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

- Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z—Selects B/W/L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

CTZ.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for CTZ.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

DEC**Decrement****DEC**

Operation: Performs the decrement operation.

Assembler Syntax: DEC.LQ(z) Rn(r)
DEC.BWLQ(z) <ea>(e)

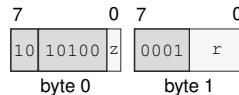
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Subtracts one from the selected-width destination and writes the modular result without changing FLAGS.

Encodings:

DEC.LQ(z) Rn(r)

Instruction Format:

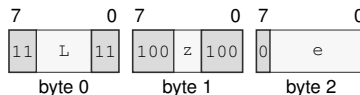
Format — Instruction format for DEC.LQ(z) Rn(r)

Instruction Fields:

Size field z—Selects L/Q.

Register field r—Selects the operand.

DEC.BWLQ(z) <ea>(e)

Instruction Format:

Format — Instruction format for DEC.BWLQ(z) <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

DECF**Decrement And Set Flags****DECF**

Operation: Decrements the destination operand and updates integer FLAGS.

Assembler Syntax: `DECF.BWLQ(z) Rn(r)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Subtracts one from the selected-width destination, writes the modular result, and updates Z, N, C, and V. REPcc observes the written destination result rather than architectural FLAGS.

FLAGS Effects

Z	N	C	V
a	b	c	d

a: result == 0.

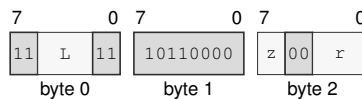
b: most_significant_bit(result).

c: unsigned borrow.

d: signed overflow.

Encodings:

`DECF.BWLQ(z) Rn(r)`

Instruction Format:

Format — Instruction format for DECF.BWLQ(z) Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field r—Selects the operand.

DIVMODS**Signed Divide With Remainder****DIVMODS**

Operation:	Performs the signed divide with remainder operation.
Assembler Syntax:	DIVMODS.BWLQ(z) <ea>(e), Rn(q), Rn(r) DIVMODS.BWLQ(z) Rn(e), Rn(q), Rn(r)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	DIVIDE_ERROR: when the divisor is zero or the most-negative value is divided by minus one ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION: when the declared destination-overlap constraint is violated

Detailed Semantics

The source operand supplies the signed divisor, and the quotient register Rn(q) supplies the signed dividend before the operation. On success, Rn(q) receives their quotient truncated toward zero and the remainder register Rn(r) receives the signed remainder satisfying dividend = quotient * divisor + remainder. Sub-Q quotient and remainder results are each sign-extended to 64 bits.

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

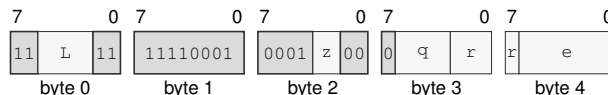
$$q = \text{trunc}(d/s), \quad r = d - qs.$$

Thus a nonzero remainder has the sign of the dividend. The original quotient operand supplies the dividend. On success the quotient operand receives q , and the separate remainder operand receives r .

A zero divisor and the most-negative value divided by -1 raise `DIVIDE_ERROR`; no destination is changed.

Encodings:

DIVMODS.BWLQ(z) <ea>(e), Rn(q), Rn(r)

Instruction Format:**Format — Instruction format for DIVMODS.BWLQ(z) <ea>(e), Rn(q), Rn(r)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

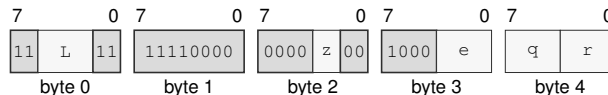
Register field q—Selects operand 2.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

Destination overlap—When the quotient and remainder operands designate the same architectural register, the instruction raises ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION before architectural effects.

DIVMODS.BWLQ(z) Rn(e), Rn(q), Rn(r)

Instruction Format:**Format — Instruction format for DIVMODS.BWLQ(z) Rn(e), Rn(q), Rn(r)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field e—Selects operand 1.

Register field q—Selects operand 2.

Register field r—Selects operand 3.

Destination overlap—When the quotient and remainder operands designate the same architectural register, the instruction raises ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION before architectural effects.

DIVMODU**Unsigned Divide With Remainder****DIVMODU**

Operation:	Performs the unsigned divide with remainder operation.
Assembler Syntax:	DIVMODU.BWLQ(z) <ea>(e), Rn(q), Rn(r) DIVMODU.BWLQ(z) Rn(e), Rn(q), Rn(r)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	DIVIDE_ERROR: when the divisor is zero ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION: when the declared destination-overlap constraint is violated

Detailed Semantics

The source operand supplies the unsigned divisor, and the quotient register Rn(q) supplies the unsigned dividend before the operation. On success, Rn(q) receives their unsigned quotient and the remainder register Rn(r) receives the unsigned remainder satisfying dividend = quotient * divisor + remainder.

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

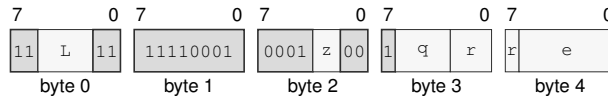
$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The original quotient operand supplies the dividend. On success the quotient operand receives q , and the separate remainder operand receives r .

A zero divisor raises DIVIDE_ERROR; no destination is changed.

Encodings:

`DIVMODU.BWLQ(z) <ea>(e), Rn(q), Rn(r)`

Instruction Format:

Format — Instruction format for `DIVMODU.BWLQ(z) <ea>(e), Rn(q), Rn(r)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

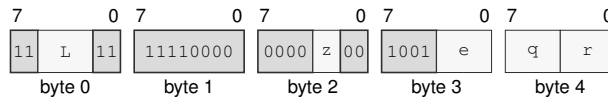
Register field *q*—Selects operand 2.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

Destination overlap—When the quotient and remainder operands designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

`DIVMODU.BWLQ(z) Rn(e), Rn(q), Rn(r)`

Instruction Format:

Format — Instruction format for `DIVMODU.BWLQ(z) Rn(e), Rn(q), Rn(r)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *e*—Selects operand 1.

Register field *q*—Selects operand 2.

Register field *r*—Selects operand 3.

Destination overlap—When the quotient and remainder operands designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

DIVS

Signed Divide

DIVS

Operation:	Performs the signed divide operation.
Assembler Syntax:	<code>DIVS.BWLQ(z) Rn(s), Rn(d)</code> <code>DIVS.BWLQ(z) <ea>(e), Rn(d)</code>
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	<code>DIVIDE_ERROR</code> : when the divisor is zero or the most-negative value is divided by minus one

Detailed Semantics

Interprets both selected-width operands as signed integers, divides the destination by the source with truncation toward zero, and writes the quotient. A sub-Q quotient is sign-extended to 64 bits. Division by zero and the most-negative value divided by minus one raise `DIVIDE_ERROR`.

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

$$q = \text{trunc}(d/s), \quad r = d - qs.$$

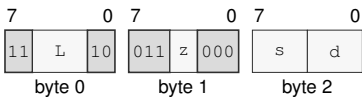
Thus a nonzero remainder has the sign of the dividend. The destination supplies the dividend and receives q .

A zero divisor and the most-negative value divided by -1 raise `DIVIDE_ERROR`; no destination is changed.

Encodings:

`DIVS.BWLQ(z) Rn(s), Rn(d)`

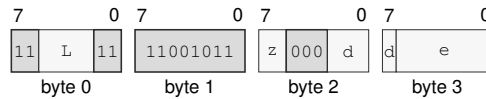
Instruction Format:



Format — Instruction format for `DIVS.BWLQ(z) Rn(s), Rn(d)`

Instruction Fields:

- Length field** L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z —Selects B/W/L/Q.
- Register field** s —Selects the source operand.
- Register field** d —Selects the destination operand.

DIVS.BWLQ(*z*) <ea>(*e*), Rn(*d*)**Instruction Format:****Format — Instruction format for DIVS.BWLQ(*z*) <ea>(*e*), Rn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

DIVU

Unsigned Divide

DIVU

Operation:	Performs the unsigned divide operation.
Assembler Syntax:	<code>DIVU.BWLQ(z) Rn(s), Rn(d)</code> <code>DIVU.BWLQ(z) <ea>(e), Rn(d)</code>
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	DIVIDE_ERROR: when the divisor is zero

Detailed Semantics

Divides the selected-width unsigned destination value by the unsigned source value and writes the quotient. A zero divisor raises DIVIDE_ERROR.

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

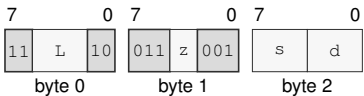
For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The destination supplies the dividend and receives q .

A zero divisor raises DIVIDE_ERROR; no destination is changed.

Encodings:

`DIVU.BWLQ(z) Rn(s), Rn(d)`

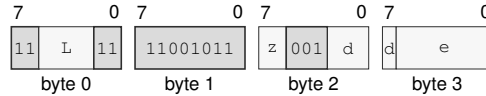
Instruction Format:



Format — Instruction format for DIVU.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

DIVU.BWLQ(*z*) <ea>(*e*), Rn(*d*)**Instruction Format:****Format — Instruction format for DIVU.BWLQ(*z*) <ea>(*e*), Rn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

DJcc**Decrement And Jump Conditional****DJcc**

Operation: Performs the decrement and jump conditional operation.

Assembler Syntax: DJcc Rn(r), <ea>(e)

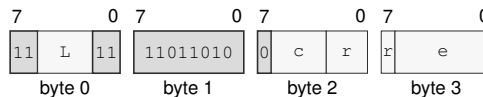
DJcc Rn(r), Rn(e)

Detailed Semantics

Decrements the selected counter and transfers control only when the new count is nonzero and the encoded condition is true. The target EA has Q interpretation width: a memory form reads a 64-bit target and a register or immediate form supplies its value directly.

Encodings:

DJcc Rn(r), <ea>(e)

Instruction Format:

Format — Instruction format for DJcc Rn(r), <ea>(e)

Instruction Fields:

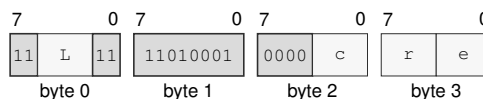
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the source operand.

Effective Address field e—Specifies the control target.

DJcc Rn(r), Rn(e)

Instruction Format:

Format — Instruction format for DJcc Rn(r), Rn(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the source operand.

Register field e—Selects the destination operand.

ERET**Event Return****ERET**

Operation:	Return through a staged user context or from an architectural event.
Assembler Syntax:	ERET
Exceptions:	INVALID_CONTROL_STATE: when the live return-source state, selected U bank, event frame, or return image fails validation

Detailed Semantics

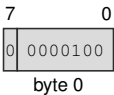
Classify the return source before stack access, then atomically return through a valid staged or outer-event U bank, or restore a valid active supervisor event frame.

1. Classify the return source before any stack access. A staged direct user entry requires supervisor mode, EA clear, EDEPTH zero, UO clear, and UCTL.V set. An outer user-event return requires EA set, EDEPTH one, UO set, and UCTL.V set. Other active supervisor returns use the memory frame.
2. For a U-bank return, validate UCTL, UINFO, the user segments and pointers, and the executable UPC, then atomically restore user state, clear UCTL.V, EDEPTH, UO, EA, and current DFA, and perform no stack access.
3. For a stacked return, read and decode FRAME_CONTROL at SS:SP without changing SP. Accept only the defined BASIC, ERROR, PAGE_FAULT, and AUXILIARY sizes, with all reserved bits zero.
4. Validate that saved STATUS.EDEPTH plus one equals the current depth and that saved and current UO preserve the outer-origin chain.
5. Read the complete selected frame and validate FLAGS, STATUS, CS, DS, SS, SP, and an executable canonical PC without changing architectural state.
6. Ignore the value in the fixed-header padding slot at offset +0x38.
7. Atomically restore the validated architectural state and current DFA from FRAME_CONTROL.SAVED_DFA. Hidden repeat state remains clear; if SAVED_PC identifies a repeat prefix, that prefix is decoded and executed normally after return.
8. After restoration, admit a pending NMI before the next instruction boundary when STATUS.NI is clear and ECR.NMI_P is set.

Encodings:

ERET

Instruction Format:



Format — Instruction format for ERET

EXTRACT

Extract From Register Pair

EXTRACT

Operation: Extracts a selected-width value from a concatenated pair of Rn registers.

Assembler Syntax: `EXTRACT.BWLQ(z) imm7(i), Rn(h), Rn(l)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Concatenates $Rn(\text{high})$ as the upper half and the original $Rn(\text{low}/\text{dest})$ value as the lower half, shifts the concatenated value right by an unsigned 7-bit immediate offset, and writes the low selected-size result back to $Rn(\text{low}/\text{dest})$.

Let n be the selected operand width and form the $2n$ -bit concatenation

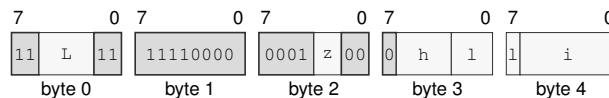
$$T = \text{concat}(Rn(\text{high})[n-1:0], Rn(\text{low})[n-1:0]).$$

The immediate is an unsigned seven-bit shift count. The destination receives the low n bits of T after a logical right shift by that count. A count greater than or equal to $2n$ produces zero. The high register is not changed.

Encodings:

`EXTRACT.BWLQ(z) imm7(i), Rn(h), Rn(l)`

Instruction Format:



Format — Instruction format for EXTRACT.BWLQ(z) imm7(i), Rn(h), Rn(l)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field h—Selects operand 2.

Register field l—Selects operand 3.

Immediate field i—Encodes the immediate value.

EXTSL

Sign Extend To Long

EXTSL

Operation:	Performs the sign extend to long operation.
Assembler Syntax:	EXTSL.B Rn(s), <ea>(e) EXTSL.W Rn(s), <ea>(e) EXTSL.BW(z) <ea>(s), <ea>(d)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

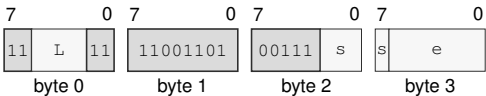
Detailed Semantics

Sign-extends the source operand from the instruction size to long size and stores the four-byte result to an EA destination. The low source-size bytes are copied into the result, and the remaining high result bits are filled with the source sign bit. EXTSL has no Rn destination form; use EXTSQ.B or EXTSQ.W for a sign-extended Rn result.

Encodings:

EXTSL.B Rn(s), <ea>(e)

Instruction Format:

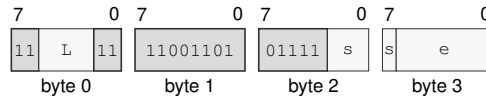


Format — Instruction format for EXTSL.B Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSL.W Rn(s), <ea>(e)

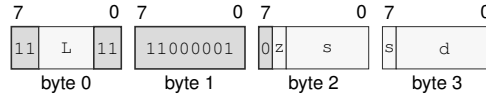
Instruction Format:**Format — Instruction format for EXTSL.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSL.BW(z) <ea>(s), <ea>(d)

Instruction Format:**Format — Instruction format for EXTSL.BW(z) <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ

Sign Extend To Quad

EXTSQ

Operation: Performs the sign extend to quad operation.

Assembler Syntax:

```
EXTSQ.L Rn(s), Rn(d)
EXTSQ.B Rn(s), Rn(d)
EXTSQ.W Rn(s), Rn(d)
EXTSQ.B <ea>(e), Rn(d)
EXTSQ.W <ea>(e), Rn(d)
EXTSQ.L <ea>(e), Rn(d)
EXTSQ.B Rn(s), <ea>(e)
EXTSQ.W Rn(s), <ea>(e)
EXTSQ.L Rn(s), <ea>(e)
EXTSQ.B <ea>(s), <ea>(d)
EXTSQ.W <ea>(s), <ea>(d)
EXTSQ.L <ea>(s), <ea>(d)
```

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

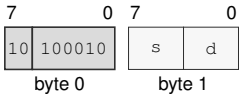
Detailed Semantics

Sign-extends the source operand from the instruction size to quad size. The low source-size bytes are copied into the result, and the remaining high result bits are filled with the source sign bit. An Rn destination receives the complete 64-bit result; an EA destination stores all eight result bytes.

Encodings:

EXTSQ.L Rn(s), Rn(d)

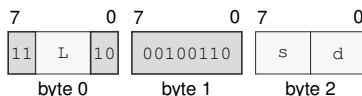
Instruction Format:



Format — Instruction format for EXTSQ.L Rn(s), Rn(d)

Instruction Fields:

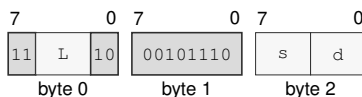
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

EXTSQ.B $Rn(s)$, $Rn(d)$ **Instruction Format:****Format — Instruction format for EXTSQ.B $Rn(s)$, $Rn(d)$** **Instruction Fields:**

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s —Selects the source operand.

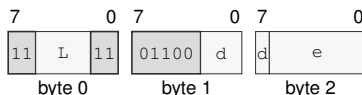
Register field d —Selects the destination operand.

EXTSQ.W $Rn(s)$, $Rn(d)$ **Instruction Format:****Format — Instruction format for EXTSQ.W $Rn(s)$, $Rn(d)$** **Instruction Fields:**

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s —Selects the source operand.

Register field d —Selects the destination operand.

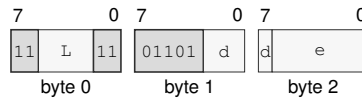
EXTSQ.B $\langle ea \rangle(e)$, $Rn(d)$ **Instruction Format:****Format — Instruction format for EXTSQ.B $\langle ea \rangle(e)$, $Rn(d)$** **Instruction Fields:**

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

EXTSQ.W <ea>(e), Rn(d)

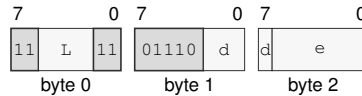
Instruction Format:**Format — Instruction format for EXTSQ.W <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EXTSQ.L <ea>(e), Rn(d)

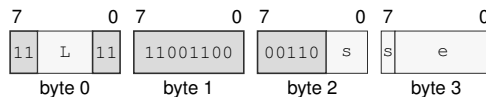
Instruction Format:**Format — Instruction format for EXTSQ.L <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EXTSQ.B Rn(s), <ea>(e)

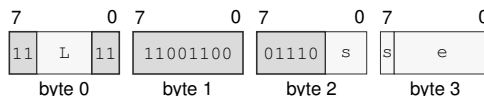
Instruction Format:**Format — Instruction format for EXTSQ.B Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ.W Rn(s), <ea>(e)

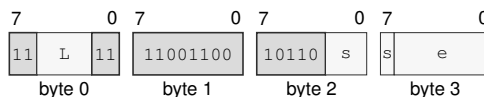
Instruction Format:**Format — Instruction format for EXTSQ.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ.L Rn(s), <ea>(e)

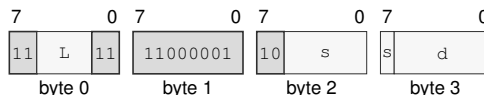
Instruction Format:**Format — Instruction format for EXTSQ.L Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ.B <ea>(s), <ea>(d)

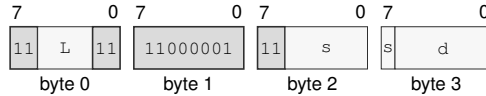
Instruction Format:**Format — Instruction format for EXTSQ.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ.W <ea>(s), <ea>(d)

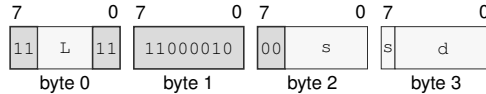
Instruction Format:**Format — Instruction format for EXTSQ.W <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSQ.L <ea>(s), <ea>(d)

Instruction Format:**Format — Instruction format for EXTSQ.L <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSW**Sign Extend To Word****EXTSW**

Operation: Performs the sign extend to word operation.

Assembler Syntax: `EXTSW.B Rn(s), <ea>(e)`
`EXTSW.B <ea>(s), <ea>(d)`

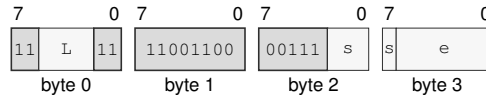
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Sign-extends the source operand from byte size to word size and stores the two-byte result to an EA destination. The low byte of the source is copied into the result, and the remaining high result bits are filled with the source sign bit. EXTSW has no Rn destination form; use EXTSQ.B for a sign-extended Rn result.

Encodings:

`EXTSW.B Rn(s), <ea>(e)`

Instruction Format:

Format — Instruction format for EXTSW.B Rn(s), <ea>(e)

Instruction Fields:

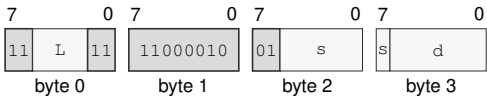
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTSW.B <ea>(s), <ea>(d)

Instruction Format:



Format — Instruction format for EXTSW.B <ea>(s), <ea>(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field** s—Specifies the source operand.
- Effective Address field** d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZL**Zero Extend To Long****EXTZL**

Operation: Performs the zero extend to long operation.

Assembler Syntax: EXTZL.B Rn(s), <ea>(e)
 EXTZL.W Rn(s), <ea>(e)
 EXTZL.BW(z) <ea>(s), <ea>(d)

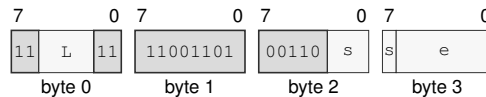
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Zero-extends the source operand from the instruction size to long size and stores the four-byte result to an EA destination. The low source-size bytes are copied into the result, and the remaining high result bits are filled with zero. EXTZL has no Rn destination form; use MOV.B or MOV.W for a zero-extended Rn result.

Encodings:

EXTZL.B Rn(s), <ea>(e)

Instruction Format:

Format — Instruction format for EXTZL.B Rn(s), <ea>(e)

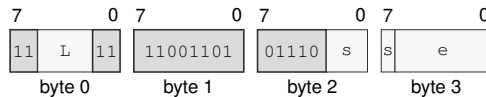
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZL.W Rn(s), <ea>(e)

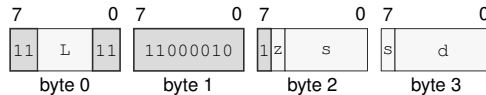
Instruction Format:**Format — Instruction format for EXTZL.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZL.BW(z) <ea>(s), <ea>(d)

Instruction Format:**Format — Instruction format for EXTZL.BW(z) <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ

Zero Extend To Quad

EXTZQ

Operation: Performs the zero extend to quad operation.

Assembler Syntax: EXTZQ.B Rn(s), <ea>(e)
 EXTZQ.W Rn(s), <ea>(e)
 EXTZQ.L Rn(s), <ea>(e)
 EXTZQ.B <ea>(s), <ea>(d)
 EXTZQ.W <ea>(s), <ea>(d)
 EXTZQ.L <ea>(s), <ea>(d)

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

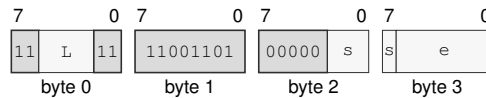
Detailed Semantics

Zero-extends the source operand from the instruction size to quad size and stores the eight-byte result to an EA destination. The low source-size bytes are copied into the result, and the remaining high result bits are filled with zero. EXTZQ has no Rn destination form; use MOV.B, MOV.W, or MOV.L for a zero-extended Rn result.

Encodings:

EXTZQ.B Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for EXTZQ.B Rn(s), <ea>(e)

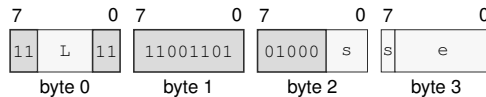
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ.W Rn(s), <ea>(e)

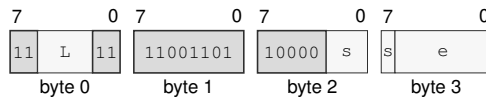
Instruction Format:**Format — Instruction format for EXTZQ.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ.L Rn(s), <ea>(e)

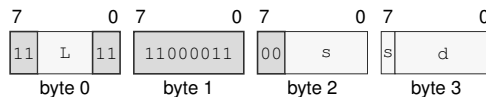
Instruction Format:**Format — Instruction format for EXTZQ.L Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ.B <ea>(s), <ea>(d)

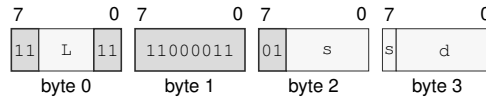
Instruction Format:**Format — Instruction format for EXTZQ.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ.W <ea>(s), <ea>(d)

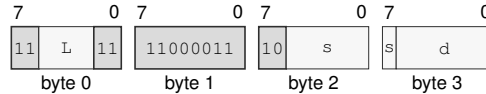
Instruction Format:**Format — Instruction format for EXTZQ.W <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZQ.L <ea>(s), <ea>(d)

Instruction Format:**Format — Instruction format for EXTZQ.L <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZW

Zero Extend To Word

EXTZW

Operation:

Performs the zero extend to word operation.

Assembler Syntax:

EXTZW.B Rn(s), <ea>(e)
EXTZW.B <ea>(s), <ea>(d)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

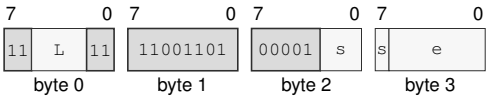
Detailed Semantics

Zero-extends the source operand from byte size to word size and stores the two-byte result to an EA destination. The low byte of the source is copied into the result, and the remaining high result bits are filled with zero. EXTZW has no Rn destination form; use MOV.B for a zero-extended Rn result.

Encodings:

EXTZW.B Rn(s), <ea>(e)

Instruction Format:

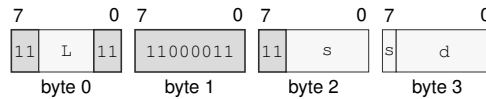


Format — Instruction format for EXTZW.B Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the destination operand. Immediate addressing is unavailable in this form.

EXTZW.B <ea>(s), <ea>(d)

Instruction Format:**Format — Instruction format for EXTZW.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field *s*—Specifies the source operand.

Effective Address field *d*—Specifies the destination operand. Immediate addressing is unavailable in this form.

FETCHADD

Atomic Fetch Add

FETCHADD

Operation: Performs the atomic fetch add operation.

Assembler Syntax: `FETCHADD.BWLQ(z)/order(o) Rn(s), <ea>(e)`

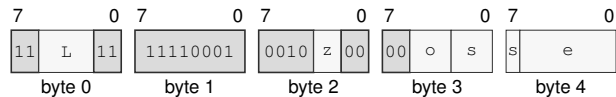
Detailed Semantics

Atomically replaces the memory operand with the selected-width sum of the old memory value and the source, and returns the old memory value in the source register.

Encodings:

`FETCHADD.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `FETCHADD.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects B/W/L/Q.
- Memory-order field** *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.
- Register field** *s*—Selects the source operand.
- Effective Address field** *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

FETCHAND

Atomic Fetch And

FETCHAND

Operation: Performs the atomic fetch and operation.

Assembler Syntax: `FETCHAND.BWLQ(z)/order(o) Rn(s), <ea>(e)`

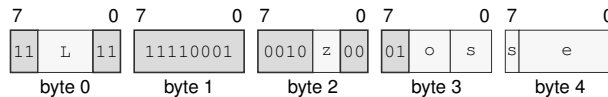
Detailed Semantics

Atomically replaces the memory operand with the selected-width bitwise conjunction of the old memory value and the source, and returns the old memory value in the source register.

Encodings:

`FETCHAND.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `FETCHAND.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

FETCHOR

Atomic Fetch Or

FETCHOR

Operation: Performs the atomic fetch or operation.

Assembler Syntax: `FETCHOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

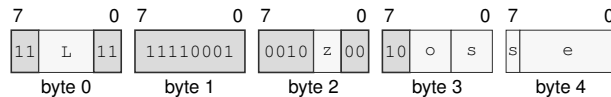
Detailed Semantics

Atomically replaces the memory operand with the selected-width bitwise disjunction of the old memory value and the source, and returns the old memory value in the source register.

Encodings:

`FETCHOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `FETCHOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

FETCHSUB

Atomic Fetch Subtract

FETCHSUB

Operation: Performs the atomic fetch subtract operation.

Assembler Syntax: `FETCHSUB.BWLQ(z)/order(o) Rn(s), <ea>(e)`

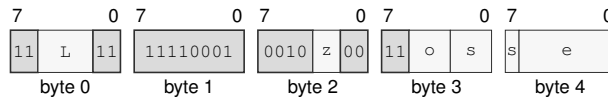
Detailed Semantics

Atomically replaces the memory operand with the selected-width result of old memory minus the source, and returns the old memory value in the source register.

Encodings:

`FETCHSUB.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `FETCHSUB.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

FETCHXOR

Atomic Fetch Xor

FETCHXOR

Operation: Performs the atomic fetch xor operation.

Assembler Syntax: `FETCHXOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

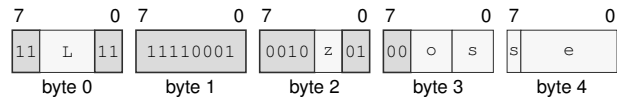
Detailed Semantics

Atomically replaces the memory operand with the selected-width bitwise exclusive-or of the old memory value and the source, and returns the old memory value in the source register.

Encodings:

`FETCHXOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for `FETCHXOR.BWLQ(z)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects B/W/L/Q.
- Memory-order field** *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.
- Register field** *s*—Selects the source operand.
- Effective Address field** *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

FLSHDCACHE

Flush Data Cache

FLSHDCACHE

Operation: Write back and invalidate one cache-maintenance block.

Assembler Syntax: FLSHDCACHE <ea>(e)

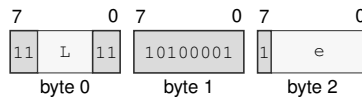
Detailed Semantics

Computes and translates the address-role EA without reading an operand value, selects its CPUID-sized maintenance block, writes dirty bytes from every data or unified cache copy in the coherence domain into normal-memory coherence, invalidates those copies, and retires after the block is complete.

Encodings:

FLSHDCACHE <ea>(e)

Instruction Format:



Format — Instruction format for FLSHDCACHE <ea>(e)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the address operand.

HALT

Halt

HALT

Operation: Halt the executing logical processor until an asynchronous architectural event is admitted.

Assembler Syntax: HALT

Detailed Semantics

Retires with the next instruction as its continuation and stops instruction fetch on the executing logical processor until an asynchronous architectural event is admitted and delivered.

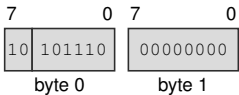
HALT retires with the following instruction as its continuation, then applies the Architectural Event Processing Model's event priority and admission rules at that instruction boundary. If the model selects an event at that boundary, it is delivered with the continuation as its saved PC before the logical processor enters the halted state.

Otherwise, the executing logical processor stops instruction fetch. When an asynchronous architectural event later becomes admissible, the logical processor leaves the halted state by delivering that event with the continuation as its saved PC. Event delivery precedes execution of the continuation. Only the executing logical processor enters the halted state.

Encodings:

HALT

Instruction Format:



Format — Instruction format for HALT

IJcc**Increment And Jump Conditional****IJcc**

Operation: Performs the increment and jump conditional operation.

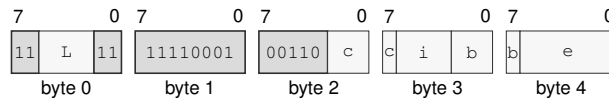
Assembler Syntax: `IJcc Rn(i), Rn(b), <ea>(e)`
`IJcc Rn(i), Rn(b), Rn(e)`

Detailed Semantics

Increments the index and transfers control only when the new index differs from the bound and the encoded condition is true. The target EA has Q interpretation width: a memory form reads a 64-bit target and a register or immediate form supplies its value directly.

Encodings:

`IJcc Rn(i), Rn(b), <ea>(e)`

Instruction Format:

Format — Instruction format for IJcc Rn(i), Rn(b), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

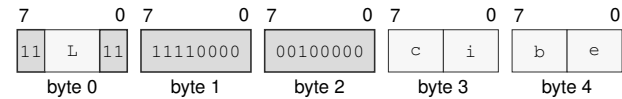
Register field i—Selects operand 1.

Register field b—Selects operand 2.

Effective Address field e—Specifies the control target.

IJcc Rn(i), Rn(b), Rn(e)

Instruction Format:



Format — Instruction format for IJcc Rn(i), Rn(b), Rn(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Condition field c**—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.
- Register field i**—Selects operand 1.
- Register field b**—Selects operand 2.
- Register field e**—Selects operand 3.

ILLEGAL**Illegal Instruction****ILLEGAL**

Operation: Raise illegal-instruction exception.

Assembler Syntax: ILLEGAL

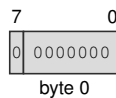
Exceptions: ILLEGAL_INSTRUCTION.EXPLICIT_ILLEGAL: when the instruction executes

Detailed Semantics

Raises ILLEGAL_INSTRUCTION with the EXPLICIT_ILLEGAL cause at the current instruction boundary without retiring or changing architectural destination state before event entry.

Encodings:

ILLEGAL

Instruction Format:

Format — Instruction format for ILLEGAL

INC

Increment

INC

- Operation:
- Performs the increment operation.
- Assembler Syntax:
- INC.LQ(z) Rn(r)
INC.BWLQ(z) <ea>(e)
- Repeat eligibility:
- Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

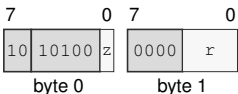
Detailed Semantics

Adds one to the selected-width destination and writes the modular result without changing FLAGS.

Encodings:

INC.LQ(z) Rn(r)

Instruction Format:



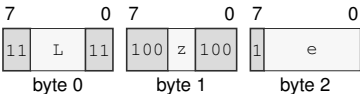
Format — Instruction format for INC.LQ(z) Rn(r)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field r—Selects the operand.

INC.BWLQ(z) <ea>(e)

Instruction Format:



Format — Instruction format for INC.BWLQ(z) <ea>(e)

Instruction Fields:

- Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z—Selects B/W/L/Q.
- Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

INCF**Increment And Set Flags****INCF**

Operation: Increments the destination operand and updates integer FLAGS.

Assembler Syntax: `INCF.BWLQ(z) Rn(r)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Adds one to the selected-width destination, writes the modular result, and updates Z, N, C, and V. REPcc observes the written destination result rather than architectural FLAGS.

FLAGS Effects

Z	N	C	V
a	b	c	d

a: result == 0.

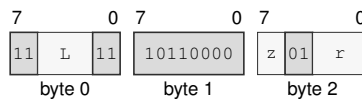
b: most_significant_bit(result).

c: unsigned carry out.

d: signed overflow.

Encodings:

`INCF.BWLQ(z) Rn(r)`

Instruction Format:

Format — Instruction format for INCF.BWLQ(z) Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field r—Selects the operand.

INVASID

Invalidate TLB By ASID

INVASID

Operation: Invalidate TLB entries for ASID.

Assembler Syntax: INVASID <imm16>

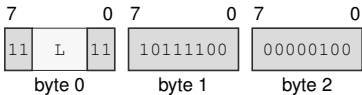
Detailed Semantics

Invalidates every non-global cached translation for the selected 16-bit ASID on the current logical processor, discards matching in-progress results, and prevents later use of stale entries.

Encodings:

INVASID <imm16>

Instruction Format:



Format — Instruction format for INVASID <imm16>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

INVDCACHE

Invalidate Data Cache

INVDCACHE

Operation: Invalidate one data-cache maintenance block.

Assembler Syntax: INVDCACHE <ea>(e)

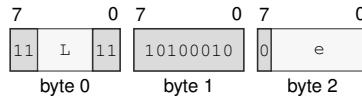
Detailed Semantics

Computes and translates the address-role EA without reading an operand value, selects its CPUID-sized maintenance block, discards every data or unified cache copy in the coherence domain without writeback, and retires after the block is complete.

Encodings:

INVDCACHE <ea>(e)

Instruction Format:



Format — Instruction format for INVDCACHE <ea>(e)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the address operand.

INVICACHE

Invalidate Instruction Cache

INVICACHE

Operation: Invalidate one local instruction-cache maintenance block.

Assembler Syntax: INVICACHE <ea>(e)

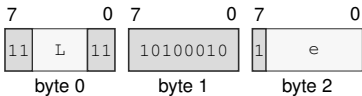
Detailed Semantics

Computes and translates the address-role EA without fetching from it, selects its CPUID-sized maintenance block, invalidates the executing logical processor’s matching instruction-cache, de-coded, and prefetch state, and serializes later instruction fetch.

Encodings:

INVICACHE <ea>(e)

Instruction Format:



Format — Instruction format for INVICACHE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.

INVPAGE

Invalidate TLB Page

INVPAGE

Operation: Invalidate TLB mapping containing a linear address.

Assembler Syntax: INVPAGE <ea>(e)
INVPAGE Rn(r)

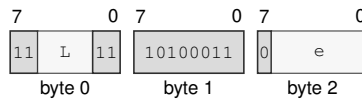
Detailed Semantics

Computes the source linear address without reading memory and invalidates every matching global or current-ASID cached leaf-aligned mapping range that contains it, plus matching in-progress results, before retirement.

Encodings:

INVPAGE <ea>(e)

Instruction Format:



Format — Instruction format for INVPAGE <ea>(e)

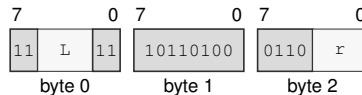
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the address operand.

INVPAGE Rn(r)

Instruction Format:



Format — Instruction format for INVPAGE Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field r—Selects the operand.

INVTLB

Invalidate TLB

INVTLB

Operation: Invalidate all TLB entries.

Assembler Syntax: INVTLB

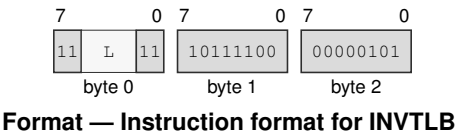
Detailed Semantics

Invalidates all cached instruction and data translations and discards in-progress results on the current logical processor before later accesses may proceed.

Encodings:

INVTLB

Instruction Format:



Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Jcc**Jump Conditional****Jcc**

Operation: Transfers control when the encoded condition is true.

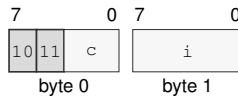
Assembler Syntax: `Jcc imm8s(i)`
`Jcc <imm16s>`
`Jcc <imm32s>`
`Jcc.LQ(z) <ea>(e)`
`Jcc.LQ(z) Rn(r)`

Detailed Semantics

Transfers execution to the target when the encoded condition is true and otherwise continues at the next instruction. Compact direct-relative encodings select a W- or L-width displacement. Extended EA encodings use L/Q. Jcc.L reads the low 32-bit target from its register or memory EA and zero-extends it to the 64-bit near target; Jcc.Q uses the complete 64-bit target.

Encodings:

`Jcc imm8s(i)`

Instruction Format:

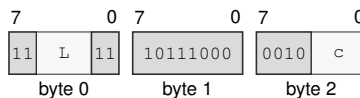
Format — Instruction format for Jcc imm8s(i)

Instruction Fields:

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Immediate field *i*—Encodes the immediate value.

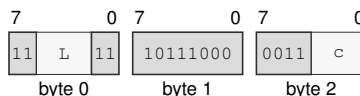
Jcc <imm16s>

Instruction Format:**Format — Instruction format for Jcc <imm16s>****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

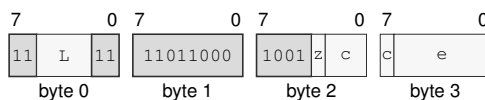
Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Jcc <imm32s>

Instruction Format:**Format — Instruction format for Jcc <imm32s>****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Jcc.LQ(*z*) <ea>(*e*)**Instruction Format:****Format — Instruction format for Jcc.LQ(*z*) <ea>(*e*)****Instruction Fields:**

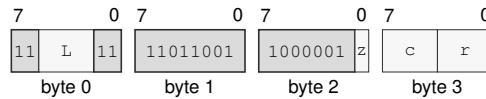
Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Effective Address field *e*—Specifies the control target. Immediate addressing is unavailable in this form.

Jcc.LQ(z) Rn(r)

Instruction Format:**Format — Instruction format for Jcc.LQ(z) Rn(r)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the operand.

JMP

Jump

JMP

Operation: Transfers control to the target address.

Assembler Syntax:

```
JMP imm8s(i)
JMP <imm16s>
JMP <imm32s>
JMP.LQ(z) <ea>(e)
JMP.LQ(z) Rn(r)
```

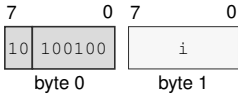
Detailed Semantics

Validates the target under the current control context and transfers execution to it. For the extended EA encoding, JMP.L reads the low 32-bit target from its register or memory EA and zero-extends it to the 64-bit near target; JMP.Q uses the complete 64-bit target.

Encodings:

JMP imm8s(i)

Instruction Format:



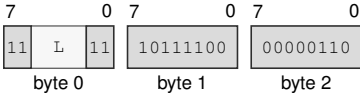
Format — Instruction format for JMP imm8s(i)

Instruction Fields:

Immediate field *i*—Encodes the immediate value.

JMP <imm16s>

Instruction Format:



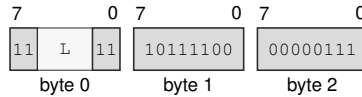
Format — Instruction format for JMP <imm16s>

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

JMP <imm32s>

Instruction Format:



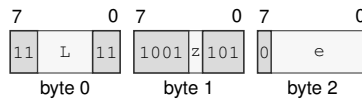
Format — Instruction format for JMP <imm32s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

JMP.LQ(z) <ea>(e)

Instruction Format:



Format — Instruction format for JMP.LQ(z) <ea>(e)

Instruction Fields:

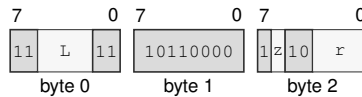
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the control target. Immediate addressing is unavailable in this form.

JMP.LQ(z) Rn(r)

Instruction Format:



Format — Instruction format for JMP.LQ(z) Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field r—Selects the operand.

LCALL

Long Call

LCALL

Operation: Pushes a long return frame and atomically transfers control to a new CS:PC pair.

Assembler Syntax: `LCALL Rn(s), Rn(d)`
`LCALL Rn(r), <ea>(e)`

Detailed Semantics

Long Call is a segment-changing control transfer. It first constructs the long return frame on SS:SP, then commits the new CS and PC together. The two-operand encoding takes the new CS image from an Rn register. Its target EA has Q interpretation width: a memory operand reads a 64-bit offset and a register or immediate operand supplies its value directly.

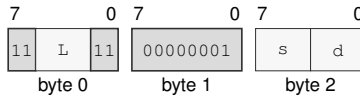
1. Evaluate the target operand completely in the old architectural context.
2. Validate the complete new CS image.
3. Validate the target offset against the new CS and probe executable translation at the resulting linear address.
4. Probe both 8-byte long-return-frame stores through the old SS context.
5. Commit the two frame stores, decremented SP, new CS, and new PC as one successful control-transfer operation.

Fault atomicity. No frame word, SP, CS, or PC update is visible when any prior step faults.

The long return frame is 16 bytes in the old SS context: the continuation PC occupies offset 0 and the return CS image occupies offset 8. An invalid new CS image raises `INVALID_CONTROL_STATE`; target translation or either frame-store translation failure raises `PAGE_FAULT`.

Encodings:

LCALL Rn(s), Rn(d)

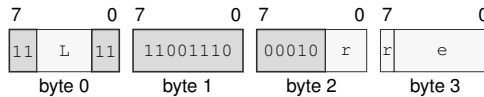
Instruction Format:**Format — Instruction format for LCALL Rn(s), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

LCALL Rn(r), <ea>(e)

Instruction Format:**Format — Instruction format for LCALL Rn(r), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field r—Selects the source operand.

Effective Address field e—Specifies the control target.

LEA

Load Effective Address

LEA

Operation:	Performs the load effective address operation.
Assembler Syntax:	LEA.BWLQ(z) <ea>(e), Rn(d) LEA.BWLQ(z) Rn(s), Rn(d)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

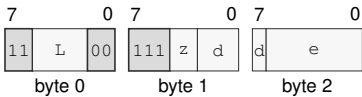
Detailed Semantics

Computes the source effective address without reading the addressed memory and writes that address to the destination.

Encodings:

LEA.BWLQ(z) <ea>(e), Rn(d)

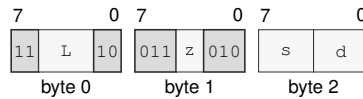
Instruction Format:



Format — Instruction format for LEA.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the address operand.

LEA.BWLQ(*z*) Rn(*s*), Rn(*d*)**Instruction Format:****Format — Instruction format for LEA.BWLQ(*z*) Rn(*s*), Rn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

LJMP

Long Jump

LJMP

Operation: Atomically transfers control to a new CS:PC pair.

Assembler Syntax: LJMP Rn(s), Rn(d)
LJMP Rn(r), <ea>(e)

Detailed Semantics

Long Jump is a segment-changing control transfer. It commits the new CS and PC as a single architectural control-transfer step. The two-operand encoding takes the new CS image from an Rn register. Its target EA has Q interpretation width: a memory operand reads a 64-bit offset and a register or immediate operand supplies its value directly.

- 1. Evaluate the target operand completely in the old architectural context.
- 2. Validate the complete new CS image.
- 3. Validate the target offset against the new CS and probe executable translation at the resulting linear address.
- 4. Commit new CS and new PC as one successful control-transfer operation.

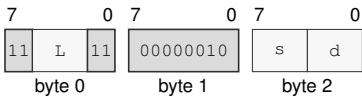
Fault atomicity. CS and PC remain unchanged when operand evaluation or target validation faults.

An invalid new CS image raises INVALID_CONTROL_STATE; target translation failure raises PAGE_FAULT.

Encodings:

LJMP Rn(s), Rn(d)

Instruction Format:



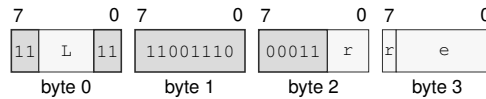
Format — Instruction format for LJMP Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

LJMP Rn(*r*), <ea>(*e*)

Instruction Format:



Format — Instruction format for LJMP Rn(*r*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field *r*—Selects the source operand.

Effective Address field *e*—Specifies the control target.

LRET

Long Return

LRET

Operation: Pops a long return frame and atomically restores CS:PC.

Assembler Syntax: LRET

Detailed Semantics

Long Return is a segment-changing control transfer that reads the long return frame from SS:SP and commits CS and PC together. The frame contains the continuation PC Q value at the current SP and the return CS Q value at SP+8; after both values are fetched, SP is advanced by 16 bytes and the control state is committed.

- 1. Read both 8-byte long-return-frame words through the old SS context without changing SP.
- 2. Validate the complete return CS image.
- 3. Validate the continuation PC offset against the return CS and probe executable translation at the resulting linear address.
- 4. Commit advanced SP, return CS, and continuation PC as one successful control-transfer operation.

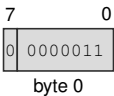
Fault atomicity. SP, CS, and PC remain unchanged when a frame read or return-target validation faults.

The 16-byte long return frame is read through the old SS context: the continuation PC occupies offset 0 and the return CS image occupies offset 8. A frame translation or return-target translation failure raises PAGE_FAULT; an invalid return CS image raises INVALID_CONTROL_STATE.

Encodings:

LRET

Instruction Format:



Format — Instruction format for LRET

MAXS**Signed Maximum****MAXS**

Operation: Performs the signed maximum operation.

Assembler Syntax: MAXS.BWLQ(*z*) Rn(*s*), Rn(*d*)
 MAXS.BWLQ(*z*) <ea>(*e*), Rn(*d*)
 MAXS.BWLQ(*z*) Rn(*s*), <ea>(*e*)

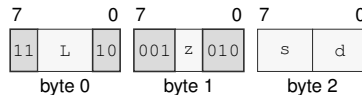
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Compares the selected-width operands as signed integers and writes the greater value to the destination. A sub-Q Rn result is sign-extended to 64 bits.

Encodings:

MAXS.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Format:

Format — Instruction format for MAXS.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

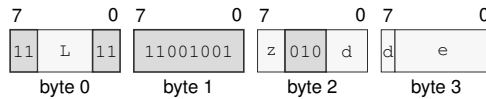
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MAXS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for MAXS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

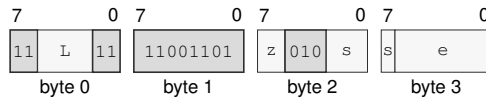
Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

MAXS.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Format:



Format — Instruction format for MAXS.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

MAXU**Unsigned Maximum****MAXU**

Operation: Performs the unsigned maximum operation.

Assembler Syntax: MAXU.BWLQ(*z*) Rn(*s*), Rn(*d*)
 MAXU.BWLQ(*z*) <ea>(*e*), Rn(*d*)
 MAXU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

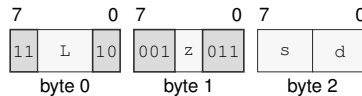
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Compares the selected-width operands as unsigned integers and writes the greater value to the destination.

Encodings:

MAXU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Format:

Format — Instruction format for MAXU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

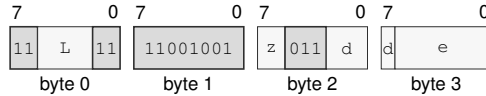
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MAXU.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for MAXU.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

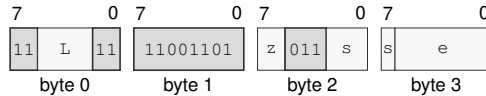
Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

MAXU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Format:



Format — Instruction format for MAXU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

MINS

Signed Minimum

MINS

Operation: Performs the signed minimum operation.

Assembler Syntax: MINS.BWLQ(z) Rn(s), Rn(d)
MINS.BWLQ(z) <ea>(e), Rn(d)
MINS.BWLQ(z) Rn(s), <ea>(e)

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

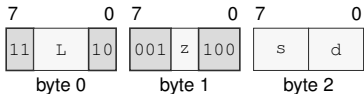
Detailed Semantics

Compares the selected-width operands as signed integers and writes the lesser value to the destination. A sub-Q Rn result is sign-extended to 64 bits.

Encodings:

MINS.BWLQ(z) Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for MINS.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

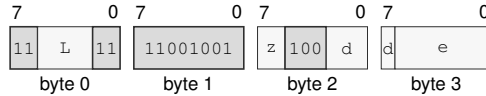
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MINS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for MINS.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

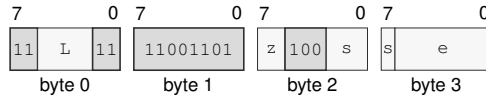
Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

MINS.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Format:



Format — Instruction format for MINS.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

MINU**Unsigned Minimum****MINU**

Operation: Performs the unsigned minimum operation.

Assembler Syntax: MINU.BWLQ(*z*) Rn(*s*), Rn(*d*)
 MINU.BWLQ(*z*) <ea>(*e*), Rn(*d*)
 MINU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

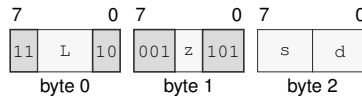
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Compares the selected-width operands as unsigned integers and writes the lesser value to the destination.

Encodings:

MINU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Format:

Format — Instruction format for MINU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

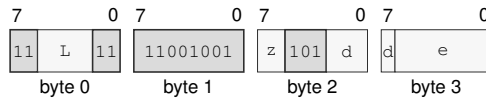
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MINU.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for MINU.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

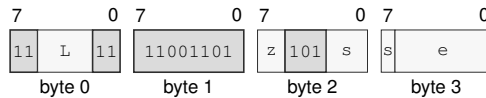
Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

MINU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Format:



Format — Instruction format for MINU.BWLQ(*z*) Rn(*s*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

MODS**Signed Modulo****MODS**

Operation:	Performs the signed modulo operation.
Assembler Syntax:	MODS.BWLQ(<i>z</i>) Rn(<i>s</i>), Rn(<i>d</i>) MODS.BWLQ(<i>z</i>) <ea>(<i>e</i>), Rn(<i>d</i>)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	DIVIDE_ERROR: when the divisor is zero or the most-negative value is divided by minus one

Detailed Semantics

Interprets both selected-width operands as signed integers and writes the remainder associated with truncation-toward-zero division. A sub-Q remainder is sign-extended to 64 bits. Invalid signed division cases raise DIVIDE_ERROR.

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

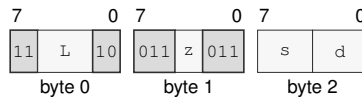
$$q = \text{trunc}(d/s), \quad r = d - qs.$$

Thus a nonzero remainder has the sign of the dividend. The destination supplies the dividend and receives r .

A zero divisor and the most-negative value divided by -1 raise DIVIDE_ERROR; no destination is changed.

Encodings:

MODS.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Format:

Format — Instruction format for MODS.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

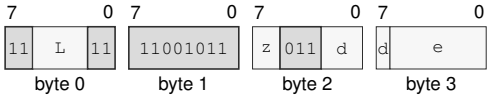
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MODS.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for MODS.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field d**—Selects the destination operand.
- Effective Address field e**—Specifies the source operand.

MODU**Unsigned Modulo****MODU**

Operation:	Performs the unsigned modulo operation.
Assembler Syntax:	MODU.BWLQ(<i>z</i>) Rn(<i>s</i>), Rn(<i>d</i>) MODU.BWLQ(<i>z</i>) <ea>(<i>e</i>), Rn(<i>d</i>)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
Exceptions:	DIVIDE_ERROR: when the divisor is zero

Detailed Semantics

Writes the remainder of selected-width unsigned destination-by-source division. A zero divisor raises DIVIDE_ERROR.

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

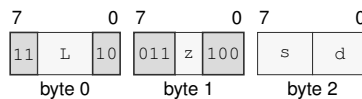
$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The destination supplies the dividend and receives r .

A zero divisor raises DIVIDE_ERROR; no destination is changed.

Encodings:

MODU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Format:

Format — Instruction format for MODU.BWLQ(*z*) Rn(*s*), Rn(*d*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

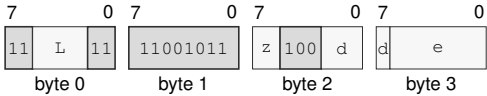
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MODU.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for MODU.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field d**—Selects the destination operand.
- Effective Address field e**—Specifies the source operand.

MOV**Move****MOV**

Operation: Copies the source operand to the destination operand.

Assembler Syntax: `MOV.Q Rn(r), SP`
`MOV.Q SP, Rn(r)`
`MOV.LQ(z) Rn(s), Rn(d)`
`MOV.B Rn(s), Rn(d)`
`MOV.W Rn(s), Rn(d)`
`MOV.BWLQ(z) Rn(s), <ea>(e)`
`MOV.BWLQ(z) <ea>(e), Rn(d)`
`MOV.BWLQ(z) <ea>(s), <ea>(d)`

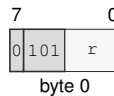
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Reads the source using its addressing mode and selected size and writes that value to the destination. A sub-Q Rn destination receives the selected source bits zero-extended to 64 bits.

Encodings:

`MOV.Q Rn(r), SP`

Instruction Format:

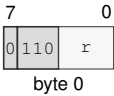
Format — Instruction format for MOV.Q Rn(r), SP

Instruction Fields:

Register field *r*—Selects the source operand.

MOV.Q SP, Rn(r)

Instruction Format:



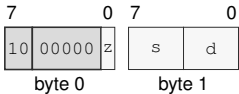
Format — Instruction format for MOV.Q SP, Rn(r)

Instruction Fields:

Register field r—Selects the destination operand.

MOV.LQ(z) Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for MOV.LQ(z) Rn(s), Rn(d)

Instruction Fields:

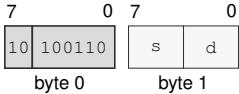
Size field z—Selects L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MOV.B Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for MOV.B Rn(s), Rn(d)

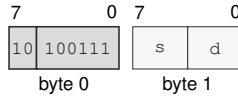
Instruction Fields:

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MOV.W Rn(s), Rn(d)

Instruction Format:



Format — Instruction format for MOV.W Rn(s), Rn(d)

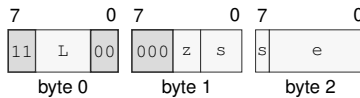
Instruction Fields:

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MOV.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for MOV.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

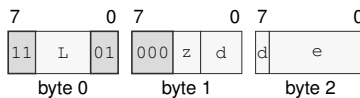
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOV.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for MOV.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

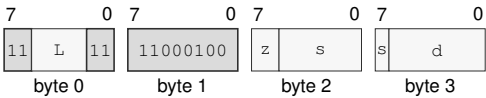
Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

MOV.BWLQ(z) <ea>(s), <ea>(d)

Instruction Format:



Format — Instruction format for MOV.BWLQ(z) <ea>(s), <ea>(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Effective Address field** s—Specifies the source operand.
- Effective Address field** d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVcc**Move Conditional****MOVcc**

Operation: Performs the move conditional operation.

Assembler Syntax: `MOVcc.BWLQ(z) Rn(s), Rn(d)`
`MOVcc.BWLQ(z) Rn(s), <ea>(e)`
`MOVcc.BWLQ(z) <ea>(e), Rn(d)`

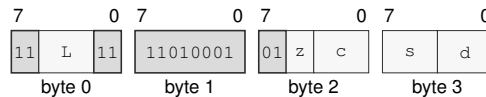
Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Copies the source value to the destination only when the encoded condition is true. When the condition is true, a sub-Q Rn destination receives the selected source bits zero-extended to 64 bits.

Encodings:

`MOVcc.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for MOVcc.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

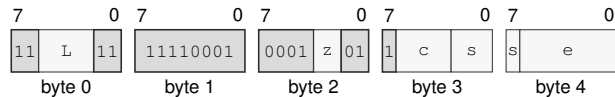
Condition field *c*—Selects the condition code.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MOVcc.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for MOVcc.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

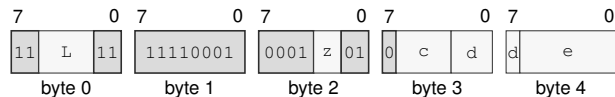
Condition field c—Selects the condition code.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVcc.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for MOVcc.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Condition field c—Selects the condition code.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

MOVCU

Move Current To User

MOVCU

Operation: Copies from a current-domain source to user-domain memory.

Assembler Syntax: `MOVCU.BWLQ(z) Rn(s), Rn(d)`
`MOVCU.BWLQ(z) <ea>(s), <ea>(d)`
`MOVCU.BWLQ(z) Rn(s), <ea>(d)`
`MOVCU.BWLQ(z) <ea>(s), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

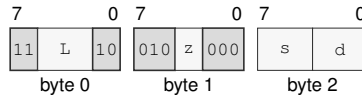
Detailed Semantics

MOVCU is a supervisor-only checked user-access move. The source is a current-domain memory operand, register, or immediate, and the destination is always user-domain memory.

Encodings:

`MOVCU.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:



Format — Instruction format for MOVCU.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

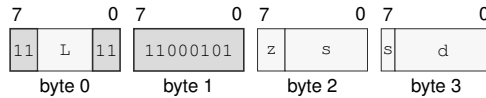
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

MOVCU.BWLQ(z) <ea>(s), <ea>(d)

Instruction Format:



Format — Instruction format for MOVCU.BWLQ(z) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

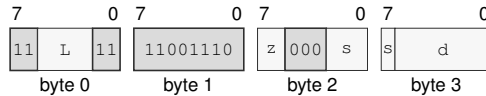
Size field z—Selects B/W/L/Q.

Effective Address field s—Specifies the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVCU.BWLQ(z) Rn(s), <ea>(d)

Instruction Format:



Format — Instruction format for MOVCU.BWLQ(z) Rn(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

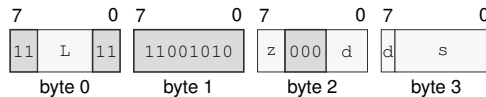
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVCU.BWLQ(z) <ea>(s), Rn(d)

Instruction Format:



Format — Instruction format for MOVCU.BWLQ(z) <ea>(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field s—Specifies the source operand.

MOVNT

Move Non-Temporal

MOVNT

Operation: Stores an Rn register value to memory with a non-temporal access hint.

Assembler Syntax: `MOVNT.BWLQ(z) Rn(s), <ea>(e)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value read from the source operand.

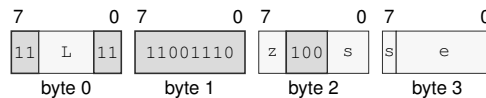
Detailed Semantics

MOVNT is a store-only non-temporal move. The source is an Rn register and the destination must be a memory operand.

Encodings:

`MOVNT.BWLQ(z) Rn(s), <ea>(e)`

Instruction Format:



Format — Instruction format for MOVNT.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVUC

Move User To Current

MOVUC

Operation:

Copies from user-domain memory to a current-domain destination.

Assembler Syntax:

MOVUC.BWLQ(z) Rn(s), Rn(d)
MOVUC.BWLQ(z) <ea>(s), <ea>(d)
MOVUC.BWLQ(z) Rn(s), <ea>(d)
MOVUC.BWLQ(z) <ea>(s), Rn(d)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

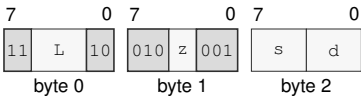
Detailed Semantics

MOVUC is a supervisor-only checked user-access move. If the source operand is memory, that access uses the user domain. The destination is a current-domain memory operand or an Rn register.

Encodings:

MOVUC.BWLQ(z) Rn(s), Rn(d)

Instruction Format:



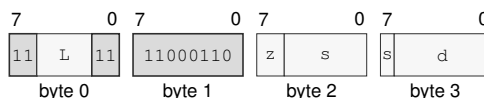
Format — Instruction format for MOVUC.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

MOVUC.BWLQ(z) <ea>(s), <ea>(d)

Instruction Format:



Format — Instruction format for MOVUC.BWLQ(z) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

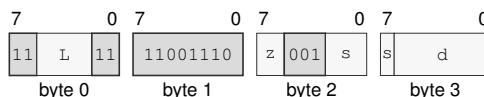
Size field z—Selects B/W/L/Q.

Effective Address field s—Specifies the source operand. Immediate addressing is unavailable in this form.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVUC.BWLQ(z) Rn(s), <ea>(d)

Instruction Format:



Format — Instruction format for MOVUC.BWLQ(z) Rn(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

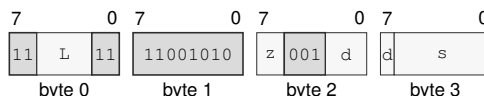
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MOVUC.BWLQ(z) <ea>(s), Rn(d)

Instruction Format:



Format — Instruction format for MOVUC.BWLQ(z) <ea>(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field s—Specifies the source operand. Immediate addressing is unavailable in this form.

MOVUU

Move User To User

MOVUU

Operation:

Copies from user-domain memory to user-domain memory.

Assembler Syntax:

MOVUU.BWLQ(z) <ea>(s) , <ea>(d)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

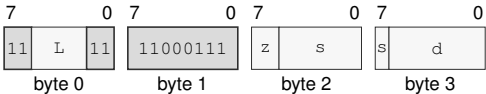
Detailed Semantics

MOVUU is a supervisor-only checked user-access move. Both source and destination operands must be memory operands, and both memory accesses use the user domain.

Encodings:

MOVUU.BWLQ(z) <ea>(s) , <ea>(d)

Instruction Format:



Format — Instruction format for MOVUU.BWLQ(z) <ea>(s), <ea>(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Effective Address field** s—Specifies the source operand. Immediate addressing is unavailable in this form.
- Effective Address field** d—Specifies the destination operand. Immediate addressing is unavailable in this form.

MUL**Multiply Low****MUL**

Operation: Multiplies the destination by the source and keeps the low half.

Assembler Syntax: `MUL.BWLQ(z) Rn(s), Rn(d)`
`MUL.BWLQ(z) <ea>(e), Rn(d)`

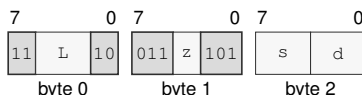
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

MUL multiplies the selected-width operands and writes the low N bits of the product.

Encodings:

`MUL.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for MUL.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

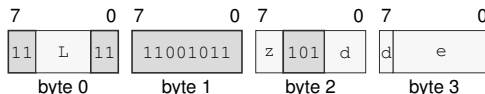
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

`MUL.BWLQ(z) <ea>(e), Rn(d)`

Instruction Format:

Format — Instruction format for MUL.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

MULHS

Signed Multiply High

MULHS

Operation:

Performs the signed multiply high operation.

Assembler Syntax:

MULHS.Q Rn(s) , Rn(d)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Multiplies the 64-bit operands as signed integers and writes the high 64 bits of the 128-bit product.

Encodings:

MULHS.Q Rn(s) , Rn(d)

Instruction Format:



Format — Instruction format for MULHS.Q Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

MULHSU**Signed By Unsigned Multiply High****MULHSU**

Operation: Performs the signed by unsigned multiply high operation.

Assembler Syntax: `MULHSU.Q Rn(s), Rn(d)`

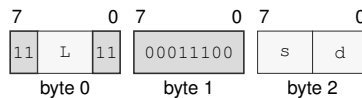
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Multiplies the signed 64-bit destination by the unsigned 64-bit source and writes the high 64 bits of the 128-bit product.

Encodings:

`MULHSU.Q Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for MULHSU.Q Rn(s), Rn(d)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MULHU

Unsigned Multiply High

MULHU

- Operation: Performs the unsigned multiply high operation.
- Assembler Syntax: `MULHU.Q Rn(s), Rn(d)`
- Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Multiplies the 64-bit operands as unsigned integers and writes the high 64 bits of the 128-bit product.

Encodings:

`MULHU.Q Rn(s), Rn(d)`

Instruction Format:



Format — Instruction format for MULHU.Q Rn(s), Rn(d)

Instruction Fields:

- Length field** `L`—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** `s`—Selects the source operand.
- Register field** `d`—Selects the destination operand.

NEG**Negate****NEG**

Operation: Performs the negate operation.

Assembler Syntax: `NEG.LQ(z) Rn(r)`
`NEG.BWLQ(z) <ea>(e)`

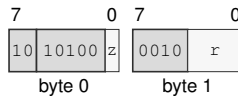
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Writes the selected-width two's-complement negation of the destination value.

Encodings:

`NEG.LQ(z) Rn(r)`

Instruction Format:

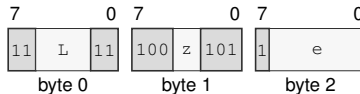
Format — Instruction format for NEG.LQ(z) Rn(r)

Instruction Fields:

Size field *z*—Selects L/Q.

Register field *r*—Selects the operand.

`NEG.BWLQ(z) <ea>(e)`

Instruction Format:

Format — Instruction format for NEG.BWLQ(z) <ea>(e)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

NOP

No Operation

NOP

Operation: No architectural state change.

Assembler Syntax: NOP

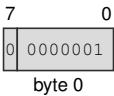
Detailed Semantics

Retires normally without changing architectural state.

Encodings:

NOP

Instruction Format:



Format — Instruction format for NOP

NOT**Bitwise Not****NOT**

Operation: Performs the bitwise not operation.

Assembler Syntax: NOT.LQ(z) Rn(r)
NOT.BWLQ(z) <ea>(e)

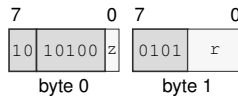
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Writes the selected-width bitwise complement of the destination value.

Encodings:

NOT.LQ(z) Rn(r)

Instruction Format:

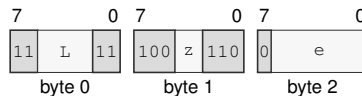
Format — Instruction format for NOT.LQ(z) Rn(r)

Instruction Fields:

Size field z—Selects L/Q.

Register field r—Selects the operand.

NOT.BWLQ(z) <ea>(e)

Instruction Format:

Format — Instruction format for NOT.BWLQ(z) <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

OR

Bitwise Or

OR

Operation: Computes the bitwise OR of the source and destination operands.

Assembler Syntax: `OR.LQ(z) Rn(s), Rn(d)`
`OR.BWLQ(z) Rn(s), <ea>(e)`
`OR.BWLQ(z) <ea>(e), Rn(d)`
`OR.BWLQ(z) <imm8s>, <ea>(e)`
`OR.WLQ(z) <imm16s>, <ea>(e)`
`OR.LQ(z) <imm32s>, <ea>(e)`
`OR.Q <imm64>, <ea>(e)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

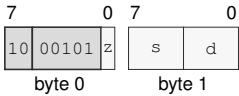
Detailed Semantics

Writes the selected-width bitwise disjunction of the source and destination values to the destination.

Encodings:

`OR.LQ(z) Rn(s), Rn(d)`

Instruction Format:

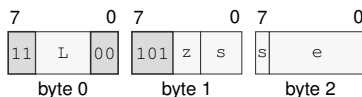


Format — Instruction format for OR.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field** `z`—Selects L/Q.
- Register field** `s`—Selects the source operand.
- Register field** `d`—Selects the destination operand.

OR.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:**Format — Instruction format for OR.BWLQ(z) Rn(s), <ea>(e)****Instruction Fields:**

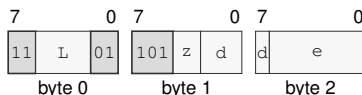
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

OR.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:**Format — Instruction format for OR.BWLQ(z) <ea>(e), Rn(d)****Instruction Fields:**

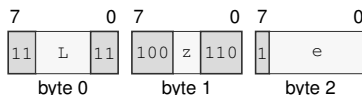
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

OR.BWLQ(z) <imm8s>, <ea>(e)

Instruction Format:**Format — Instruction format for OR.BWLQ(z) <imm8s>, <ea>(e)****Instruction Fields:**

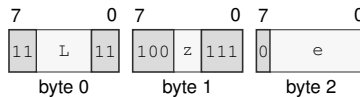
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

OR.WLQ(z) <imm16s>, <ea>(e)

Instruction Format:



Format — Instruction format for OR.WLQ(z) <imm16s>, <ea>(e)

Instruction Fields:

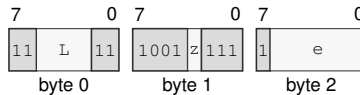
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

OR.LQ(z) <imm32s>, <ea>(e)

Instruction Format:



Format — Instruction format for OR.LQ(z) <imm32s>, <ea>(e)

Instruction Fields:

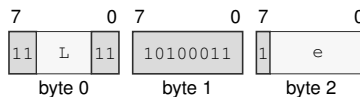
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

OR.Q <imm64>, <ea>(e)

Instruction Format:



Format — Instruction format for OR.Q <imm64>, <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

PARITY

Parity Test

PARITY

Operation: Computes operand parity and writes the predicate to an Rn register.

Assembler Syntax: `PARITY.BWLQ(z) Rn(s), Rn(d)`
`PARITY.BWLQ(z) <ea>(e), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

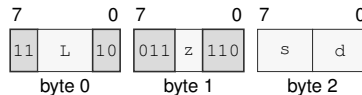
Detailed Semantics

Computes the parity of the selected operand size and writes `popcount(src) & 1` to the destination register. The written value is 1 for odd parity and 0 for even parity. FLAGS are unchanged.

Encodings:

`PARITY.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:



Format — Instruction format for `PARITY.BWLQ(z) Rn(s), Rn(d)`

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

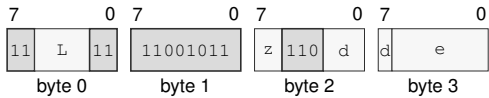
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

PARITY.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for PARITY.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects B/W/L/Q.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

POP

Pop

POP

Operation:	Pops one 64-bit stack value into an Rn or SREG destination.
Assembler Syntax:	POP Rn(r) POP SREG(s)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

POP loads one 64-bit stack value into an Rn or SREG destination and advances SP. The destination and updated SP commit together.

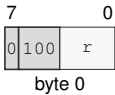
POP reads and validates the stack value without changing architectural state. For an SREG destination, segment-image validation completes before commit. It then updates the destination and SP together; any preceding access or validation fault leaves both unchanged.

POP SS performs the read using the old SS and exposes the new SS only at the final commit.

Encodings:

POP Rn(r)

Instruction Format:



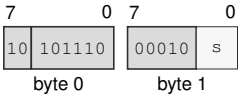
Format — Instruction format for POP Rn(r)

Instruction Fields:

Register field r—Selects the operand.

POP SREG(s)

Instruction Format:



Format — Instruction format for POP SREG(s)

Instruction Fields:

Register field *s*—Selects the operand using the SREG encoding.

POPCNT

Population Count

POPCNT

Operation: Performs the population count operation.

Assembler Syntax: `POPCNT.BWLQ(z) Rn(s), Rn(d)`
`POPCNT.BWLQ(z) <ea>(e), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

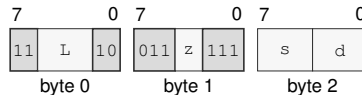
Detailed Semantics

Counts set bits within the selected B, W, L, or Q source width and writes that count to the destination.

Encodings:

`POPCNT.BWLQ(z) Rn(s), Rn(d)`

Instruction Format:



Format — Instruction format for POPCNT.BWLQ(z) Rn(s), Rn(d)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

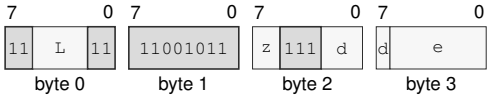
Size field *z*—Selects B/W/L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

POPCNT.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Format:



Format — Instruction format for POPCNT.BWLQ(*z*) <ea>(*e*), Rn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects B/W/L/Q.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

POPP**Pop Register Pair****POPP**

Operation: Pops one canonical Rn register pair selected by an inline pair index.

Assembler Syntax: POPP PAIRn(i)

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

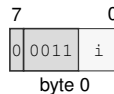
Pops the canonical register pair selected by the pair ID. The canonical pair sequence is pair 0 (R0, R1), pair 1 (R2, R3), pair 2 (R4, R5), pair 3 (R6, R7), pair 4 (R8, R9), pair 5 (R10, R11), pair 6 (R12, R13), and pair 7 (R14, R15). All eight pair IDs are valid and each selects one pair.

POPP selects one canonical pair using the unsigned 3-bit pair index and reads the complete two-slot stack range through the old SS:SP context before either destination register or SP is changed. For a listed pair (*first*, *second*), the second register is read from old SP and the first register is read from old SP plus 8. Both destination registers and the SP update to old SP plus 16 commit together. A preceding fault changes neither destination register nor SP.

Multiple POPP instructions can be used in reverse canonical epilogue order to restore multiple pairs.

Encodings:

POPP PAIRn(i)

Instruction Format:

Format — Instruction format for POPP PAIRn(i)

Instruction Fields:

Immediate field i—Encodes the immediate value.

PREFETCH

Prefetch

PREFETCH

- Operation: Hints that EA will be read soon; faults only under the configured prefetch-fault policy.
- Assembler Syntax: PREFETCH <ea>(e)
- Repeat eligibility: Eligible as a REP body.

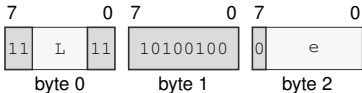
Detailed Semantics

Computes the byte-addressed source without an architectural read and issues an optional temporal read-prefetch hint. Ignoring the hint has no architectural effect except under an explicitly configured prefetch-fault policy.

Encodings:

PREFETCH <ea>(e)

Instruction Format:



Format — Instruction format for PREFETCH <ea>(e)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field** *e*—Specifies the address operand.

PREFETCHNT

Prefetch Non-Temporal

PREFETCHNT

Operation: Hint that EA will be read with non-temporal locality.

Assembler Syntax: `PREFETCHNT <ea>(e)`

Repeat eligibility: Eligible as a REP body.

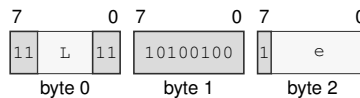
Detailed Semantics

Computes the byte-addressed source without an architectural read and issues an optional non-temporal read-prefetch hint. Ignoring the hint has no architectural effect except under an explicitly configured prefetch-fault policy.

Encodings:

`PREFETCHNT <ea>(e)`

Instruction Format:



Format — Instruction format for PREFETCHNT <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the address operand.

Operation: Non-destructively reads the selected raw page-table descriptor.

Assembler Syntax: PTQUERY pt_level(i), <ea>(e), Rn(d)
PTQUERY pt_level(i), Rn(s), Rn(d)

Detailed Semantics

PTQUERY reads the raw descriptor selected by a linear address and requested page-table level, or returns zero when the requested entry cannot be read. Z reports structural validity under the layout selected by T; N, C, and V are zero.

The requested level is the low three bits of the level operand. Values outside 1 through 5 raise ILLEGAL_INSTRUCTION; level 5 is reported as an ordinary unsuccessful query in LA48 mode.

1. Compute the linear address without reading the source memory and reject a noncanonical address as an unsuccessful query.
2. Starting at the PTCR root, read one 64-bit PTE per level. Every preceding entry must be a valid table pointer; a valid earlier leaf prevents traversal to a lower requested level.
3. Stop after reading the requested level, even when that requested entry is structurally malformed or is a valid leaf rather than a table pointer.
4. Return the requested raw PTE and set Z only when it is structurally valid under the table-pointer or leaf layout selected by T at that level. N, C, and V are zero.

Failure before the requested entry is read returns zero and clears all four flags. The walk ignores PTCR.PE, does not set PTE A or D, does not modify a TLB, and does not raise PAGE_FAULT for walk failure.

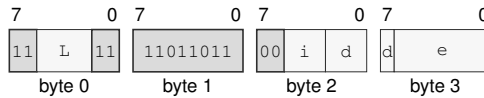
FLAGS Effects

Z	N	C	V
*	0	0	0

- *: requested PTE was read and is structurally valid for its level.
- 0: writes zero.

Encodings:

PTQUERY pt_level(i), <ea>(e), Rn(d)

Instruction Format:**Format — Instruction format for PTQUERY pt_level(i), <ea>(e), Rn(d)****Instruction Fields:**

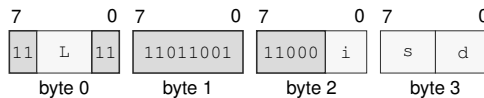
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Register field d—Selects operand 3.

Effective Address field e—Specifies the address operand.

PTQUERY pt_level(i), Rn(s), Rn(d)

Instruction Format:**Format — Instruction format for PTQUERY pt_level(i), Rn(s), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Register field s—Selects operand 2.

Register field d—Selects operand 3.

PUSH

Push

PUSH

Operation:	Pushes one Rn register value, SREG image, or the current CS image onto the stack.
Assembler Syntax:	PUSH CS PUSH Rn(r) PUSH SREG(s)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value read from the source operand.

Detailed Semantics

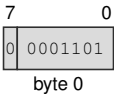
PUSH stores one 64-bit Rn register value, selected SREG image, or current CS image on the stack. The SREG encoding accepts DS, SS, and GS0-GS5. PUSH SS captures the old SS image and uses that same old SS as the stack context for the store.

PUSH captures the source value and current SS:SP before changing architectural state. After the destination stack slot has been validated, it commits the 64-bit store and decremented SP together. A fault before commit leaves memory and SP unchanged. For PUSH SS, both the captured value and the stack access use the old SS image.

Encodings:

PUSH CS

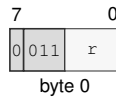
Instruction Format:



Format — Instruction format for PUSH CS

PUSH Rn(r)

Instruction Format:



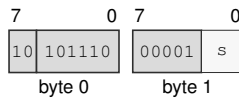
Format — Instruction format for PUSH Rn(r)

Instruction Fields:

Register field r—Selects the operand.

PUSH SREG(s)

Instruction Format:



Format — Instruction format for PUSH SREG(s)

Instruction Fields:

Register field s—Selects the operand using the SREG encoding.

PUSHP

Push Register Pair

PUSHP

- Operation:
- Pushes one canonical Rn register pair selected by an inline pair index.
- Assembler Syntax:
- PUSHP PAIRn(i)
- Repeat eligibility:
- Eligible as a REP body.

Detailed Semantics

Pushes the canonical register pair selected by the pair ID. The canonical pair sequence is pair 0 (R0, R1), pair 1 (R2, R3), pair 2 (R4, R5), pair 3 (R6, R7), pair 4 (R8, R9), pair 5 (R10, R11), pair 6 (R12, R13), and pair 7 (R14, R15). All eight pair IDs are valid and each selects one pair.

PUSHP selects one canonical pair using the unsigned 3-bit pair index and captures both register values and the old SS:SP context. It validates the complete two-slot stack range before either slot or SP is changed. For a listed pair (*first*, *second*), the second register is stored at old SP minus 16 and the first register is stored at old SP minus 8. Both stores and the SP update to old SP minus 16 commit together. A preceding fault changes neither slot nor SP.

Multiple PUSHP instructions can be used in canonical prologue order to save multiple pairs.

Encodings:

PUSHP PAIRn(i)

Instruction Format:



Format — Instruction format for PUSHP PAIRn(i)

Instruction Fields:

Immediate field *i*—Encodes the immediate value.

RDCR**Read Control Register****RDCR**

Operation: Read a privileged control register.

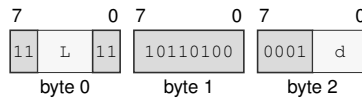
Assembler Syntax: RDCR <imm16>, Rn(d)

Detailed Semantics

RDCR reads a selected control register in supervisor mode.

Encodings:

RDCR <imm16>, Rn(d)

Instruction Format:

Format — Instruction format for RDCR <imm16>, Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

RDFLAGS

Read Flags

RDFLAGS

Operation: Read the integer condition-code flags into an Rn register.

Assembler Syntax: RDFLAGS Rn(d)

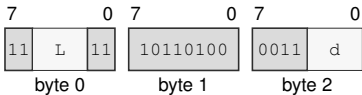
Detailed Semantics

RDFLAGS reads the architectural 16-bit FLAGS register, zero-extends it to 64 bits, and writes the result to the destination Rn register.

Encodings:

RDFLAGS Rn(d)

Instruction Format:



Format — Instruction format for RDFLAGS Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** d—Selects the operand.

RDPMC**Read Performance Counter****RDPMC**

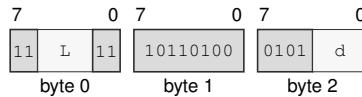
Operation: RDPMC reads a mandatory or implementation-discovered performance counter from any privilege level. Counter ID zero and every other unimplemented counter ID raise INVALID_CONTROL_STATE with the INVALID_SELECTOR cause.

Assembler Syntax: RDPMC <imm16>, Rn(d)

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 5 bytes
 Repeat = none

Encodings:

RDPMC <imm16>, Rn(d)

Instruction Format:

Format — Instruction format for RDPMC <imm16>, Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

RDSEG

Read Segment Register

RDSEG

Operation: Read an SREG or the current CS image into an Rn register.

Assembler Syntax: RDSEG SREG(s) , Rn(d)
RDSEG CS, Rn(d)

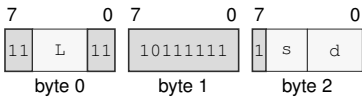
Detailed Semantics

RDSEG reads the 64-bit image of an SREG-class segment register into the destination Rn register. The RDSEG CS encoding reads the current CS image.

Encodings:

RDSEG SREG(s) , Rn(d)

Instruction Format:



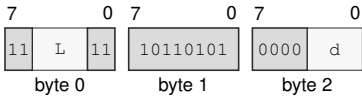
Format — Instruction format for RDSEG SREG(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand using the SREG encoding.
- Register field** d—Selects the destination operand.

RDSEG CS, Rn(d)

Instruction Format:



Format — Instruction format for RDSEG CS, Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** d—Selects the destination operand.

RDSTATUS

Read Status

RDSTATUS

Operation: Read the processor STATUS register into an Rn register.

Assembler Syntax: RDSTATUS Rn(d)

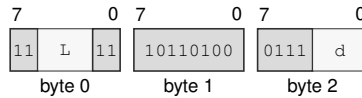
Detailed Semantics

RDSTATUS reads the architectural 16-bit STATUS register, zero-extends it to 64 bits, and writes the result to the destination Rn register. Reading STATUS is unprivileged; writing STATUS remains supervisor-only.

Encodings:

RDSTATUS Rn(d)

Instruction Format:



Format — Instruction format for RDSTATUS Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

REPcc**Repeat Conditional****REPcc**

Operation: Repeats the next instruction while the counter and condition remain active.

Assembler Syntax: `REPcc Rn(r), (<instruction>)`

Detailed Semantics

REPcc installs repeat state for the following single instruction. The selected counter register is interpreted as an unsigned remaining count. REP is the unconditional spelling and uses condition T. Condition F is reserved and is not a valid encoding. A faulting iteration does not decrement the count. A successful iteration decrements it before an event may be admitted. An event between iterations saves the repeat-prefix address and clears hidden repeat state; ERET returns to the prefix and restores no repeat continuation.

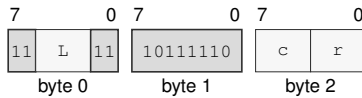
The body instruction is fetched, decoded, and checked for repeatability before the zero-count rule is applied. If the counter is zero, the decoded body is annulled and has no architectural side effects.

If the counter is nonzero, the first body iteration executes before any condition check. The repeat condition is tested against a temporary ZNCV image derived from the body instruction's repeat-observed value: Z reports zero, N is the most significant bit at the observation width, and C and V are zero. REPcc never uses architectural FLAGS as the condition input and does not write architectural FLAGS beyond any ordinary FLAGS effect of the body itself.

Body completion, observation, temporary-condition construction, architectural commit, counter decrement, and continuation PC selection are one precise atomic architectural commit step. An interrupt handler therefore cannot alter the continuation decision of an iteration that reached this step. Every completed body iteration decrements the counter by one, including the terminating condition-false iteration. Writes by the body to the selected counter register have no architectural effect while repeat state is active.

Encodings:

REPcc Rn(r), (<instruction>)

Instruction Format:**Format — Instruction format for REPcc Rn(r), (<instruction>)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the source operand.

RESET

Reset

RESET

Operation: Perform warm reset; preserve BOOTPC and BOOTCFG.

Assembler Syntax: RESET

Detailed Semantics

RESET serializes earlier memory effects, applies the complete architectural warm-reset state to the executing logical processor, and resumes execution at BOOTPC. BOOTPC and BOOTCFG are preserved.

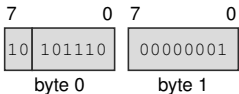
RESET is supervisor-only and affects only the executing logical processor. Before the reset-state transition commits, all earlier memory effects and pending write-combining stores from that logical processor complete. No younger instruction effect becomes visible.

The instruction installs the complete Architectural Reset State table. R0 through R15 and SP become zero, PC becomes BOOTPC, STATUS contains only PM set, and BOOTPC and BOOTCFG retain their values. Architectural repeat state, the load reservation, translation and page-walk caches, decoded instruction state, and prefetch state are invalidated. Coherent data- and instruction-cache contents remain coherent. Instruction-fetch synchronization completes before the first instruction at BOOTPC is fetched.

Encodings:

RESET

Instruction Format:



Format — Instruction format for RESET

RESTORE

Restore Processor State

RESTORE

Operation: Restore processor state from a CPUID-defined save area.

Assembler Syntax: `RESTORE <ea>(e)`

`RESTORE Rn(r)`

Detailed Semantics

RESTORE reconstructs base architectural state and enabled extension state from a 4-KiB-aligned, CPUID-defined save area.

RESTORE treats its address-role EA as the base of the save area without first reading an EA-sized operand. It restores R0–R15 and FLAGS from the fixed base block. A set GS_VALID bit selects the corresponding GS image for validation and restoration; a clear bit preserves the current GS_n. User-mode RESTORE ignores saved STATUS and DFA. Supervisor RESTORE validates and restores the complete saved supervisor STATUS, including EDEPTH and UO, together with current DFA as an architectural context transition; this is not an ordinary WRSTATUS update. A set state-block bitmap bit restores its CPUID-described extension component. A clear bit for a described component installs that component's initial state and makes the component clean; a restored non-initial image makes the component modified.

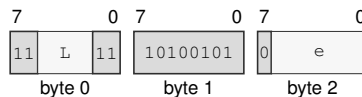
Before reading any extension component or changing architectural state, RESTORE validates the complete base header and bitmap. An unrecognized format, a nonzero reserved bit, or any set bitmap bit without a matching CPUID component descriptor raises INVALID_CONTROL_STATE.

The complete SAVE_AREA_SIZE_BYTES range reported by CPUID, every selected GS image, and every selected extension component are validated before architectural state is committed. Any address, access, or validation fault leaves the architectural register state and save area unchanged.

Encodings:

RESTORE <ea>(e)

Instruction Format:



Format — Instruction format for RESTORE <ea>(e)

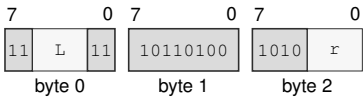
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the address operand.

RESTORE Rn(r)

Instruction Format:



Format — Instruction format for RESTORE Rn(r)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field r**—Selects the operand.

RET**Return****RET**

Operation: Performs the return operation.

Assembler Syntax: RET

Detailed Semantics

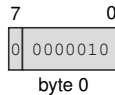
Reads one 64-bit return address from SS:SP, advances SP by eight bytes, validates the target, and transfers control to it.

1. Read the 8-byte return address through the old SS:SP context without changing SP.
2. Validate that address as an executable target under the current CS.
3. Commit advanced SP and the new PC as one successful control-transfer operation.

A stack-read or target-validation fault leaves SP and PC unchanged.

Encodings:

RET

Instruction Format:

Format — Instruction format for RET

REVBYTE

Reverse Bytes

REVBYTE

Operation:

Performs the reverse bytes operation.

Assembler Syntax:

REVBYTE.W Rn(r)
REVBYTE.L Rn(r)
REVBYTE.Q Rn(r)
REVBYTE.W <ea>(e)
REVBYTE.L <ea>(e)
REVBYTE.Q <ea>(e)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

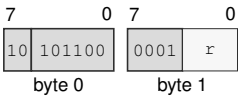
Detailed Semantics

Reverses the byte order within the selected operand width and writes the result to the destination.

Encodings:

REVBYTE.W Rn(r)

Instruction Format:

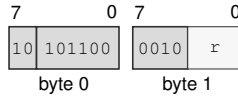


Format — Instruction format for REVBYTE.W Rn(r)

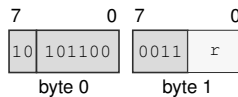
Instruction Fields:

Register field r—Selects the operand.

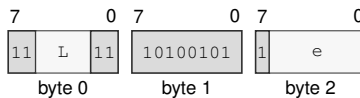
REVBYTE.L Rn(r)

Instruction Format:**Format — Instruction format for REVBYTE.L Rn(r)****Instruction Fields:****Register field** r—Selects the operand.

REVBYTE.Q Rn(r)

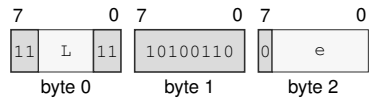
Instruction Format:**Format — Instruction format for REVBYTE.Q Rn(r)****Instruction Fields:****Register field** r—Selects the operand.

REVBYTE.W <ea>(e)

Instruction Format:**Format — Instruction format for REVBYTE.W <ea>(e)****Instruction Fields:****Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.**Effective Address field** e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

REVBYTE.L <ea>(e)

Instruction Format:



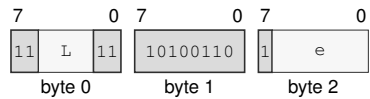
Format — Instruction format for REVBYTE.L <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field e**—Specifies the read/write operand. Immediate addressing is unavailable in this form.

REVBYTE.Q <ea>(e)

Instruction Format:



Format — Instruction format for REVBYTE.Q <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field e**—Specifies the read/write operand. Immediate addressing is unavailable in this form.

RFENCE

Read Fence

RFENCE

Operation: Order prior reads before later reads.

Assembler Syntax: RFENCE

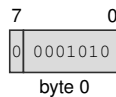
Detailed Semantics

Prevents retirement until every earlier memory read is ordered before every later memory read on the same logical processor. It performs no memory access and changes no register or flag.

Encodings:

RFENCE

Instruction Format:



Format — Instruction format for RFENCE

ROL

Rotate Left

ROL

Operation:	Performs the rotate left operation.
Assembler Syntax:	ROL.LQ(z) Rn(s), Rn(d) ROL.BWLQ(z) Rn(s), <ea>(e) ROL.BWLQ(z) imm6(i), <ea>(e)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

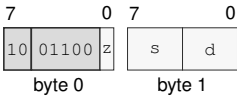
Detailed Semantics

Rotates the selected-width destination value left by the effective count and writes the result to the destination. The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the selected-width result equals the original selected-width value; an Rn destination is still written and a sub-Q result is zero-extended to 64 bits, while a memory destination rewrites the same selected bytes. FLAGS are unchanged for all counts.

Encodings:

ROL.LQ(z) Rn(s), Rn(d)

Instruction Format:



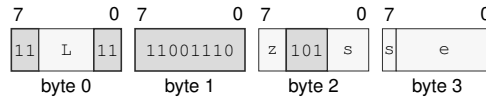
Format — Instruction format for ROL.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

ROL.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for ROL.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

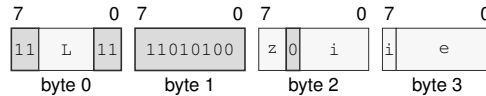
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ROL.BWLQ(z) imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for ROL.BWLQ(z) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ROR

Rotate Right

ROR

Operation:

Performs the rotate right operation.

Assembler Syntax:

ROR.LQ(z) Rn(s), Rn(d)
ROR.BWLQ(z) Rn(s), <ea>(e)
ROR.BWLQ(z) imm6(i), <ea>(e)

Repeat eligibility:

Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

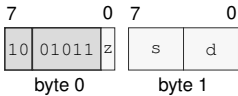
Detailed Semantics

Rotates the selected-width destination value right by the effective count and writes the result to the destination. The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the selected-width result equals the original selected-width value; an Rn destination is still written and a sub-Q result is zero-extended to 64 bits, while a memory destination rewrites the same selected bytes. FLAGS are unchanged for all counts.

Encodings:

ROR.LQ(z) Rn(s), Rn(d)

Instruction Format:

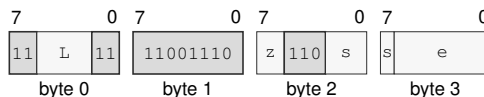


Format — Instruction format for ROR.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

ROR.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:**Format — Instruction format for ROR.BWLQ(z) Rn(s), <ea>(e)****Instruction Fields:**

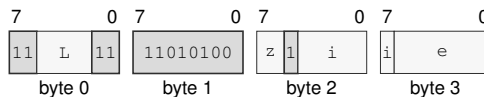
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

ROR.BWLQ(z) imm6(i), <ea>(e)

Instruction Format:**Format — Instruction format for ROR.BWLQ(z) imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SAR

Shift Arithmetic Right

SAR

Operation:	Performs the shift arithmetic right operation.
Assembler Syntax:	SAR.LQ(z) Rn(s), Rn(d) SAR.BWLQ(z) Rn(s), <ea>(e) SAR.BWLQ(z) imm6(i), <ea>(e)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

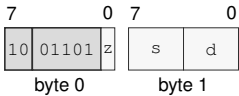
Detailed Semantics

Shifts the selected-width destination value right by the effective count, sign-fills the vacated high bits, and writes the result to the destination. The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the selected-width result equals the original selected-width value; an Rn destination is still written and a sub-Q result is sign-extended to 64 bits, while a memory destination rewrites the same selected bytes. FLAGS are unchanged for all counts.

Encodings:

SAR.LQ(z) Rn(s), Rn(d)

Instruction Format:



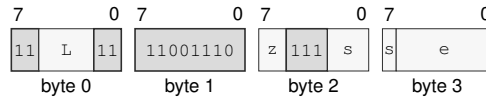
Format — Instruction format for SAR.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

SAR.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for SAR.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

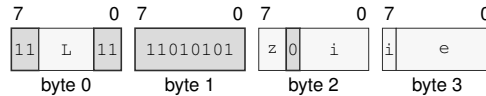
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SAR.BWLQ(z) imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for SAR.BWLQ(z) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SAVE

Save Processor State

SAVE

Operation: Saves processor state to a CPUID-defined save area.

Assembler Syntax: `SAVE <ea>(e)`
`SAVE Rn(r)`

Detailed Semantics

SAVE writes base architectural state and modified enabled extension state to a 4-KiB-aligned, CPUID-defined save area.

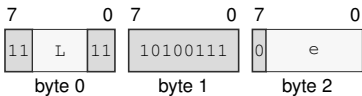
SAVE treats its address-role EA as the base of the save area without first reading an EA-sized operand. It writes the fixed base block, FMT=0, GS_VALID=0x3f, the state-block bitmap, R0–R15, GS0–GS5, FLAGS, STATUS including EDEPTH and UO, and the hidden current-DFA bit according to the Processor-State Save and Restore layout. Enabled modified extension components are written to their CPUID-defined offsets; a component known to equal its initial state may be omitted and reported clear in the bitmap. SAVE does not change a component’s clean or modified state.

The complete SAVE_AREA_SIZE_BYTES range reported by CPUID is validated before any byte is written. Any address or access fault leaves the architectural register state and save area unchanged.

Encodings:

SAVE <ea>(e)

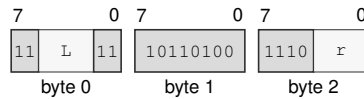
Instruction Format:



Format — Instruction format for SAVE <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field e**—Specifies the address operand.

SAVE R_n(r)**Instruction Format:****Format — Instruction format for SAVE R_n(r)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field r—Selects the operand.

SBB**Subtract With Borrow****SBB**

Operation: Performs the subtract with borrow operation.

Assembler Syntax: `SBB.BWLQ(z) Rn(s), Rn(d)`
`SBB.BWLQ(z) Rn(s), <ea>(e)`
`SBB.BWLQ(z) <ea>(e), Rn(d)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Subtracts the selected-width source value and incoming C borrow from the destination, writes the truncated difference, and updates Z, N, C, and V. REPcc observes the written destination result rather than architectural FLAGS.

Let n be the selected width, d and s the unsigned operand bit patterns, and b the incoming borrow represented by the C flag. The written result is

$$r = (d - s - b) \bmod 2^n.$$

Z is set when $r = 0$, and N receives the most-significant bit of r . C reports an unsigned borrow from either subtraction. V reports signed overflow for the complete $d - s - b$ operation.

FLAGS Effects

Z	N	C	V
a	b	c	d

a: `result == 0`.

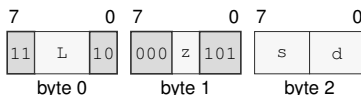
b: `most_significant_bit(result)`.

c: unsigned borrow.

d: signed overflow.

Encodings:

SBB.BWLQ(z) Rn(s), Rn(d)

Instruction Format:**Format — Instruction format for SBB.BWLQ(z) Rn(s), Rn(d)****Instruction Fields:**

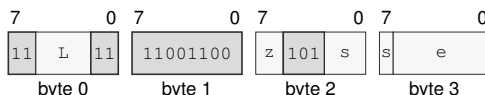
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SBB.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:**Format — Instruction format for SBB.BWLQ(z) Rn(s), <ea>(e)****Instruction Fields:**

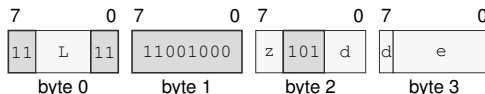
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SBB.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:**Format — Instruction format for SBB.BWLQ(z) <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

SEGLEA

Segment Load Effective Address

SEGLEA

- Operation:
- Performs the segment load effective address operation.
- Assembler Syntax:
- SEGLEA.BWLQ(z) Rn(s), Rn(d)
SEGLEA.BWLQ(z) <ea>(e), Rn(d)
- Repeat eligibility:
- Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

SEGLEA computes the source effective-address point and applies the selected segment's pre-translation to that point. The size suffix selects the index scale used during effective-address calculation. On success, SEGLEA writes the segment result to the destination and clears FLAGS. A failed segment-bound check sets V while clearing Z, N, and C, suppresses the destination write, commits effective-address auto-update effects, and completes without an exception. REPcc observes the failure result as a B-sized 0 or 1 rather than architectural FLAGS.

Let a be the source effective address before a memory read. The effective-address form selects the segment image, which supplies the *base*, *span*, and *limit* quantities defined in Segment Registers. A zero mantissa disables bounds and returns a . In translated mode, a must satisfy $0 \leq a < span$ and the result is $base + a$. In bounds-only mode, a must satisfy $base \leq a < limit$ and the result remains a . An out-of-bounds address writes FLAGS as Z=N=C=0 and V=1, in which case the destination is not written. A successful address writes FLAGS as Z=N=C=V=0.

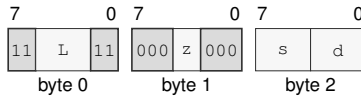
FLAGS Effects

Z	N	C	V
0	0	0	*

- 0: writes zero.
- *: cleared on success; set when the segment-point bounds check fails.

Encodings:

SEGLEA.BWLQ(z) Rn(s), Rn(d)

Instruction Format:**Format — Instruction format for SEGLEA.BWLQ(z) Rn(s), Rn(d)****Instruction Fields:**

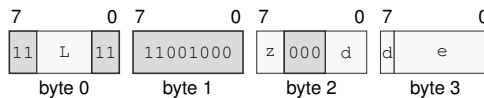
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SEGLEA.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:**Format — Instruction format for SEGLEA.BWLQ(z) <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the address operand.

SET

Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.
----------------------------	---

Writes integer one to the selected-width destination.

SET R_n(r)

Diagram illustrating the bit layout of two bytes:

- byte 0:** Bit 7 is 1, bits 6 to 0 are 000000.
- byte 1:** Bits 7 to 4 are 0000, bit 0 is r .

Format — Instruction format for SET Rn(r)

Register field r —Selects the operand.

SETcc**Set Conditional****SETcc**

Operation: Performs the set conditional operation.

Assembler Syntax: SETcc Rn(r)

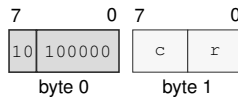
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Writes one when the encoded condition is true and zero when it is false.

Encodings:

SETcc Rn(r)

Instruction Format:

Format — Instruction format for SETcc Rn(r)

Instruction Fields:

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field r—Selects the operand.

SETF

Set Flags Image

SETF

Operation: Replaces integer FLAGS from an immediate bitmap.

Assembler Syntax: SETF flags_bitmap(m)

Detailed Semantics

SETF replaces the complete architectural integer FLAGS image from flags_bitmap. Bit 3 writes Z, bit 2 writes N, bit 1 writes C, and bit 0 writes V; a one writes the corresponding flag to one and a zero writes it to zero. The assembler may provide symbolic bitmap names such as Z, N, C, V, and combinations using |.

FLAGS Effects

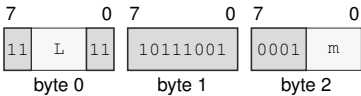
Z	N	C	V
a	b	c	d

- a: mask bit 3.
- b: mask bit 2.
- c: mask bit 1.
- d: mask bit 0.

Encodings:

SETF flags_bitmap(m)

Instruction Format:



Format — Instruction format for SETF flags_bitmap(m)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Immediate field** m—Encodes the immediate value.

SHL**Shift Left****SHL**

Operation: Performs the shift left operation.

Assembler Syntax: `SHL.LQ(z) Rn(s), Rn(d)`
`SHL.BWLQ(z) Rn(s), <ea>(e)`
`SHL.BWLQ(z) imm6(i), <ea>(e)`

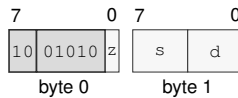
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Shifts the selected-width destination value left by the effective count, zero-fills the vacated low bits, and writes the result to the destination. The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the selected-width result equals the original selected-width value; an Rn destination is still written and a sub-Q result is zero-extended to 64 bits, while a memory destination rewrites the same selected bytes. FLAGS are unchanged for all counts.

Encodings:

`SHL.LQ(z) Rn(s), Rn(d)`

Instruction Format:

Format — Instruction format for SHL.LQ(z) Rn(s), Rn(d)

Instruction Fields:

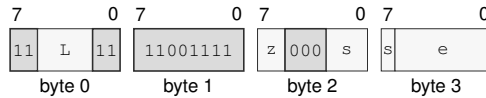
Size field z—Selects L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SHL.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for SHL.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

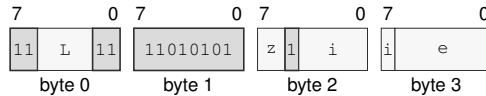
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SHL.BWLQ(z) imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for SHL.BWLQ(z) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SHR

Shift Right

SHR

Operation: Performs the shift right operation.

Assembler Syntax: `SHR.LQ(z) Rn(s), Rn(d)`
`SHR.BWLQ(z) Rn(s), <ea>(e)`
`SHR.BWLQ(z) imm6(i), <ea>(e)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

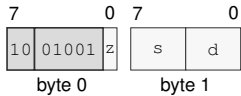
Detailed Semantics

Shifts the selected-width destination value right by the effective count, zero-fills the vacated high bits, and writes the result to the destination. The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the selected-width result equals the original selected-width value; an Rn destination is still written and a sub-Q result is zero-extended to 64 bits, while a memory destination rewrites the same selected bytes. FLAGS are unchanged for all counts.

Encodings:

`SHR.LQ(z) Rn(s), Rn(d)`

Instruction Format:



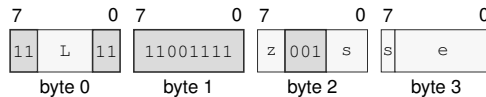
Format — Instruction format for SHR.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z**—Selects L/Q.
- Register field s**—Selects the source operand.
- Register field d**—Selects the destination operand.

SHR.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for SHR.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

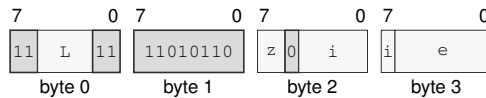
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SHR.BWLQ(z) imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for SHR.BWLQ(z) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SUB**Subtract****SUB**

Operation: Subtracts the source operand from the destination operand.

Assembler Syntax: SUB.Q 8, SP
 SUB.LQ(z) Rn(s), Rn(d)
 SUB.Q imm8(i), SP
 SUB.Q <imm16s>, SP
 SUB.Q <imm32s>, SP
 SUB.BWLQ(z) Rn(s), <ea>(e)
 SUB.BWLQ(z) <ea>(e), Rn(d)
 SUB.Q <imm64>, <ea>(e)
 SUB.BWLQ(z) <imm8s>, <ea>(e)
 SUB.WLQ(z) <imm16s>, <ea>(e)
 SUB.LQ(z) <imm32s>, <ea>(e)

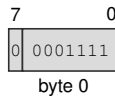
Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

Detailed Semantics

Performs selected-width modular subtraction of the source from the destination and writes the low result bits back to the destination.

Encodings:

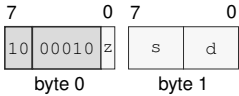
SUB.Q 8, SP

Instruction Format:

Format — Instruction format for SUB.Q 8, SP

SUB.LQ(z) Rn(s), Rn(d)

Instruction Format:



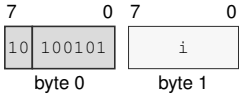
Format — Instruction format for SUB.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

SUB.Q imm8(i), SP

Instruction Format:



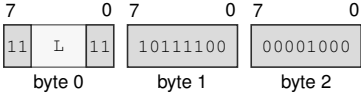
Format — Instruction format for SUB.Q imm8(i), SP

Instruction Fields:

- Immediate field i—Encodes the immediate value. Allowed values: 1-255.

SUB.Q <imm16s>, SP

Instruction Format:

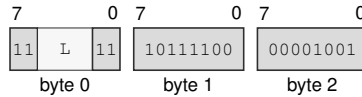


Format — Instruction format for SUB.Q <imm16s>, SP

Instruction Fields:

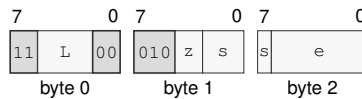
- Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

SUB.Q <imm32s>, SP

Instruction Format:**Format — Instruction format for SUB.Q <imm32s>, SP****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

SUB.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:**Format — Instruction format for SUB.BWLQ(z) Rn(s), <ea>(e)****Instruction Fields:**

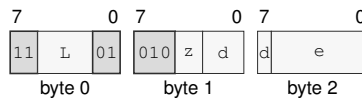
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SUB.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:**Format — Instruction format for SUB.BWLQ(z) <ea>(e), Rn(d)****Instruction Fields:**

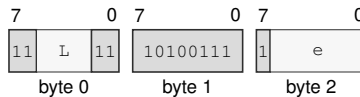
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

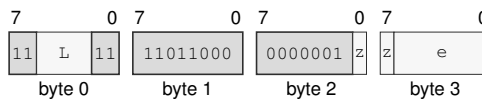
SUB.Q <imm64>, <ea>(e)

Instruction Format:**Format — Instruction format for SUB.Q <imm64>, <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SUB.BWLQ(z) <imm8s>, <ea>(e)

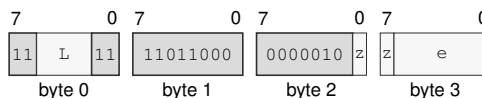
Instruction Format:**Format — Instruction format for SUB.BWLQ(z) <imm8s>, <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SUB.WLQ(z) <imm16s>, <ea>(e)

Instruction Format:**Format — Instruction format for SUB.WLQ(z) <imm16s>, <ea>(e)****Instruction Fields:**

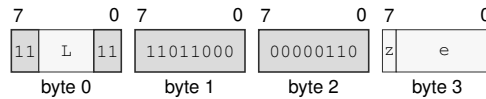
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SUB.LQ(z) <imm32s>, <ea>(e)

Instruction Format:



Format — Instruction format for SUB.LQ(z) <imm32s>, <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

SWPT

Switch Page Table

SWPT

Operation: Performs the switch page table operation.

Assembler Syntax: SWPT Rn(p)

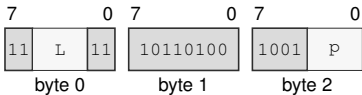
Detailed Semantics

Replaces PTCR with the supplied page-table control value and clears ASCR as one architectural page-table switch.

Encodings:

SWPT Rn(p)

Instruction Format:



Format — Instruction format for SWPT Rn(p)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** p—Selects the operand.

SWPTA

Switch Page Table With Context

SWPTA

Operation: Switches the page table and enables address-space context matching.

Assembler Syntax: SWPTA Rn(p), Rn(a)

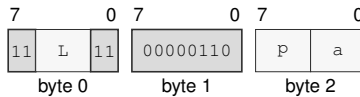
Detailed Semantics

Updates PTCR from the first Rn register, writes the low 16 bits of the second Rn register into ASCR[31:16], and sets ASCR.AE.

Encodings:

SWPTA Rn(p), Rn(a)

Instruction Format:



Format — Instruction format for SWPTA Rn(p), Rn(a)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field p—Selects the source operand.

Register field a—Selects the destination operand.

SYNCCACHE

Synchronize Caches

SYNCCACHE

Operation: Synchronize one cache-maintenance block for local instruction fetch.

Assembler Syntax: SYNCCACHE <ea>(e)

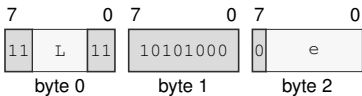
Detailed Semantics

Computes and translates the address-role EA, writes dirty data for its CPUID-sized maintenance block into normal-memory coherence throughout the coherence domain, invalidates the executing logical processor’s matching instruction, decoded, and prefetch state, and serializes later instruction fetch.

Encodings:

SYNCCACHE <ea>(e)

Instruction Format:



Format — Instruction format for SYNCCACHE <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field e**—Specifies the address operand.

SYSCALL**System Call****SYSCALL**

Operation:	Raise the SYSTEM_CALL architectural event.
Assembler Syntax:	SYSCALL
Exceptions:	INVALID_CONTROL_STATE: when executed outside user mode or while architectural event state is active

Detailed Semantics

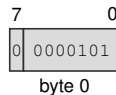
Raise the explicit synchronous SYSTEM_CALL event (EXC:0x20), saving the following instruction as UPC and entering the common EPC/ECS/EDS event handler on SSS:SSP without a memory frame.

Execution is valid only in user mode with no active architectural event. It selects the explicit synchronous SYSTEM_CALL event, whose event code is EXC:0x20, priority is 4, frame class is BASIC, and saved PC is the following instruction.

On successful entry the common event mechanism atomically saves the user continuation in UPC, USP, UCS, UDS, USS, UCTL, and UINFO; loads CS, DS, and PC from ECS, EDS, and EPC; selects SSS:SSP; and sets STATUS.PM, STATUS.EA, STATUS.EDEPTH=1, and STATUS.UO. Because SYSTEM_CALL has no payload, entry performs no stack-memory access. Return uses ERET. Software restarts the one-byte instruction by subtracting one from UPC before ERET.

Encodings:

SYSCALL

Instruction Format:**Format — Instruction format for SYSCALL**

TEST

Test

TEST

Operation:	Computes a bitwise conjunction and updates condition codes without storing the temporary value.
Assembler Syntax:	TEST.LQ(z) Rn(s), Rn(d) TEST.BWLQ(z) <ea>(e), Rn(d) TEST.BWLQ(z) Rn(s), <ea>(e)
Repeat eligibility:	Eligible as a REP or REPcc body. REPcc observes the operation-defined completion value.

Detailed Semantics

Computes the selected-width bitwise conjunction only to set Z and N, clears C and V, and does not write the temporary result. REPcc observes that temporary conjunction rather than architectural FLAGS.

FLAGS Effects

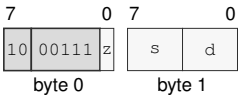
Z	N	C	V
a	b	0	0

- a: result == 0.
- b: most_significant_bit(result).
- 0: writes zero.

Encodings:

TEST.LQ(z) Rn(s), Rn(d)

Instruction Format:



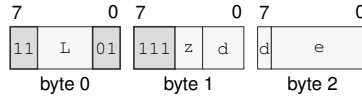
Format — Instruction format for TEST.LQ(z) Rn(s), Rn(d)

Instruction Fields:

- Size field z—Selects L/Q.
- Register field s—Selects the source operand.
- Register field d—Selects the destination operand.

TEST.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for TEST.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

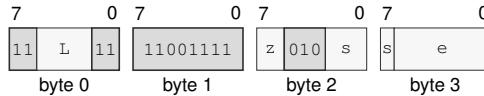
Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

TEST.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for TEST.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the source operand.

TESTJcc

Test And Jump Conditional

TESTJcc

Operation: Tests two Rn registers and branches on the selected condition without writing FLAGS.

Assembler Syntax: TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>
TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>

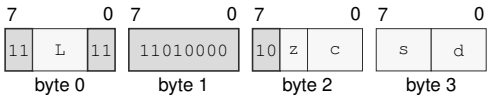
Detailed Semantics

TESTJcc computes the same temporary logical-test result as TEST for the selected operand size, evaluates the encoded condition from that temporary result, and branches to the relative target when the condition is true. The computed condition is consumed only by the branch decision; architectural FLAGS are not read or written. Conditions T and F are reserved and are not valid encodings.

Encodings:

TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>

Instruction Format:



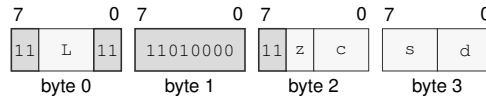
Format — Instruction format for TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm8s>

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Condition field** c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.
- Register field** s—Selects operand 1.
- Register field** d—Selects operand 2.

TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>

Instruction Format:



Format — Instruction format for TESTJcc.BWLQ(z) Rn(s), Rn(d), <imm16s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field s—Selects operand 1.

Register field d—Selects operand 2.

TRACE

Trace Stream Marker

TRACE

Operation: Records a marker and instruction boundary in the implementation trace stream.

Assembler Syntax: TRACE <imm16>

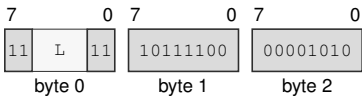
Detailed Semantics

When the implementation trace stream is enabled, appends the low 16-bit marker and current instruction boundary to that stream; otherwise it completes without changing architectural state. The tracing terminology distinguishes this marker instruction from a TF-selected trace unit and the DEBUG_TRACE architectural event.

Encodings:

TRACE <imm16>

Instruction Format:



Format — Instruction format for TRACE <imm16>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

VTOP**Virtual To Physical Query****VTOP**

Operation: Non-destructively queries the current translation of a linear address.

Assembler Syntax: `VTOP Rn(v), Rn(p)`

Detailed Semantics

With paging disabled, VTOP returns the supplied address when it fits implementation PABITS. With paging enabled, it returns the current physical byte-address translation without accessing the translated target. Z reports success; N, C, and V are zero. The query does not modify A or D.

The source address is checked for canonicity before the page-table walk. The walk begins at the PTCR root and follows the active LA48 or LA57 hierarchy using the virtual page number.

Every continuing entry must be a valid table pointer. R, W, X, and U are intersected across table pointers and the terminating leaf AM. A valid leaf at level L combines its naturally aligned physical base with the original low $12 + 9(L - 1)$ address bits to form a byte address. Success sets Z and clears N, C, and V. Any canonicity, PABITS, or walk failure returns zero and clears all four flags without raising an access exception, setting PTE A or D, or modifying the TLB.

FLAGS Effects

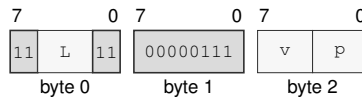
Z	N	C	V
*	0	0	0

*: translation query succeeded.

0: writes zero.

Encodings:

`VTOP Rn(v), Rn(p)`

Instruction Format:

Format — Instruction format for VTOP Rn(v), Rn(p)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field v—Selects the source operand.

Register field p—Selects the destination operand.

WAIT

Wait

WAIT

Operation: Permits a finite implementation-selected delay, including zero.

Assembler Syntax: WAIT

Detailed Semantics

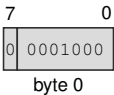
Retires normally with the next instruction as its continuation and permits a finite implementation-selected delay, including zero, before subsequent execution.

WAIT is a PAUSE-like execution hint. It retires as an ordinary instruction with the following instruction as its continuation. The implementation selects a finite delay before subsequent execution; the selected delay may be zero. The architectural result of WAIT is normal retirement to the recorded continuation.

Encodings:

WAIT

Instruction Format:



Format — Instruction format for WAIT

WFENCE

Write Fence

WFENCE

Operation: Order prior writes before later writes.

Assembler Syntax: WFENCE

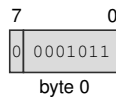
Detailed Semantics

Prevents retirement until every earlier memory write is ordered before every later memory write on the same logical processor. It performs no memory access and changes no register or flag.

Encodings:

WFENCE

Instruction Format:



Format — Instruction format for WFENCE

WRBKDCACHE

Write Back Data Cache

WRBKDCACHE

Operation: Write back one data-cache maintenance block.

Assembler Syntax: WRBKDCACHE <ea>(e)

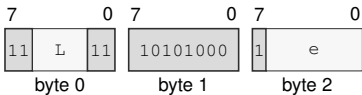
Detailed Semantics

Computes and translates the address-role EA without reading an operand value, selects its CPUID-sized maintenance block, writes dirty bytes from every data or unified cache copy in the coherence domain into normal-memory coherence, retains clean copies, and retires after the block is complete.

Encodings:

WRBKDCACHE <ea>(e)

Instruction Format:



Format — Instruction format for WRBKDCACHE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.

WRCR**Write Control Register****WRCR**

Operation: Write a privileged control register from an Rn register.

Assembler Syntax: WRCR Rn(s), <imm16>

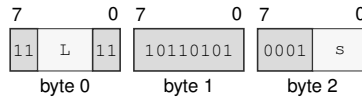
Exceptions: INVALID_CONTROL_STATE: when the selector, reserved bits, image, or requested transition fails the WRCR selector rule

Detailed Semantics

WRCR validates and atomically writes the selected control register in supervisor mode. The WRCR Selector Rules table defines each writable image, validation condition, and translation or event-state side effect.

Encodings:

WRCR Rn(s), <imm16>

Instruction Format:

Format — Instruction format for WRCR Rn(s), <imm16>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

WRFLAGS

Write Flags

WRFLAGS

Operation: Write the integer condition-code flags from an Rn register.

Assembler Syntax: WRFLAGS Rn(s)

Detailed Semantics

WRFLAGS writes the low 16 bits from the source Rn register into the architectural FLAGS register. Reserved FLAGS bits must be zero.

FLAGS Effects

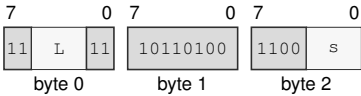
z	n	c	v
a	b	c	d

- a: source bit 3.
- b: source bit 2.
- c: source bit 1.
- d: source bit 0.

Encodings:

WRFLAGS Rn(s)

Instruction Format:



Format — Instruction format for WRFLAGS Rn(s)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the operand.

WRSEG**Write Segment Register****WRSEG**

Operation: Write an SREG segment image from an Rn register.

Assembler Syntax: WRSEG Rn(d) , SREG(s)

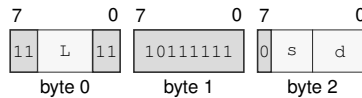
Detailed Semantics

WRSEG validates a 64-bit segment register image from the source Rn register and writes it to the selected SREG.

The complete source image is validated before architectural state changes. An invalid image raises `INVALID_CONTROL_STATE`; the selected segment register remains unchanged when validation fails.

Encodings:

WRSEG Rn(d) , SREG(s)

Instruction Format:

Format — Instruction format for WRSEG Rn(d), SREG(s)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the destination operand using the SREG encoding.

Register field d—Selects the source operand.

WRSTATUS

Write Status

WRSTATUS

Operation:	Write the processor STATUS register from an Rn register.
Assembler Syntax:	WRSTATUS Rn(s)
Exceptions:	INVALID_CONTROL_STATE: when reserved bits are nonzero or a supplied hardware-managed PM, EA, EDEPTH, or UO field differs from the current value

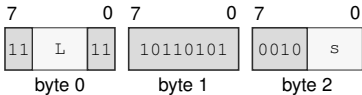
Detailed Semantics

WRSTATUS validates the low 16 bits from the source Rn register and atomically updates IE, NI, TF, and RF. Reserved bits must be zero, and the supplied PM, EA, EDEPTH, and UO values must equal the current values. Clearing NI with a pending NMI admits that NMI at the next instruction boundary.

Encodings:

WRSTATUS Rn(s)

Instruction Format:



Format — Instruction format for WRSTATUS Rn(s)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field s**—Selects the operand.

XCHG**Exchange****XCHG**

Operation: Performs the exchange operation.

Assembler Syntax: XCHG.LQ(z) Rn(s), Rn(d)
 XCHG.BWLQ(z) Rn(s), <ea>(e)
 XCHG.BWLQ(z) <ea>(e), Rn(d)

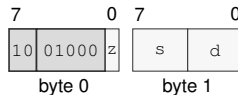
Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Exchanges the two selected-width operand values as one instruction. A memory form performs an ordinary selected-width load followed by an ordinary selected-width store, with both architectural destinations committed by the instruction.

Encodings:

XCHG.LQ(z) Rn(s), Rn(d)

Instruction Format:

Format — Instruction format for XCHG.LQ(z) Rn(s), Rn(d)

Instruction Fields:

Size field z—Selects L/Q.

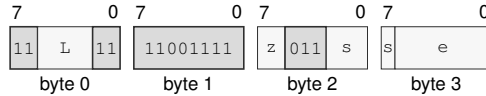
Register field s—Selects the source operand.

Register field d—Selects the destination operand.

Destination overlap—When the lhs and rhs operands designate the same architectural register, the final value equals that register's initial value.

XCHG.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for XCHG.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

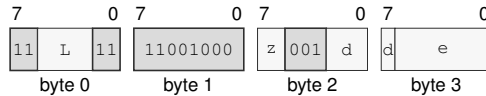
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XCHG.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for XCHG.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XOR

Bitwise Xor

XOR

Operation: Computes the bitwise XOR of the source and destination operands.

Assembler Syntax: `XOR.LQ(z) Rn(s), Rn(d)`
`XOR.BWLQ(z) Rn(s), <ea>(e)`
`XOR.BWLQ(z) <ea>(e), Rn(d)`
`XOR.Q <imm64>, <ea>(e)`
`XOR.BWLQ(z) <imm8s>, <ea>(e)`
`XOR.WLQ(z) <imm16s>, <ea>(e)`
`XOR.LQ(z) <imm32s>, <ea>(e)`

Repeat eligibility: Eligible as a REP or REPcc body. REPcc observes the value written to the destination operand.

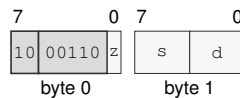
Detailed Semantics

Writes the selected-width bitwise exclusive-or of the source and destination values to the destination.

Encodings:

`XOR.LQ(z) Rn(s), Rn(d)`

Instruction Format:



Format — Instruction format for XOR.LQ(z) Rn(s), Rn(d)

Instruction Fields:

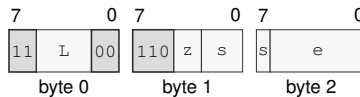
Size field *z*—Selects L/Q.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

XOR.BWLQ(z) Rn(s), <ea>(e)

Instruction Format:



Format — Instruction format for XOR.BWLQ(z) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

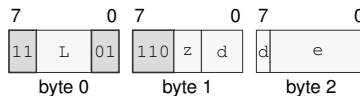
Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XOR.BWLQ(z) <ea>(e), Rn(d)

Instruction Format:



Format — Instruction format for XOR.BWLQ(z) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

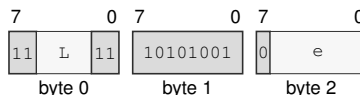
Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

XOR.Q <imm64>, <ea>(e)

Instruction Format:



Format — Instruction format for XOR.Q <imm64>, <ea>(e)

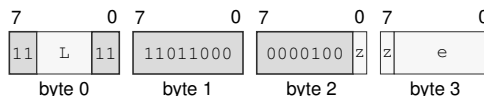
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XOR.BWLQ(z) <imm8s>, <ea>(e)

Instruction Format:



Format — Instruction format for XOR.BWLQ(z) <imm8s>, <ea>(e)

Instruction Fields:

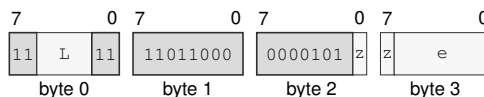
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XOR.WLQ(z) <imm16s>, <ea>(e)

Instruction Format:



Format — Instruction format for XOR.WLQ(z) <imm16s>, <ea>(e)

Instruction Fields:

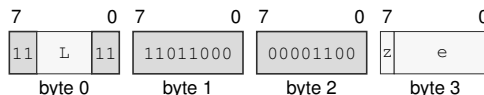
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

XOR.LQ(z) <imm32s>, <ea>(e)

Instruction Format:



Format — Instruction format for XOR.LQ(z) <imm32s>, <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

YIELD

Yield

YIELD

Operation: Hints that the current logical processor may be deprioritized.

Assembler Syntax: YIELD

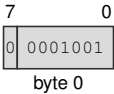
Detailed Semantics

Notifies the implementation that the current logical processor may be deprioritized or descheduled. YIELD affects scheduling only; memory accesses retain the ordinary instruction-ordering rules, and execution resumes at the next instruction.

Encodings:

YIELD

Instruction Format:



Format — Instruction format for YIELD

18 FLOATING-POINT INSTRUCTIONS

18.1 Common Floating-Point Semantics

The S and D formats are IEEE-754 binary32 and binary64. S uses sign bit 31, exponent bits 30..23, and fraction bits 22..0. D uses sign bit 63, exponent bits 62..52, and fraction bits 51..0. An all-ones exponent and nonzero fraction is a NaN. The most significant fraction bit distinguishes a quiet NaN from a signaling NaN: zero is signaling and one is quiet. The architectural default quiet NaNs are 0x7fc00000 for S and 0x7ff8000000000000 for D. Figure 5-4 shows both encodings in their 64-bit Fn register view.

An S operation reads bits 31..0 of each Fn operand. Writing an S result to Fn writes the result to bits 31..0 and writes zero to bits 63..32. A D operation reads and writes all 64 bits. Operations use the selected format for their intermediate and final values. FMADD, FMSUB, FNMADD, and FNMSUB form the complete multiply-add expression with unbounded intermediate range and precision and round only the final result. Other arithmetic operations round once after computing their exact mathematical result.

An encoded FEA source `immsf` is an exact IEEE 754 binary32 payload, and `immdf` is an exact binary64 payload. Either immediate may be used by either an S or D instruction. If the payload format differs from the operation format selected by the suffix, the source is converted to the operation format using the current `FSTATUS.RM` before the operation. All conversion causes, including causes from NaN handling and narrowing, are combined with the operation's causes. The combined cause set follows the same enabled-cause fault test, `FFLAGS` accrual, and all-or-nothing destination commit rules below. Floating-point immediate forms are source-only.

18.1.1 Input, NaN, Rounding, and Result Order

Each ordinary floating-point operation applies the following order.

1. Read all operands and classify zero, subnormal, normal, infinity, quiet NaN, and signaling NaN.
2. If `FSTATUS.DAZ` is set, replace each subnormal input with a zero having the same sign.
3. Determine signaling-NaN and operation-specific floating-point exception causes and form the exact mathematical result.
4. For a NaN result, select the first signaling NaN in source-operand order, or the first quiet NaN when there is no signaling NaN, and set its quiet bit. If the operation produces a NaN without a NaN operand, use the default quiet NaN. When `FSTATUS.DN` is set, replace the selected NaN with the default quiet NaN.
5. Round a numeric result to the selected format using `FSTATUS.RM`, except where an instruction names a fixed rounding direction.
6. Detect overflow and tininess after rounding. UF is generated when the rounded result is tiny and inexact.
7. If `FSTATUS.FTZ` is set and the rounded result is a nonzero subnormal, replace it with a zero of the same sign and generate both UF and NX.

8. Test all generated causes against their FSTATUS enable bits and then perform the architectural commit.

A signaling NaN generates NV. A quiet NaN does not generate NV unless the operation itself is invalid. When two NaN operands have the same priority, source-operand order determines the selected sign and payload. Quieting sets the quiet bit and preserves the remaining selected sign and payload bits.

Overflow generates OF and NX. Nearest-even produces signed infinity. Toward zero produces the signed maximum finite value. Toward positive produces positive infinity for a positive result and the negative maximum finite value for a negative result. Toward negative produces negative infinity for a negative result and the positive maximum finite value for a positive result.

18.1.2 Floating-Point Exception Causes and Commit

Cause	Generated when
NV	an input is a signaling NaN; an arithmetic operation has no defined numeric result, including infinity minus the same infinity, zero times infinity, zero divided by zero, infinity divided by infinity, or square root of a negative nonzero value; or a floating-to-integer conversion is invalid
DZ	a finite nonzero value is divided by zero
OF	the rounded numeric magnitude exceeds the selected format's finite range
UF	the result is tiny after rounding and inexact, or FTZ flushes a nonzero subnormal result
NX	rounding or a valid conversion changes the exact value, or OF or FTZ generates NX

If any generated floating-point exception cause has its matching FSTATUS enable bit set, the instruction raises the `FLOATING_POINT_FAULT` architectural event. Its error-code cause bitmap contains every cause generated by the operation. The destination, integer `FLAGS`, and `FFLAGS` remain unchanged, and arithmetic instructions never modify `FSTATUS`.

If no generated floating-point exception cause is enabled, the instruction commits its destination and any complete integer `FLAGS` image defined by the instruction, and ORs every generated cause into `FFLAGS`. `FFLAGS` fields not named by the instruction remain unchanged. An instruction-specific rule may exclude a cause, as the `FPTRANSA` contracts exclude `NX`.

18.1.3 Comparison, Minimum, and Maximum

`FCMP` and `FTEST` write `Z`, `N`, `C`, `V` as follows.

Relation	Z	N	C	V
greater	0	0	0	0
equal	1	0	0	0
less	0	1	1	0
unordered	0	0	0	1

A quiet NaN produces `unordered` without `NV`. A signaling NaN produces `unordered` and `NV`. `FTEST` compares its operand with positive zero under these rules.

For FMIN and FMAX, when exactly one operand is a NaN the numeric operand is the result. When both operands are NaNs, the common NaN-selection rule supplies the result. A signaling NaN generates NV even when the other operand is numeric. Between positive and negative zero, FMIN selects negative zero and FMAX selects positive zero.

18.1.4 Conversions

For Fn-to-Fn FCVT or FCVTU, the .S encoding converts D to S and the .D encoding converts S to D. The two mnemonics have identical behavior for this conversion family. Rn-to-Fn FCVT interprets all 64 source bits as a signed integer; FCVTU interprets them as an unsigned integer. Integer-to-floating conversion uses FSTATUS.RM.

Fn-to-Rn conversion truncates toward zero. FCVT produces a signed 64-bit result and FCVTU produces an unsigned 64-bit result. A valid conversion that discards a fractional part generates NX. An invalid conversion generates NV without OF or NX. When NV is not enabled, the committed invalid result is:

Instruction	Input	Result
FCVT	negative overflow or negative infinity	0x8000000000000000
FCVT	positive overflow or positive infinity	0x7fffffffffffffff
FCVT	NaN	0x0000000000000000
FCVTU	negative value or negative infinity	0x0000000000000000
FCVTU	positive overflow or positive infinity	0xfffffffffffffff
FCVTU	NaN	0x0000000000000000

18.2 Summary

Table 18-1. Floating-Point Instructions Summary (Informative)

Mnemonic	Brief description
FABS	Performs the floating-point absolute value operation.
FADD	Adds the source operand to the destination operand.
FBNDII	Checks floating-point value membership in the inclusive-inclusive interval [lo, hi].
FBNDIX	Checks floating-point value membership in the inclusive-exclusive interval [lo, hi).
FBNDXI	Checks floating-point value membership in the exclusive-inclusive interval (lo, hi].
FBNDXX	Checks floating-point value membership in the exclusive-exclusive interval (lo, hi).
FCEIL	Performs the floating-point ceiling operation.
FCLASS	Performs the floating-point classify operation.
FCLR	Performs the floating-point clear operation.
FCMP	Performs the floating-point compare operation.
FCOPYSIGN	Performs the floating-point copy sign operation.
FCVT	Performs the floating-point convert signed operation.
FCVTU	Performs the floating-point convert unsigned operation.
FDIV	Performs the floating-point divide operation.
FFLOOR	Performs the floating-point floor operation.
FGETEXP	Performs the floating-point get exponent operation.
FGETMAN	Performs the floating-point get mantissa operation.

Table 18-1 (continued)

Mnemonic	Brief description
FINT	Performs the floating-point integral value operation.
FINTRZ	Performs the floating-point integral toward zero operation.
FMADD	Performs the floating-point fused multiply add operation.
FMAX	Performs the floating-point maximum operation.
FMIN	Performs the floating-point minimum operation.
FMOD	Performs the floating-point modulo operation.
FMOV	Copies the source operand to the destination operand.
FMOVcc	Performs the floating-point conditional move operation.
FMOVCR	Loads a named architectural binary64 constant selected by an unsigned 16-bit ID.
FMSUB	Performs the floating-point fused multiply subtract operation.
FMUL	Performs the floating-point multiply operation.
FNEG	Performs the floating-point negate operation.
FNMADD	Performs the floating-point fused negative multiply add operation.
FNMSUB	Performs the floating-point fused negative multiply subtract operation.
FPOPP	Pops one canonical floating-point register pair selected by an inline pair index.
FPUSHP	Pushes one canonical floating-point register pair selected by an inline pair index.
FREM	Performs the floating-point remainder operation.
FROUND	Performs the floating-point round operation.
FSCALE	Performs the floating-point scale operation.
FSQRT	Performs the floating-point square root operation.
FSUB	Subtracts the source operand from the destination operand.
FTEST	Performs the floating-point test operation.
FTRUNC	Performs the floating-point truncate operation.
FXCHG	Performs the floating-point exchange operation.
RDFFLAGS	Read the accrued floating-point exception flags into an Rn register.
RDFSTATUS	Read the FPU status/control register into an Rn register.
WRFFLAGS	Write the accrued floating-point exception flags from an Rn register.
WRFSTATUS	Write the FPU status/control register from an Rn register.

FABS**Floating-Point Absolute Value****FABS**

Operation: Performs the floating-point absolute value operation.

Assembler Syntax: `FABS.SD(z) Fn(s), Fn(d)`
`FABS.SD(z) <ea>(e), Fn(d)`
`FABS.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Clears the sign of the selected floating-point source value and writes the resulting magnitude to the destination.

FFLAGS Effects

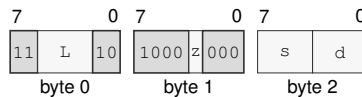
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual from FEA source conversion.

-: preserved.

Encodings:

`FABS.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FABS.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

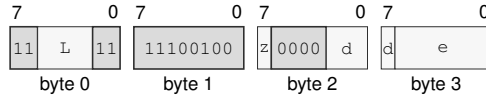
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FABS.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FABS.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

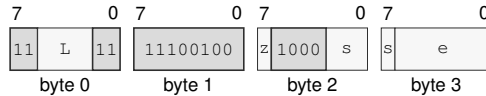
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FABS.SD(z) Fn(s), <ea>(e)

Instruction Format:



Format — Instruction format for FABS.SD(z) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FADD**Floating-Point Add****FADD**

Operation:	Adds the source operand to the destination operand.
Assembler Syntax:	FADD.SD(z) Fn(s), Fn(d) FADD.SD(z) <ea>(e), Fn(d)
Required CPUID flags:	FP
Repeat eligibility:	Eligible as a REP body.
Exceptions:	FLOATING_POINT_FAULT: when a generated floating-point exception is enabled

Detailed Semantics

FADD adds the selected-format floating-point source to the destination and rounds the result according to the floating-point environment. A completed operation writes the rounded value to the destination and accrues any generated NV, OF, UF, and NX causes in FFLAGS; DZ is unchanged.

If a generated floating-point exception is enabled, FADD raises FLOATING_POINT_FAULT. The faulting operation commits neither its destination write nor its accrued FFLAGS causes.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	*

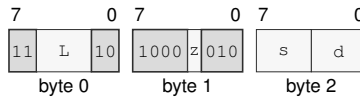
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FADD.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FADD.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

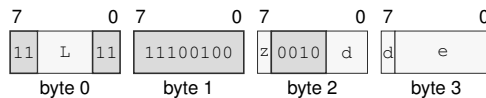
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FADD.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FADD.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

FBNDII**Floating-Point Bounds Check Inclusive-Inclusive****FBNDII**

Operation: Checks floating-point value membership in the inclusive-inclusive interval [lo, hi].

Assembler Syntax: FBNDII.SD(z) Fn(l), Fn(v), Fn(h)
 FBNDII.SD(z) Fn(l), <ea>(v), Fn(h)
 FBNDII.SD(z) <ea>(l), Fn(v), <ea>(h)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

FBNDII treats both bounds as inclusive. It sets V when the value is unordered or outside [lo, hi] and clears Z, N, and C. IEEE-754 FFLAGS causes continue to accrue independently.

FLAGS Effects

Z	N	C	V
0	0	0	*

0: writes zero.

*: set when the value is out of bounds or unordered.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

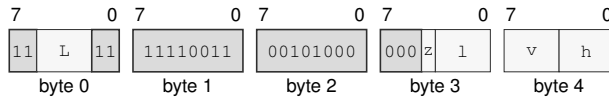
-: preserved.

b: accrual from FEA source conversion.

Encodings:

FBNDII.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)

Instruction Format:



Format — Instruction format for FBNDII.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

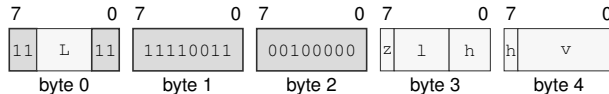
Register field *l*—Selects operand 1.

Register field *v*—Selects operand 2.

Register field *h*—Selects operand 3.

FBNDII.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)

Instruction Format:



Format — Instruction format for FBNDII.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

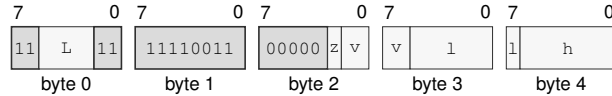
Register field *l*—Selects operand 1.

Register field *h*—Selects operand 3.

Effective Address field *v*—Specifies the source operand.

FBNDII.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Format:



Format — Instruction format for FBNDII.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *v*—Selects operand 2.

Effective Address field *l*—Specifies the source operand.

Effective Address field *h*—Specifies the source operand.

FBNDIX

Floating-Point Bounds Check Inclusive-Exclusive

FBNDIX

- Operation:

Checks floating-point value membership in the inclusive-exclusive interval [lo, hi).
- Assembler Syntax:

FBNDIX.SD(z) Fn(l), Fn(v), Fn(h)
FBNDIX.SD(z) Fn(l), <ea>(v), Fn(h)
FBNDIX.SD(z) <ea>(l), Fn(v), <ea>(h)
- Required CPUID flags:

FP
- Repeat eligibility:

Eligible as a REP body.

Detailed Semantics

FBNDIX treats the low bound as inclusive and the high bound as exclusive. It sets V when the value is unordered or outside [lo, hi) and clears Z, N, and C. IEEE-754 FFLAGS causes continue to accrue independently.

FLAGS Effects

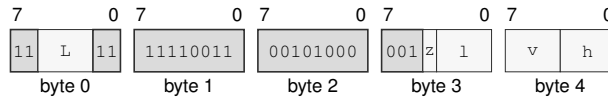
Z	N	C	V
0	0	0	*

- 0: writes zero.
- *: set when the value is out of bounds or unordered.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

- a: accrual of an operation-generated cause.
- : preserved.
- b: accrual from FEA source conversion.

Encodings:FBNDIX.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)**Instruction Format:****Format — Instruction format for FBNDIX.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)****Instruction Fields:**

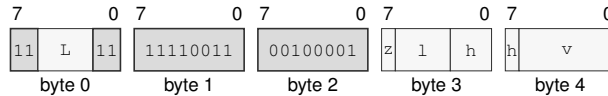
Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *l*—Selects operand 1.

Register field *v*—Selects operand 2.

Register field *h*—Selects operand 3.

FBNDIX.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)**Instruction Format:****Format — Instruction format for FBNDIX.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

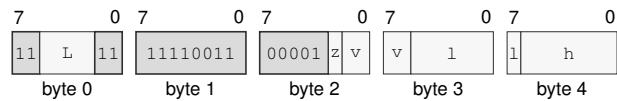
Register field *l*—Selects operand 1.

Register field *h*—Selects operand 3.

Effective Address field *v*—Specifies the source operand.

FBNDIX.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Format:



Format — Instruction format for FBNDIX.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Fields:

- Length field *L***—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field *z***—Selects S/D.
- Register field *v***—Selects operand 2.
- Effective Address field *l***—Specifies the source operand.
- Effective Address field *h***—Specifies the source operand.

FBNDXI**Floating-Point Bounds Check Exclusive-Inclusive****FBNDXI**

Operation: Checks floating-point value membership in the exclusive-inclusive interval [lo, hi].

Assembler Syntax: FBNDXI.SD(z) Fn(l), Fn(v), Fn(h)
 FBNDXI.SD(z) Fn(l), <ea>(v), Fn(h)
 FBNDXI.SD(z) <ea>(l), Fn(v), <ea>(h)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

FBNDXI treats the low bound as exclusive and the high bound as inclusive. It sets V when the value is unordered or outside [lo, hi] and clears Z, N, and C. IEEE-754 FFLAGS causes continue to accrue independently.

FLAGS Effects

Z	N	C	V
0	0	0	*

0: writes zero.

*: set when the value is out of bounds or unordered.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

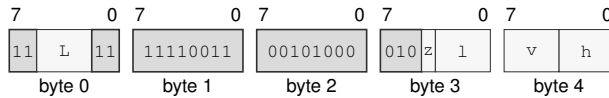
-: preserved.

b: accrual from FEA source conversion.

Encodings:

FBNDXI.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)

Instruction Format:



Format — Instruction format for FBNDXI.SD(*z*) Fn(*l*), Fn(*v*), Fn(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

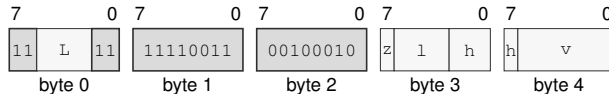
Register field *l*—Selects operand 1.

Register field *v*—Selects operand 2.

Register field *h*—Selects operand 3.

FBNDXI.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)

Instruction Format:



Format — Instruction format for FBNDXI.SD(*z*) Fn(*l*), <ea>(*v*), Fn(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

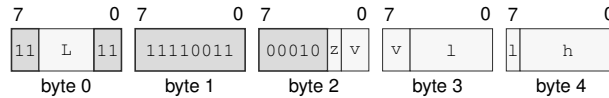
Register field *l*—Selects operand 1.

Register field *h*—Selects operand 3.

Effective Address field *v*—Specifies the source operand.

FBNDXI.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Format:



Format — Instruction format for FBNDXI.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *v*—Selects operand 2.

Effective Address field *l*—Specifies the source operand.

Effective Address field *h*—Specifies the source operand.

FBNDXX

Floating-Point Bounds Check Exclusive-Exclusive

FBNDXX

Operation: Checks floating-point value membership in the exclusive-exclusive interval (lo, hi).

Assembler Syntax: FBNDXX.SD(z) Fn(l), Fn(v), Fn(h)
FBNDXX.SD(z) Fn(l), <ea>(v), Fn(h)
FBNDXX.SD(z) <ea>(l), Fn(v), <ea>(h)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

FBNDXX treats both bounds as exclusive. It sets V when the value is unordered or outside (lo, hi) and clears Z, N, and C. IEEE-754 FFLAGS causes continue to accrue independently.

FLAGS Effects

Z	N	C	V
0	0	0	*

- 0: writes zero.
- *: set when the value is out of bounds or unordered.

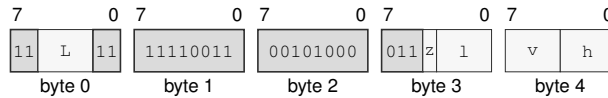
FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

- a: accrual of an operation-generated cause.
- : preserved.
- b: accrual from FEA source conversion.

Encodings:

FBNDXX.SD(z) Fn(l), Fn(v), Fn(h)

Instruction Format:**Format — Instruction format for FBNDXX.SD(z) Fn(l), Fn(v), Fn(h)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

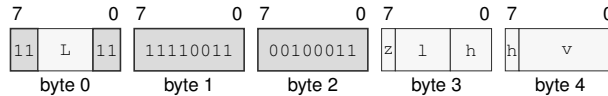
Size field z—Selects S/D.

Register field l—Selects operand 1.

Register field v—Selects operand 2.

Register field h—Selects operand 3.

FBNDXX.SD(z) Fn(l), <ea>(v), Fn(h)

Instruction Format:**Format — Instruction format for FBNDXX.SD(z) Fn(l), <ea>(v), Fn(h)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

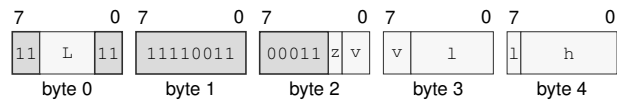
Register field l—Selects operand 1.

Register field h—Selects operand 3.

Effective Address field v—Specifies the source operand.

FBNDXX.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Format:



Format — Instruction format for FBNDXX.SD(*z*) <ea>(*l*), Fn(*v*), <ea>(*h*)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Register field v**—Selects operand 2.
- Effective Address field l**—Specifies the source operand.
- Effective Address field h**—Specifies the source operand.

FCEIL**Floating-Point Ceiling****FCEIL**

Operation: Performs the floating-point ceiling operation.

Assembler Syntax: FCEIL.SD(*z*) Fn(*s*), Fn(*d*)
 FCEIL.SD(*z*) <ea>(*e*), Fn(*d*)
 FCEIL.SD(*z*) Fn(*s*), <ea>(*e*)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source toward positive infinity to an integral floating-point value.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

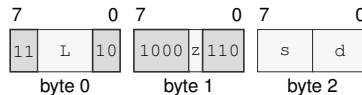
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

FCEIL.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:

Format — Instruction format for FCEIL.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

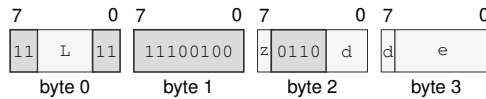
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

$\text{FCEIL.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for $\text{FCEIL.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

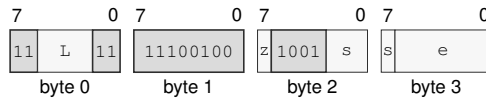
Size field z —Selects S/D.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

$\text{FCEIL.SD}(z) \text{Fn}(s), \text{<ea>}(e)$

Instruction Format:



Format — Instruction format for $\text{FCEIL.SD}(z) \text{Fn}(s), \text{<ea>}(e)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand. Immediate addressing is unavailable in this form.

FCLASS**Floating-Point Classify****FCLASS**

Operation: Performs the floating-point classify operation.

Assembler Syntax: `FCLASS.SD(z) Fn(s), Rn(d)`

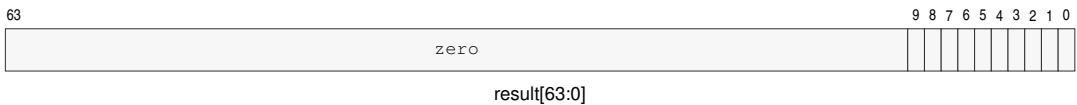
Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Examines the selected floating-point encoding without generating a floating-point exception condition and writes a one-hot class bitmap to the destination Rn register.

The destination is a one-hot bitmap selected directly from the sign, exponent, fraction, and NaN quiet bit of the S or D source encoding.

**FCLASS Result Bitmap**

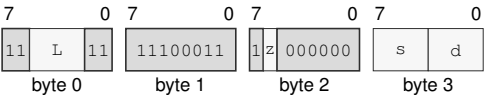
Bit	Class
0	negative infinity
1	negative normal
2	negative subnormal
3	negative zero
4	positive zero
5	positive subnormal
6	positive normal
7	positive infinity
8	signaling NaN
9	quiet NaN

Bits 63..10 of the destination Rn register are zero. No floating-point exception flag is changed.

Encodings:

FCLASS.SD(*z*) Fn(*s*), Rn(*d*)

Instruction Format:



Format — Instruction format for FCLASS.SD(*z*) Fn(*s*), Rn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *s*—Selects the source operand.
- Register field** *d*—Selects the destination operand.

FCLR

Floating-Point Clear

FCLR

Operation: Performs the floating-point clear operation.

Assembler Syntax: FCLR Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

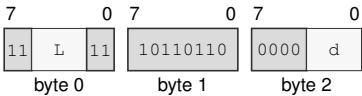
Detailed Semantics

Writes all zero bits to the complete 64-bit Fn destination, representing positive zero in both S and D interpretations.

Encodings:

FCLR Fn(d)

Instruction Format:



Format — Instruction format for FCLR Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

FCMP

Floating-Point Compare

FCMP

Operation: Performs the floating-point compare operation.

Assembler Syntax: FCMP.SD(z) Fn(s), Fn(d)
FCMP.SD(z) <ea>(e), Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Compares the selected-format operands and writes Z,N,C,V as greater=0000, equal=1000, less=0110, or unordered=0001. A quiet NaN is unordered; a signaling NaN is unordered and generates NV.

FLAGS Effects

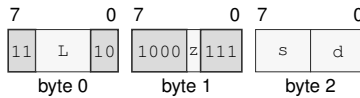
Z	N	C	V
a	b	b	c

- a: set only when equal.
- b: set only when less.
- c: set when unordered.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

- a: accrual of an operation-generated cause.
- : preserved.
- b: accrual from FEA source conversion.

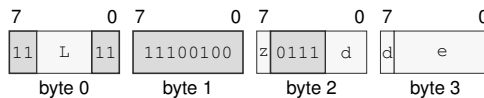
Encodings:FCMP.SD(*z*) Fn(*s*), Fn(*d*)**Instruction Format:****Format — Instruction format for FCMP.SD(*z*) Fn(*s*), Fn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FCMP.SD(*z*) <ea>(*e*), Fn(*d*)**Instruction Format:****Format — Instruction format for FCMP.SD(*z*) <ea>(*e*), Fn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

FCOPYSIGN

Floating-Point Copy Sign

FCOPYSIGN

Operation: Performs the floating-point copy sign operation.

Assembler Syntax: FCOPYSIGN.SD(*z*) Fn(*s*), Fn(*m*), Fn(*d*)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

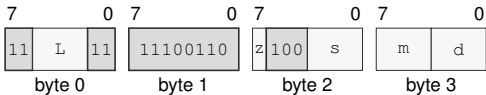
Detailed Semantics

Combines the magnitude bits of the destination with the sign bit of the source and writes the selected-format result.

Encodings:

FCOPYSIGN.SD(*z*) Fn(*s*), Fn(*m*), Fn(*d*)

Instruction Format:



Format — Instruction format for FCOPYSIGN.SD(*z*) Fn(*s*), Fn(*m*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *s*—Selects operand 1.
- Register field** *m*—Selects operand 2.
- Register field** *d*—Selects operand 3.

FCVT**Floating-Point Convert Signed****FCVT**

Operation: Performs the floating-point convert signed operation.

Assembler Syntax: FCVT.SD(z) Fn(s), Fn(d)
 FCVT.SD(z) Fn(s), Rn(d)
 FCVT.SD(z) Rn(s), Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Converts between supported signed integer and floating-point forms, or between the supported floating-point widths, according to the operand classes.

The operand classes select one of three conversion families.

- Fn to Fn .S converts D to S, while .D converts S to D.
- Fn to Rn truncates toward zero and writes a signed 64-bit integer. Negative overflow and negative infinity produce 0x8000000000000000; positive overflow and positive infinity produce 0x7fffffffffffffff; NaN produces zero.
- Rn to Fn converts the full signed 64-bit source to the selected floating-point format using FSTATUS.RM.

An invalid Fn-to-Rn conversion generates NV without OF or NX. A valid conversion that discards a fractional part generates NX. Fn-to-Fn and Rn-to-Fn conversion use the common floating-point rounding, floating-point exception-cause generation, condition-enable testing, and commit rules.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	*

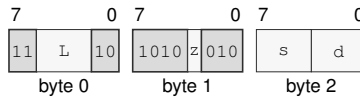
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FCVT.SD(z) Fn(s), Fn(d)

Instruction Format:



Format — Instruction format for FCVT.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

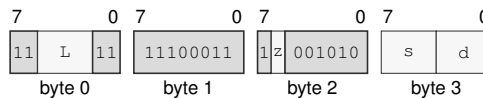
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVT.SD(z) Fn(s), Rn(d)

Instruction Format:



Format — Instruction format for FCVT.SD(z) Fn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

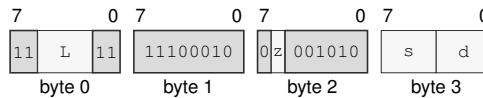
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVT.SD(z) Rn(s), Fn(d)

Instruction Format:



Format — Instruction format for FCVT.SD(z) Rn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU**Floating-Point Convert Unsigned****FCVTU**

Operation: Performs the floating-point convert unsigned operation.

Assembler Syntax: FCVTU.SD(z) Fn(s), Fn(d)
 FCVTU.SD(z) Fn(s), Rn(d)
 FCVTU.SD(z) Rn(s), Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Converts between supported unsigned integer and floating-point forms, or between the supported floating-point widths, according to the operand classes.

The operand classes select one of three conversion families.

- Fn to Fn .S converts D to S, while .D converts S to D.
- Fn to Rn truncates toward zero and writes an unsigned 64-bit integer. A negative value or negative infinity produces zero; positive overflow or positive infinity produces 0xffffffffffffffff; NaN produces zero.
- Rn to Fn converts the full unsigned 64-bit source range to the selected floating-point format using FSTATUS.RM.

An invalid Fn-to-Rn conversion generates NV without OF or NX. A valid conversion that discards a fractional part generates NX. Fn-to-Fn and Rn-to-Fn conversion use the common floating-point rounding, floating-point exception-cause generation, condition-enable testing, and commit rules.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	*

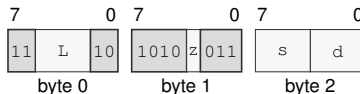
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FCVTU.SD(z) Fn(s), Fn(d)

Instruction Format:



Format — Instruction format for FCVTU.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

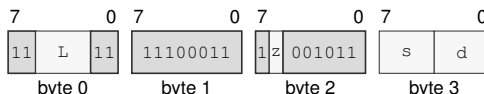
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU.SD(z) Fn(s), Rn(d)

Instruction Format:



Format — Instruction format for FCVTU.SD(z) Fn(s), Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

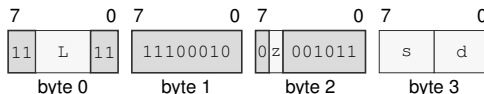
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU.SD(z) Rn(s), Fn(d)

Instruction Format:



Format — Instruction format for FCVTU.SD(z) Rn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FDIV**Floating-Point Divide****FDIV**

Operation: Performs the floating-point divide operation.

Assembler Syntax: `FDIV.SD(z) Fn(s), Fn(d)`
`FDIV.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Divides the selected-format floating-point destination value by the source and writes the rounded result under the architectural floating-point environment.

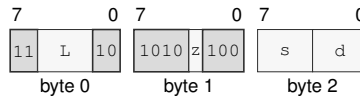
FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	*	*	*

*: accrual of an operation-generated cause.

Encodings:

`FDIV.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for `FDIV.SD(z) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

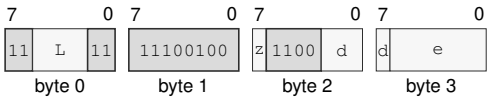
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FDIV.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FDIV.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FFLOOR**Floating-Point Floor****FFLOOR**

Operation: Performs the floating-point floor operation.

Assembler Syntax: `FFLOOR.SD(z) Fn(s), Fn(d)`
`FFLOOR.SD(z) <ea>(e), Fn(d)`
`FFLOOR.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source toward negative infinity to an integral floating-point value.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

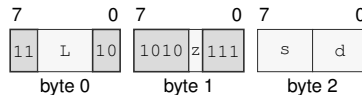
a: accrual of an operation-generated cause.

-: preserved.

b: accrual from FEA source conversion.

Encodings:

`FFLOOR.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FFLOOR.SD(z) Fn(s), Fn(d)

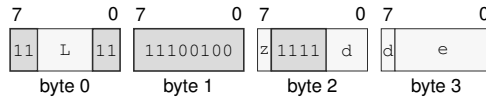
Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

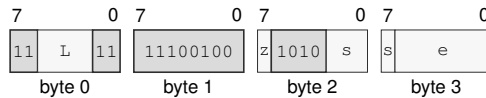
FFLOOR.SD(z) <ea>(e), Fn(d)**Instruction Format:****Format — Instruction format for FFLOOR.SD(z) <ea>(e), Fn(d)****Instruction Fields:**

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

FFLOOR.SD(z) Fn(s), <ea>(e)**Instruction Format:****Format — Instruction format for FFLOOR.SD(z) Fn(s), <ea>(e)****Instruction Fields:**

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand. Immediate addressing is unavailable in this form.

FGETEXP**Floating-Point Get Exponent****FGETEXP**

Operation: Performs the floating-point get exponent operation.

Assembler Syntax: `FGETEXP.SD(z) Fn(s), Fn(d)`
`FGETEXP.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Derives a signed exponent from the selected source's exponent field and converts that integer to the selected floating-point destination format.

For S format, exponent field zero is treated as biased exponent one and the bias 127 is subtracted. For D format, exponent field zero is likewise treated as one and bias 1023 is subtracted. The resulting signed integer exponent is converted to the selected floating-point destination format. This rule gives zero and subnormal inputs the minimum normal exponent and leaves infinity and NaN encodings at the exponent produced by their all-ones field.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

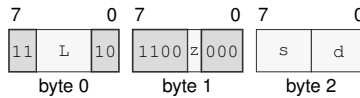
-: preserved.

b: accrual from FEA source conversion.

Encodings:

FGETEXP.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FGETEXP.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

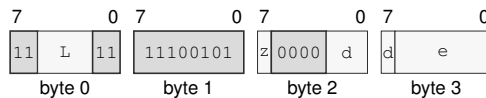
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FGETEXP.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FGETEXP.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

FGETMAN**Floating-Point Get Mantissa****FGETMAN**

Operation: Performs the floating-point get mantissa operation.

Assembler Syntax: `FGETMAN.SD(z) Fn(s), Fn(d)`
`FGETMAN.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Extracts the normalized significand of the selected floating-point source and writes the specified floating-point result for finite and special values.

For a normal finite source, the result preserves the sign and fraction and replaces the exponent with the bias, producing a magnitude in $[1, 2)$. A zero, subnormal, infinity, or NaN source retains its source encoding, subject to the architectural signaling-NaN and default-NaN rules. The S and D encodings use their native exponent and fraction widths.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

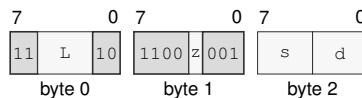
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

`FGETMAN.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FGETMAN.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

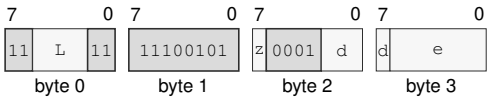
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FGETMAN.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FGETMAN.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FINT**Floating-Point Integral Value****FINT**

Operation: Performs the floating-point integral value operation.

Assembler Syntax: `FINT.SD(z) Fn(s), Fn(d)`
`FINT.SD(z) <ea>(e), Fn(d)`
`FINT.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source to an integral floating-point value using the active FS-TATUS rounding mode.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

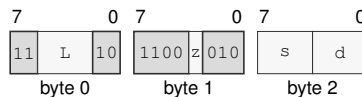
a: accrual of an operation-generated cause.

-: preserved.

b: accrual from FEA source conversion.

Encodings:

`FINT.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FINT.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

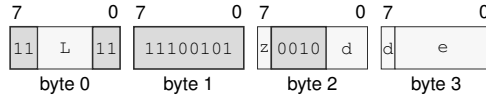
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FINT.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FINT.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

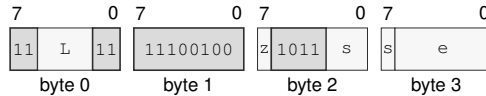
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FINT.SD(z) Fn(s), <ea>(e)

Instruction Format:



Format — Instruction format for FINT.SD(z) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FINTRZ**Floating-Point Integral Toward Zero****FINTRZ**

Operation: Performs the floating-point integral toward zero operation.

Assembler Syntax: `FINTRZ.SD(z) Fn(s), Fn(d)`
`FINTRZ.SD(z) <ea>(e), Fn(d)`
`FINTRZ.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source toward zero to an integral floating-point value.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

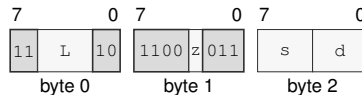
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

`FINTRZ.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FINTRZ.SD(z) Fn(s), Fn(d)

Instruction Fields:

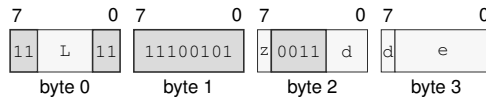
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FINTRZ.SD(z) <ea>(e), Fn(d)

Instruction Format:**Format — Instruction format for FINTRZ.SD(z) <ea>(e), Fn(d)****Instruction Fields:**

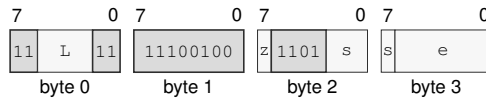
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FINTRZ.SD(z) Fn(s), <ea>(e)

Instruction Format:**Format — Instruction format for FINTRZ.SD(z) Fn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FMADD**Floating-Point Fused Multiply Add****FMADD**

Operation: Performs the floating-point fused multiply add operation.

Assembler Syntax: `FMADD.SD(z) Fn(l), Fn(r), Fn(d)`
`FMADD.SD(z) <ea>(l), Fn(r), Fn(d)`
`FMADD.SD(z) Fn(l), <ea>(r), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the selected-format fused multiply-add with one final rounding and writes the result to the destination.

FFLAGS Effects

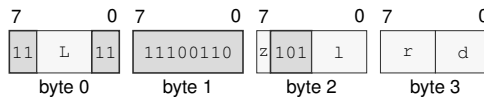
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

`FMADD.SD(z) Fn(l), Fn(r), Fn(d)`

Instruction Format:

Format — Instruction format for FMADD.SD(z) Fn(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

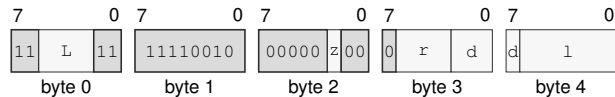
Register field 1—Selects operand 1.

Register field r—Selects operand 2.

Register field d—Selects operand 3.

FMADD.SD(z) <ea>(l), Fn(r), Fn(d)

Instruction Format:



Format — Instruction format for FMADD.SD(z) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

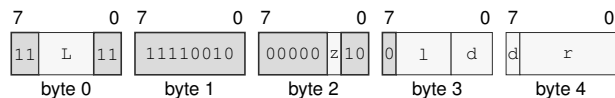
Register field r—Selects operand 2.

Register field d—Selects operand 3.

Effective Address field l—Specifies the source operand.

FMADD.SD(z) Fn(l), <ea>(r), Fn(d)

Instruction Format:



Format — Instruction format for FMADD.SD(z) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects operand 1.

Register field d—Selects operand 3.

Effective Address field r—Specifies the source operand.

FMAX**Floating-Point Maximum****FMAX**

Operation: Performs the floating-point maximum operation.

Assembler Syntax: `FMAX.SD(z) Fn(s), Fn(d)`
`FMAX.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Selects the greater numeric operand; one NaN selects the numeric operand, two NaNs use the common NaN-selection rule, and a positive-zero/negative-zero pair selects positive zero. A signaling NaN generates NV.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

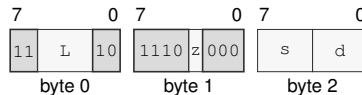
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

`FMAX.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FMAX.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

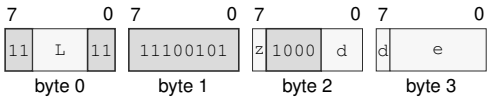
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FMAX.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FMAX.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FMIN**Floating-Point Minimum****FMIN**

Operation: Performs the floating-point minimum operation.

Assembler Syntax: `FMIN.SD(z) Fn(s), Fn(d)`
`FMIN.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Selects the lesser numeric operand; one NaN selects the numeric operand, two NaNs use the common NaN-selection rule, and a positive-zero/negative-zero pair selects negative zero. A signaling NaN generates NV.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

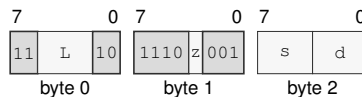
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

`FMIN.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FMIN.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

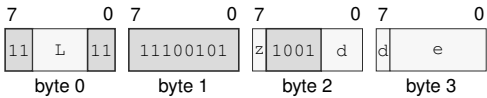
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FMIN.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FMIN.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FMOD**Floating-Point Modulo****FMOD**

Operation: Performs the floating-point modulo operation.

Assembler Syntax: `FMOD.SD(z) Fn(s), Fn(d)`
`FMOD.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the IEEE-style floating-point modulus defined by a quotient rounded toward zero and writes the selected-format remainder.

After DAZ processing, let x be the original destination and y the source. For finite valid operands, choose q as x/y truncated toward zero and compute

$$r = x - qy$$

with exact intermediate arithmetic. A zero result has the sign of x .

A signaling NaN generates NV; a NaN input produces the selected quiet NaN. Infinite x or zero y generates NV and produces the architectural default quiet NaN. Infinite y with finite x returns x . An enabled floating-point exception condition raises the FLOATING_POINT_FAULT architectural event before the destination write.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

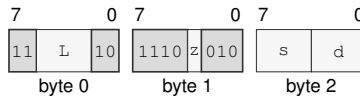
–: preserved.

b: accrual from FEA source conversion.

Encodings:

$\text{FMOD.SD}(z) \text{ Fn}(s), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for FMOD.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

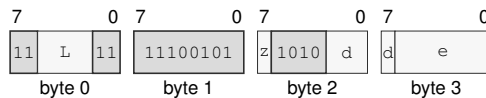
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

$\text{FMOD.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for FMOD.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

FMOV**Floating-Point Move****FMOV**

Operation: Copies the source operand to the destination operand.

Assembler Syntax: `FMOV.SD(z) Fn(s), Fn(d)`
`FMOV.SD(z) <ea>(e), Fn(d)`
`FMOV.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Copies the selected-format floating-point source value to the destination.

FFLAGS Effects

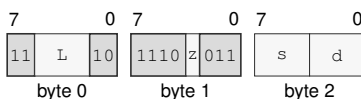
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual from FEA source conversion.

-: preserved.

Encodings:

`FMOV.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FMOV.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

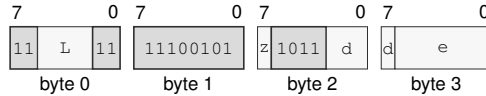
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FMOV.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FMOV.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

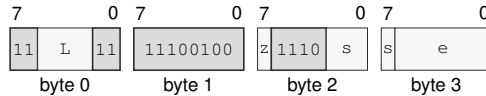
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FMOV.SD(z) Fn(s), <ea>(e)

Instruction Format:



Format — Instruction format for FMOV.SD(z) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FMOVcc**Floating-Point Conditional Move****FMOVcc**

Operation: Performs the floating-point conditional move operation.

Assembler Syntax: `FMOVcc Fn(s), Fn(d)`
`FMOVcc.SD(z) <ea>(e), Fn(d)`
`FMOVcc.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Copies the selected-format floating-point source value to the destination only when the encoded integer condition is true.

FFLAGS Effects

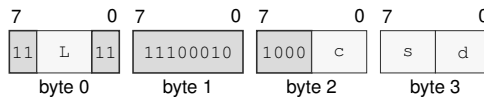
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual from FEA source conversion.

-: preserved.

Encodings:

`FMOVcc Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FMOVcc Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

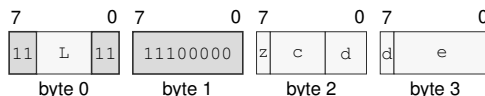
Condition field c—Selects the condition code.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

$\text{FMOVcc.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for $\text{FMOVcc.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

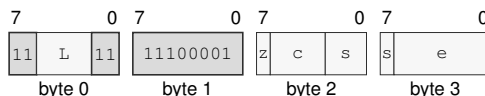
Condition field c —Selects the condition code.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

$\text{FMOVcc.SD}(z) \text{ Fn}(s), \text{ <ea>}(e)$

Instruction Format:



Format — Instruction format for $\text{FMOVcc.SD}(z) \text{ Fn}(s), \text{ <ea>}(e)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Condition field c —Selects the condition code.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand. Immediate addressing is unavailable in this form.

FMOVCR**Floating-Point Move Constant****FMOVCR**

Operation: Loads a named architectural binary64 constant selected by an unsigned 16-bit ID.

Assembler Syntax: FMOVCR.SD(z) <fconst_id>, Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Table 18-2. FMOVCR Constant IDs (IEEE-754 binary64)

ID	Constant	Result bits
0x0000	positive zero	0x0000000000000000
0x0001	negative zero	0x8000000000000000
0x0002	positive one	0x3ff0000000000000
0x0003	negative one	0xbff0000000000000
0x0004	positive one half	0x3fe0000000000000
0x0005	negative one half	0xbfe0000000000000
0x0006	positive two	0x4000000000000000
0x0007	negative two	0xc000000000000000
0x0008	positive ten	0x4024000000000000
0x0009	negative ten	0xc024000000000000
0x0010	pi	0x400921fb54442d18
0x0011	pi over 2	0x3ff921fb54442d18
0x0012	pi over 4	0x3fe921fb54442d18
0x0013	two pi	0x401921fb54442d18
0x0014	reciprocal pi	0x3fd45f306dc9c883
0x0015	two over pi	0x3fe45f306dc9c883
0x0016	sqrt 2	0x3ff6a09e667f3bcd
0x0017	reciprocal sqrt 2	0x3fe6a09e667f3bcc
0x0020	e	0x4005bf0a8b145769
0x0021	log2 e	0x3ff71547652b82fe
0x0022	log10 e	0x3fdbcb7b1526e50e
0x0023	ln 2	0x3fe62e42fef39ef
0x0024	ln 10	0x40026bb1bbb55516
0x0025	log2 10	0x400a934f0979a371
0x0026	log10 2	0x3fd34413509f79ff
0x0100	positive infinity	0x7ff0000000000000
0x0101	negative infinity	0xfff0000000000000
0x0102	canonical positive quiet nan	0x7ff8000000000000
0x0103	canonical negative quiet nan	0xfff8000000000000
0x0104	canonical positive signaling nan	0x7ff0000000000001
0x0105	canonical negative signaling nan	0xfff0000000000001
0x0110	maximum positive finite	0x7fefffffffffffffff

Table 18-2 (continued)

ID	Constant	Result bits
0x0111	maximum negative finite	0xffefffffffffffffff
0x0112	minimum positive normal	0x0010000000000000
0x0113	minimum negative normal	0x8010000000000000
0x0114	minimum positive subnormal	0x0000000000000001
0x0115	minimum negative subnormal	0x8000000000000001
0x0116	machine epsilon	0x3cb0000000000000
0x0117	maximum positive subnormal	0x000fffffffffffffff
0x0118	maximum negative subnormal	0x800fffffffffffffff

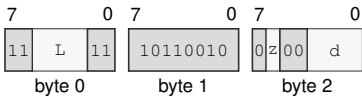
Detailed Semantics

FMOVCR maps an unsigned 16-bit constant ID to a fixed IEEE-754 binary64 bit pattern and writes that pattern to the destination F register. It is independent of FSTATUS rounding, DAZ, FTZ, and default-NaN modes. Loading a signaling NaN is a bitwise move and does not itself raise NV; a later consuming floating-point operation applies its normal NaN rules.

Encodings:

FMOVCR.SD(z) <fconst_id>, Fn(d)

Instruction Format:



Format — Instruction format for FMOVCR.SD(z) <fconst_id>, Fn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Register field d**—Selects the destination operand.

FMSUB**Floating-Point Fused Multiply Subtract****FMSUB**

Operation: Performs the floating-point fused multiply subtract operation.

Assembler Syntax: `FMSUB.SD(z) Fn(l), Fn(r), Fn(d)`
`FMSUB.SD(z) <ea>(l), Fn(r), Fn(d)`
`FMSUB.SD(z) Fn(l), <ea>(r), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the selected-format fused multiply-subtract with one final rounding and writes the result to the destination.

FFLAGS Effects

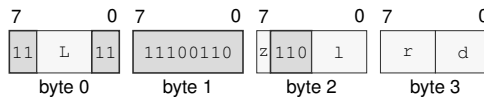
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

`FMSUB.SD(z) Fn(l), Fn(r), Fn(d)`

Instruction Format:

Format — Instruction format for FMSUB.SD(z) Fn(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

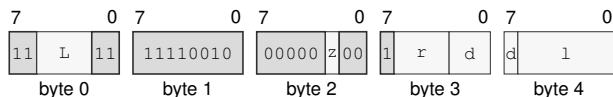
Register field l—Selects operand 1.

Register field r—Selects operand 2.

Register field d—Selects operand 3.

FMSUB.SD(z) <ea>(l), Fn(r), Fn(d)

Instruction Format:



Format — Instruction format for FMSUB.SD(z) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

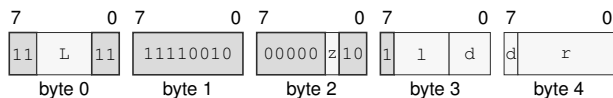
Register field r—Selects operand 2.

Register field d—Selects operand 3.

Effective Address field l—Specifies the source operand.

FMSUB.SD(z) Fn(l), <ea>(r), Fn(d)

Instruction Format:



Format — Instruction format for FMSUB.SD(z) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects operand 1.

Register field d—Selects operand 3.

Effective Address field r—Specifies the source operand.

FMUL**Floating-Point Multiply****FMUL**

Operation: Performs the floating-point multiply operation.

Assembler Syntax: FMUL.SD(z) Fn(s), Fn(d)
FMUL.SD(z) <ea>(e), Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Multiplies the selected-format floating-point source and destination values and writes the rounded result.

FFLAGS Effects

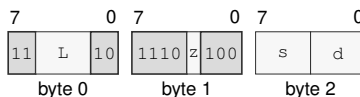
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FMUL.SD(z) Fn(s), Fn(d)

Instruction Format:

Format — Instruction format for FMUL.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

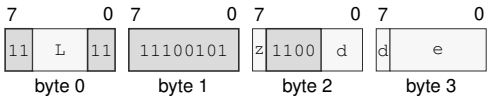
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FMUL.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FMUL.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FNEG**Floating-Point Negate****FNEG**

Operation: Performs the floating-point negate operation.

Assembler Syntax: `FNEG.SD(z) Fn(s), Fn(d)`
`FNEG.SD(z) <ea>(e), Fn(d)`
`FNEG.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Complements the sign bit of the selected floating-point source value and writes the result.

FFLAGS Effects

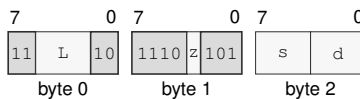
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual from FEA source conversion.

-: preserved.

Encodings:

`FNEG.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FNEG.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

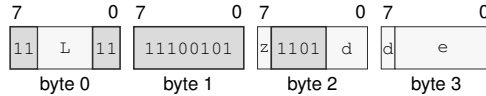
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FNEG.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FNEG.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

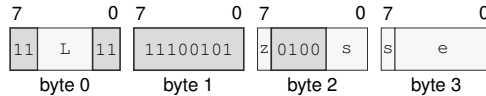
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FNEG.SD(z) Fn(s), <ea>(e)

Instruction Format:



Format — Instruction format for FNEG.SD(z) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FNMADD**Floating-Point Fused Negative Multiply Add****FNMADD**

Operation: Performs the floating-point fused negative multiply add operation.

Assembler Syntax: `FNMADD.SD(z) Fn(l), Fn(r), Fn(d)`
`FNMADD.SD(z) <ea>(l), Fn(r), Fn(d)`
`FNMADD.SD(z) Fn(l), <ea>(r), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the negated selected-format fused multiply-add with one final rounding and writes the result.

FFLAGS Effects

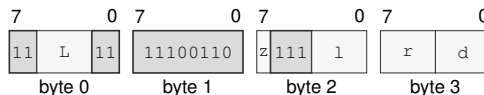
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

`FNMADD.SD(z) Fn(l), Fn(r), Fn(d)`

Instruction Format:

Format — Instruction format for FNMADD.SD(z) Fn(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

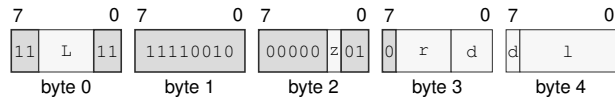
Register field l—Selects operand 1.

Register field r—Selects operand 2.

Register field d—Selects operand 3.

FNMADD.SD(*z*) <ea>(*l*), Fn(*r*), Fn(*d*)

Instruction Format:



Format — Instruction format for FNMADD.SD(*z*) <ea>(*l*), Fn(*r*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

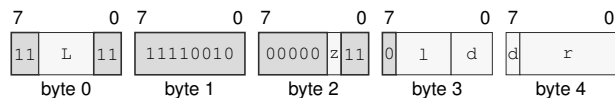
Register field *r*—Selects operand 2.

Register field *d*—Selects operand 3.

Effective Address field *l*—Specifies the source operand.

FNMADD.SD(*z*) Fn(*l*), <ea>(*r*), Fn(*d*)

Instruction Format:



Format — Instruction format for FNMADD.SD(*z*) Fn(*l*), <ea>(*r*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *l*—Selects operand 1.

Register field *d*—Selects operand 3.

Effective Address field *r*—Specifies the source operand.

FNMSUB**Floating-Point Fused Negative Multiply Subtract****FNMSUB**

Operation: Performs the floating-point fused negative multiply subtract operation.

Assembler Syntax: FNMSUB.SD(z) Fn(l), Fn(r), Fn(d)
 FNMSUB.SD(z) <ea>(l), Fn(r), Fn(d)
 FNMSUB.SD(z) Fn(l), <ea>(r), Fn(d)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the negated selected-format fused multiply-subtract with one final rounding and writes the result.

FFLAGS Effects

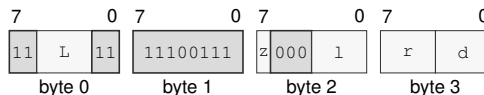
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FNMSUB.SD(z) Fn(l), Fn(r), Fn(d)

Instruction Format:

Format — Instruction format for FNMSUB.SD(z) Fn(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

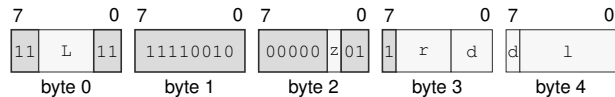
Register field l—Selects operand 1.

Register field r—Selects operand 2.

Register field d—Selects operand 3.

FNMSUB.SD(*z*) <ea>(*l*), Fn(*r*), Fn(*d*)

Instruction Format:



Format — Instruction format for FNMSUB.SD(*z*) <ea>(*l*), Fn(*r*), Fn(*d*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

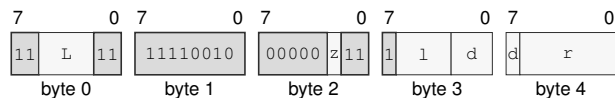
Register field r—Selects operand 2.

Register field d—Selects operand 3.

Effective Address field l—Specifies the source operand.

FNMSUB.SD(*z*) Fn(*l*), <ea>(*r*), Fn(*d*)

Instruction Format:



Format — Instruction format for FNMSUB.SD(*z*) Fn(*l*), <ea>(*r*), Fn(*d*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects operand 1.

Register field d—Selects operand 3.

Effective Address field r—Specifies the source operand.

FPOPP**Floating-Point Pop Register Pair****FPOPP**

Operation:	Pops one canonical floating-point register pair selected by an inline pair index.
Assembler Syntax:	FPOPP FPAIRn(i)
Required CPUID flags:	FP
Repeat eligibility:	Eligible as a REP body.

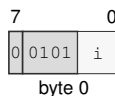
Detailed Semantics

Pops the canonical floating-point register pair selected by the pair ID. The canonical pair sequence is pair 0 (F0, F1), pair 1 (F2, F3), pair 2 (F4, F5), pair 3 (F6, F7), pair 4 (F8, F9), pair 5 (F10, F11), pair 6 (F12, F13), and pair 7 (F14, F15). All eight pair IDs are valid and each selects one pair.

FPOPP selects one canonical floating-point pair using the unsigned 3-bit pair index and processes that pair's registers in reverse listed order. The complete two-slot stack range is validated and read before either floating-point register or SP is changed. Each register pop loads the 64-bit stack slot at SP and then increments SP by 8.

Encodings:

FPOPP FPAIRn(i)

Instruction Format:**Format — Instruction format for FPOPP FPAIRn(i)****Instruction Fields:**

Immediate field i—Encodes the immediate value.

FPUSHP

Floating-Point Push Register Pair

FPUSHP

Operation:	Pushes one canonical floating-point register pair selected by an inline pair index.
Assembler Syntax:	FPUSHP FPAIRn(i)
Required CPUID flags:	FP
Repeat eligibility:	Eligible as a REP body.

Detailed Semantics

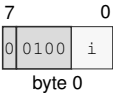
Pushes the canonical floating-point register pair selected by the pair ID. The canonical pair sequence is pair 0 (F0, F1), pair 1 (F2, F3), pair 2 (F4, F5), pair 3 (F6, F7), pair 4 (F8, F9), pair 5 (F10, F11), pair 6 (F12, F13), and pair 7 (F14, F15). All eight pair IDs are valid and each selects one pair.

FPUSHP selects one canonical floating-point pair using the unsigned 3-bit pair index and processes that pair's registers in the listed order. The complete two-slot stack range is validated before either slot or SP is changed. Each register push decrements SP by 8 and then stores the 64-bit floating-point register value at the new SP.

Encodings:

FPUSHP FPAIRn(i)

Instruction Format:



Format — Instruction format for FPUSHP FPAIRn(i)

Instruction Fields:

Immediate field i—Encodes the immediate value.

FREM**Floating-Point Remainder****FREM**

Operation: Performs the floating-point remainder operation.

Assembler Syntax: `FREM.SD(z) Fn(s), Fn(d)`
`FREM.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes and writes the IEEE floating-point remainder to the destination using a quotient rounded to the nearest integer with ties to even.

After DAZ processing, let x be the original destination and y the source. For finite valid operands, choose q as x/y rounded to the nearest integer with ties to even and compute

$$r = x - qy$$

with exact intermediate arithmetic. A zero result has the sign of x .

A signaling NaN generates NV; a NaN input produces the selected quiet NaN. Infinite x or zero y generates NV and produces the architectural default quiet NaN. Infinite y with finite x returns x . An enabled floating-point exception condition raises the `FLOATING_POINT_FAULT` architectural event before the destination write.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

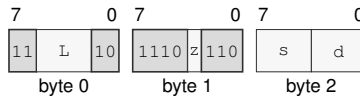
-: preserved.

b: accrual from FEA source conversion.

Encodings:

$\text{FREM.SD}(z) \text{ Fn}(s), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for FREM.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

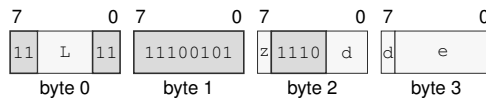
Size field z —Selects S/D.

Register field s —Selects the source operand.

Register field d —Selects the destination operand.

$\text{FREM.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for FREM.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

FROUND**Floating-Point Round****FROUND**

Operation: Performs the floating-point round operation.

Assembler Syntax: `FROUND.SD(z) Fn(s), Fn(d)`
`FROUND.SD(z) <ea>(e), Fn(d)`
`FROUND.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source to the nearest integral value in the same format using nearest-even, independently of FSTATUS.RM.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

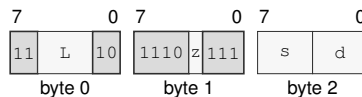
a: accrual of an operation-generated cause.

-: preserved.

b: accrual from FEA source conversion.

Encodings:

`FROUND.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FROUND.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

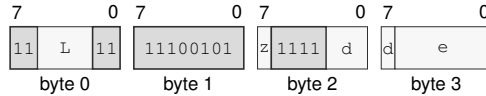
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

$\text{FROUND.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Format:



Format — Instruction format for $\text{FROUND.SD}(z) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

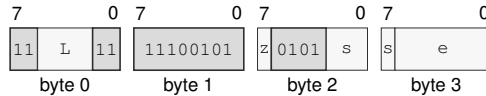
Size field z —Selects S/D.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

$\text{FROUND.SD}(z) \text{Fn}(s), \text{<ea>}(e)$

Instruction Format:



Format — Instruction format for $\text{FROUND.SD}(z) \text{Fn}(s), \text{<ea>}(e)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand. Immediate addressing is unavailable in this form.

FSCALE**Floating-Point Scale****FSCALE**

Operation: Performs the floating-point scale operation.

Assembler Syntax: `FSCALE.SD(z) Fn(s), Fn(d)`
`FSCALE.SD(z) <ea>(e), Fn(d)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Scales the selected floating-point destination by an integral power of two derived from the source and writes the rounded result.

Let x be the destination value after DAZ processing and let k be the source value rounded to an integer using the current rounding mode. The mathematical result before format rounding is

$$x \times 2^k.$$

A NaN operand is propagated according to the common floating-point rules. Multiplying zero or infinity by the power of two preserves that value and its sign. Conversion of the scale operand to k can generate NX; final rounding can generate OF, UF, or NX. An enabled floating-point exception condition raises the `FLOATING_POINT_FAULT` architectural event before the destination is written.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	*

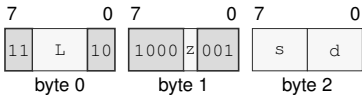
*: accrual of an operation-generated cause.

–: preserved.

Encodings:

FSCALE.SD(z) Fn(s), Fn(d)

Instruction Format:



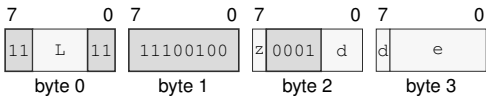
Format — Instruction format for FSCALE.SD(z) Fn(s), Fn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Register field s**—Selects the source operand.
- Register field d**—Selects the destination operand.

FSCALE.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FSCALE.SD(z) <ea>(e), Fn(d)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Register field d**—Selects the destination operand.
- Effective Address field e**—Specifies the source operand.

FSQRT**Floating-Point Square Root****FSQRT**

Operation: Performs the floating-point square root operation.

Assembler Syntax: FSQRT.SD(*z*) Fn(*s*), Fn(*d*)
 FSQRT.SD(*z*) <ea>(*e*), Fn(*d*)
 FSQRT.SD(*z*) Fn(*s*), <ea>(*e*)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the selected-format square root of the source and writes the rounded result.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

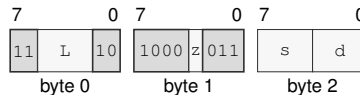
a: accrual of an operation-generated cause.

–: preserved.

b: accrual from FEA source conversion.

Encodings:

FSQRT.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:

Format — Instruction format for FSQRT.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

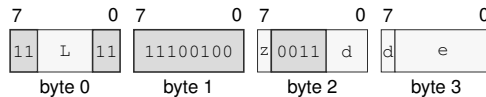
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FSQRT.SD(z) <ea>(e), Fn(d)

Instruction Format:



Format — Instruction format for FSQRT.SD(z) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

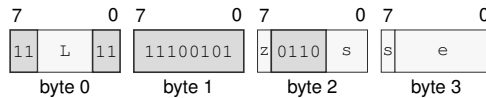
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

FSQRT.SD(z) Fn(s), <ea>(e)

Instruction Format:



Format — Instruction format for FSQRT.SD(z) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

FSUB**Floating-Point Subtract****FSUB**

Operation: Subtracts the source operand from the destination operand.

Assembler Syntax: FSUB.SD(*z*) Fn(*s*), Fn(*d*)
 FSUB.SD(*z*) <ea>(*e*), Fn(*d*)

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Subtracts the selected-format floating-point source from the destination and writes the rounded result.

FFLAGS Effects

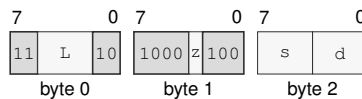
NV	DZ	OF	UF	NX
*	-	*	*	*

*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FSUB.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:

Format — Instruction format for FSUB.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

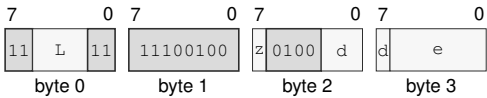
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FSUB.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FSUB.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects S/D.
- Register field** *d*—Selects the destination operand.
- Effective Address field** *e*—Specifies the source operand.

FTEST**Floating-Point Test****FTEST**

Operation: Performs the floating-point test operation.

Assembler Syntax: `FTEST.SD(z) Fn(s)`
`FTEST.SD(z) <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Compares the selected-format operand with positive zero and writes Z,N,C,V as greater=0000, equal=1000, less=0110, or unordered=0001. A quiet NaN is unordered; a signaling NaN is unordered and generates NV.

FLAGS Effects

Z	N	C	V
a	b	b	c

a: set only when equal.

b: set only when less.

c: set when unordered.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	b

a: accrual of an operation-generated cause.

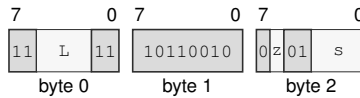
-: preserved.

b: accrual from FEA source conversion.

Encodings:

FTEST.SD(z) Fn(s)

Instruction Format:



Format — Instruction format for FTEST.SD(z) Fn(s)

Instruction Fields:

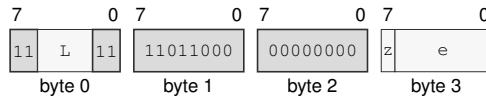
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the operand.

FTEST.SD(z) <ea>(e)

Instruction Format:



Format — Instruction format for FTEST.SD(z) <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Effective Address field e—Specifies the source operand.

FTRUNC**Floating-Point Truncate****FTRUNC**

Operation: Performs the floating-point truncate operation.

Assembler Syntax: `FTRUNC.SD(z) Fn(s), Fn(d)`
`FTRUNC.SD(z) <ea>(e), Fn(d)`
`FTRUNC.SD(z) Fn(s), <ea>(e)`

Required CPUID flags: FP

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Rounds the selected floating-point source toward zero to an integral value in the same floating-point format.

FFLAGS Effects

NV	DZ	OF	UF	NX
a	-	b	b	a

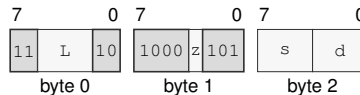
a: accrual of an operation-generated cause.

-: preserved.

b: accrual from FEA source conversion.

Encodings:

`FTRUNC.SD(z) Fn(s), Fn(d)`

Instruction Format:

Format — Instruction format for FTRUNC.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

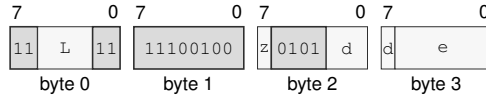
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FTRUNC.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Format:



Format — Instruction format for FTRUNC.SD(*z*) <ea>(*e*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

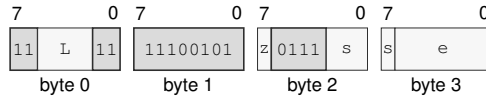
Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

FTRUNC.SD(*z*) Fn(*s*), <ea>(*e*)

Instruction Format:



Format — Instruction format for FTRUNC.SD(*z*) Fn(*s*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand. Immediate addressing is unavailable in this form.

FXCHG**Floating-Point Exchange****FXCHG**

Operation: Performs the floating-point exchange operation.

Assembler Syntax: `FXCHG Fn(l), Fn(r)`

Required CPUID flags: FP

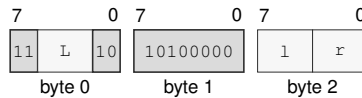
Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Exchanges the two selected-format floating-point register values.

Encodings:

`FXCHG Fn(l), Fn(r)`

Instruction Format:

Format — Instruction format for FXCHG Fn(l), Fn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field l—Selects the source operand.

Register field r—Selects the destination operand.

Destination overlap—When the lhs and rhs operands designate the same architectural register, the final value equals that register's initial value.

RDFFLAGS

Read Floating-Point Flags

RDFFLAGS

Operation: Read the accrued floating-point exception flags into an Rn register.

Assembler Syntax: RDFFLAGS Rn(d)

Required CPUID flags: FP

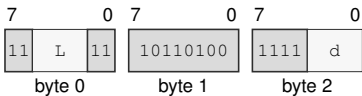
Detailed Semantics

RDFFLAGS reads the architectural FFLAGS register, zero-extends it to the destination register, and writes only that destination. RDFFLAGS requires the floating-point extension.

Encodings:

RDFFLAGS Rn(d)

Instruction Format:



Format — Instruction format for RDFFLAGS Rn(d)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** d—Selects the operand.

RDFSTATUS**Read Floating-Point Status****RDFSTATUS**

Operation: Read the FPU status/control register into an Rn register.

Assembler Syntax: RDFSTATUS Rn(d)

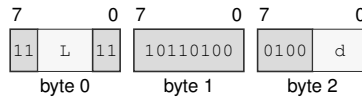
Required CPUID flags: FP

Detailed Semantics

RDFSTATUS reads the architectural 16-bit FSTATUS register, zero-extends it to 64 bits, and writes the result to the destination Rn register. FSTATUS contains floating-point exception-condition enables, rounding mode, and non-default mode bits. FFLAGS holds accrued floating-point exception flags. RDFSTATUS requires the floating-point extension.

Encodings:

RDFSTATUS Rn(d)

Instruction Format:

Format — Instruction format for RDFSTATUS Rn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

WRFFLAGS

Write Floating-Point Flags

WRFFLAGS

Operation: Write the accrued floating-point exception flags from an Rn register.

Assembler Syntax: WRFFLAGS Rn(s)

Required CPUID flags: FP

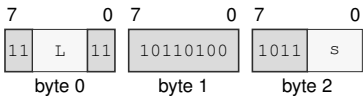
Detailed Semantics

WRFFLAGS writes the architectural FFLAGS register from the low source bits. Reserved FFLAGS bits must be zero. WRFFLAGS requires the floating-point extension.

Encodings:

WRFFLAGS Rn(s)

Instruction Format:



Format — Instruction format for WRFFLAGS Rn(s)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Register field** s—Selects the operand.

WRFSTATUS

Write Floating-Point Status

WRFSTATUS

Operation: Write the FPU status/control register from an Rn register.

Assembler Syntax: WRFSTATUS Rn(s)

Required CPUID flags: FP

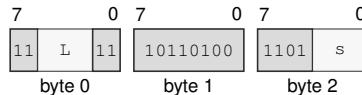
Detailed Semantics

WRFSTATUS writes only the architectural 16-bit FSTATUS register from the low 16 bits of the source Rn register. Reserved FSTATUS bits must be zero. WRFSTATUS requires the floating-point extension.

Encodings:

WRFSTATUS Rn(s)

Instruction Format:



Format — Instruction format for WRFSTATUS Rn(s)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Register field s—Selects the operand.

19 APPROXIMATE FLOATING-POINT TRANSCENDENTAL INSTRUCTIONS

19.1 FPTRANSA Common Model

For an approximate transcendental instruction, x' is the source after `FSTATUS.DAZ`, $y = f(x')$ is the exact real reference value named by its accuracy contract, and a is the approximate result before `FSTATUS.FTZ`. For a format with precision p and minimum normal exponent $e_{\min}$, define

$$\text{ulp}_F(y) = \begin{cases} 2^{\max(\lfloor \log_2 |y| \rfloor - p + 1, e_{\min} - p + 1)}, & y \neq 0, \\ 2^{e_{\min} - p + 1}, & y = 0. \end{cases}$$

The measured error is $|a - y| / \text{ulp}_F(y)$. The bound applies only to finite reference values within the selected format's finite range; NaN, infinity, and overflow follow the instruction's special-value rules. Every present contract guarantees at most 4 ULP for both S and D, corresponding to at least 21 and 50 worst-case relative precision bits respectively for nonzero normal results. CPUID may advertise a smaller certified bound.

FPTRANSA applies DAZ before forming the exact reference input and measures its advertised ULP error before FTZ. It ignores `FSTATUS.RM` and formats its implementation-defined approximation with nearest-even rounding. The approximation is deterministic for the same implementation, input, format, and relevant `FSTATUS` mode, but need not be bit-identical across implementations.

A signaling NaN generates NV. NaN results are quieted and then processed by `FSTATUS.DN`. `FSTATUS.FTZ` replaces a subnormal result with the required signed zero and generates UF. Each instruction generates NV and DZ according to its special-value rules, and generates OF and UF when required by its exact result; NX is never generated, remains unchanged in `FFLAGS`, and cannot cause the `FLOATING_POINT_FAULT` architectural event. Overflow means that the exact result magnitude exceeds the selected format's maximum finite value. Underflow means that the exact nonzero result magnitude is below the selected format's minimum normal value. If any generated floating-point exception cause has its matching `FSTATUS` enable bit set, the instruction raises the `FLOATING_POINT_FAULT` architectural event before writing any destination. Otherwise it commits its result or results and accrues the generated causes in `FFLAGS` as one architectural commit.

19.2 Summary

Table 19-1. Approximate Floating-Point Transcendental Instructions Summary (Informative)

Mnemonic	Brief description
FACOSA	Performs the approximate floating-point arc cosine operation.
FASINA	Performs the approximate floating-point arc sine operation.
FATANA	Performs the approximate floating-point arc tangent operation.
FATANHA	Performs the approximate floating-point hyperbolic arc tangent operation.
FCOSA	Performs the approximate floating-point cosine operation.
FCOSHA	Performs the approximate floating-point hyperbolic cosine operation.
FETOXA	Performs the approximate floating-point e to x operation.
FETOXM1A	Performs the approximate floating-point e to x minus one operation.
FLOG10A	Performs the approximate floating-point log base 10 operation.
FLOG2A	Performs the approximate floating-point log base 2 operation.
FLOGNA	Performs the approximate floating-point natural log operation.
FLOGNP1A	Performs the approximate floating-point natural log plus one operation.
FSINA	Performs the approximate floating-point sine operation.
FSINCOSA	Performs the approximate floating-point sine and cosine operations.
FSINHA	Performs the approximate floating-point hyperbolic sine operation.
FTANA	Performs the approximate floating-point tangent operation.
FTANHA	Performs the approximate floating-point hyperbolic tangent operation.
FTENTOX A	Performs the approximate floating-point 10 to x operation.
FTWOTOXA	Performs the approximate floating-point 2 to x operation.

FACOSA

Approximate Floating-Point Arc Cosine

FACOSA

Operation:	Performs the approximate floating-point arc cosine operation.
Assembler Syntax:	<code>FACOSA.SD(z) Fn(s), Fn(d)</code>
Required CPUID flags:	FP FPTRANSA
Repeat eligibility:	Eligible as a REP body.

Detailed Semantics

Computes the `acos(x)` reference in radians over finite `x` with `abs(x) <= 1` after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is `0x0012`. If that accuracy contract's PRESENT field is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is `acos(x)` in radians on finite `x` with `abs(x) <= 1`. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source equals 1, the result is positive zero.

Exact anchors

- `acos(+1) = +0`

Required properties

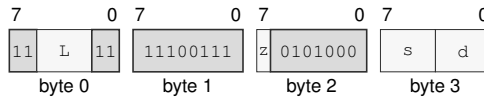
- monotonically decreasing on `[-1, 1]`
- result is in `[0, pi]`

NaN handling, floating-point exception-cause generation, matching condition-enable testing, `FLOATING_POINT_FAULT` architectural-event delivery, and commit behavior follow the common FPTRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

*: accrual of an operation-generated cause.
-: preserved.

Encodings:FACOSA.SD(*z*) Fn(*s*), Fn(*d*)**Instruction Format:****Format — Instruction format for FACOSA.SD(*z*) Fn(*s*), Fn(*d*)****Instruction Fields:**

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FASINA**Approximate Floating-Point Arc Sine****FASINA**

Operation: Performs the approximate floating-point arc sine operation.

Assembler Syntax: `FASINA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\text{asin}(x)$ reference in radians over finite x with $\text{abs}(x) \leq 1$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0011. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\text{asin}(x)$ in radians on finite x with $\text{abs}(x) \leq 1$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\text{asin}(+0) = +0$
- $\text{asin}(-0) = -0$

Required properties

- odd
- monotonically increasing on $[-1, 1]$
- result is in $[-\pi/2, \pi/2]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

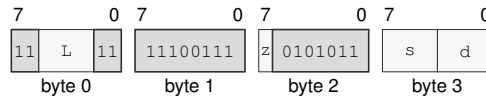
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FASINA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FASINA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FATANA**Approximate Floating-Point Arc Tangent****FATANA**

Operation: Performs the approximate floating-point arc tangent operation.

Assembler Syntax: `FATANA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\text{atan}(x)$ reference in radians over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0013. If that accuracy contract's PRESENT field is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\text{atan}(x)$ in radians on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\text{atan}(+0) = +0$
- $\text{atan}(-0) = -0$

Required properties

- odd
- monotonically increasing
- finite results are in $[-\pi/2, \pi/2]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, `FLOATING_POINT_FAULT` architectural-event delivery, and commit behavior follow the common FPTRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

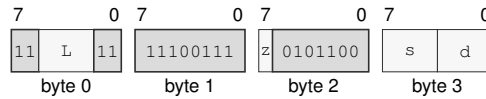
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FATANA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FATANA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FATANHA**Approximate Floating-Point Hyperbolic Arc Tangent****FATANHA**

Operation: Performs the approximate floating-point hyperbolic arc tangent operation.

Assembler Syntax: `FATANHA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\operatorname{atanh}(x)$ reference over finite x with $\operatorname{abs}(x) \leq 1$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0024. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\operatorname{atanh}(x)$ on finite x with $\operatorname{abs}(x) \leq 1$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\operatorname{atanh}(+0) = +0$
- $\operatorname{atanh}(-0) = -0$
- $\operatorname{atanh}(\text{signed } 1) = \text{signed infinity}$

Required properties

- odd
- monotonically increasing on $(-1, 1)$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	-	*	-

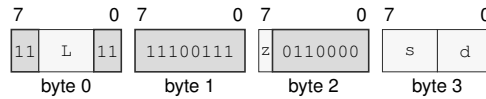
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FATANHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FATANHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FCOSA**Approximate Floating-Point Cosine****FCOSA**

Operation: Performs the approximate floating-point cosine operation.

Assembler Syntax: `FCOSA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\cos(x)$ reference in radians over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0002. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\cos(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is generated and the result is the architectural default quiet NaN.
- When the source is zero, the result is positive one.

Exact anchors

- $\cos(+0) = +1$
- $\cos(-0) = +1$

Required properties

- even
- nondecreasing on $[-\pi/4, 0]$ and nonincreasing on $[0, \pi/4]$
- result is in $[\sqrt{2}/2, 1]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	-	-

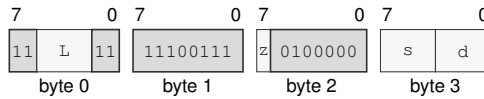
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FCOSA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FCOSA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FCOSHA**Approximate Floating-Point Hyperbolic Cosine****FCOSHA**

Operation: Performs the approximate floating-point hyperbolic cosine operation.

Assembler Syntax: `FCOSHA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\cosh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0022. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\cosh(x)$ on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite, the result is +infinity.
- When the source is zero, the result is positive one.

Exact anchors

- $\cosh(+0) = +1$
- $\cosh(-0) = +1$
- $\cosh(\text{infinity}) = +\text{infinity}$

Required properties

- even
- nonincreasing below zero and nondecreasing above zero
- result is at least 1

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	-	-

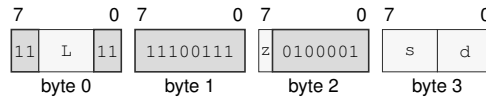
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FCOSHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FCOSHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FETOXA**Approximate Floating-Point e To x****FETOXA**

Operation: Performs the approximate floating-point e to x operation.

Assembler Syntax: `FETOXA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $e ** x$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0031. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $e ** x$ on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- $e ** \text{signed } 0 = +1$
- $e ** +\text{infinity} = +\text{infinity}$
- $e ** -\text{infinity} = +0$

Required properties

- strictly increasing for finite inputs
- result is nonnegative

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	-

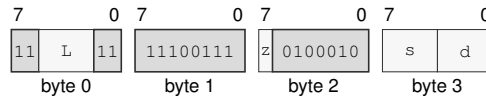
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FETOXA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FETOXA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FETOXM1A**Approximate Floating-Point e To x Minus One****FETOXM1A**

Operation: Performs the approximate floating-point e to x minus one operation.

Assembler Syntax: `FETOXM1A.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $(e ** x) - 1$ reference without first rounding $e ** x$ over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0032. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $(e ** x) - 1$ without an intermediate rounded exponential on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is -1.
- When the source is zero, the result is the source.

Exact anchors

- $\text{expm1}(+0) = +0$
- $\text{expm1}(-0) = -0$
- $\text{expm1}(-\text{infinity}) = -1$

Required properties

- strictly increasing for finite inputs
- result is at least -1

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	-

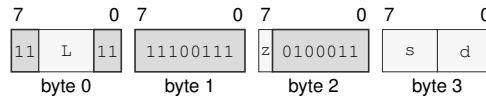
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FETOXM1A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FETOXM1A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FLOG10A**Approximate Floating-Point Log Base 10****FLOG10A**

Operation: Performs the approximate floating-point log base 10 operation.

Assembler Syntax: `FLOG10A.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

After DAZ, computes an implementation-bounded S- or D-format approximation of the $\log_{10}(x)$ reference under the selected FPTRANSA accuracy contract. The reference domain includes positive finite values and positive infinity; signed zero is the DZ boundary.

The required FPTRANSA_ACCURACY contract is 0x0044. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\log_{10}(x)$ on positive finite values and positive infinity; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is generated and the result is -infinity.
- When the source < 0 , NV is generated and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\log_{10}(+1) = +0$
- $\log_{10}(\text{signed } 0) = -\text{infinity}$
- $\log_{10}(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	-	*	-

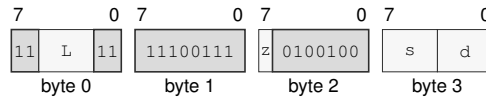
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FLOG10A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FLOG10A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FLOG2A**Approximate Floating-Point Log Base 2****FLOG2A**

Operation: Performs the approximate floating-point log base 2 operation.

Assembler Syntax: `FLOG2A.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

After DAZ, computes an implementation-bounded S- or D-format approximation of the $\log_2(x)$ reference under the selected FPTRANSA accuracy contract. The reference domain includes positive finite values and positive infinity; signed zero is the DZ boundary.

The required FPTRANSA_ACCURACY contract is 0x0043. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\log_2(x)$ on positive finite values and positive infinity; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is generated and the result is -infinity.
- When the source < 0 , NV is generated and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\log_2(+1) = +0$
- $\log_2(\text{signed } 0) = -\text{infinity}$
- $\log_2(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	-	*	-

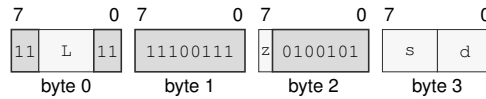
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FLOG2A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FLOG2A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FLOGNA**Approximate Floating-Point Natural Log****FLOGNA**

Operation: Performs the approximate floating-point natural log operation.

Assembler Syntax: `FLOGNA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

After DAZ, computes an implementation-bounded S- or D-format approximation of the $\ln(x)$ reference under the selected FPTRANSA accuracy contract. The reference domain includes positive finite values and positive infinity; signed zero is the DZ boundary.

The required FPTRANSA_ACCURACY contract is 0x0041. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\ln(x)$ on positive finite values and positive infinity; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is generated and the result is -infinity.
- When the source < 0 , NV is generated and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\ln(+1) = +0$
- $\ln(\text{signed } 0) = -\text{infinity}$
- $\ln(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FPTRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	-	*	-

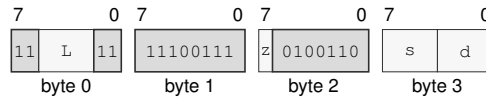
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FLOGNA.SD(z) Fn(s), Fn(d)

Instruction Format:



Format — Instruction format for FLOGNA.SD(z) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FLOGNP1A Approximate Floating-Point Natural Log Plus One FLOGNP1A

Operation: Performs the approximate floating-point natural log plus one operation.

Assembler Syntax: FLOGNP1A.SD(*z*) *F_n*(*s*), *F_n*(*d*)

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

After DAZ, computes an implementation-bounded S- or D-format approximation of the $\ln(1 + x)$ reference without an intermediate rounded addition under the selected FPTRANSA accuracy contract. The reference domain includes finite $x \geq -1$ and positive infinity; -1 is the DZ boundary.

The required FPTRANSA_ACCURACY contract is 0x0042. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\ln(1 + x)$ without an intermediate rounded addition on finite $x \geq -1$ and positive infinity; -1 is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source equals -1 , DZ is generated and the result is $-\infty$.
- When the source < -1 , NV is generated and the result is the architectural default quiet NaN.
- When the source is $+\infty$, the result is $+\infty$.
- When the source is zero, the result is the source.

Exact anchors

- $\log_1 p(+0) = +0$
- $\log_1 p(-0) = -0$
- $\log_1 p(-1) = -\infty$

Required properties

- strictly increasing on $(-1, +\infty)$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FPTRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	*	-	*	-

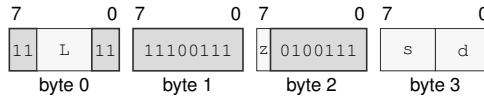
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FLOGNP1A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FLOGNP1A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FSINA**Approximate Floating-Point Sine****FSINA**

Operation: Performs the approximate floating-point sine operation.

Assembler Syntax: `FSINA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\sin(x)$ reference in radians over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0001. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\sin(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is generated and the result is the architectural default quiet NaN.
- When the source is zero, the result is the source.

Exact anchors

- $\sin(+0) = +0$
- $\sin(-0) = -0$

Required properties

- odd
- monotonically increasing on the contract domain
- result is in $[-\sqrt{2}/2, \sqrt{2}/2]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

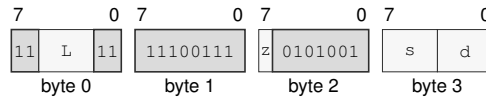
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FSINA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FSINA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FSINCOSA**Approximate Floating-Point Sine And Cosine****FSINCOSA**

Operation:	Performs the approximate floating-point sine and cosine operations.
Assembler Syntax:	<code>FSINCOSA.SD(z) Fn(s), Fn(d), Fn(c)</code>
Required CPUID flags:	FP FPTRANSA
Repeat eligibility:	Eligible as a REP body.
Exceptions:	<code>ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION</code>: when the declared destination-overlap constraint is violated <code>FLOATING_POINT_FAULT</code>: when a generated floating-point exception is enabled

Detailed Semantics

Computes the paired $\sin(x)$ and $\cos(x)$ references in radians over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes implementation-bounded S- or D-format approximations under the selected FPTRANSA accuracy contract.

The required `FPTRANSA_ACCURACY` contract is `0x0004`. If that accuracy contract's `PRESENT` field is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After `FSTATUS.DAZ` processing, the reference function is paired $\sin(x)$ and $\cos(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the two destinations name the same F register, `ILLEGAL_INSTRUCTION` is raised before operands or floating-point state are read.
- When the source is infinite or its magnitude exceeds $\pi / 4$, NV is generated and both results are the architectural default quiet NaN.
- When the source is zero, the sine result preserves the source zero and the cosine result is positive one.

Exact anchors

- $\sin(+0) = +0$ and $\cos(+0) = +1$
- $\sin(-0) = -0$ and $\cos(-0) = +1$

Required properties

- the ULP bound applies independently to both results
- results need not match separate FSINA and FCOSA executions

For nontrapping execution, the sine and cosine destinations are written atomically.

NaN handling, floating-point exception-cause generation, matching condition-enable testing, `FLOATING_POINT_FAULT` architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

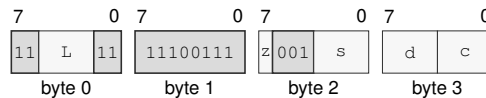
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

`FSINCOSA.SD(z) Fn(s), Fn(d), Fn(c)`

Instruction Format:



Format — Instruction format for `FSINCOSA.SD(z) Fn(s), Fn(d), Fn(c)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects operand 1.

Register field *d*—Selects operand 2.

Register field *c*—Selects operand 3.

Destination overlap—When the `sin_dst` and `cos_dst` operands designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

FSINHA**Approximate Floating-Point Hyperbolic Sine****FSINHA**

Operation: Performs the approximate floating-point hyperbolic sine operation.

Assembler Syntax: `FSINHA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\sinh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0021. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\sinh(x)$ on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or zero, the result is the source.

Exact anchors

- $\sinh(+0) = +0$
- $\sinh(-0) = -0$
- $\sinh(\text{infinity}) = \text{infinity}$ with the input sign

Required properties

- odd
- monotonically increasing

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	-

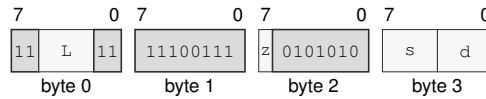
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FSINHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FSINHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FTANA**Approximate Floating-Point Tangent****FTANA**

Operation: Performs the approximate floating-point tangent operation.

Assembler Syntax: `FTANA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\tan(x)$ reference in radians over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0003. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\tan(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is generated and the result is the architectural default quiet NaN.
- When the source is zero, the result is the source.

Exact anchors

- $\tan(+0) = +0$
- $\tan(-0) = -0$

Required properties

- odd
- monotonically increasing on the contract domain
- result is in $[-1, 1]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

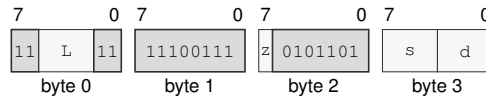
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FTANA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FTANA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FTANHA**Approximate Floating-Point Hyperbolic Tangent****FTANHA**

Operation: Performs the approximate floating-point hyperbolic tangent operation.

Assembler Syntax: `FTANHA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the $\tanh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0023. If that accuracy contract's PRESENT field is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\tanh(x)$ on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite, the result is one with the sign of the source.
- When the source is zero, the result is the source.

Exact anchors

- $\tanh(+0) = +0$
- $\tanh(-0) = -0$
- $\tanh(\text{signed infinity}) = \text{signed } 1$

Required properties

- odd
- monotonically increasing
- result is in $[-1, 1]$

NaN handling, floating-point exception-cause generation, matching condition-enable testing, `FLOATING_POINT_FAULT` architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	-	*	-

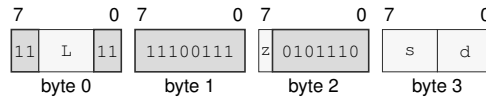
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FTANHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FTANHA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FTENTOX**Approximate Floating-Point 10 To x****FTENTOX**

Operation: Performs the approximate floating-point 10 to x operation.

Assembler Syntax: `FTENTOX.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the 10^{**x} reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0034. If that accuracy contract's PRESENT field is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is 10^{**x} on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- $10^{**\text{signed } 0} = +1$
- $10^{**+\text{infinity}} = +\text{infinity}$
- $10^{**-\text{infinity}} = +0$

Required properties

- strictly increasing for finite inputs
- result is nonnegative

NaN handling, floating-point exception-cause generation, matching condition-enable testing, `FLOATING_POINT_FAULT` architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	-

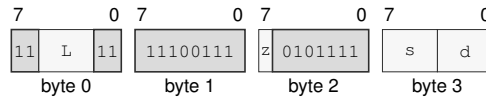
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FTENTOX.A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FTENTOX.A.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FTWOTOXA**Approximate Floating-Point 2 To x****FTWOTOXA**

Operation: Performs the approximate floating-point 2 to x operation.

Assembler Syntax: `FTWOTOXA.SD(z) Fn(s), Fn(d)`

Required CPUID flags: FP
FPTRANSA

Repeat eligibility: Eligible as a REP body.

Detailed Semantics

Computes the 2^x reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

The required FPTRANSA_ACCURACY contract is 0x0033. If that accuracy contract's PRESENT field is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is 2^x on all finite values and signed infinity. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- $2^{\text{signed } 0} = +1$
- $2^{\text{+infinity}} = \text{+infinity}$
- $2^{\text{-infinity}} = +0$

Required properties

- strictly increasing for finite inputs
- result is nonnegative

NaN handling, floating-point exception-cause generation, matching condition-enable testing, FLOATING_POINT_FAULT architectural-event delivery, and commit behavior follow the common FP-TRANSA model in Section 19.1.

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	-

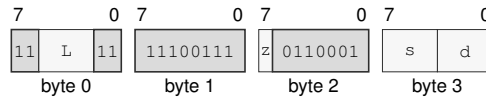
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

FTWOTOXA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Format:



Format — Instruction format for FTWOTOXA.SD(*z*) Fn(*s*), Fn(*d*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

20 SCALABLE-VECTOR INSTRUCTIONS

The scalable-vector extension provides 32 vector registers, V_0 through V_{31} , and 16 predicate registers, P_0 through P_{15} . A vector register contains $VLEN$ bits. $VLEN$ is an implementation-selected power of two from 128 through 2048 bits and remains stable until reset. A predicate register contains one bit for each byte of a vector register; its packed image therefore occupies $VLEN/64$ bytes. Reset clears every vector and predicate bit.

The `VECTOR` discovery predicate and the `VECTOR_PARAMETERS` CPUID leaf report the extension and $VLEN$. Integer and raw-bit vector forms require `VECTOR`. A form whose selected element type is `H`, `S`, or `D` additionally requires the base floating-point extension. The requirement follows the decoded element type, including a shared integer/floating-point opcode; disabling FP does not disable the integer forms of that opcode. Vector floating-point operations use the ordinary `FSTATUS` environment and accrue active-lane causes into `FFLAGS`.

20.1 Element Types and Assembly Spelling

A size-bearing vector instruction has one suffix. The integer element suffixes `B`, `W`, `L`, and `Q` select 1-, 2-, 4-, and 8-byte elements. The floating-point suffixes `H`, `S`, and `D` select IEEE 754 binary16, binary32, and binary64 elements. Binary16 is part of the vector FP facility; its default quiet NaN is `0x7e00`.

An opcode family that supports both integer and floating-point arithmetic uses the three-bit `tzz` selector. Values zero through three select `B/W/L/Q`, value four is invalid, and values five through seven select `H/S/D`. A family whose numeric domain is fixed by its opcode uses a two-bit `zz` selector: integer families assign all four values to `B/W/L/Q`, while FP families reserve zero and assign one through three to `H/S/D`. A reserved selector does not name another data format.

The suffix is the source width for a width-changing instruction; the destination width and signedness are part of the mnemonic. For example, `VEXTZQ.B` zero-extends a byte into a quadword. An instruction whose suffix controls only lane width accepts `H/S/D` as assembly aliases for `W/L/Q`; canonical disassembly uses `B/W/L/Q`. Predicate operations, `VLNT`, `VLCADD`, `VGATHER1`, `VSCATTER1`, `PLPOP`, `VMOV`, `VMOVZ`, and the permute, slide, slice, and zip families use this width-only rule.

20.2 Predicates and Destination Images

For an element size of E bytes, logical lane i is governed by predicate bit iE . All other predicate bits are nonsignificant for that operation. Unless an instruction explicitly produces a complete predicate image or uses zeroing movement, a false governing bit annuls that lane: it performs no memory access, produces no exception, and preserves the old destination lane.

An instruction that writes a predicate result writes a complete packed predicate image. It sets or clears each significant result bit and clears all nonsignificant bits. Integer comparisons support `EQ`, `NE`, `ULT`, `UGE`, `ULE`, `UGT`, `LT`, `GE`, `LE`, and `GT`. FP comparisons support `EQ`, `NE`, `LT`, `LE`, `GE`, `GT`, `VS` for unordered, and `VC` for ordered. `VTESTZ` and `VTESTNZ` are distinct integer test operations. Vector integer operations do not update scalar `FLAGS`.

`PLPOP` consumes a remaining-count register and an offset register. If the remaining count is zero, it takes its encoded branch target and does not change the predicate or either register. Otherwise, an offset greater than or equal to the selected lane count raises `VECTOR_RANGE_ERROR` with error

code zero and leaves the instruction uncommitted. A valid nonzero iteration activates the next $\min(\text{remaining}, \text{lane_count} - \text{offset})$ lanes, subtracts that number from the remaining count, clears the offset register, and falls through. Software advances the offset between chunks when it needs a nonzero subsequent chunk origin.

20.3 Lane Mapping and Scalar Bridges

Vector bytes use increasing-address little-endian order. A same-width operation treats a vector as VLEN divided by the selected element width lanes. For widening, narrowing, FP format conversion, and integer/FP conversion, the operation container is the larger of the source and destination widths. Each active container reads the source value from its low source-width part and writes the result into its low destination-width part. Other bits of an active destination container and every inactive container remain unchanged.

`VDUP` broadcasts a scalar register, FP register, or immediate into every lane. `VEXTRACT` and `VINSERT` use a scalar lane index. An index at or above the lane count raises `VECTOR_RANGE_ERROR` with error code one and commits no state. `VLCNT` returns VLEN in bytes divided by the selected element width in bytes. `VLCADD` adds its signed eight-bit immediate multiplied by that lane count to its scalar destination modulo 2^{64} . Both operations preserve `FLAGS`; their H/S/D spellings are width-only aliases for W/L/Q and do not require the floating-point extension.

20.4 Vector Memory Operations

A vector memory operand uses the ordinary compact, `EXT1`, and `EXT2` descriptor formats in vector context. The descriptor computes one scalar anchor address. A contiguous access uses

$$\text{address}[i] = \text{anchor} + iE,$$

All arithmetic is modulo 2^{64} . Existing effective-address auto-update is applied once to the scalar anchor in the ordinary architectural order; it is not repeated per lane.

Inactive lanes issue no memory transaction. `VMOV` merges a memory source into its destination, while `VMOVZ` writes zero to inactive destination lanes. A memory-destination arithmetic form is a lane-wise read-modify-write, not an atomic operation. A vector instruction nevertheless has one architectural commit point: address evaluation, all active reads, computation, and all active writes must succeed before any register, predicate, memory, auto-update, `FFLAGS`, or PC effect becomes visible. After range, wrap, and canonicity checks, the implementation completes translation of every mapping covered by every active lane in increasing linear-address order. Only after all such translations succeed does it apply physical-class checks, the vector/gather/scatter MMIO-operation prohibition, alignment checks, and target or bus accesses, in that order. Consequently, a translation fault in a higher-address active lane takes precedence over an MMIO condition discovered in a lower-address active lane. When logical lane order differs from address order, translation and translation-fault selection still follow address order. The reported fault address identifies the lane whose range supplies the selected failure.

20.4.1 One-Lane Gather and Scatter Steps

VGATHER1 and VSCATTER1 are one-lane vector memory instructions eligible for unconditional REP and not for REPcc. Their canonical operand order is

```
VGATHER1.T Pn, [address], Vd
VSCATTER1.T Pn, Vs, [address]
```

The read-write lane cursor R_i appears inside every address expression. The vector address operand V_a has no suffix; the instruction suffix selects both the transferred element width and the width of $V_a[R_i]$. The encoded widths are B/W/L/Q. The H/S/D spellings are width-only aliases for W/L/Q, preserve raw element bits, require no floating-point extension, and disassemble canonically as W/L/Q.

For element width E bytes, let $i = \text{unsigned}(R_i)$ and $N = VLEN/E$. The range check precedes predicate evaluation. If $i \geq N$, the instruction raises VECTOR_RANGE_ERROR with error code one and commits nothing. Otherwise predicate bit $P_n[iE]$ governs the step. A false bit issues no memory transaction, increments R_i modulo 2^{64} , and commits. A true bit performs exactly one E -byte access through the default data segment. On success, gather replaces only lane i of V_d , or scatter stores lane i of V_s ; the instruction then increments R_i and commits. A memory fault leaves R_i , vector state, and memory unchanged. Both instructions preserve FLAGS.

The address forms, before default-data-segment translation, are:

Form	Offset
scalar stride	$R_b + \text{unsigned}(R_i) \cdot \text{signed}_{64}(R_s)$
absolute 32	$\text{zero_extend}_{64}(V_a.L[i])$
absolute 64	$V_a.Q[i]$
vector byte displacement	$R_b + \text{sign_extend}_{64}(V_a.E[i])$
vector index	$R_b + \text{zero_extend}_{64}(V_a.E[i]) E$
indexed plus disp8s/16s/32s	preceding indexed offset plus the signed payload
indexed plus disp64	preceding indexed offset plus the exact 64-bit payload

Address arithmetic is modulo 2^{64} . The absolute forms are offsets in the default data segment; they do not bypass segmentation. Absolute 32 is the fixed L/S width and zero-extends $V_a.L[i]$; absolute 64 is the fixed Q/D width. In indexed syntax the scale is a literal equal to E and is not a separately encoded field. The signed 8-, 16-, and 32-bit displacements and exact 64-bit displacement are appended payloads.

Every base form requires $R_i \neq R_b$; scalar stride also requires $R_i \neq R_s$, while $R_b == R_s$ is permitted. $V_a == V_d$ for gather and $V_a == V_s$ for scatter are permitted. The existing vector/gather/scatter MMIO-operation prohibition applies to these one-element transactions.

20.5 Floating-Point Exceptions, Conversions, and Reductions

Each active FP lane applies the corresponding scalar rounding, NaN, signed-zero, default-NaN, denormal, and exception rules. Inactive lanes generate no cause. The instruction unions all

active-lane causes before testing the enables in the original FSTATUS. If any generated cause is enabled, one FLOATING_POINT_FAULT occurs and neither a destination effect nor newly accrued FFLAGS becomes visible. Otherwise the union is accrued once and all destination effects commit together. FP-to-integer conversion rounds toward zero and follows the scalar saturation and invalid result rules.

Integer reductions return a zero-extended result in an R_n . For an empty predicate, ADD, OR, and XOR return zero; AND and unsigned MIN return the all-ones value of the selected width; signed MIN returns the signed maximum; signed MAX returns the signed minimum; and unsigned MAX returns zero. FP reductions return an F_n . Empty FP ADD returns positive zero, empty FP MIN returns positive infinity, and empty FP MAX returns negative infinity. FP ADD visits active lanes in increasing logical-lane order and may not be reassociated.

20.6 Execution and Saved State

The vector SAVE/RESTORE component contains the complete scalable vector and packed predicate images. A committed V_n or P_n write marks that component modified. Architectural event entry does not otherwise change vector state.

An implementation may internally coalesce eligible scalar REP or REP_{CC} iterations into vector-sized work. This permission creates no vector instruction, predicate, or additional repeat state. The observable counter, condition, FLAGS, memory, auto-update, fault, event, and restart behavior must equal the scalar iteration sequence, and only the scalar prefix preceding the first fault or terminating REP_{CC} observation may commit.

20.7 Summary

Table 20-1. Scalable-Vector Instructions Summary (Informative)

Mnemonic	Brief description
VDUP	Broadcast the selected R_n subfield to every lane.
VMOV	Copy the complete scalable register image.
PHEAD	Sets predicate positions from the count remainder through the final lane.
PTAIL	Set significant positions before 'unsigned(Rcount) mod lane_count'.
PFIRST	Return the first selected element index at a significant position.
PLAST	Return the last selected element index at a significant position.
PCOUNT	Count set significant positions for the selected element size.
PAND	Destructive predicate AND: ' $P_d = P_d \text{ AND } P_s$ '.
POR	Destructive predicate OR: ' $P_d = P_d \text{ OR } P_s$ '.
PXOR	Destructive predicate XOR: ' $P_d = P_d \text{ XOR } P_s$ '.
PUNPKLO	Unpacks the lower half of source predicate positions into wider destination positions.
PUNPKHI	Unpacks the upper half of source predicate positions into wider destination positions.
PPACKLO	Packs source predicate positions into the lower half of narrower destination positions.
PPACKHI	Packs source predicate positions into the upper half of narrower destination positions.
VCLR	Clear every bit of ' V_d '.
VINDEX	Write each lane's logical index.
VLCNT	Write the number of lanes of the selected element width.
VLCADD	Add a signed immediate multiple of the selected lane count to ' R_n '.
VGATHER1	Conditionally gather one vector lane and advance its lane cursor.

Table 20-1 (continued)

Mnemonic	Brief description
VSCATTER1	Conditionally scatter one vector lane and advance its lane cursor.
PTRUE	Set every significant position for the selected size.
PFALSE	Clear the complete predicate image.
PNOT	Complement 'Pd'.
BPANY	Branch if any bit is set.
BPNONE	Branch if no bit is set.
BPALL	Branch if every significant position for the selected size is set.
VNEG	Integer two's-complement negation or FP sign toggle.
VABS	Integer signed absolute value or FP sign clear.
VNOT	Bitwise complement.
VCLZ	Count leading zeros.
VCTZ	Count trailing zeros.
VCLS	Count leading ones.
VCTS	Count trailing ones.
VPOPCNT	Population count.
VREVBYTE	Reverse bytes within each element.
VSQRT	Square root.
VROUND	Integral FP result under the current rounding mode.
VTRUNC	Integral FP result toward zero.
VFLOOR	Integral FP result toward negative infinity.
VCEIL	Integral FP result toward positive infinity.
VCLASS	Write a per-lane FP classification bitmap to a vector.
PPERM	Select predicate lanes using unsigned vector indices.
PSLIDEUP	Move predicate lanes toward higher indices and insert zero at the low end.
PSLIDEDN	Move predicate lanes toward lower indices and insert zero at the high end.
VCMPcc	Compare active integer or FP lanes and write a complete predicate result.
VTESTZ	Test 'Va AND Vb' for zero and write a complete predicate result.
VTESTNZ	Test 'Va AND Vb' for nonzero and write a complete predicate result.
VADD	Adds corresponding active vector elements using integer or floating-point arithmetic.
VSUB	Modular integer or IEEE floating-point subtraction.
VMUL	Low integer product or IEEE floating-point multiplication.
VAND	Bitwise AND.
VOR	Bitwise OR.
VXOR	Bitwise XOR.
VMINS	Signed minimum.
VMINU	Unsigned minimum.
VMAXS	Signed maximum.
VMAXU	Unsigned maximum.
VMULHS	High half of a signed product.
VMULHU	High half of an unsigned product.
VMULHSU	High half of signed-by-unsigned product.
VSHL	Logical left shift.
VSHR	Logical right shift.
VSAR	Arithmetic right shift.
VROL	Rotate left.

Table 20-1 (continued)

Mnemonic	Brief description
VROR	Rotate right.
VMIN	FP minimum using the scalar FMIN NaN and signed-zero rules.
VMAX	FP maximum using the scalar FMAX NaN and signed-zero rules.
VDIV	Division.
VCOPYSIGN	Combine magnitude and sign operands.
VEXTZW	Zero-extends B source elements to W destination elements.
VEXTSW	Sign-extends B source elements to W destination elements.
VEXTZL	Zero-extends B or W source elements to L destination elements.
VEXTSL	Sign-extends B or W source elements to L destination elements.
VEXTZQ	Zero-extends B, W, or L source elements to Q destination elements.
VEXTSQ	Sign-extends B, W, or L source elements to Q destination elements.
VTRUNCB	Copies the low B field from each W, L, or Q source container.
VTRUNCW	Copies the low W field from each L or Q source container.
VTRUNCL	Copies the low L field from each Q source container.
VCVTS	Convert H or D to S.
VCVTD	Convert H or S to D.
VCVTUS	Unsigned integer to S.
VCVTUD	Unsigned integer to D.
VCVTL	FP to signed L.
VCVTUL	FP to unsigned L.
VCVTQ	FP to signed Q.
VCVTUQ	FP to unsigned Q.
VPERM	Select source lanes with vector indices; out-of-range indices produce zero.
VZIPL0	Interleave the lower halves of the old destination and source.
VZIPHI	Interleave the upper halves of the old destination and source.
VUZIPL0	Gather even lanes from the old destination and source into separate result halves.
VUZIPHI	Gather odd lanes from the old destination and source into separate result halves.
VTRNLO	Interleave even lanes from the old destination and source by adjacent result pairs.
VTRNHI	Interleave odd lanes from the old destination and source by adjacent result pairs.
VCVTH	Convert S or D to H.
VCVTUH	Unsigned integer to H.
VMADD	Fused multiply-add with read/write accumulator destination.
VMSUB	Fused multiply-subtract.
VNMADD	Negated fused multiply-add.
VNMSUB	Negated fused multiply-subtract.
VSLICE	Extract a lane window from the old destination followed by a source vector.
VSLIDEUP	Move old destination lanes toward higher indices and zero vacated lanes.
VSLIDEDN	Move old destination lanes toward lower indices and zero vacated lanes.
VEXTRACT	Extract one integer lane.
VINSERT	Replace one integer lane.
VREDADD	Modular sum of active lanes to Rn.
VREDMINS	Signed minimum.
VREDMINU	Unsigned minimum.
VREDMAXS	Signed maximum.
VREDMAXU	Unsigned maximum.

Table 20-1 (continued)

Mnemonic	Brief description
VREDAND	Bitwise AND.
VREDOR	Bitwise OR.
VREDXOR	Bitwise XOR.
VREDMIN	Active-lane FP minimum to Fn.
VREDMAX	Active-lane FP maximum to Fn.
PSEL	Replace bits of 'Pd' selected by 'Pc' with bits from 'Ps'.
PZIPLO	Interleave the lower halves of two predicate-lane sequences.
PZIPHI	Interleave the upper halves of two predicate-lane sequences.
PUZIPLO	Concatenate even-position predicate lanes from two sources.
PUZIPHI	Concatenate odd-position predicate lanes from two sources.
PTRNLO	Interleave even-position predicate lanes from two sources.
PTRNHI	Interleave odd-position predicate lanes from two sources.
PSLICE	Extract a predicate-lane window from the old destination followed by a source predicate.
VMOVZ	Move active memory elements into 'Vd'; inactive lanes become zero.
PLOOP	Performs the PLOOP scalable-vector operation.
PMOV	Read a complete packed predicate image from memory.

VDUP**VDUP (Vector)****VDUP**

Operation: Broadcast the selected Rn subfield to every lane. Broadcast an immediate value. Broadcast the selected FP-format value from 'Fn'.

Assembler Syntax: VDUP.BWLQ(z) Rn(r), Vn(v)
 VDUP.HSD(z) Fn(r), Vn(v)
 VDUP.B <imm8>, Vn(v)
 VDUP.W <imm16>, Vn(v)
 VDUP.L <imm32>, Vn(v)
 VDUP.Q <imm64>, Vn(v)

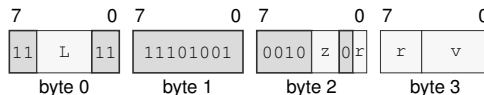
Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-12 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VDUP.BWLQ(z) Rn(r), Vn(v)

Instruction Format:

Format — Instruction format for VDUP.BWLQ(z) Rn(r), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

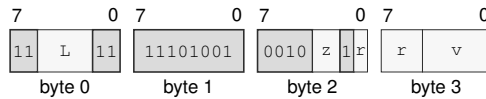
Size field z—Selects B/W/L/Q.

Register field r—Selects the source operand.

Bits field v—Encodes the bits value.

VDUP.HSD(z) Fn(r), Vn(v)

Instruction Format:



Format — Instruction format for VDUP.HSD(z) Fn(r), Vn(v)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

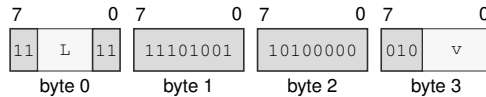
Size field z —Selects H/S/D.

Register field r —Selects the source operand.

Bits field v —Encodes the bits value.

VDUP.B <imm8>, Vn(v)

Instruction Format:



Format — Instruction format for VDUP.B <imm8>, Vn(v)

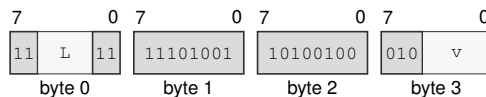
Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field v —Encodes the bits value.

VDUP.W <imm16>, Vn(v)

Instruction Format:



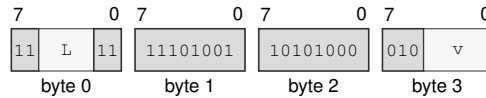
Format — Instruction format for VDUP.W <imm16>, Vn(v)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field v —Encodes the bits value.

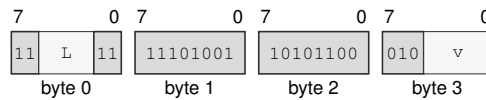
VDUP.L <imm32>, Vn(v)

Instruction Format:**Format — Instruction format for VDUP.L <imm32>, Vn(v)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field v—Encodes the bits value.

VDUP.Q <imm64>, Vn(v)

Instruction Format:**Format — Instruction format for VDUP.Q <imm64>, Vn(v)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field v—Encodes the bits value.

VMOV

VMOV (Vector)

VMOV

Operation: Copy the complete scalable register image. Replace lanes of ‘Vd’ selected by ‘Pn’ with lanes from ‘Vs’. Move active memory elements into ‘Vd’; inactive lanes preserve ‘Vd’. Move active vector elements to memory.

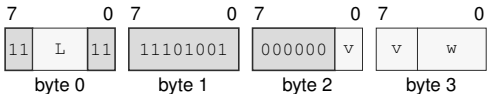
Assembler Syntax: VMOV Vn(v), Vn(w)
VMOV.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMOV.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMOV.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 4-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMOV Vn(v), Vn(w)

Instruction Format:



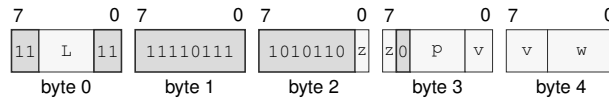
Format — Instruction format for VMOV Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMOV.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VMOV.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

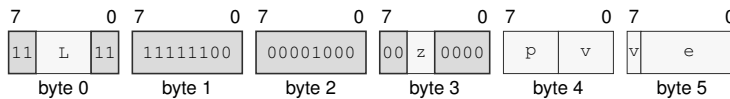
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VMOV.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMOV.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

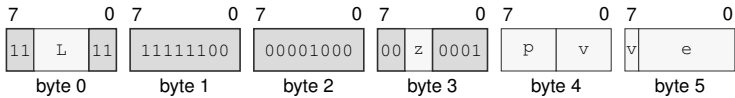
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMOV.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMOV.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Bits field p**—Encodes the bits value.
- Bits field v**—Encodes the bits value.
- Effective Address field e**—Specifies the destination operand. Immediate addressing is unavailable in this form.

PHEAD

Predicate Head

PHEAD

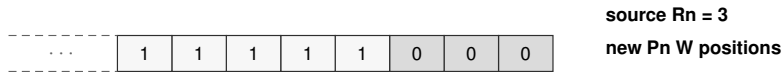
Operation: Sets predicate positions from the count remainder through the final lane.

Assembler Syntax: PHEAD.BWLQ(*z*) Rn(*r*), Pn(*p*)

Required CPUID flags: VECTOR

Detailed Semantics

PHEAD computes the selected-element lane count and starts at the unsigned Rcount remainder modulo that count. It clears predicate positions below the start position and sets positions from the start through the final lane.

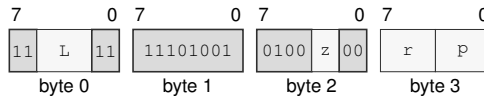


Example W-position fill beginning at count remainder 3.

Encodings:

PHEAD.BWLQ(*z*) Rn(*r*), Pn(*p*)

Instruction Format:



Format — Instruction format for PHEAD.BWLQ(*z*) Rn(*r*), Pn(*p*)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field r—Selects the source operand.

Bits field p—Encodes the bits value.

PTAIL

PTAIL (Vector)

PTAIL

Operation: Set significant positions before ‘unsigned(Rcount) mod lane_count’.

Assembler Syntax: PTAIL.BWLQ(z) Rn(r), Pn(p)

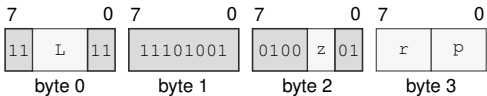
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 4 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

PTAIL.BWLQ(z) Rn(r), Pn(p)

Instruction Format:



Format — Instruction format for PTAIL.BWLQ(z) Rn(r), Pn(p)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field r**—Selects the source operand.
- Bits field p**—Encodes the bits value.

PFIRST**PFIRST (Vector)****PFIRST**

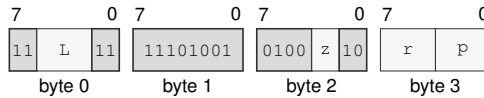
Operation: Return the first selected element index at a significant position.

Assembler Syntax: `PFIRST.BWLQ(z) Pn(p), Rn(r)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

`PFIRST.BWLQ(z) Pn(p), Rn(r)`

Instruction Format:

Format — Instruction format for PFIRST.BWLQ(z) Pn(p), Rn(r)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *r*—Selects the destination operand.

Bits field *p*—Encodes the bits value.

PLAST

PLAST (Vector)

PLAST

Operation: Return the last selected element index at a significant position.

Assembler Syntax: PLAST.BWLQ(z) Pn(p), Rn(r)

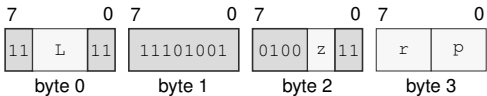
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 4 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

PLAST.BWLQ(z) Pn(p), Rn(r)

Instruction Format:



Format — Instruction format for PLAST.BWLQ(z) Pn(p), Rn(r)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field r**—Selects the destination operand.
- Bits field p**—Encodes the bits value.

PCOUNT**PCOUNT (Vector)****PCOUNT**

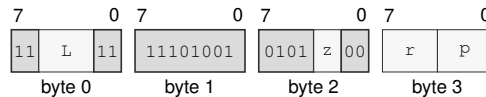
Operation: Count set significant positions for the selected element size.

Assembler Syntax: `PCOUNT.BWLQ(z) Pn(p), Rn(r)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

`PCOUNT.BWLQ(z) Pn(p), Rn(r)`

Instruction Format:

Format — Instruction format for PCOUNT.BWLQ(z) Pn(p), Rn(r)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *r*—Selects the destination operand.

Bits field *p*—Encodes the bits value.

PAND

PAND (Vector)

PAND

Operation: Destructive predicate AND: ‘Pd = Pd AND Ps’.

Assembler Syntax: PAND Pn(p) , Pn(q)

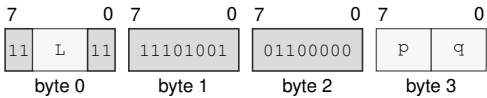
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 4 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

PAND Pn(p) , Pn(q)

Instruction Format:



Format — Instruction format for PAND Pn(p), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.

POR

POR (Vector)

POR

Operation: Destructive predicate OR: 'Pd = Pd OR Ps'.

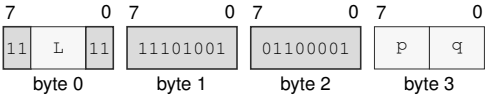
Assembler Syntax: POR Pn(p), Pn(q)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

POR Pn(p), Pn(q)

Instruction Format:



Format — Instruction format for POR Pn(p), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

PXOR

PXOR (Vector)

PXOR

Operation: Destructive predicate XOR: ‘Pd = Pd XOR Ps’.

Assembler Syntax: PXOR Pn(p), Pn(q)

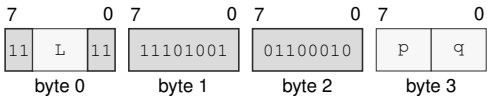
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 4 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

PXOR Pn(p), Pn(q)

Instruction Format:



Format — Instruction format for PXOR Pn(p), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.

PUNPKLO

Predicate Unpack from Lower Half

PUNPKLO

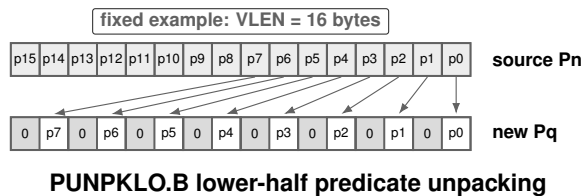
Operation: Unpacks the lower half of source predicate positions into wider destination positions.

Assembler Syntax: PUNPKLO.BWL(z) Pn(p), Pn(q)

Required CPUID flags: VECTOR

Detailed Semantics

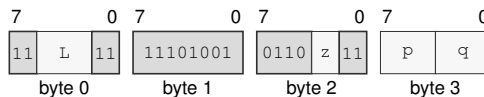
PUNPKLO reads the significant predicate positions for the selected B, W, or L source element width. It selects the lower half of those positions and copies them in lane order to the significant destination positions for elements twice that width. The complete destination predicate image is written, and every nonsignificant destination bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

PUNPKLO.BWL(z) Pn(p), Pn(q)

Instruction Format:



Format — Instruction format for PUNPKLO.BWL(z) Pn(p), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

PUNPKHI

Predicate Unpack from Upper Half

PUNPKHI

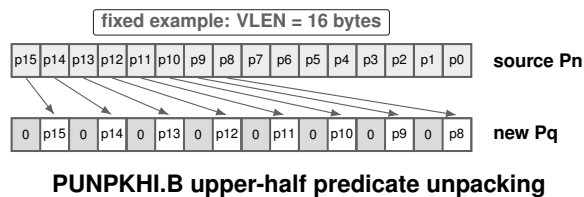
Operation: Unpacks the upper half of source predicate positions into wider destination positions.

Assembler Syntax: `PUNPKHI.BWL(z) Pn(p), Pn(q)`

Required CPUID flags: VECTOR

Detailed Semantics

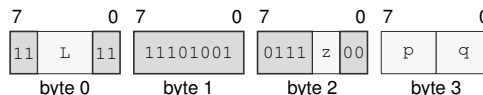
PUNPKHI reads the significant predicate positions for the selected B, W, or L source element width. It selects the upper half of those positions and copies them in lane order to the significant destination positions for elements twice that width. The complete destination predicate image is written, and every nonsignificant destination bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

`PUNPKHI.BWL(z) Pn(p), Pn(q)`

Instruction Format:



Format — Instruction format for PUNPKHI.BWL(z) Pn(p), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

PPACKLO**Predicate Pack to Lower Half****PPACKLO**

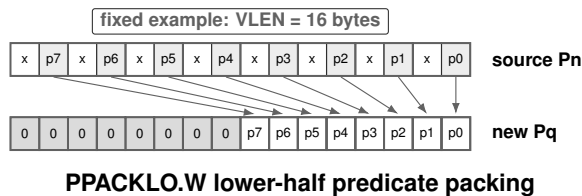
Operation: Packs source predicate positions into the lower half of narrower destination positions.

Assembler Syntax: PPACKLO.WLQ(z) Pn(p), Pn(q)

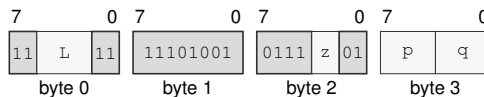
Required CPUID flags: VECTOR

Detailed Semantics

PPACKLO reads the significant predicate positions for the selected W, L, or Q source element width. It copies those positions in lane order to the lower half of the significant destination positions for elements half that width. The complete destination predicate image is written: the upper half of its significant positions and every nonsignificant bit are cleared. The operation leaves FLAGS unchanged.

**Encodings:**

PPACKLO.WLQ(z) Pn(p), Pn(q)

Instruction Format:

Format — Instruction format for PPACKLO.WLQ(z) Pn(p), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

PPACKHI

Predicate Pack to Upper Half

PPACKHI

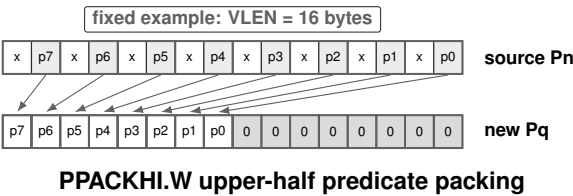
Operation: Packs source predicate positions into the upper half of narrower destination positions.

Assembler Syntax: PPACKHI.WLQ(z) Pn(p), Pn(q)

Required CPUID flags: VECTOR

Detailed Semantics

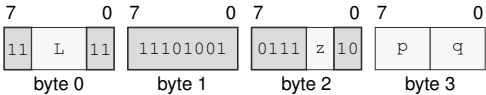
PPACKHI reads the significant predicate positions for the selected W, L, or Q source element width. It copies those positions in lane order to the upper half of the significant destination positions for elements half that width. The complete destination predicate image is written: the lower half of its significant positions and every nonsignificant bit are cleared. The operation leaves FLAGS unchanged.



Encodings:

PPACKHI.WLQ(z) Pn(p), Pn(q)

Instruction Format:



Format — Instruction format for PPACKHI.WLQ(z) Pn(p), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.

VCLR**VCLR (Vector)****VCLR**

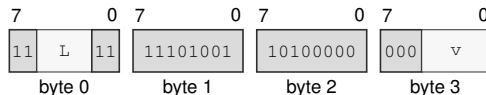
Operation: Clear every bit of 'Vd'.

Assembler Syntax: VCLR V_n(v)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCLR V_n(v)

Instruction Format:

Format — Instruction format for VCLR V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field v—Encodes the bits value.

VINDEX

VINDEX (Vector)

VINDEX

Operation: Write each lane's logical index.

Assembler Syntax: VINDEX.BWLQ(z) Vn(v)

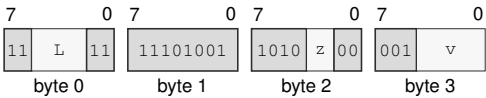
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 4 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

VINDEX.BWLQ(z) Vn(v)

Instruction Format:



Format — Instruction format for VINDEX.BWLQ(z) Vn(v)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** v—Encodes the bits value.

VLCNT**Vector Lane Count****VLCNT**

Operation: Write the number of lanes of the selected element width.

Assembler Syntax: `VLCNT.BWLQ(z) Rn(r)`

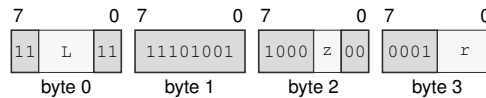
Required CPUID flags: VECTOR

Detailed Semantics

Write VLEN in bytes divided by the selected element width in bytes to 'Rn'.

Encodings:

`VLCNT.BWLQ(z) Rn(r)`

Instruction Format:

Format — Instruction format for VLCNT.BWLQ(z) Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field r—Selects the operand.

VLCADD

Vector Lane-Count Add

VLCADD

Operation: Add a signed immediate multiple of the selected lane count to ‘Rn’.

Assembler Syntax: VLCADD.BWLQ(z) <imm8s>, Rn(r)

Required CPUID flags: VECTOR

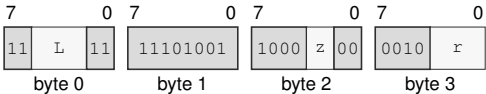
Detailed Semantics

Add signed ‘imm8’ multiplied by VLEN in bytes divided by the selected element width in bytes, modulo 2^64.

Encodings:

VLCADD.BWLQ(z) <imm8s>, Rn(r)

Instruction Format:



Format — Instruction format for VLCADD.BWLQ(z) <imm8s>, Rn(r)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field r**—Selects the destination operand.

VGATHER1**One-Lane Vector Gather Step****VGATHER1**

Operation: Conditionally gather one vector lane and advance its lane cursor.

Assembler Syntax: VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Rn(i) * Rn(s)], Vn(v)
 VGATHER1.L Pn(p), [Vn(x)[Rn(i)]], Vn(v)
 VGATHER1.Q Pn(p), [Vn(x)[Rn(i)]], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)]], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale>], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp8s>], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp16s>], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp32s>], Vn(v)
 VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp64>], Vn(v)

Required CPUID flags: VECTOR

Repeat eligibility: Eligible as a REP body.

Exceptions: VECTOR_RANGE_ERROR: when the unsigned lane cursor is greater than or equal to the selected lane count

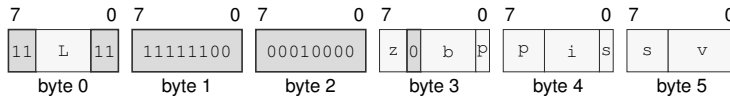
Detailed Semantics

Range-check the lane cursor, then conditionally load one selected element through the default data segment and advance the cursor.

Encodings:

$\text{VGATHER1.BWLQ}(z) \text{ Pn}(p), [\text{Rn}(b) + \text{Rn}(i) * \text{Rn}(s)], \text{Vn}(v)$

Instruction Format:



Format — Instruction format for $\text{VGATHER1.BWLQ}(z) \text{ Pn}(p), [\text{Rn}(b) + \text{Rn}(i) * \text{Rn}(s)], \text{Vn}(v)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects B/W/L/Q.

Register field b —Selects operand 2.

Bits field p —Encodes the bits value.

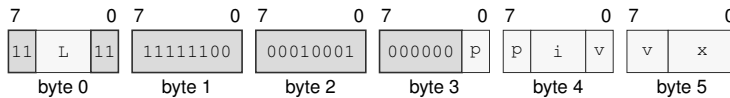
Register field i —Selects operand 2.

Register field s —Selects operand 2.

Bits field v —Encodes the bits value.

$\text{VGATHER1.L Pn}(p), [\text{Vn}(x)[\text{Rn}(i)]], \text{Vn}(v)$

Instruction Format:



Format — Instruction format for $\text{VGATHER1.L Pn}(p), [\text{Vn}(x)[\text{Rn}(i)]], \text{Vn}(v)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p —Encodes the bits value.

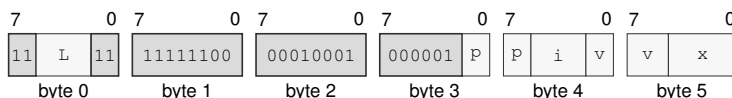
Register field i —Selects operand 2.

Bits field v —Encodes the bits value.

Bits field x —Encodes the bits value.

VGATHER1.Q Pn(p), [Vn(x)[Rn(i)]], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.Q Pn(p), [Vn(x)[Rn(i)]], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

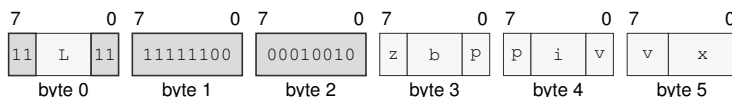
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)]], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)]], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 2.

Bits field p—Encodes the bits value.

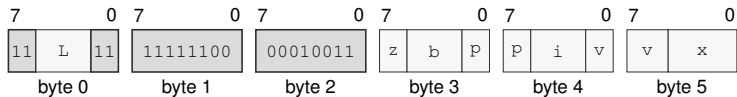
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale>], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale>], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 2.

Bits field p—Encodes the bits value.

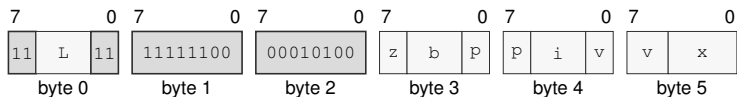
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp8s>], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp8s>], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 2.

Bits field p—Encodes the bits value.

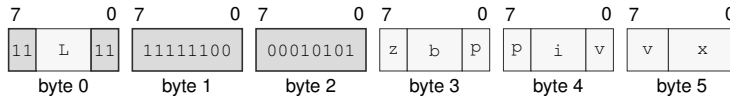
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp16s>], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp16s>], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 2.

Bits field p—Encodes the bits value.

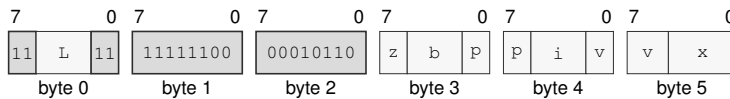
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp32s>], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp32s>], Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 2.

Bits field p—Encodes the bits value.

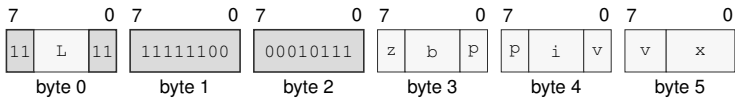
Register field i—Selects operand 2.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp64>], Vn(v)

Instruction Format:



Format — Instruction format for VGATHER1.BWLQ(z) Pn(p), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp64>], Vn(v)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field b**—Selects operand 2.
- Bits field p**—Encodes the bits value.
- Register field i**—Selects operand 2.
- Bits field v**—Encodes the bits value.
- Bits field x**—Encodes the bits value.

VSCATTER1**One-Lane Vector Scatter Step****VSCATTER1**

Operation: Conditionally scatter one vector lane and advance its lane cursor.

Assembler Syntax: VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Rn(i) * Rn(s)]
 VSCATTER1.L Pn(p), Vn(v), [Vn(x)[Rn(i)]]
 VSCATTER1.Q Pn(p), Vn(v), [Vn(x)[Rn(i)]]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)]]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] *
 <scale>]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] *
 <scale> + <disp8s>]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] *
 <scale> + <disp16s>]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] *
 <scale> + <disp32s>]
 VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] *
 <scale> + <disp64>]

Required CPUID flags: VECTOR

Repeat eligibility: Eligible as a REP body.

Exceptions: VECTOR_RANGE_ERROR: when the unsigned lane cursor is greater than or equal to the selected lane count

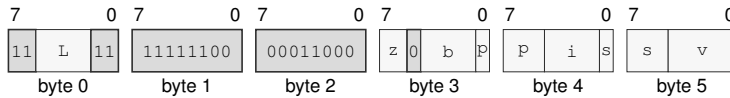
Detailed Semantics

Range-check the lane cursor, then conditionally store one selected element through the default data segment and advance the cursor.

Encodings:

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [$Rn(b) + Rn(i) * Rn(s)$]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [$Rn(b) + Rn(i) * Rn(s)$]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

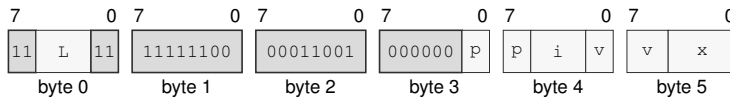
Register field i—Selects operand 3.

Register field s—Selects operand 3.

Bits field v—Encodes the bits value.

VSCATTER1.L Pn(p), Vn(v), [$Vn(x)[Rn(i)]$]

Instruction Format:



Format — Instruction format for VSCATTER1.L Pn(p), Vn(v), [$Vn(x)[Rn(i)]$]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

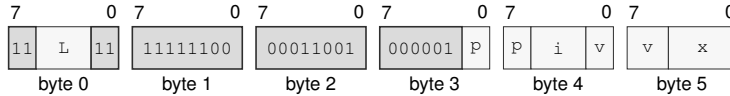
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.Q Pn(p), Vn(v), [Vn(x) [Rn(i)]]

Instruction Format:



Format — Instruction format for VSCATTER1.Q Pn(p), Vn(v), [Vn(x)[Rn(i)]]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

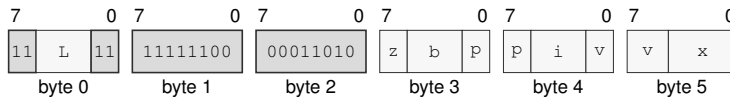
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x) [Rn(i)]]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)]]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

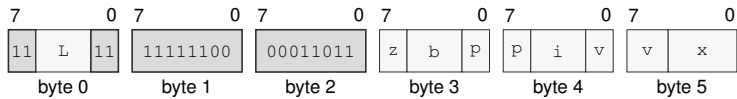
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale>]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale>]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

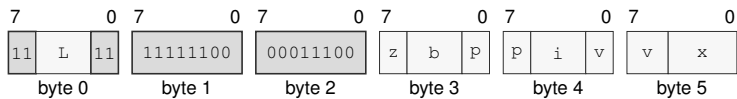
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp8s>]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp8s>]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

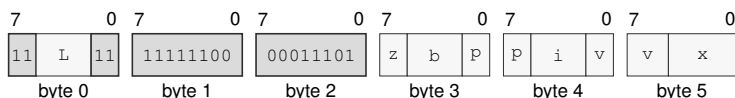
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp16s>]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp16s>]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

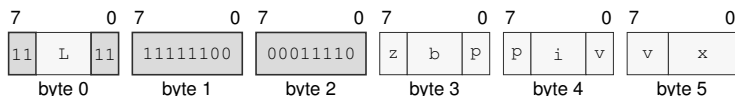
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp32s>]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp32s>]

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field b—Selects operand 3.

Bits field p—Encodes the bits value.

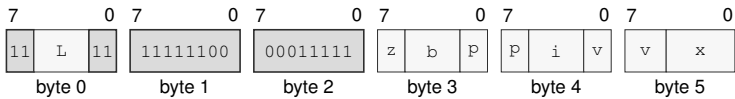
Register field i—Selects operand 3.

Bits field v—Encodes the bits value.

Bits field x—Encodes the bits value.

VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x) [Rn(i)] * <scale> + <disp64>]

Instruction Format:



Format — Instruction format for VSCATTER1.BWLQ(z) Pn(p), Vn(v), [Rn(b) + Vn(x)[Rn(i)] * <scale> + <disp64>]

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Register field b**—Selects operand 3.
- Bits field p**—Encodes the bits value.
- Register field i**—Selects operand 3.
- Bits field v**—Encodes the bits value.
- Bits field x**—Encodes the bits value.

PTRUE

PTRUE (Vector)

PTRUE

Operation: Set every significant position for the selected size.

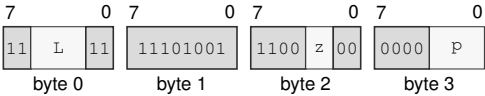
Assembler Syntax: PTRUE.BWLQ(z) Pn(p)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

PTRUE.BWLQ(z) Pn(p)

Instruction Format:



Format — Instruction format for PTRUE.BWLQ(z) Pn(p)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Bits field p**—Encodes the bits value.

PFALSE

PFALSE (Vector)

PFALSE

Operation: Clear the complete predicate image.

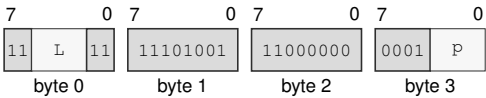
Assembler Syntax: PFALSE Pn(p)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 4 bytes
Feature = VECTOR
Repeat = none

Encodings:

PFALSE Pn(p)

Instruction Format:



Format — Instruction format for PFALSE Pn(p)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.

PNOT

PNOT (Vector)

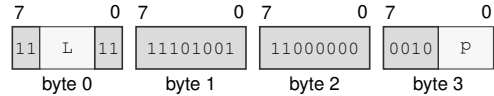
PNOT

Operation:	Complement 'Pd'.
Assembler Syntax:	PNOT P _n (p)
Attributes:	Class = vector Family = vector Privilege = unprivileged Length = 4 bytes Feature = VECTOR Repeat = none

Encodings:

PNOT P_n(p)

Instruction Format:



Format — Instruction format for PNOT P_n(p)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

BPANY

BPANY (Vector)

BPANY

Operation: Branch if any bit is set.

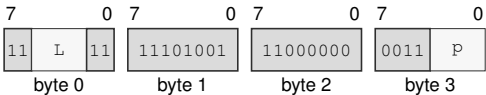
Assembler Syntax: BPANY Pn(p), <imm32s>
BPANY Pn(p), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 6-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

BPANY Pn(p), <imm32s>

Instruction Format:



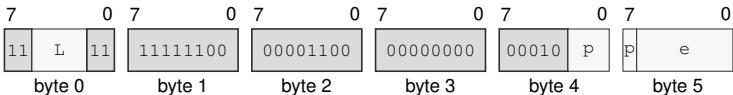
Format — Instruction format for BPANY Pn(p), <imm32s>

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.

BPANY Pn(p), <ea>(e)

Instruction Format:



Format — Instruction format for BPANY Pn(p), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.
- Effective Address field** e—Specifies the control target.

BPNONE**BPNONE (Vector)****BPNONE**

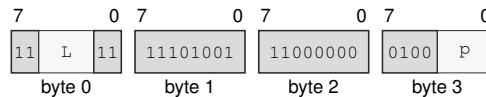
Operation: Branch if no bit is set.

Assembler Syntax: BPNONE Pn(p), <imm32s>
BPNONE Pn(p), <ea>(e)

Attributes:
Class = vector
Family = vector
Privilege = unprivileged
Length = 6-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

BPNONE Pn(p), <imm32s>

Instruction Format:

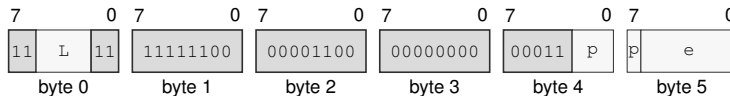
Format — Instruction format for BPNONE Pn(p), <imm32s>

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

BPNONE Pn(p), <ea>(e)

Instruction Format:

Format — Instruction format for BPNONE Pn(p), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

Effective Address field e—Specifies the control target.

BPALL

BPALL (Vector)

BPALL

Operation: Branch if every significant position for the selected size is set.

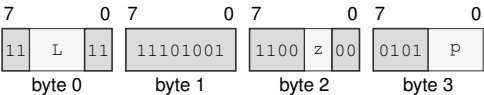
Assembler Syntax: BPALL.BWLQ(z) Pn(p), <imm32s>
BPALL.BWLQ(z) Pn(p), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 6-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

BPALL.BWLQ(z) Pn(p), <imm32s>

Instruction Format:



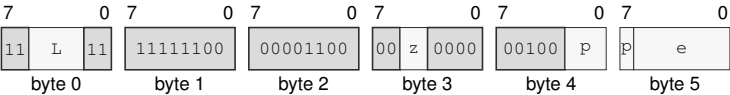
Format — Instruction format for BPALL.BWLQ(z) Pn(p), <imm32s>

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Bits field p**—Encodes the bits value.

BPALL.BWLQ(z) Pn(p), <ea>(e)

Instruction Format:



Format — Instruction format for BPALL.BWLQ(z) Pn(p), <ea>(e)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Bits field p**—Encodes the bits value.
- Effective Address field e**—Specifies the control target.

VNEG**VNEG (Vector)****VNEG**

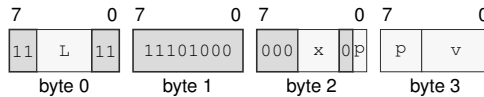
Operation: Integer two's-complement negation or FP sign toggle.

Assembler Syntax: VNEG.VTYPE(x) P_n(p), V_n(v)
 VNEG.VTYPE(x) P_n(p), <ea>(e), V_n(v)
 VNEG.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VNEG.VTYPE(x) P_n(p), V_n(v)

Instruction Format:

Format — Instruction format for VNEG.VTYPE(x) P_n(p), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

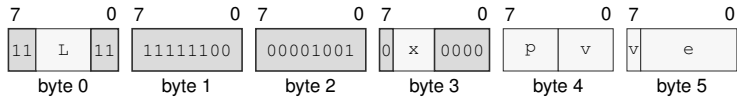
Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VNEG.VTYPE(x) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VNEG.VTYPE(x) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

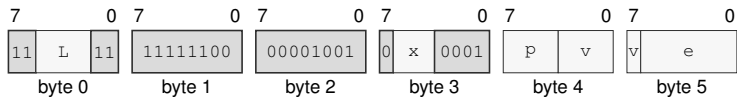
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VNEG.VTYPE(x) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VNEG.VTYPE(x) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VABS**VABS (Vector)****VABS**

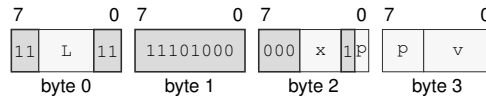
Operation: Integer signed absolute value or FP sign clear.

Assembler Syntax: VABS.VTYPE(x) P_n(p), V_n(v)
 VABS.VTYPE(x) P_n(p), <ea>(e), V_n(v)
 VABS.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VABS.VTYPE(x) P_n(p), V_n(v)

Instruction Format:

Format — Instruction format for VABS.VTYPE(x) P_n(p), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

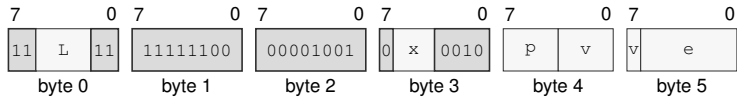
Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VABS.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VABS.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

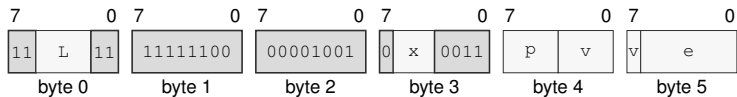
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VABS.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VABS.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VNOT**VNOT (Vector)****VNOT**

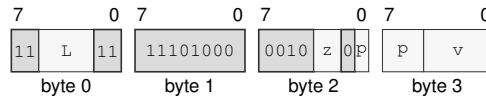
Operation: Bitwise complement.

Assembler Syntax: VNOT.BWLQ(z) Pn(p), Vn(v)
 VNOT.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VNOT.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VNOT.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VNOT.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

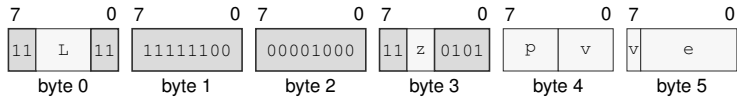
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VNOT.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VNOT.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

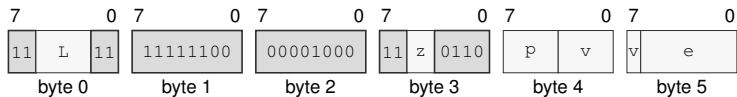
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VNOT.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VNOT.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCLZ**VCLZ (Vector)****VCLZ**

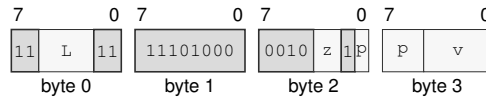
Operation: Count leading zeros.

Assembler Syntax: VCLZ.BWLQ(z) Pn(p), Vn(v)
 VCLZ.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VCLZ.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCLZ.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VCLZ.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

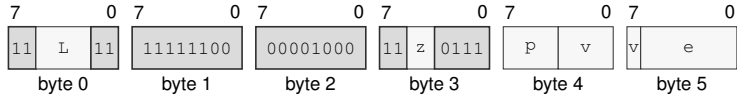
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VCLZ.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VCLZ.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

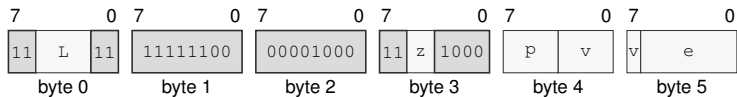
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCLZ.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VCLZ.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCTZ**VCTZ (Vector)****VCTZ**

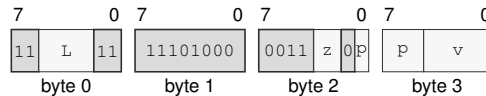
Operation: Count trailing zeros.

Assembler Syntax: VCTZ.BWLQ(z) Pn(p), Vn(v)
 VCTZ.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VCTZ.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCTZ.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VCTZ.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

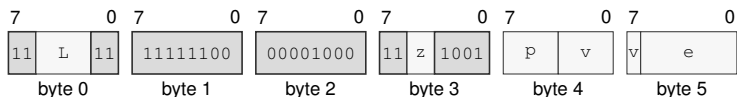
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VCTZ.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VCTZ.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

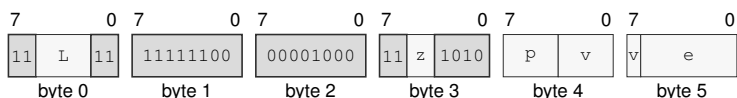
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCTZ.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VCTZ.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCLS**VCLS (Vector)****VCLS**

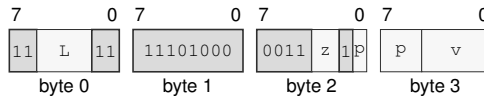
Operation: Count leading ones.

Assembler Syntax: VCLS.BWLQ(z) Pn(p), Vn(v)
 VCLS.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VCLS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCLS.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VCLS.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

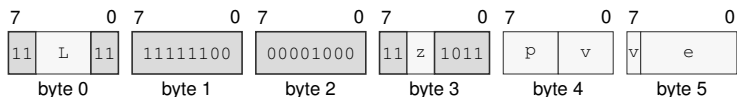
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VCLS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VCLS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

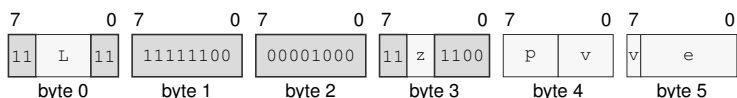
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCLS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VCLS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCTS**VCTS (Vector)****VCTS**

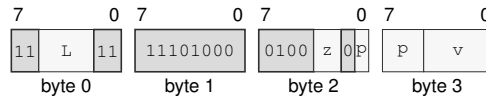
Operation: Count trailing ones.

Assembler Syntax: VCTS.BWLQ(z) Pn(p), Vn(v)
 VCTS.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VCTS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCTS.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VCTS.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

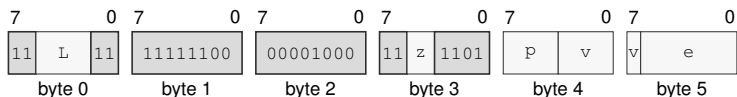
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VCTS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VCTS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

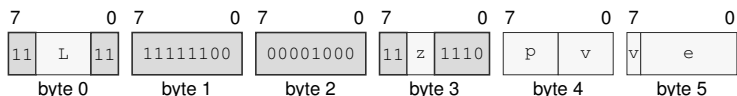
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCTS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VCTS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VPOPCNT**VPOPCNT (Vector)****VPOPCNT**

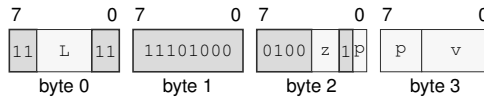
Operation: Population count.

Assembler Syntax: VPOPCNT.BWLQ(z) Pn(p), Vn(v)
 VPOPCNT.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VPOPCNT.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VPOPCNT.BWLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VPOPCNT.BWLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

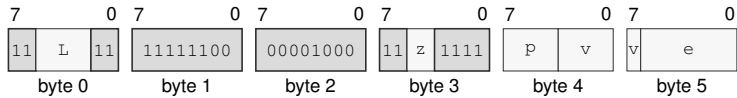
Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VPOPCNT.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VPOPCNT.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

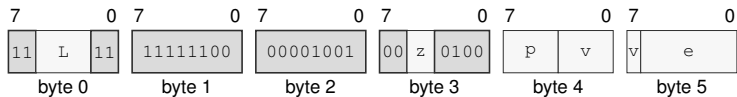
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VPOPCNT.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VPOPCNT.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VREVBYTE**VREVBYTE (Vector)****VREVBYTE**

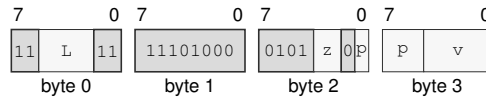
Operation: Reverse bytes within each element.

Assembler Syntax: VREVBYTE.WLQ(z) Pn(p), Vn(v)
 VREVBYTE.WLQ(z) Pn(p), <ea>(e), Vn(v)
 VREVBYTE.WLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VREVBYTE.WLQ(z) Pn(p), Vn(v)

Instruction Format:

Format — Instruction format for VREVBYTE.WLQ(z) Pn(p), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

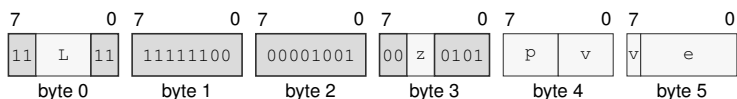
Size field z—Selects W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

VREVBYTE.WLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VREVBYTE.WLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

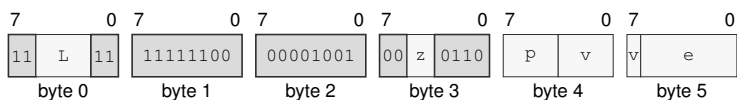
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VREVBYTE.WLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VREVBYTE.WLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSQRT

VSQRT (Vector)

VSQRT

Operation: Square root.

Assembler Syntax: VSQRT.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)
VSQRT.HSD(*z*) P_{*n*}(*p*), <ea>(*e*), V_{*n*}(*v*)
VSQRT.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), <ea>(*e*)

Attributes:
Class = vector
Family = vector
Privilege = unprivileged
Length = 4-16 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

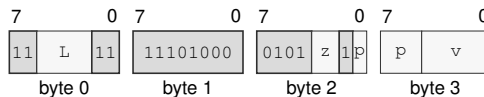
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VSQRT.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

Instruction Format:



Format — Instruction format for VSQRT.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

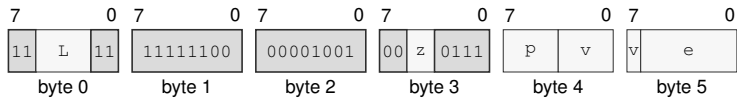
Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

VSQRT.HSD(z) $P_n(p)$, $\langle ea \rangle(e)$, $V_n(v)$

Instruction Format:



Format — Instruction format for VSQRT.HSD(z) $P_n(p)$, $\langle ea \rangle(e)$, $V_n(v)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects H/S/D.

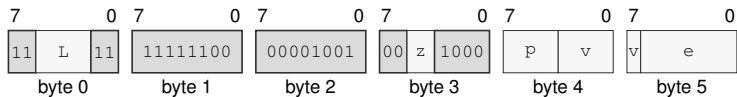
Bits field p —Encodes the bits value.

Bits field v —Encodes the bits value.

Effective Address field e —Specifies the source operand.

VSQRT.HSD(z) $P_n(p)$, $V_n(v)$, $\langle ea \rangle(e)$

Instruction Format:



Format — Instruction format for VSQRT.HSD(z) $P_n(p)$, $V_n(v)$, $\langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects H/S/D.

Bits field p —Encodes the bits value.

Bits field v —Encodes the bits value.

Effective Address field e —Specifies the read/write operand. Immediate addressing is unavailable in this form.

VROUND

VROUND (Vector)

VROUND

Operation: Integral FP result under the current rounding mode.

Assembler Syntax: `VROUND.HSD(z) Pn(p), Vn(v)`
`VROUND.HSD(z) Pn(p), <ea>(e), Vn(v)`
`VROUND.HSD(z) Pn(p), Vn(v), <ea>(e)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics

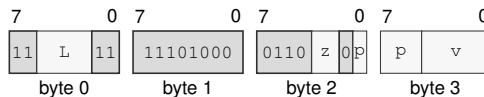
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

`VROUND.HSD(z) Pn(p), Vn(v)`

Instruction Format:



Format — Instruction format for `VROUND.HSD(z) Pn(p), Vn(v)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

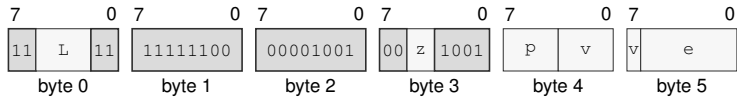
Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

VROUND.HSD(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VROUND.HSD(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

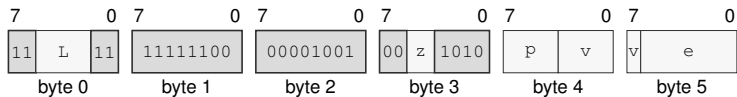
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VROUND.HSD(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VROUND.HSD(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VTRUNC

VTRUNC (Vector)

VTRUNC

Operation: Integral FP result toward zero.

Assembler Syntax: `VTRUNC.HSD(z) Pn(p), Vn(v)`
`VTRUNC.HSD(z) Pn(p), <ea>(e), Vn(v)`
`VTRUNC.HSD(z) Pn(p), Vn(v), <ea>(e)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics

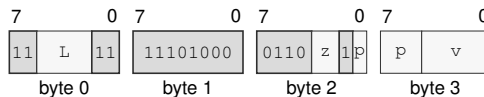
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

`VTRUNC.HSD(z) Pn(p), Vn(v)`

Instruction Format:



Format — Instruction format for `VTRUNC.HSD(z) Pn(p), Vn(v)`

Instruction Fields:

Length field `L`—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

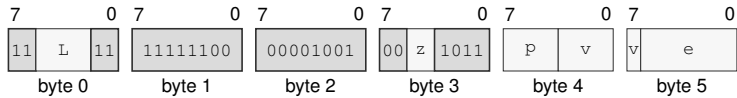
Size field `z`—Selects H/S/D.

Bits field `p`—Encodes the bits value.

Bits field `v`—Encodes the bits value.

VTRUNC.HSD(z) P $_n$ (p), <ea>(e), V $_n$ (v)

Instruction Format:



Format — Instruction format for VTRUNC.HSD(z) P $_n$ (p), <ea>(e), V $_n$ (v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

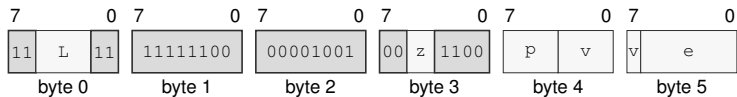
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VTRUNC.HSD(z) P $_n$ (p), V $_n$ (v), <ea>(e)

Instruction Format:



Format — Instruction format for VTRUNC.HSD(z) P $_n$ (p), V $_n$ (v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VFLOOR

VFLOOR (Vector)

VFLOOR

Operation: Integral FP result toward negative infinity.

Assembler Syntax: VFLOOR.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)
 VFLOOR.HSD(*z*) P_{*n*}(*p*), <ea>(*e*), V_{*n*}(*v*)
 VFLOOR.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), <ea>(*e*)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics

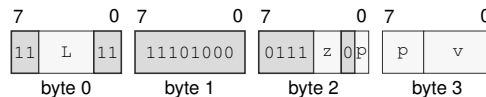
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VFLOOR.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

Instruction Format:



Format — Instruction format for VFLOOR.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

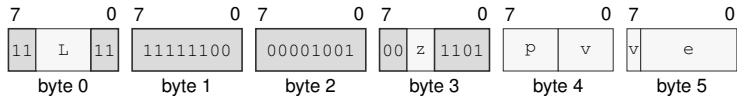
Instruction Fields:

Length field *L*—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

VFLOOR.HSD(z) P $_n$ (p), <ea>(e), V $_n$ (v)**Instruction Format:****Format — Instruction format for VFLOOR.HSD(z) P $_n$ (p), <ea>(e), V $_n$ (v)****Instruction Fields:**

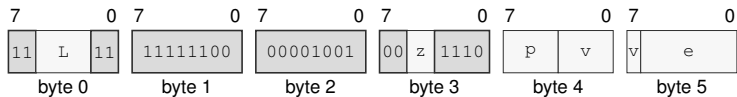
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VFLOOR.HSD(z) P $_n$ (p), V $_n$ (v), <ea>(e)**Instruction Format:****Format — Instruction format for VFLOOR.HSD(z) P $_n$ (p), V $_n$ (v), <ea>(e)****Instruction Fields:**

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCEIL**VCEIL (Vector)****VCEIL**

Operation: Integral FP result toward positive infinity.

Assembler Syntax: VCEIL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)
 VCEIL.HSD(*z*) P_{*n*}(*p*), <ea>(*e*), V_{*n*}(*v*)
 VCEIL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), <ea>(*e*)

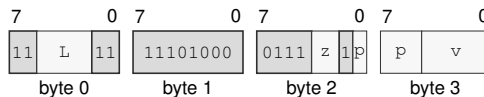
Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCEIL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

Instruction Format:

Format — Instruction format for VCEIL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

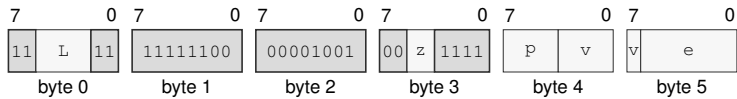
Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

VCEIL.HSD(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VCEIL.HSD(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

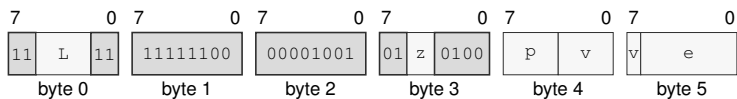
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCEIL.HSD(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VCEIL.HSD(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCLASS

VCLASS (Vector)

VCLASS

Operation: Write a per-lane FP classification bitmap to a vector.

Assembler Syntax: `VCLASS.HSD(z) Pn(p), Vn(v)`
`VCLASS.HSD(z) Pn(p), <ea>(e), Vn(v)`
`VCLASS.HSD(z) Pn(p), Vn(v), <ea>(e)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 4-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics

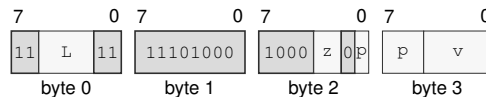
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

`VCLASS.HSD(z) Pn(p), Vn(v)`

Instruction Format:



Format — Instruction format for `VCLASS.HSD(z) Pn(p), Vn(v)`

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

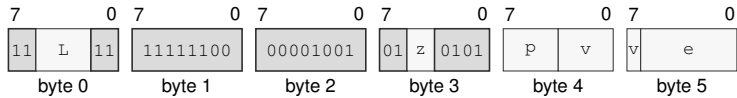
Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

VCLASS.HSD(*z*) P_n(*p*), <ea>(*e*), V_n(*v*)

Instruction Format:



Format — Instruction format for VCLASS.HSD(*z*) P_n(*p*), <ea>(*e*), V_n(*v*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

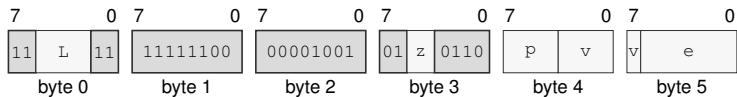
Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Effective Address field *e*—Specifies the source operand.

VCLASS.HSD(*z*) P_n(*p*), V_n(*v*), <ea>(*e*)

Instruction Format:



Format — Instruction format for VCLASS.HSD(*z*) P_n(*p*), V_n(*v*), <ea>(*e*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Effective Address field *e*—Specifies the read/write operand. Immediate addressing is unavailable in this form.

PPERM**Predicate Permute****PPERM**

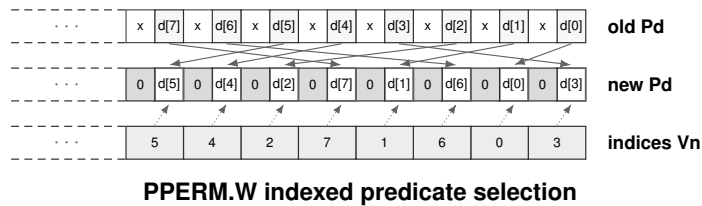
Operation: Select predicate lanes using unsigned vector indices.

Assembler Syntax: PPERM.BWLQ(z) Vn(v), Pn(p)

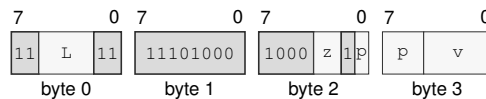
Required CPUID flags: VECTOR

Detailed Semantics

PPERM reads an unsigned index from each selected B, W, L, or Q vector lane. Each index selects the old destination's significant predicate position whose typed-lane index has that value. An index outside the architectural typed-lane range produces zero. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.

**Encodings:**

PPERM.BWLQ(z) Vn(v), Pn(p)

Instruction Format:

Format — Instruction format for PPERM.BWLQ(z) Vn(v), Pn(p)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

PSLIDEUP

Predicate Slide Up

PSLIDEUP

Operation:

Move predicate lanes toward higher indices and insert zero at the low end.

Assembler Syntax:

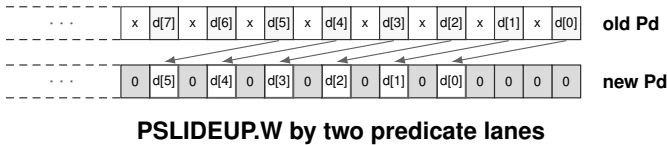
PSLIDEUP.BWLQ(z) imm6(i), Pn(p)

Required CPUID flags:

VECTOR

Detailed Semantics

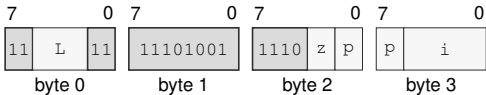
PSLIDEUP moves the significant predicate positions for the selected B, W, L, or Q width toward higher typed-lane indices by the unsigned immediate count. Vacated positions at the low end become zero. A count at least as large as the architectural typed-lane count clears every significant position. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

PSLIDEUP.BWLQ(z) imm6(i), Pn(p)

Instruction Format:



Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Immediate field** i—Encodes the immediate value.

PSLIDEDN

Predicate Slide Down

PSLIDEDN

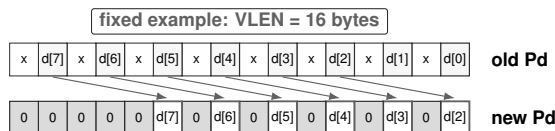
Operation: Move predicate lanes toward lower indices and insert zero at the high end.

Assembler Syntax: PSLIDEDN.BWLQ(*z*) imm6(*i*), Pn(*p*)

Required CPUID flags: VECTOR

Detailed Semantics

PSLIDEDN moves the significant predicate positions for the selected B, W, L, or Q width toward lower typed-lane indices by the unsigned immediate count. Vacated positions at the high end become zero. A count at least as large as the architectural typed-lane count clears every significant position. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.

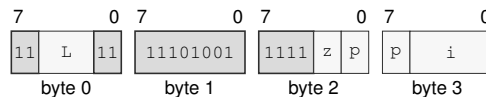


PSLIDEDN.W by two predicate lanes

Encodings:

PSLIDEDN.BWLQ(*z*) imm6(*i*), Pn(*p*)

Instruction Format:



Format — Instruction format for PSLIDEDN.BWLQ(*z*) imm6(*i*), Pn(*p*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Bits field *p*—Encodes the bits value.

Immediate field *i*—Encodes the immediate value.

VCMPcc

VCMP Conditional (Vector)

VCMPcc

Operation: Compare active integer or FP lanes and write a complete predicate result.

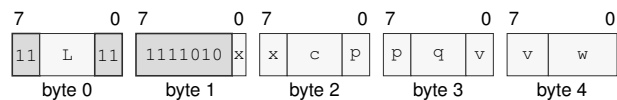
Assembler Syntax: VCMPcc.VTYPE(x) Pn(p), Vn(v), Vn(w), Pn(q)
VCMPcc.VTYPE(x) Pn(p), Vn(v), <ea>(e), Pn(q)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VCMPcc.VTYPE(x) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Format:



Format — Instruction format for VCMPcc.VTYPE(x) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, ULE, UGT, LT, GE, LE, GT.

Bits field p—Encodes the bits value.

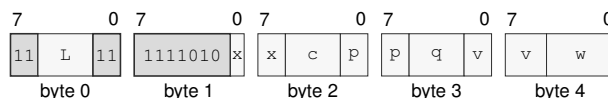
Bits field q—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VCMPcc.VTYPE(x) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Format:



Format — Instruction format for VCMPcc.VTYPE(x) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Condition field c—Selects the condition code. Allowed values: EQ, NE, VS, VC, LT, GE, LE, GT.

Bits field p—Encodes the bits value.

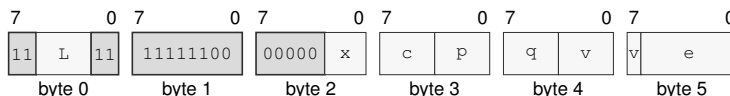
Bits field q—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VCMPcc.VTYPE(x) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Format:



Format — Instruction format for VCMPcc.VTYPE(x) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, ULE, UGT, LT, GE, LE, GT.

Bits field p—Encodes the bits value.

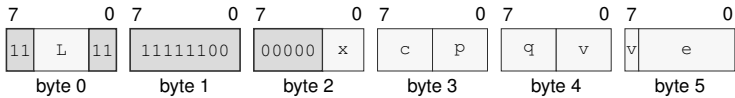
Bits field q—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VCMPcc.VTYPE(x) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Format:



Format — Instruction format for VCMPcc.VTYPE(x) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** x—Selects B/W/L/Q/H/S/D.
- Condition field** c—Selects the condition code. Allowed values: EQ, NE, VS, VC, LT, GE, LE, GT.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Effective Address field** e—Specifies the source operand.

VTESTZ**VTESTZ (Vector)****VTESTZ**

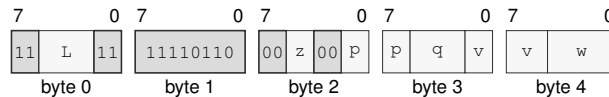
Operation: Test 'Va AND Vb' for zero and write a complete predicate result.

Assembler Syntax: `VTESTZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)`
`VTESTZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

`VTESTZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)`

Instruction Format:

Format — Instruction format for VTESTZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

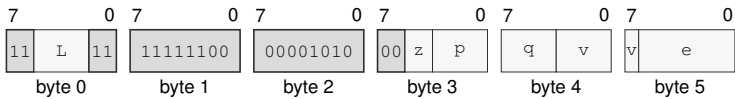
Bits field q—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VTESTZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Format:



Format — Instruction format for VTESTZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Effective Address field** e—Specifies the source operand.

VTESTNZ**VTESTNZ (Vector)****VTESTNZ**

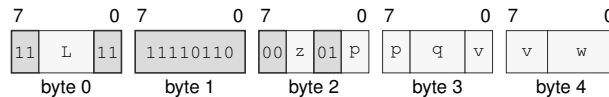
Operation: Test 'Va AND Vb' for nonzero and write a complete predicate result.

Assembler Syntax: VTESTNZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)
 VTESTNZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VTESTNZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Format:

Format — Instruction format for VTESTNZ.BWLQ(z) Pn(p), Vn(v), Vn(w), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

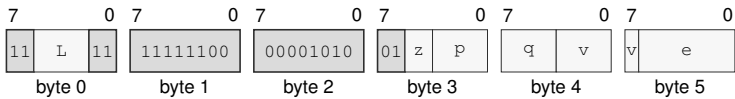
Bits field q—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VTESTNZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Format:



Format — Instruction format for VTESTNZ.BWLQ(z) Pn(p), Vn(v), <ea>(e), Pn(q)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Effective Address field** e—Specifies the source operand.

VADD**Vector Add****VADD**

Operation:	Adds corresponding active vector elements using integer or floating-point arithmetic.
Assembler Syntax:	VADD.VTYPE(x) Pn(p), Vn(v), Vn(w) VADD.VTYPE(x) Pn(p), <ea>(e), Vn(v) VADD.VTYPE(x) Pn(p), Vn(v), <ea>(e)
Required CPUID flags:	VECTOR H/S/D cases: FP
Exceptions:	FLOATING_POINT_FAULT: when a generated floating-point exception is enabled

Detailed Semantics

VADD applies the selected integer or floating-point addition independently to each active vector lane. The governing predicate selects active lanes; an inactive destination lane retains its previous value.

For B, W, L, and Q elements, each active result is the lane-width modular sum and no flag bank changes. For H, S, and D elements, each active result is rounded according to the floating-point environment, and a completed operation accrues any generated NV, OF, UF, and NX causes in FFLAGS; DZ is unchanged. If a generated floating-point exception is enabled in any active lane, VADD raises FLOATING_POINT_FAULT and commits neither destination lanes nor accrued FFLAGS causes from the faulting operation. H/S/D elements

FFLAGS Effects

NV	DZ	OF	UF	NX
*	-	*	*	*

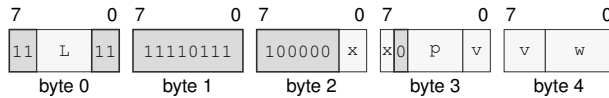
*: accrual of an operation-generated cause.

-: preserved.

Encodings:

VADD.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VADD.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

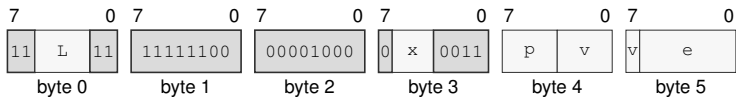
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VADD.VTYPE(x) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VADD.VTYPE(x) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

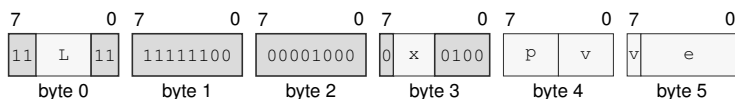
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VADD.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VADD.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSUB

VSUB (Vector)

VSUB

Operation: Modular integer or IEEE floating-point subtraction.

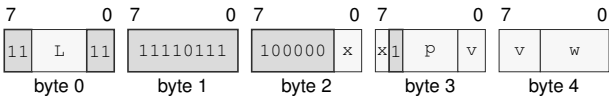
Assembler Syntax: VSUB.VTYPE(x) Pn(p), Vn(v), Vn(w)
VSUB.VTYPE(x) Pn(p), <ea>(e), Vn(v)
VSUB.VTYPE(x) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VSUB.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Format:



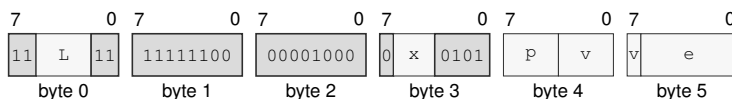
Format — Instruction format for VSUB.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** x—Selects B/W/L/Q/H/S/D.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VSUB.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VSUB.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

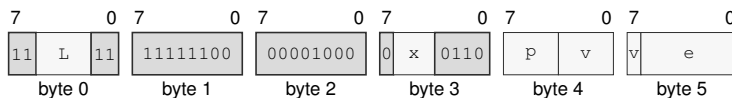
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VSUB.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VSUB.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMUL

VMUL (Vector)

VMUL

Operation: Low integer product or IEEE floating-point multiplication.

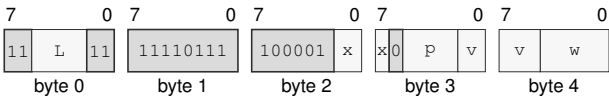
Assembler Syntax: VMUL.VTYPE(x) Pn(p), Vn(v), Vn(w)
VMUL.VTYPE(x) Pn(p), <ea>(e), Vn(v)
VMUL.VTYPE(x) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMUL.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Format:



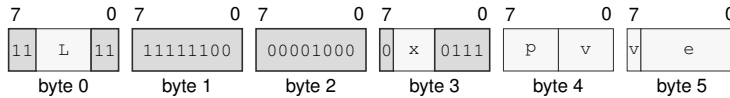
Format — Instruction format for VMUL.VTYPE(x) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** x—Selects B/W/L/Q/H/S/D.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMUL.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMUL.VTYPE(x) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

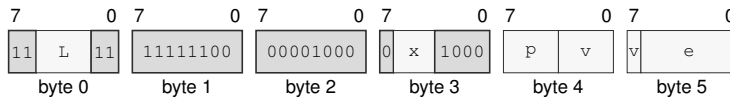
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMUL.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMUL.VTYPE(x) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field x—Selects B/W/L/Q/H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VAND

VAND (Vector)

VAND

Operation:

Bitwise AND.

Assembler Syntax:

VAND.BWLQ(z) Pn(p), Vn(v), Vn(w)
VAND.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VAND.BWLQ(z) Pn(p), Vn(v), <ea>(e)

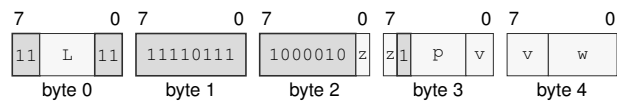
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VAND.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



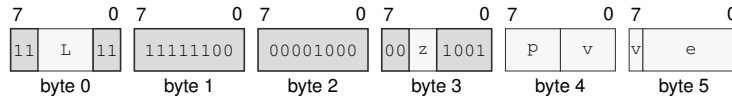
Format — Instruction format for VAND.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VAND.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VAND.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

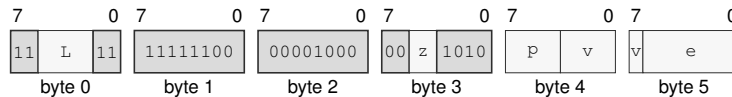
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VAND.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VAND.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VOR

VOR (Vector)

VOR

- Operation: Bitwise OR.
- Assembler Syntax:

VOR.BWLQ(z) Pn(p), Vn(v), Vn(w)

VOR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

VOR.BWLQ(z) Pn(p), Vn(v), <ea>(e)
- Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5-16 bytes

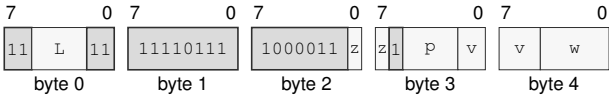
Feature = VECTOR

Repeat = none

Encodings:

VOR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



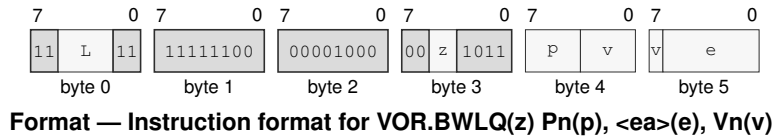
Format — Instruction format for VOR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VOR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

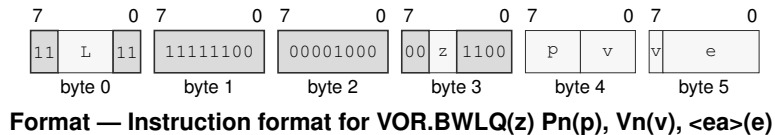
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VOR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VXOR

VXOR (Vector)

VXOR

Operation:

Bitwise XOR.

Assembler Syntax:

VXOR.BWLQ(z) Pn(p), Vn(v), Vn(w)
VXOR.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VXOR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

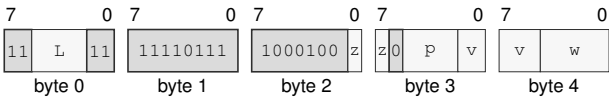
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VXOR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



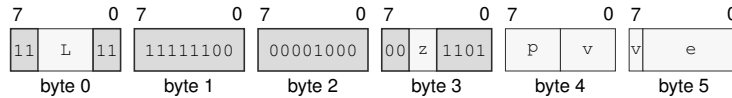
Format — Instruction format for VXOR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VXOR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VXOR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

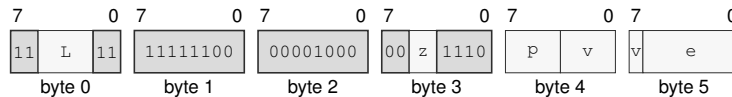
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VXOR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VXOR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMINS

VMINS (Vector)

VMINS

Operation: Signed minimum.

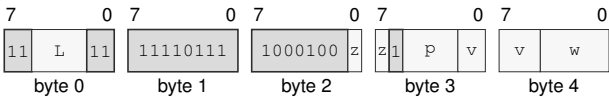
Assembler Syntax: VMINS.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMINS.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMINS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMINS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



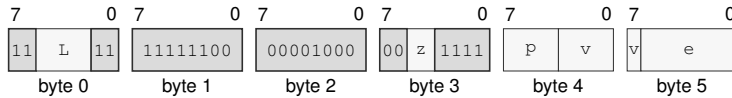
Format — Instruction format for VMINS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMINS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMINS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

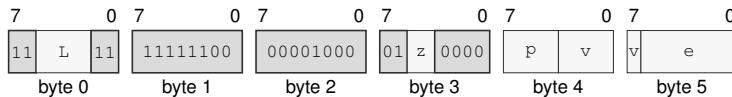
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMINS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMINS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMINU

VMINU (Vector)

VMINU

Operation: Unsigned minimum.

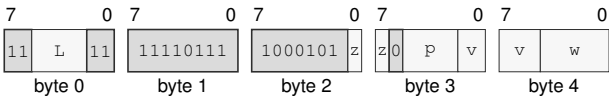
Assembler Syntax: VMINU.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMINU.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMINU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMINU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



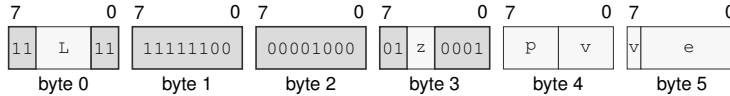
Format — Instruction format for VMINU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMINU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMINU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

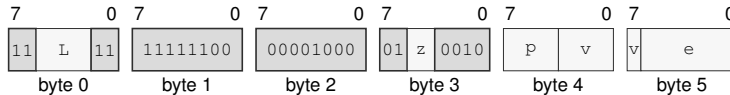
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMINU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMINU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMAXS

VMAXS (Vector)

VMAXS

Operation:

Signed maximum.

Assembler Syntax:

VMAXS.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMAXS.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMAXS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

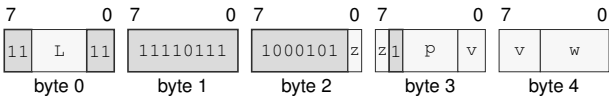
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMAXS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



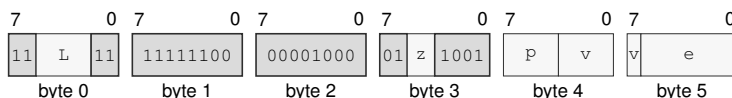
Format — Instruction format for VMAXS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMAXS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMAXS.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

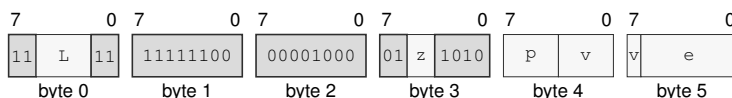
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMAXS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMAXS.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMAXU

VMAXU (Vector)

VMAXU

Operation: Unsigned maximum.

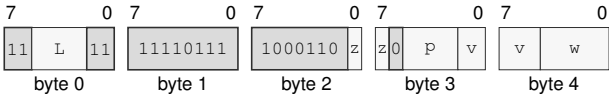
Assembler Syntax: VMAXU.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMAXU.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMAXU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMAXU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



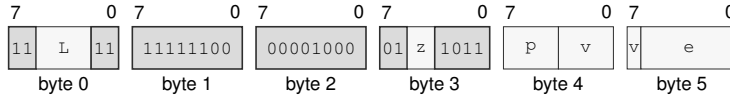
Format — Instruction format for VMAXU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMAXU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMAXU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

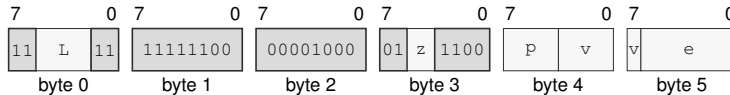
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMAXU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMAXU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMULHS

VMULHS (Vector)

VMULHS

Operation: High half of a signed product.

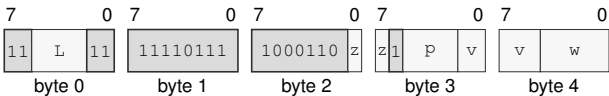
Assembler Syntax: VMULHS.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMULHS.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMULHS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMULHS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



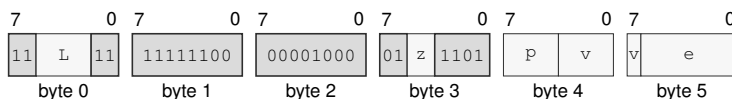
Format — Instruction format for VMULHS.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMULHS.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMULHS.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

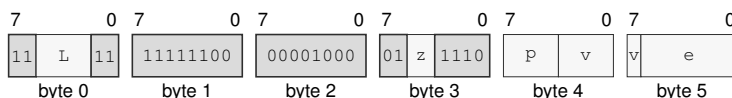
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMULHS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMULHS.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMULHU

VMULHU (Vector)

VMULHU

Operation: High half of an unsigned product.

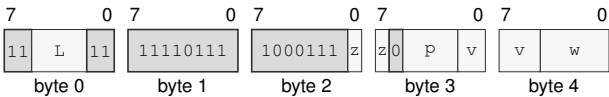
Assembler Syntax: VMULHU.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMULHU.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMULHU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMULHU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



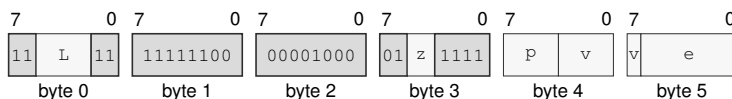
Format — Instruction format for VMULHU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMULHU.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMULHU.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

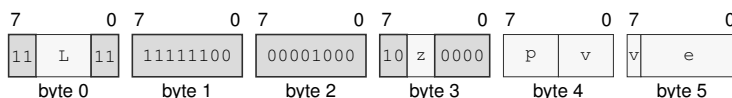
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMULHU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMULHU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMULHSU

VMULHSU (Vector)

VMULHSU

Operation: High half of signed-by-unsigned product.

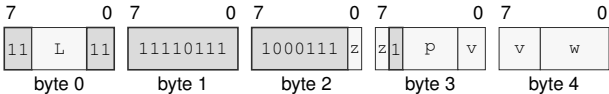
Assembler Syntax: VMULHSU.BWLQ(z) Pn(p), Vn(v), Vn(w)
VMULHSU.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VMULHSU.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VMULHSU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



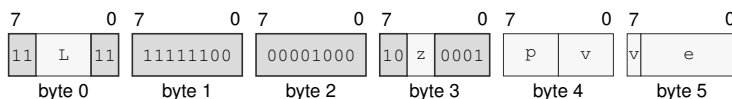
Format — Instruction format for VMULHSU.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VMULHSU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VMULHSU.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

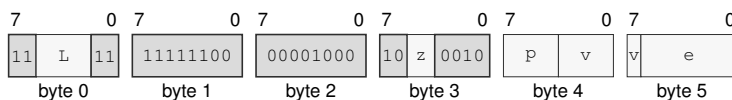
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMULHSU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMULHSU.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSHL

VSHL (Vector)

VSHL

Operation:

Logical left shift.

Assembler Syntax:

VSHL.BWLQ(z) Pn(p), Vn(v), Vn(w)
VSHL.BWLQ(z) Pn(p), imm6(i), Vn(v)
VSHL.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VSHL.BWLQ(z) Pn(p), Vn(v), <ea>(e)
VSHL.BWLQ(z) Pn(p), imm6(i), <ea>(e)

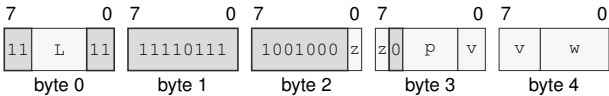
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VSHL.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



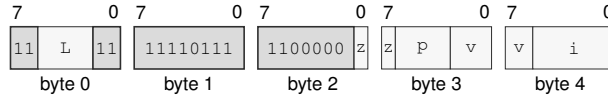
Format — Instruction format for VSHL.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VSHL.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Format:



Format — Instruction format for VSHL.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

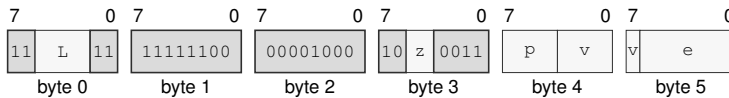
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VSHL.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VSHL.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

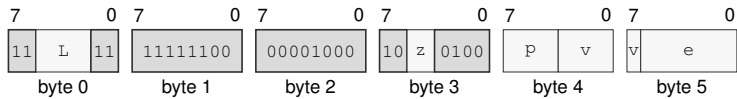
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VSHL.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VSHL.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

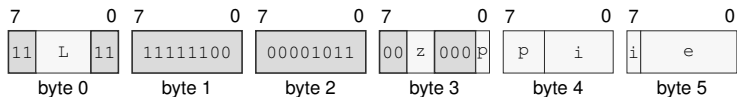
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSHL.BWLQ(z) P_n(p), imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for VSHL.BWLQ(z) P_n(p), imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSHR

VSHR (Vector)

VSHR

Operation: Logical right shift.

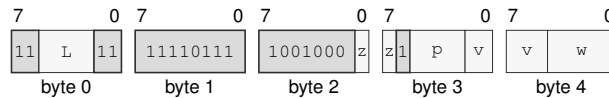
Assembler Syntax: VSHR.BWLQ(z) Pn(p), Vn(v), Vn(w)
 VSHR.BWLQ(z) Pn(p), imm6(i), Vn(v)
 VSHR.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VSHR.BWLQ(z) Pn(p), Vn(v), <ea>(e)
 VSHR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VSHR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VSHR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

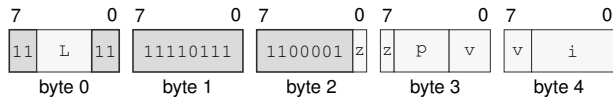
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VSHR.BWLQ(z) P_n(p), imm6(i), V_n(v)

Instruction Format:



Format — Instruction format for VSHR.BWLQ(z) P_n(p), imm6(i), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

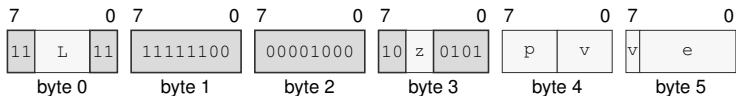
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VSHR.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VSHR.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

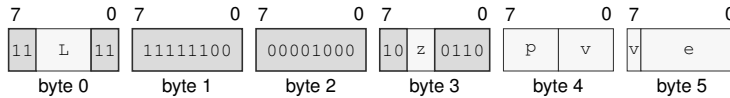
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VSHR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VSHR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

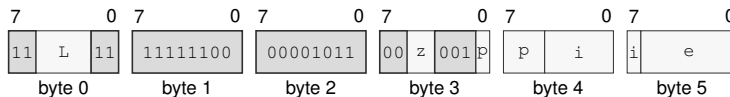
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSHR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for VSHR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSAR

VSAR (Vector)

VSAR

Operation: Arithmetic right shift.

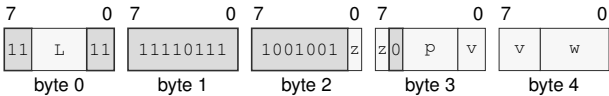
Assembler Syntax: VSAR.BWLQ(z) Pn(p), Vn(v), Vn(w)
VSAR.BWLQ(z) Pn(p), imm6(i), Vn(v)
VSAR.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VSAR.BWLQ(z) Pn(p), Vn(v), <ea>(e)
VSAR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VSAR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



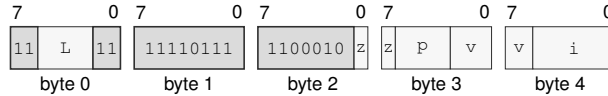
Format — Instruction format for VSAR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VSAR.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Format:



Format — Instruction format for VSAR.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

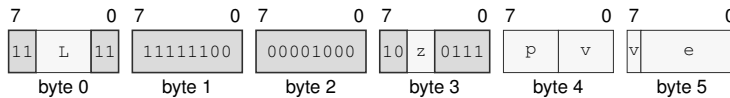
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VSAR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VSAR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

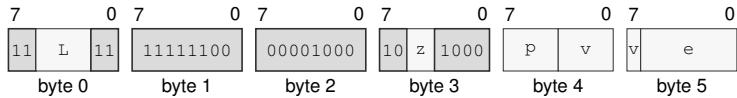
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VSAR.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Format:



Format — Instruction format for VSAR.BWLQ(z) P_n(p), V_n(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

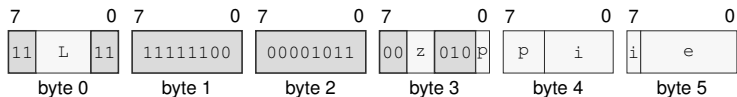
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VSAR.BWLQ(z) P_n(p), imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for VSAR.BWLQ(z) P_n(p), imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VROL**VROL (Vector)****VROL**

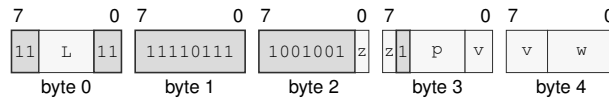
Operation: Rotate left.

Assembler Syntax: VROL.BWLQ(z) Pn(p), Vn(v), Vn(w)
 VROL.BWLQ(z) Pn(p), imm6(i), Vn(v)
 VROL.BWLQ(z) Pn(p), <ea>(e), Vn(v)
 VROL.BWLQ(z) Pn(p), Vn(v), <ea>(e)
 VROL.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VROL.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

Format — Instruction format for VROL.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

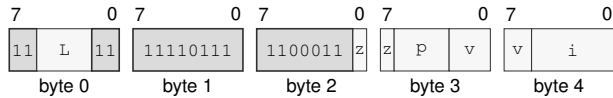
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VROL.BWLQ(z) P_n(p), imm6(i), V_n(v)

Instruction Format:



Format — Instruction format for VROL.BWLQ(z) P_n(p), imm6(i), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

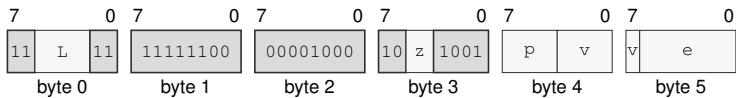
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VROL.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Format:



Format — Instruction format for VROL.BWLQ(z) P_n(p), <ea>(e), V_n(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

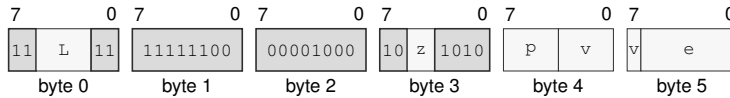
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VROL.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VROL.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

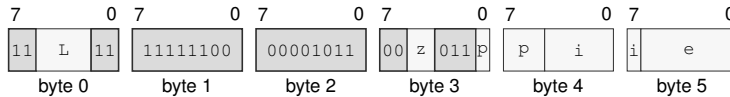
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VROL.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for VROL.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VROR

VROR (Vector)

VROR

Operation: Rotate right.

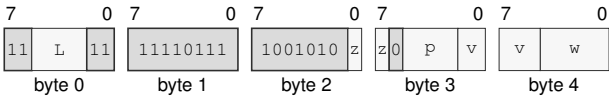
Assembler Syntax: VROR.BWLQ(z) Pn(p), Vn(v), Vn(w)
VROR.BWLQ(z) Pn(p), imm6(i), Vn(v)
VROR.BWLQ(z) Pn(p), <ea>(e), Vn(v)
VROR.BWLQ(z) Pn(p), Vn(v), <ea>(e)
VROR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VROR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VROR.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

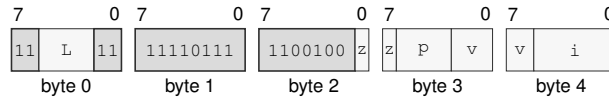
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VROR.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Format:



Format — Instruction format for VROR.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

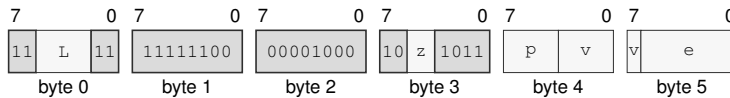
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VROR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VROR.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

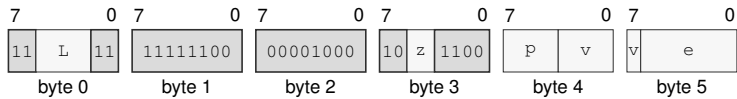
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VROR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VROR.BWLQ(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

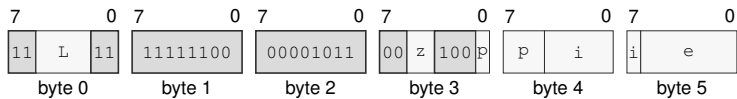
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VROR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Format:



Format — Instruction format for VROR.BWLQ(z) Pn(p), imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMIN**VMIN (Vector)****VMIN**

Operation: FP minimum using the scalar FMIN NaN and signed-zero rules.

Assembler Syntax: VMIN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)
 VMIN.HSD(*z*) Pn(*p*), <ea>(*e*), Vn(*v*)
 VMIN.HSD(*z*) Pn(*p*), Vn(*v*), <ea>(*e*)

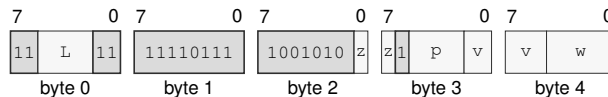
Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VMIN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Format:

Format — Instruction format for VMIN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

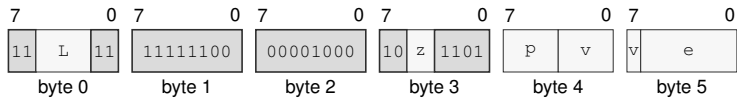
Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VMIN.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMIN.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

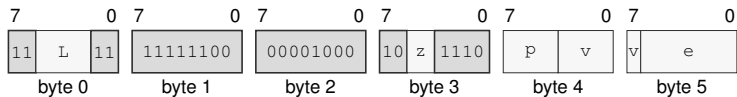
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMIN.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMIN.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VMAX**VMAX (Vector)****VMAX**

Operation: FP maximum using the scalar FMAX NaN and signed-zero rules.

Assembler Syntax: VMAX.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)
 VMAX.HSD(*z*) Pn(*p*), <ea>(*e*), Vn(*v*)
 VMAX.HSD(*z*) Pn(*p*), Vn(*v*), <ea>(*e*)

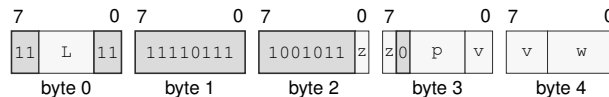
Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VMAX.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Format:

Format — Instruction format for VMAX.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

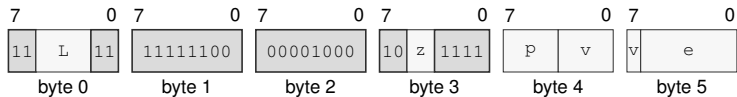
Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VMAX.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMAX.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

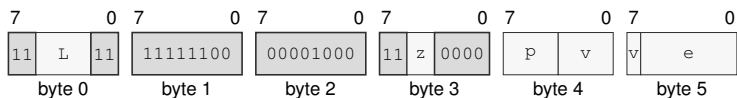
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VMAX.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VMAX.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VDIV**VDIV (Vector)****VDIV**

Operation: Division.

Assembler Syntax: VDIV.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)
VDIV.HSD(*z*) Pn(*p*), <ea>(*e*), Vn(*v*)
VDIV.HSD(*z*) Pn(*p*), Vn(*v*), <ea>(*e*)

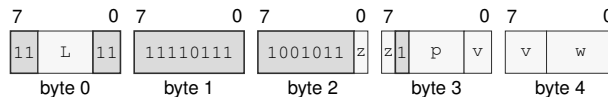
Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VDIV.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Format:

Format — Instruction format for VDIV.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

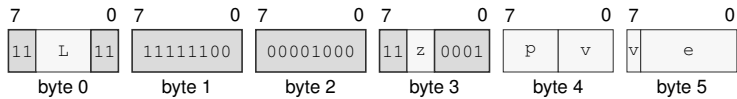
Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VDIV.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VDIV.HSD(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

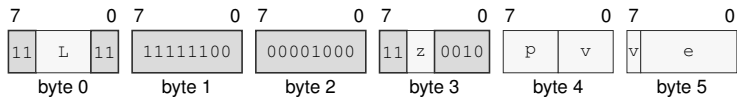
Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the source operand.

VDIV.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Format:



Format — Instruction format for VDIV.HSD(z) Pn(p), Vn(v), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Effective Address field e—Specifies the read/write operand. Immediate addressing is unavailable in this form.

VCOPYSIGN

VCOPYSIGN (Vector)

VCOPYSIGN

Operation: Combine magnitude and sign operands.

Assembler Syntax: VCOPYSIGN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)
VCOPYSIGN.HSD(*z*) Pn(*p*), <ea>(*e*), Vn(*v*)
VCOPYSIGN.HSD(*z*) Pn(*p*), Vn(*v*), <ea>(*e*)

Attributes:
Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

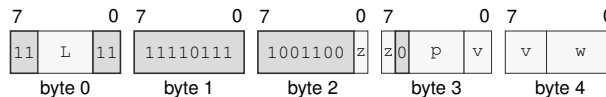
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCOPYSIGN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Format:



Format — Instruction format for VCOPYSIGN.HSD(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

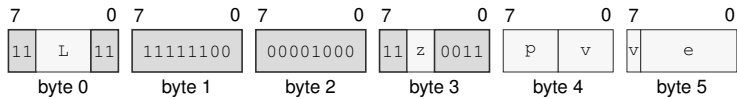
Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VCOPYSIGN.HSD(z) P n (p), <ea>(e), V n (v)

Instruction Format:



Format — Instruction format for VCOPYSIGN.HSD(z) P n (p), <ea>(e), V n (v)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects H/S/D.

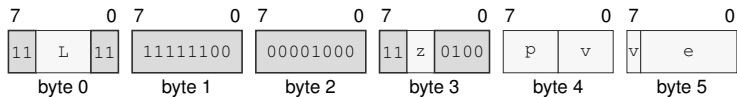
Bits field p —Encodes the bits value.

Bits field v —Encodes the bits value.

Effective Address field e —Specifies the source operand.

VCOPYSIGN.HSD(z) P n (p), V n (v), <ea>(e)

Instruction Format:



Format — Instruction format for VCOPYSIGN.HSD(z) P n (p), V n (v), <ea>(e)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects H/S/D.

Bits field p —Encodes the bits value.

Bits field v —Encodes the bits value.

Effective Address field e —Specifies the read/write operand. Immediate addressing is unavailable in this form.

VEXTZW**Vector Zero Extension to Word****VEXTZW**

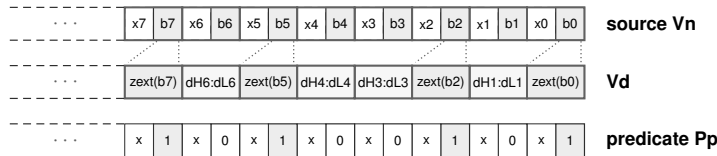
Operation: Zero-extends B source elements to W destination elements.

Assembler Syntax: VEXTZW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

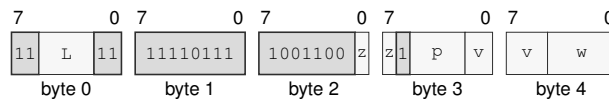
VEXTZW zero-extends the low B source element in each W container across the destination W element. The governing predicate position associated with each W container controls that container. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.



VEXTZW.B unsigned byte-to-word extension

Encodings:

VEXTZW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

Format — Instruction format for VEXTZW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VEXTSW

Vector Signed Extension to Word

VEXTSW

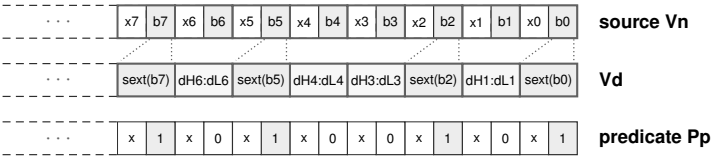
Operation: Sign-extends B source elements to W destination elements.

Assembler Syntax: VEXTSW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

VEXTSW treats the low B source element in each W container as a signed two's-complement value and sign-extends it across the destination W element. The governing predicate position associated with each W container controls that container. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.

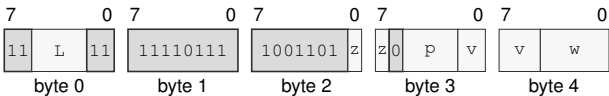


VEXTSW.B signed byte-to-word extension

Encodings:

VEXTSW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VEXTSW.B_ONLY(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B.
- Bits field p**—Encodes the bits value.
- Bits field v**—Encodes the bits value.
- Bits field w**—Encodes the bits value.

VEXTZL**Vector Zero Extension to Long****VEXTZL**

Operation: Zero-extends B or W source elements to L destination elements.

Assembler Syntax: VEXTZL.V_BW(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

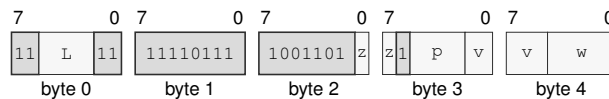
VEXTZL zero-extends the selected B or W source element into its L destination container. The governing predicate position associated with each L container controls that container. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.



VEXTZL.W unsigned word-to-long extension

Encodings:

VEXTZL.V_BW(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

Format — Instruction format for VEXTZL.V_BW(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VEXTSL

Vector Signed Extension to Long

VEXTSL

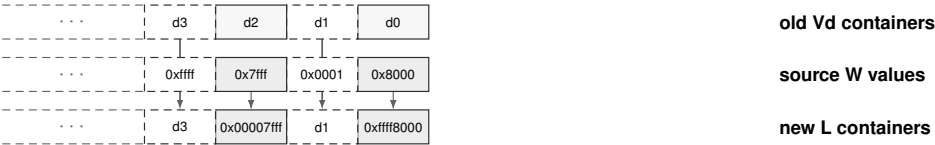
Operation: Sign-extends B or W source elements to L destination elements.

Assembler Syntax: VEXTSL.V_BW(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Required CPUID flags: VECTOR

Detailed Semantics

VEXTSL sign-extends the selected B or W source element into its L destination container for each active governing-predicate position. An inactive destination container retains its previous value. The operation leaves FLAGS unchanged.

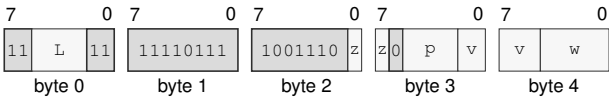


Example W-to-L signed extension in four visible containers.

Encodings:

VEXTSL.V_BW(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Format:



Format — Instruction format for VEXTSL.V_BW(*z*) Pn(*p*), Vn(*v*), Vn(*w*)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects B/W.
- Bits field** *p*—Encodes the bits value.
- Bits field** *v*—Encodes the bits value.
- Bits field** *w*—Encodes the bits value.

VEXTZQ**Vector Zero Extension to Quad****VEXTZQ**

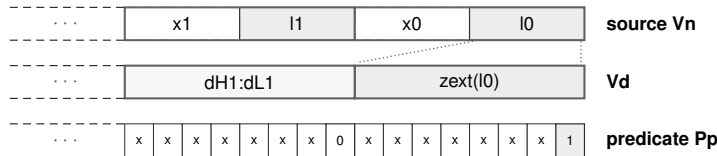
Operation: Zero-extends B, W, or L source elements to Q destination elements.

Assembler Syntax: VEXTZQ.BWL(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Required CPUID flags: VECTOR

Detailed Semantics

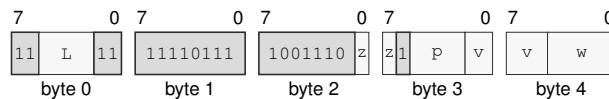
VEXTZQ zero-extends the selected B, W, or L source element into its Q destination container. The governing predicate position associated with each Q container controls that container. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.



VEXTZQ.L unsigned long-to-quad extension

Encodings:

VEXTZQ.BWL(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Instruction Format:

Format — Instruction format for VEXTZQ.BWL(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VEXTSQ

Vector Signed Extension to Quad

VEXTSQ

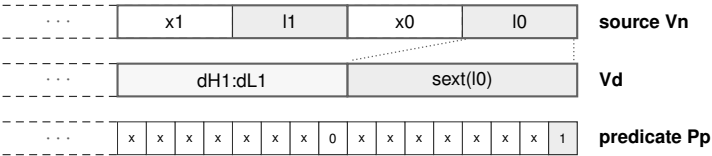
Operation: Sign-extends B, W, or L source elements to Q destination elements.

Assembler Syntax: VEXTSQ.BWL(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

VEXTSQ treats each selected B, W, or L source element as a signed two's-complement value and sign-extends it into the corresponding Q destination container. The governing predicate position associated with each Q container controls that container. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.

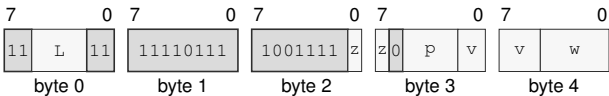


VEXTSQ.L signed long-to-quad extension

Encodings:

VEXTSQ.BWL(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VEXTSQ.BWL(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L.
- Bits field p**—Encodes the bits value.
- Bits field v**—Encodes the bits value.
- Bits field w**—Encodes the bits value.

VTRUNCB**Vector Truncation to Byte****VTRUNCB**

Operation: Copies the low B field from each W, L, or Q source container.

Assembler Syntax: `VTRUNCB.WLQ(z) Pn(p), Vn(v), Vn(w)`

Required CPUID flags: VECTOR

Detailed Semantics

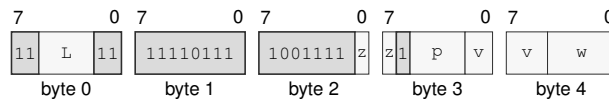
VTRUNCB copies the low B field of each selected W, L, or Q source container into the low B field of the corresponding destination container. For an active container, the destination bytes above that field retain their previous values. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.



VTRUNCB.W low-byte truncation

Encodings:

`VTRUNCB.WLQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:

Format — Instruction format for VTRUNCB.WLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VTRUNCW

Vector Truncation to Word

VTRUNCW

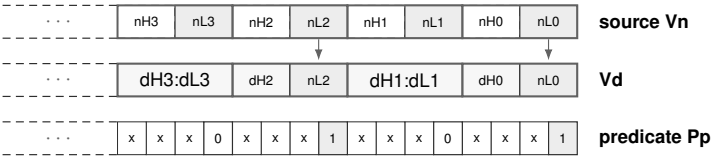
Operation: Copies the low W field from each L or Q source container.

Assembler Syntax: `VTRUNCW.V_LQ(z) Pn(p), Vn(v), Vn(w)`

Required CPUID flags: VECTOR

Detailed Semantics

VTRUNCW copies the low W field of each selected L or Q source container into the low W field of the corresponding destination container. For an active container, the destination fields above that word retain their previous values. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.

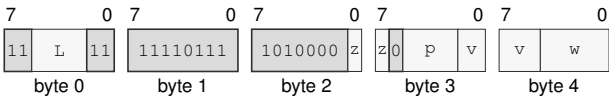


VTRUNCW.L low-word truncation

Encodings:

`VTRUNCW.V_LQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:



Format — Instruction format for VTRUNCW.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects L/Q.
- Bits field p**—Encodes the bits value.
- Bits field v**—Encodes the bits value.
- Bits field w**—Encodes the bits value.

VTRUNCL**Vector Truncation to Long****VTRUNCL**

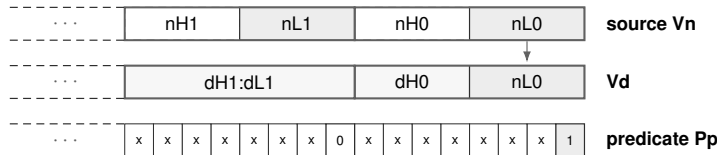
Operation: Copies the low L field from each Q source container.

Assembler Syntax: `VTRUNCL.Q_ONLY(z) Pn(p), Vn(v), Vn(w)`

Required CPUID flags: VECTOR

Detailed Semantics

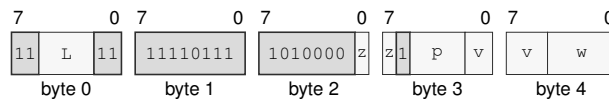
VTRUNCL copies the low L field of each Q source container into the low L field of the corresponding destination container. For an active container, the high L field of the destination retains its previous value. An inactive container retains its complete previous destination value. The operation leaves FLAGS unchanged.



VTRUNCL.Q low-longword truncation

Encodings:

`VTRUNCL.Q_ONLY(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:

Format — Instruction format for VTRUNCL.Q_ONLY(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L —Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z —Selects Q .

Bits field p —Encodes the bits value.

Bits field v —Encodes the bits value.

Bits field w —Encodes the bits value.

VCVTS

VCVTS (Vector)

VCVTS

Operation: Convert H or D to S. Signed integer to S.

Assembler Syntax: VCVTS.HD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)
VCVTS.V_LQ(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

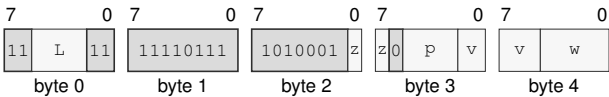
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTS.HD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

Instruction Format:



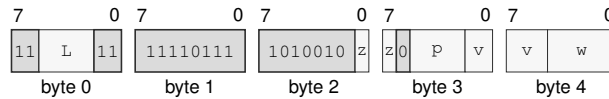
Format — Instruction format for VCVTS.HD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects H/D.
- Bits field** *p*—Encodes the bits value.
- Bits field** *v*—Encodes the bits value.
- Bits field** *w*—Encodes the bits value.

VCVTS.V_LQ(z) P_n(p), V_n(v), V_n(w)

Instruction Format:



Format — Instruction format for VCVTS.V_LQ(z) P_n(p), V_n(v), V_n(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VCVTD

VCVTD (Vector)

VCVTD

Operation: Convert H or S to D. Signed integer to D.

Assembler Syntax: VCVTD.HS(z) Pn(p), Vn(v), Vn(w)
VCVTD.V_LQ(z) Pn(p), Vn(v), Vn(w)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

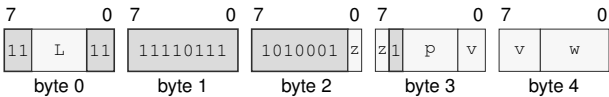
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTD.HS(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



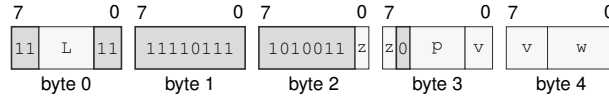
Format — Instruction format for VCVTD.HS(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VCVTD.V_LQ(z) P_n(p), V_n(v), V_n(w)

Instruction Format:



Format — Instruction format for VCVTD.V_LQ(z) P_n(p), V_n(v), V_n(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VCVTUS

VCVTUS (Vector)

VCVTUS

Operation:

Unsigned integer to S.

Assembler Syntax:

VCVTUS.V_LQ(z) Pn(p), Vn(v), Vn(w)

Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5 bytes

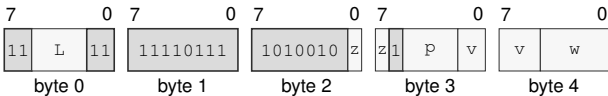
Feature = VECTOR

Repeat = none

Encodings:

VCVTUS.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VCVTUS.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z—Selects L/Q.
- Bits field p—Encodes the bits value.
- Bits field v—Encodes the bits value.
- Bits field w—Encodes the bits value.

VCVTUD**VCVTUD (Vector)****VCVTUD**

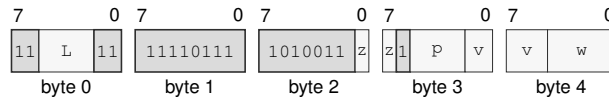
Operation: Unsigned integer to D.

Assembler Syntax: VCVTUD.V_LQ(z) Pn(p), Vn(v), Vn(w)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCVTUD.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

Format — Instruction format for VCVTUD.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VCVTL

VCVTL (Vector)

VCVTL

Operation:

FP to signed L.

Assembler Syntax:

VCVTL.HSD(z) P_n(p), V_n(v), V_n(w)

Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5 bytes

Feature = VECTOR

Repeat = none

Detailed Semantics

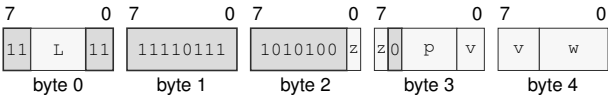
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTL.HSD(z) P_n(p), V_n(v), V_n(w)

Instruction Format:



Format — Instruction format for VCVTL.HSD(z) P_n(p), V_n(v), V_n(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S/D.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VCVTUL**VCVTUL (Vector)****VCVTUL**

Operation: FP to unsigned L.

Assembler Syntax: VCVTUL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

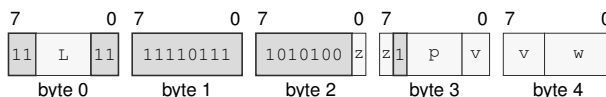
Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTUL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

Instruction Format:

Format — Instruction format for VCVTUL.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VCVTQ

VCVTQ (Vector)

VCVTQ

Operation:

FP to signed Q.

Assembler Syntax:

VCVTQ.HSD(z) P_N(p), V_N(v), V_N(w)

Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5 bytes

Feature = VECTOR

Repeat = none

Detailed Semantics

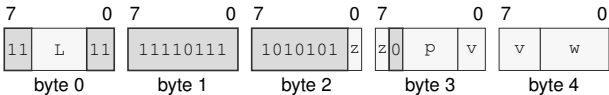
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTQ.HSD(z) P_N(p), V_N(v), V_N(w)

Instruction Format:



Format — Instruction format for VCVTQ.HSD(z) P_N(p), V_N(v), V_N(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S/D.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VCVTUQ

VCVTUQ (Vector)

VCVTUQ

Operation: FP to unsigned Q.

Assembler Syntax: VCVTUQ.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Attributes:
Class = vector
Family = vector
Privilege = unprivileged
Length = 5 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

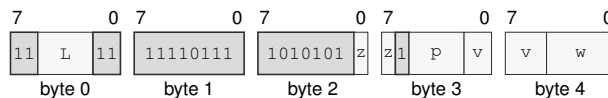
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VCVTUQ.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Instruction Format:



Format — Instruction format for VCVTUQ.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VPERM
Vector Permute
VPERM

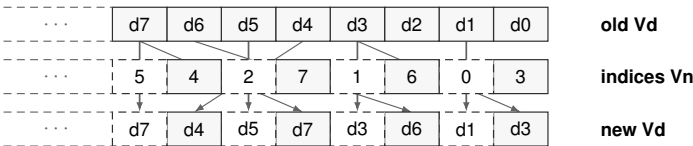
Operation: Select source lanes with vector indices; out-of-range indices produce zero.

Assembler Syntax: `VPERM.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Required CPUID flags: VECTOR

Detailed Semantics

VPERM reads an element index from each active source lane. An index within the selected vector lane count copies the correspondingly numbered old destination lane to that destination lane; an out-of-range index writes zero. The governing predicate retains the old destination value in each inactive lane.

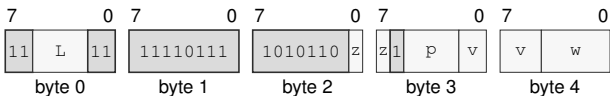


Example W-lane indexed selection with alternating active lanes.

Encodings:

`VPERM.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:



Format — Instruction format for `VPERM.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VZIPLO

Vector Lower-Half Interleave

VZIPLO

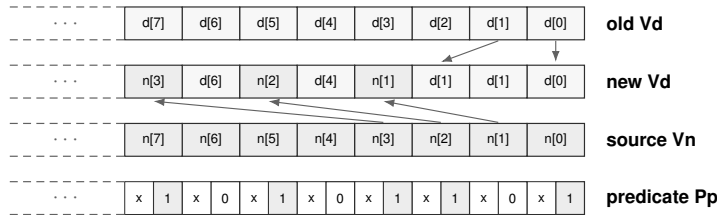
Operation: Interleave the lower halves of the old destination and source.

Assembler Syntax: VZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

VZIPLO forms each adjacent result-lane pair from one lane in the lower half of each input. In increasing order through those input halves, the even result lane receives the old destination value and the odd result lane receives the corresponding Vn value. The governing predicate retains the corresponding old destination value in each inactive result lane.

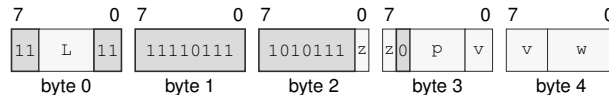


VZIPLO lower-half interleave

Encodings:

VZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VZIPHI

Vector Upper-Half Interleave

VZIPHI

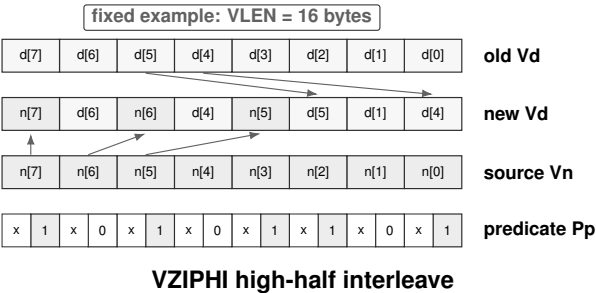
Operation: Interleave the upper halves of the old destination and source.

Assembler Syntax: VZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

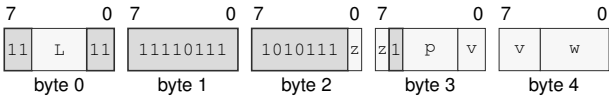
VZIPHI forms each adjacent result-lane pair from one lane in the upper half of each input. In increasing order through those input halves, the even result lane receives the old destination value and the odd result lane receives the corresponding Vn value. The governing predicate retains the corresponding old destination value in each inactive result lane.



Encodings:

VZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VUZIPLO

Vector Even-Lane Deinterleave

VUZIPLO

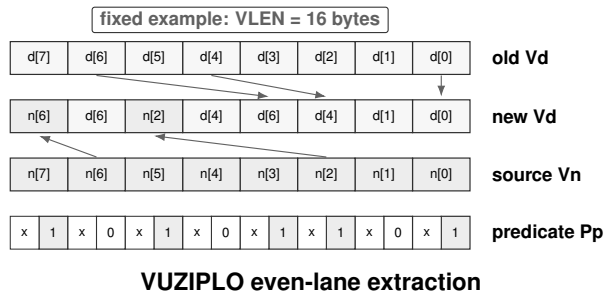
Operation: Gather even lanes from the old destination and source into separate result halves.

Assembler Syntax: VUZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

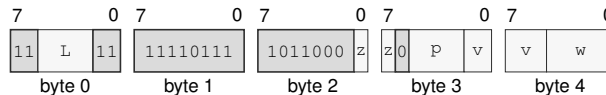
VUZIPLO gathers the even-numbered lanes from the old destination into the lower half of the result, in increasing lane order. It gathers the even-numbered Vn lanes into the upper half in the same order. The governing predicate retains the corresponding old destination value in each inactive result lane.



Encodings:

VUZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VUZIPLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VUZIPHI

Vector Odd-Lane Deinterleave

VUZIPHI

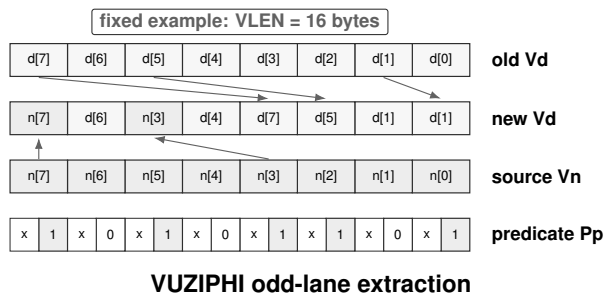
Operation: Gather odd lanes from the old destination and source into separate result halves.

Assembler Syntax: `VUZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Required CPUID flags: VECTOR

Detailed Semantics

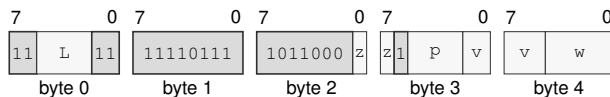
VUZIPHI gathers the odd-numbered lanes from the old destination into the lower half of the result, in increasing lane order. It gathers the odd-numbered V_n lanes into the upper half in the same order. The governing predicate retains the corresponding old destination value in each inactive result lane.



Encodings:

`VUZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:



Format — Instruction format for VUZIPHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VTRNLO**Vector Even-Lane Transpose****VTRNLO**

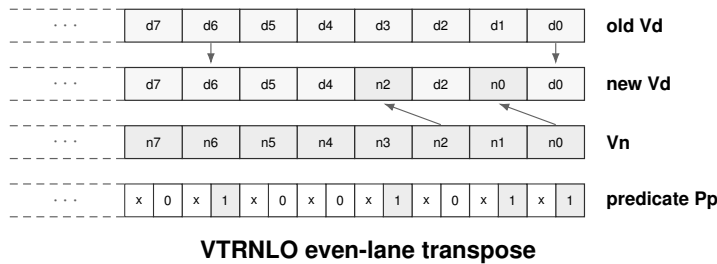
Operation: Interleave even lanes from the old destination and source by adjacent result pairs.

Assembler Syntax: `VTRNLO.BWLQ(z) Pn(p), Vn(v), Vn(w)`

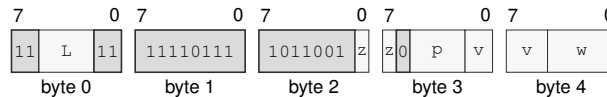
Required CPUID flags: VECTOR

Detailed Semantics

For each adjacent result-lane pair, VTRNLO selects the even lane of the corresponding pair in both inputs. The even result lane receives that old destination value, and the odd result lane receives the value from Vn. The governing predicate retains the corresponding old destination value in each inactive result lane.

**Encodings:**

`VTRNLO.BWLQ(z) Pn(p), Vn(v), Vn(w)`

Instruction Format:

Format — Instruction format for VTRNLO.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

VTRNHI

Vector Odd-Lane Transpose

VTRNHI

Operation:

Interleave odd lanes from the old destination and source by adjacent result pairs.

Assembler Syntax:

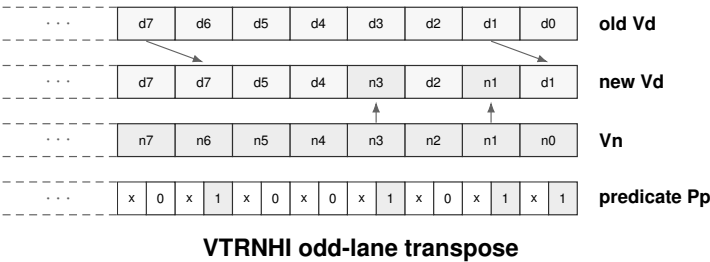
VTRNHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Required CPUID flags:

VECTOR

Detailed Semantics

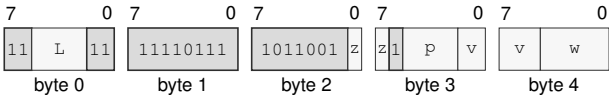
For each adjacent result-lane pair, VTRNHI selects the odd lane of the corresponding pair in both inputs. The even result lane receives that old destination value, and the odd result lane receives the value from Vn. The governing predicate retains the corresponding old destination value in each inactive result lane.



Encodings:

VTRNHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

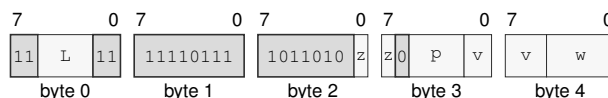


Format — Instruction format for VTRNHI.BWLQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

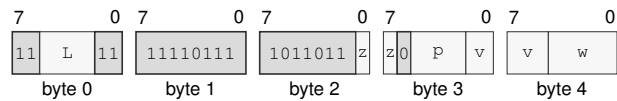
Instruction Format:



Bits field w —Encodes the bits value.

VCVTH.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:



Format — Instruction format for VCVTH.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.

VCVTUH**VCVTUH (Vector)****VCVTUH**

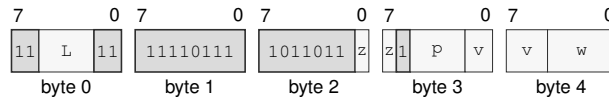
Operation: Unsigned integer to H.

Assembler Syntax: VCVTUH.V_LQ(z) Pn(p), Vn(v), Vn(w)

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Encodings:

VCVTUH.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Format:

Format — Instruction format for VCVTUH.V_LQ(z) Pn(p), Vn(v), Vn(w)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

VMADD

VMADD (Vector)

VMADD

Operation:

Fused multiply-add with read/write accumulator destination.

Assembler Syntax:

VMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5 bytes

Feature = VECTOR

Repeat = none

Detailed Semantics

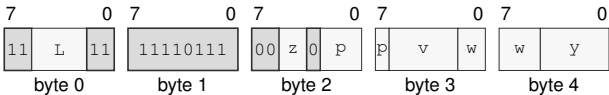
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Instruction Format:



Format — Instruction format for VMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S/D.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Bits field** w—Encodes the bits value.
- Bits field** y—Encodes the bits value.

VMSUB**VMSUB (Vector)****VMSUB**

Operation: Fused multiply-subtract.

Assembler Syntax: VMSUB.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*), V_{*n*}(*y*)

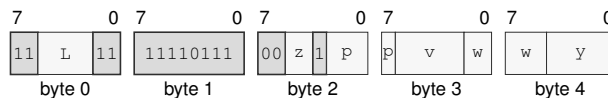
Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**VFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VMSUB.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*), V_{*n*}(*y*)

Instruction Format:

Format — Instruction format for VMSUB.HSD(*z*) P_{*n*}(*p*), V_{*n*}(*v*), V_{*n*}(*w*), V_{*n*}(*y*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

Bits field *y*—Encodes the bits value.

VNMADD

VNMADD (Vector)

VNMADD

Operation:

Negated fused multiply-add.

Assembler Syntax:

VNMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Attributes:

Class = vector

Family = vector

Privilege = unprivileged

Length = 5 bytes

Feature = VECTOR

Repeat = none

Detailed Semantics

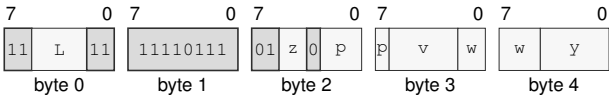
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VNMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Instruction Format:



Format — Instruction format for VNMADD.HSD(z) Pn(p), Vn(v), Vn(w), Vn(y)

Instruction Fields:

- Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z—Selects H/S/D.
- Bits field p—Encodes the bits value.
- Bits field v—Encodes the bits value.
- Bits field w—Encodes the bits value.
- Bits field y—Encodes the bits value.

VNMSUB

VNMSUB (Vector)

VNMSUB

Operation: Negated fused multiply-subtract.

Assembler Syntax: VNMSUB.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*), V_n(*y*)

Attributes:
Class = vector
Family = vector
Privilege = unprivileged
Length = 5 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

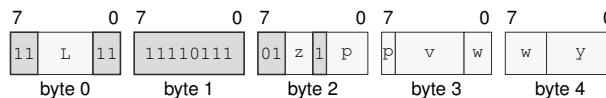
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VNMSUB.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*), V_n(*y*)

Instruction Format:



Format — Instruction format for VNMSUB.HSD(*z*) P_n(*p*), V_n(*v*), V_n(*w*), V_n(*y*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Bits field *v*—Encodes the bits value.

Bits field *w*—Encodes the bits value.

Bits field *y*—Encodes the bits value.

VSLICE

Vector Slice

VSLICE

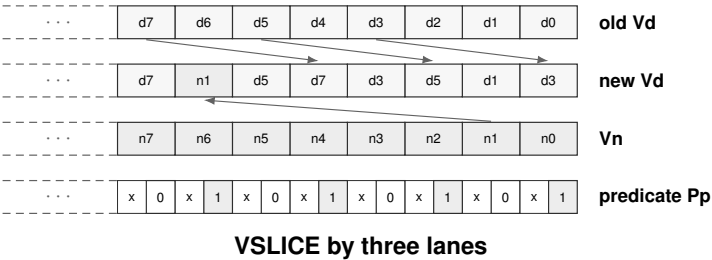
Operation: Extract a lane window from the old destination followed by a source vector.

Assembler Syntax: VSLICE.BWLQ(z) Pn(p), Vn(v), imm6(i), Vn(w)

Required CPUID flags: VECTOR

Detailed Semantics

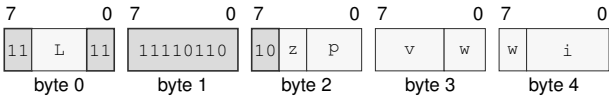
Number the lanes of the old destination followed by the lanes of Vn as one sequence. For each active destination lane, VSLICE selects the sequence lane whose index is the destination-lane index plus the unsigned count. A sequence index beyond both vectors produces zero. The governing predicate retains the corresponding old destination value in each inactive lane.



Encodings:

VSLICE.BWLQ(z) Pn(p), Vn(v), imm6(i), Vn(w)

Instruction Format:



Instruction Fields:

Length field L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Bits field w—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VSLIDEUP

Vector Slide Up

VSLIDEUP

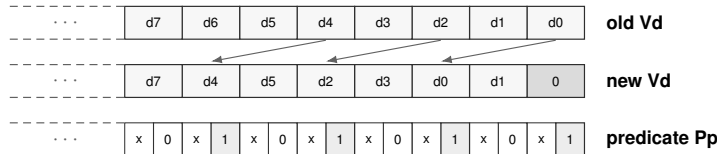
Operation: Move old destination lanes toward higher indices and zero vacated lanes.

Assembler Syntax: VSLIDEUP.BWLQ(z) Pn(p), imm6(i), Vn(v)

Required CPUID flags: VECTOR

Detailed Semantics

For each active destination lane whose index is at least the unsigned count, VSLIDEUP reads the old destination lane at the index reduced by that count. It writes zero to an active lane with a lower index. The governing predicate retains the corresponding old destination value in each inactive lane.

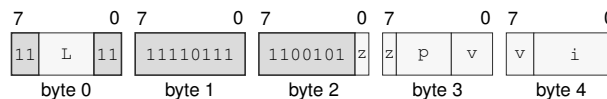


VSLIDEUP by two lanes

Encodings:

VSLIDEUP.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Format:



Format — Instruction format for VSLIDEUP.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field v—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VSLIDEDN

Vector Slide Down

VSLIDEDN

Operation:

Move old destination lanes toward lower indices and zero vacated lanes.

Assembler Syntax:

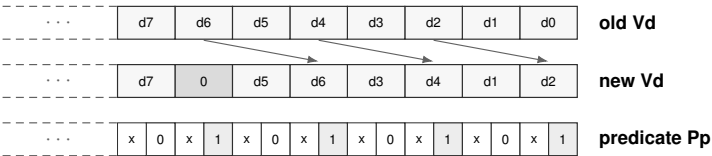
VSLIDEDN.BWLQ(z) Pn(p), imm6(i), Vn(v)

Required CPUID flags:

VECTOR

Detailed Semantics

For each active destination lane, VSLIDEDN reads the old destination lane whose index is the destination-lane index plus the unsigned count. It writes zero when that index reaches or exceeds the vector lane count. The governing predicate retains the corresponding old destination value in each inactive lane.

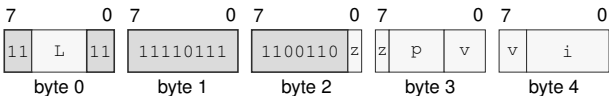


VSLIDEDN by two lanes

Encodings:

VSLIDEDN.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Format:



Format — Instruction format for VSLIDEDN.BWLQ(z) Pn(p), imm6(i), Vn(v)

Instruction Fields:

- Length field L**—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field z**—Selects B/W/L/Q.
- Bits field p**—Encodes the bits value.
- Bits field v**—Encodes the bits value.
- Immediate field i**—Encodes the immediate value.

VEEXTRACT

VEEXTRACT (Vector)

VEEXTRACT

Operation: Extract one integer lane. Extract one FP lane.

Assembler Syntax: `VEEXTRACT.BWLQ(z) Vn(v), Rn(r), Rn(s)`
`VEEXTRACT.HSD(z) Vn(v), Rn(r), Fn(s)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Exceptions: VECTOR_RANGE_ERROR: the architected vector range check fails

Detailed Semantics

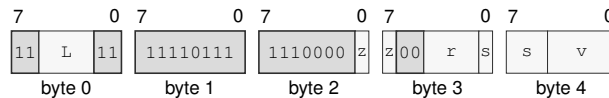
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

`VEEXTRACT.BWLQ(z) Vn(v), Rn(r), Rn(s)`

Instruction Format:



Format — Instruction format for VEEXTRACT.BWLQ(z) Vn(v), Rn(r), Rn(s)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

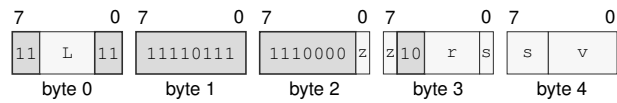
Register field r—Selects operand 2.

Register field s—Selects operand 3.

Bits field v—Encodes the bits value.

`VEXTRACT.HSD(z) Vn(v), Rn(r), Fn(s)`

Instruction Format:



Format — Instruction format for `VEXTRACT.HSD(z) Vn(v), Rn(r), Fn(s)`

Instruction Fields:

- Length field** `L`—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** `z`—Selects H/S/D.
- Register field** `r`—Selects operand 2.
- Register field** `s`—Selects operand 3.
- Bits field** `v`—Encodes the bits value.

VINSERT

VINSERT (Vector)

VINSERT

Operation: Replace one integer lane. Replace one FP lane.

Assembler Syntax: `VINSERT.BWLQ(z) Rn(r), Rn(s), Vn(v)`
`VINSERT.HSD(z) Fn(r), Rn(s), Vn(v)`

Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5 bytes
 Feature = VECTOR
 Repeat = none

Exceptions: VECTOR_RANGE_ERROR: the architected vector range check fails

Detailed Semantics

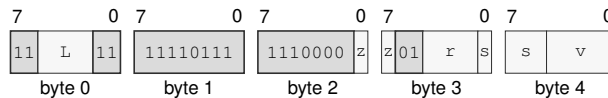
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

`VINSERT.BWLQ(z) Rn(r), Rn(s), Vn(v)`

Instruction Format:



Format — Instruction format for VINSERT.BWLQ(z) Rn(r), Rn(s), Vn(v)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

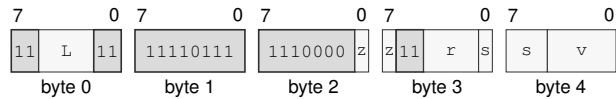
Register field r—Selects operand 1.

Register field s—Selects operand 2.

Bits field v—Encodes the bits value.

VINSERT.HSD(*z*) Fn(*r*), Rn(*s*), Vn(*v*)

Instruction Format:



Format — Instruction format for VINSERT.HSD(*z*) Fn(*r*), Rn(*s*), Vn(*v*)

Instruction Fields:

- Length field** *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** *z*—Selects H/S/D.
- Register field** *r*—Selects operand 1.
- Register field** *s*—Selects operand 2.
- Bits field** *v*—Encodes the bits value.

VREDADD**VREDADD (Vector)****VREDADD**

Operation: Modular sum of active lanes to Rn. Ordered active-lane FP sum to Fn.

Assembler Syntax: VREDADD.BWLQ(z) Pn(p), Vn(v), Rn(r)
 VREDADD.HSD(z) Pn(p), Vn(v), Fn(r)
 VREDADD.BWLQ(z) Pn(p), <ea>(e), Rn(r)
 VREDADD.HSD(z) Pn(p), <ea>(e), Fn(r)

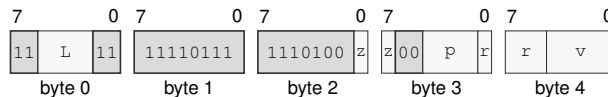
Attributes: Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 5-16 bytes
 Feature = VECTOR
 Repeat = none

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VREDADD.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:

Format — Instruction format for VREDADD.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

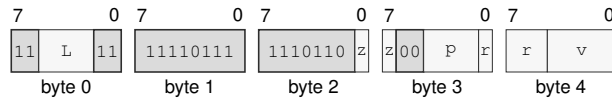
Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Bits field v—Encodes the bits value.

VREDADD.HSD(z) $P_n(p)$, $V_n(v)$, $F_n(r)$

Instruction Format:



Format — Instruction format for VREDADD.HSD(z) $P_n(p)$, $V_n(v)$, $F_n(r)$

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects H/S/D.

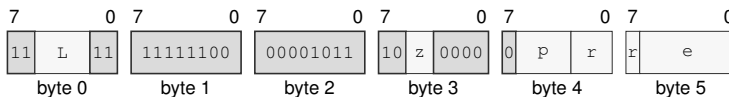
Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Bits field v—Encodes the bits value.

VREDADD.BWLQ(z) $P_n(p)$, $\langle ea \rangle(e)$, $R_n(r)$

Instruction Format:



Format — Instruction format for VREDADD.BWLQ(z) $P_n(p)$, $\langle ea \rangle(e)$, $R_n(r)$

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

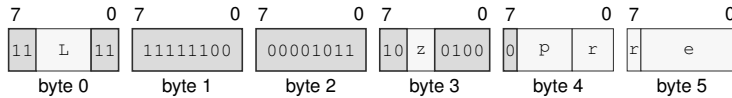
Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDADD.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Format:



Format — Instruction format for VREDADD.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

VREDMINS

VREDMINS (Vector)

VREDMINS

Operation:

Signed minimum.

Assembler Syntax:

VREDMINS.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDMINS.BWLQ(z) Pn(p), <ea>(e), Rn(r)

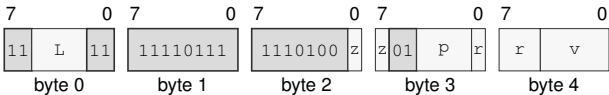
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDMINS.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



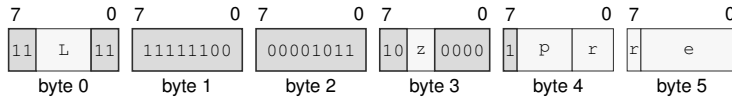
Format — Instruction format for VREDMINS.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMINS.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Format:



Format — Instruction format for VREDMINS.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDMINU

VREDMINU (Vector)

VREDMINU

Operation: Unsigned minimum.

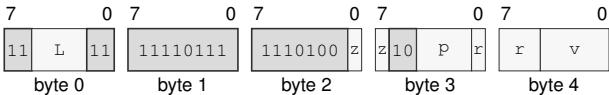
Assembler Syntax: VREDMINU.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDMINU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDMINU.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



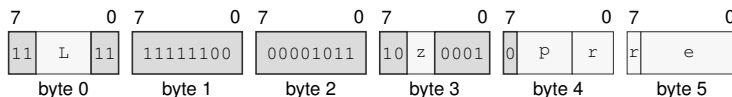
Format — Instruction format for VREDMINU.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMINU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Format:



Format — Instruction format for VREDMINU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDMAXS

VREDMAXS (Vector)

VREDMAXS

Operation: Signed maximum.

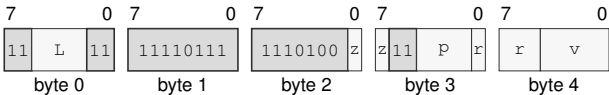
Assembler Syntax: `VREDMAXS.BWLQ(z) Pn(p), Vn(v), Rn(r)`
`VREDMAXS.BWLQ(z) Pn(p), <ea>(e), Rn(r)`

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

`VREDMAXS.BWLQ(z) Pn(p), Vn(v), Rn(r)`

Instruction Format:



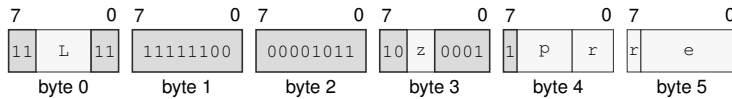
Format — Instruction format for VREDMAXS.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMAXS.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Format:



Format — Instruction format for VREDMAXS.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDMAXU

VREDMAXU (Vector)

VREDMAXU

Operation:

Unsigned maximum.

Assembler Syntax:

VREDMAXU.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDMAXU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

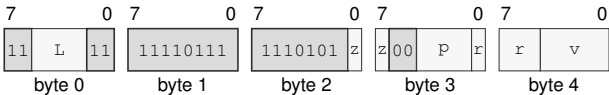
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDMAXU.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



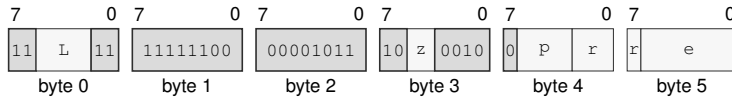
Format — Instruction format for VREDMAXU.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMAXU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Format:



Format — Instruction format for VREDMAXU.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDAND

VREDAND (Vector)

VREDAND

Operation: Bitwise AND.

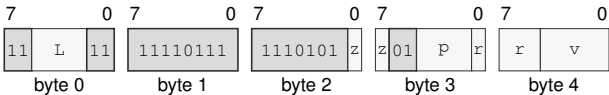
Assembler Syntax: VREDAND.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDAND.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDAND.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



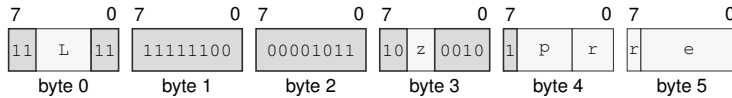
Format — Instruction format for VREDAND.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDAND.BWLQ(*z*) P_n(*p*), <ea>(*e*), R_n(*r*)

Instruction Format:



Format — Instruction format for VREDAND.BWLQ(*z*) P_n(*p*), <ea>(*e*), R_n(*r*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Bits field *p*—Encodes the bits value.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

VREDOR

VREDOR (Vector)

VREDOR

Operation:

Bitwise OR.

Assembler Syntax:

VREDOR.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDOR.BWLQ(z) Pn(p), <ea>(e), Rn(r)

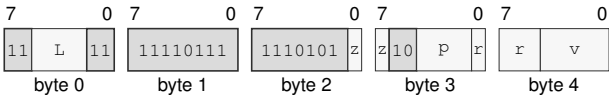
Attributes:

Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDOR.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



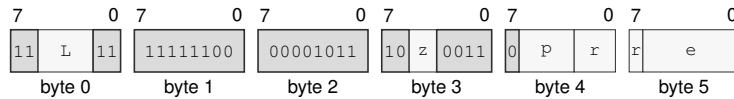
Format — Instruction format for VREDOR.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDOR.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Format:



Format — Instruction format for VREDOR.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Register field r—Selects operand 3.

Effective Address field e—Specifies the source operand.

VREDXOR

VREDXOR (Vector)

VREDXOR

Operation: Bitwise XOR.

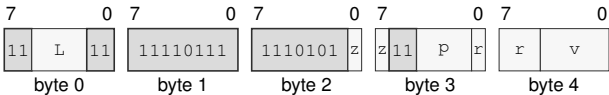
Assembler Syntax: VREDXOR.BWLQ(z) Pn(p), Vn(v), Rn(r)
VREDXOR.BWLQ(z) Pn(p), <ea>(e), Rn(r)

Attributes: Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Encodings:

VREDXOR.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Format:



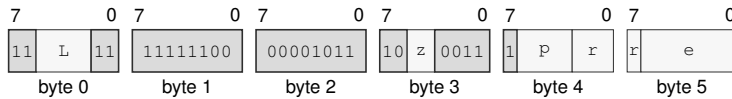
Format — Instruction format for VREDXOR.BWLQ(z) Pn(p), Vn(v), Rn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDXOR.BWLQ(*z*) P_n(*p*), <ea>(*e*), R_n(*r*)

Instruction Format:



Format — Instruction format for VREDXOR.BWLQ(*z*) P_n(*p*), <ea>(*e*), R_n(*r*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Bits field *p*—Encodes the bits value.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

VREDMIN

VREDMIN (Vector)

VREDMIN

- Operation:Active-lane FP minimum to Fn.
- Assembler Syntax:VREDMIN.HSD(z) Pn(p), Vn(v), Fn(r)
VREDMIN.HSD(z) Pn(p), <ea>(e), Fn(r)
- Attributes:Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

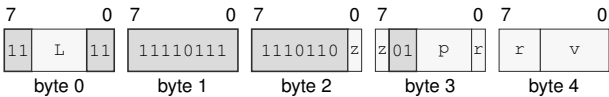
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VREDMIN.HSD(z) Pn(p), Vn(v), Fn(r)

Instruction Format:



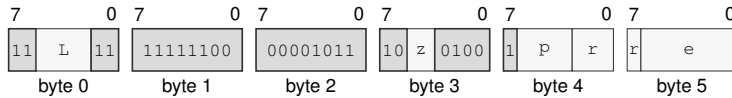
Format — Instruction format for VREDMIN.HSD(z) Pn(p), Vn(v), Fn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S/D.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMIN.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Format:



Format — Instruction format for VREDMIN.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

VREDMAX

VREDMAX (Vector)

VREDMAX

- Operation:Active-lane FP maximum to Fn.
- Assembler Syntax:VREDMAX.HSD(z) Pn(p), Vn(v), Fn(r)
VREDMAX.HSD(z) Pn(p), <ea>(e), Fn(r)
- Attributes:Class = vector
Family = vector
Privilege = unprivileged
Length = 5-16 bytes
Feature = VECTOR
Repeat = none

Detailed Semantics

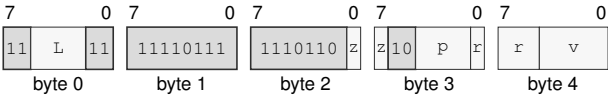
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Encodings:

VREDMAX.HSD(z) Pn(p), Vn(v), Fn(r)

Instruction Format:



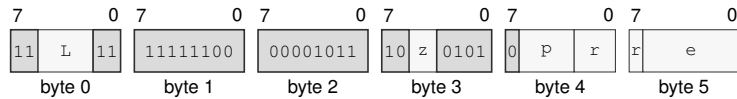
Format — Instruction format for VREDMAX.HSD(z) Pn(p), Vn(v), Fn(r)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + L bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects H/S/D.
- Bits field** p—Encodes the bits value.
- Register field** r—Selects operand 3.
- Bits field** v—Encodes the bits value.

VREDMAX.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Format:



Format — Instruction format for VREDMAX.HSD(*z*) P_n(*p*), <ea>(*e*), F_n(*r*)

Instruction Fields:

Length field *L*—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field *z*—Selects H/S/D.

Bits field *p*—Encodes the bits value.

Register field *r*—Selects operand 3.

Effective Address field *e*—Specifies the source operand.

PSEL

PSEL (Vector)

PSEL

Operation: Replace bits of ‘Pd’ selected by ‘Pc’ with bits from ‘Ps’.

Assembler Syntax: PSEL Pn(p), Pn(q), Pn(h)

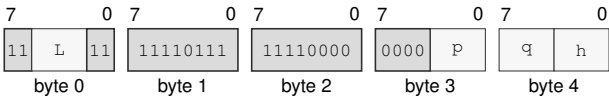
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 5 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

PSEL Pn(p), Pn(q), Pn(h)

Instruction Format:



Format — Instruction format for PSEL Pn(p), Pn(q), Pn(h)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Bits field** p—Encodes the bits value.
- Bits field** q—Encodes the bits value.
- Bits field** h—Encodes the bits value.

PZIPLO**Predicate Lower-Half Interleave****PZIPLO**

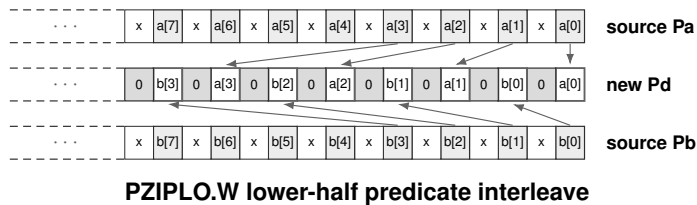
Operation: Interleave the lower halves of two predicate-lane sequences.

Assembler Syntax: PZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

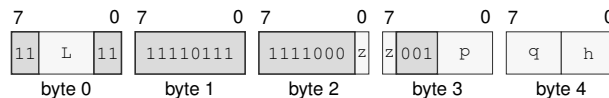
Required CPUID flags: VECTOR

Detailed Semantics

PZIPLO selects the lower half of the significant predicate lanes for the chosen B, W, L, or Q width from each source. In increasing order through those halves, each even result lane receives a left-source value and the following odd result lane receives the corresponding right-source value. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.

**Encodings:**

PZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Format:

Format — Instruction format for PZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PZIPHI**Predicate Upper-Half Interleave****PZIPHI**

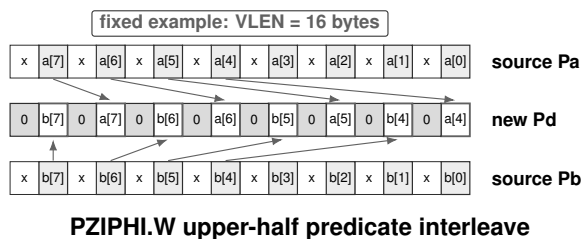
Operation: Interleave the upper halves of two predicate-lane sequences.

Assembler Syntax: PZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

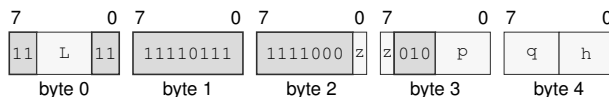
Required CPUID flags: VECTOR

Detailed Semantics

PZIPHI selects the upper half of the significant predicate lanes for the chosen B, W, L, or Q width from each source. In increasing order through those halves, each even result lane receives a left-source value and the following odd result lane receives the corresponding right-source value. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.

**Encodings:**

PZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Format:

Format — Instruction format for PZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PUZIPLO**Predicate Even-Lane Extraction****PUZIPLO**

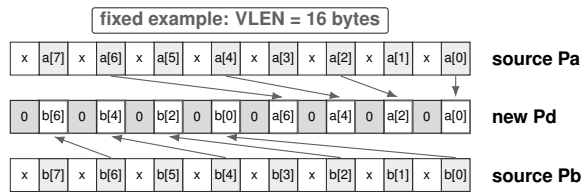
Operation: Concatenate even-position predicate lanes from two sources.

Assembler Syntax: PUZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Required CPUID flags: VECTOR

Detailed Semantics

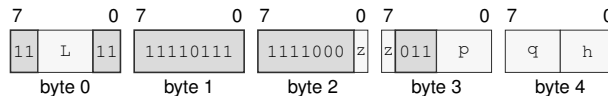
PUZIPLO selects the even-position significant predicate lanes for the chosen B, W, L, or Q width. The selected lanes from the left source form the lower half of the result, and those from the right source form the upper half, with each half retaining source order. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.



PUZIPLO.W even predicate-lane extraction

Encodings:

PUZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Format:

Format — Instruction format for PUZIPLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PUZIPHI

Predicate Odd-Lane Extraction

PUZIPHI

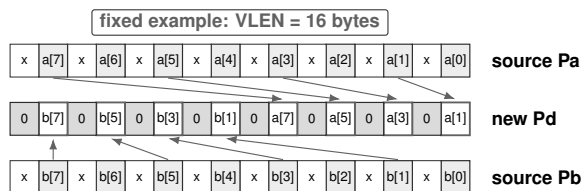
Operation: Concatenate odd-position predicate lanes from two sources.

Assembler Syntax: PUZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

Required CPUID flags: VECTOR

Detailed Semantics

PUZIPHI selects the odd-position significant predicate lanes for the chosen B, W, L, or Q width. The selected lanes from the left source form the lower half of the result, and those from the right source form the upper half, with each half retaining source order. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.

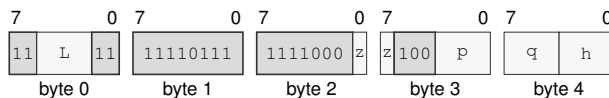


PUZIPHI.W odd predicate-lane extraction

Encodings:

PUZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Format:



Format — Instruction format for PUZIPHI.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PTRNLO

Predicate Even-Lane Transpose

PTRNLO

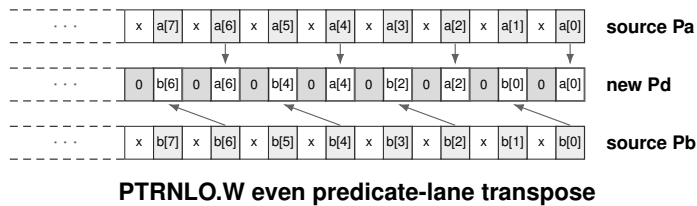
Operation: Interleave even-position predicate lanes from two sources.

Assembler Syntax: PTRNLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Required CPUID flags: VECTOR

Detailed Semantics

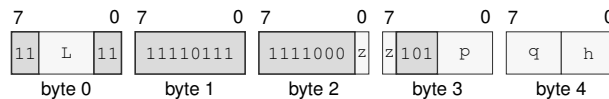
PTRNLO selects the even-position significant predicate lanes for the chosen B, W, L, or Q width from each source. For each selected source position in increasing order, the corresponding even result lane receives the left source value and the following odd result lane receives the right source value. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

PTRNLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Format:



Format — Instruction format for PTRNLO.BWLQ(z) Pn(p), Pn(q), Pn(h)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PTRNHI

Predicate Odd-Lane Transpose

PTRNHI

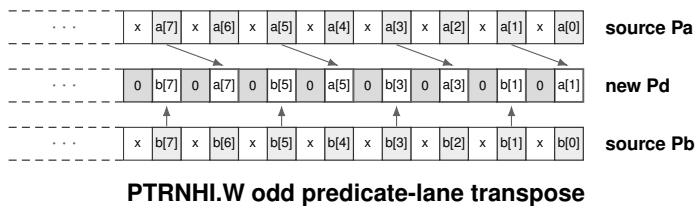
Operation: Interleave odd-position predicate lanes from two sources.

Assembler Syntax: `PTRNHI.BWLQ(z) Pn(p), Pn(q), Pn(h)`

Required CPUID flags: VECTOR

Detailed Semantics

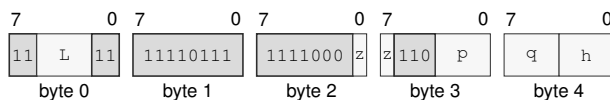
PTRNHI selects the odd-position significant predicate lanes for the chosen B, W, L, or Q width from each source. For each selected source position in increasing order, the corresponding even result lane receives the left source value and the following odd result lane receives the right source value. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

`PTRNHI.BWLQ(z) Pn(p), Pn(q), Pn(h)`

Instruction Format:



Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Bits field h—Encodes the bits value.

PSLICE

Predicate Slice

PSLICE

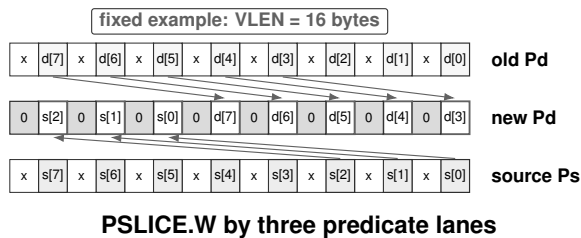
Operation: Extract a predicate-lane window from the old destination followed by a source predicate.

Assembler Syntax: PSLICE.BWLQ(z) Pn(p), imm6(i), Pn(q)

Required CPUID flags: VECTOR

Detailed Semantics

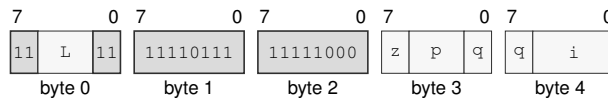
PSLICE treats the significant predicate positions of the old destination followed by those of the source as one sequence of selected B, W, L, or Q typed lanes. It writes the destination from a window of the architectural typed-lane count beginning at the unsigned immediate count. Positions beyond that concatenated sequence become zero. The complete destination predicate image is written, and every nonsignificant bit is cleared. The operation leaves FLAGS unchanged.



Encodings:

PSLICE.BWLQ(z) Pn(p), imm6(i), Pn(q)

Instruction Format:



Format — Instruction format for PSLICE.BWLQ(z) Pn(p), imm6(i), Pn(q)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Bits field p—Encodes the bits value.

Bits field q—Encodes the bits value.

Immediate field i—Encodes the immediate value.

VMOVZ

VMOVZ (Vector)

VMOVZ

Operation: Move active memory elements into ‘Vd’; inactive lanes become zero.

Assembler Syntax: VMOVZ.BWLQ(z) Pn(p), <ea>(e), Vn(v)

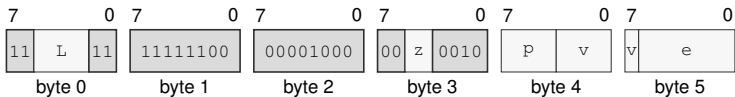
Attributes:

- Class = vector
- Family = vector
- Privilege = unprivileged
- Length = 6-16 bytes
- Feature = VECTOR
- Repeat = none

Encodings:

VMOVZ.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Format:



Format — Instruction format for VMOVZ.BWLQ(z) Pn(p), <ea>(e), Vn(v)

Instruction Fields:

- Length field** L—Encodes the encoded instruction length as 3 + *L* bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Bits field** p—Encodes the bits value.
- Bits field** v—Encodes the bits value.
- Effective Address field** e—Specifies the source operand.

PLOOP

PLOOP (Vector)

PLOOP

Operation: Implements the architected PLOOP scalable-vector operation.

Assembler Syntax: PLOOP.BWLQ(z) Rn(r), Rn(s), Pn(p), <ea>(e)

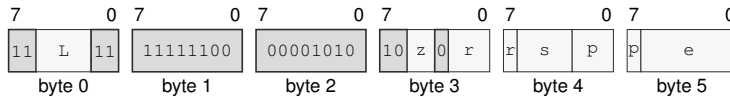
Attributes:
 Class = vector
 Family = vector
 Privilege = unprivileged
 Length = 6-16 bytes
 Feature = VECTOR
 Repeat = none

Exceptions: VECTOR_RANGE_ERROR: the architected vector range check fails

Encodings:

PLOOP.BWLQ(z) Rn(r), Rn(s), Pn(p), <ea>(e)

Instruction Format:



Format — Instruction format for PLOOP.BWLQ(z) Rn(r), Rn(s), Pn(p), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field r—Selects operand 1.

Register field s—Selects operand 2.

Bits field p—Encodes the bits value.

Effective Address field e—Specifies the control target.

Destination overlap—When the remaining and offset operands designate the same architectural register, the instruction raises ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION before architectural effects.

PMOV

PMOV (Vector)

PMOV

Operation: Read a complete packed predicate image from memory.

Assembler Syntax: `PMOV <ea>(e), Pn(p)`
`PMOV Pn(p), <ea>(e)`

Required CPUID flags: VECTOR

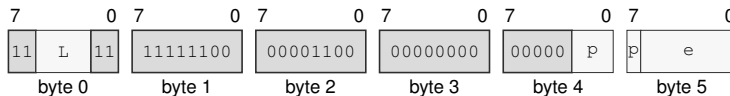
Detailed Semantics

Read a complete packed predicate image from memory. Write a complete packed predicate image to memory.

Encodings:

`PMOV <ea>(e), Pn(p)`

Instruction Format:



Format — Instruction format for PMOV <ea>(e), Pn(p)

Instruction Fields:

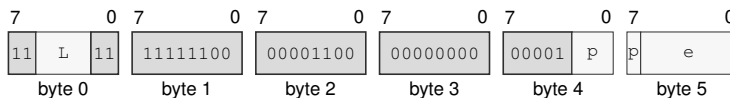
Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

Effective Address field e—Specifies the source operand.

`PMOV Pn(p), <ea>(e)`

Instruction Format:



Format — Instruction format for PMOV Pn(p), <ea>(e)

Instruction Fields:

Length field L—Encodes the encoded instruction length as $3 + L$ bytes. The encoded length must cover the required instruction length; trailing bytes are uninterpreted padding.

Bits field p—Encodes the bits value.

Effective Address field e—Specifies the destination operand. Immediate addressing is unavailable in this form.

A REFERENCE INDEXES

These indexes provide page references to architectural definitions and instruction entries.

A.1 Canonical Field Names

The following spellings are canonical within the named owner. A one-letter field is interpreted in its owner or instruction-encoding context.

Table A-1. Canonical Field Names

Owner	Canonical fields	Normative definition
FLAGS	Z, N, C, V	definition (p. 12)
STATUS	IE, PM, RF, TF, NI, EA	definition (p. 12)
PTCR	root_page, LA57, PE	definition (p. 16)
ASCR	ASID, AE	definition (p. 16)
ECR	MAX_EDEPTH, NMI_P, V	definition (p. 16)
UCTL	V, STATUS, FLAGS	definition (p. 16)
PMC	EN	definition (p. 16)
PTE	P, T, AM, R, W, X, U, G, A, D, CP, SW, PFN	definition (p. 53)
instruction_header	L	definition (p. 24)

A.2 Architectural State Index

Reader and writer columns identify the architectural interfaces or execution classes that access each state group. Reset values are the architectural reset values for each state group.

Table A-2. Architectural State Index

State	Fields	Readers	Writers	Reset
R0--R15 (p. 12)	—	Rn source operands and effective-address base or index terms	writable Rn destinations and effective-address auto-update	0
SP (p. 12)	—	stack operations and SP-relative effective addresses	stack operations and writable SP destinations	0
PC (p. 12)	—	instruction fetch and PC-relative effective addresses	control transfers, event return, and RESET	BOOTPC
CS, DS, SS, GS0, GS1, GS2, GS3, GS4, GS5 (p. 14)	base_page, e, m, b	segment pre-translation, RDSEG, fixed CS reads, and SAVE	WRSEG, segment-changing control transfers, event return, and RESTORE	0
FLAGS, STATUS (p. 12)	Z, N, C, V, IE, PM, RF, TF, NI, EA	condition evaluation, RDFLAGS, RDSTATUS, event entry, and SAVE	instruction flag results, WRFLAGS, WRSTATUS, event transitions, and RESTORE	FLAGS: 0; STATUS: PM=1; all other STATUS bits 0
F0--F15 (p. 14)	—	floating-point source operands and SAVE	floating-point destinations and RESTORE	0
FFLAGS, FSTATUS (p. 12)	NV, DZ, OF, UF, NX, NV_EN, DZ_EN, OF_EN, UF_EN, NX_EN, RM, FTZ, DAZ, DN	floating-point execution, dedicated reads, and SAVE	floating-point results, dedicated writes, RESET, and RESTORE	0

Table A-2 (continued)

State	Fields	Readers	Writers	Reset
V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31 (p. 464)	—	vector operands, vector effective-address-independent computation, and SAVE	vector destinations, RESET, and RESTORE	not listed
P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15 (p. 464)	—	governing predicates, predicate operations, predicate branches, PLOOP, and SAVE	predicate destinations, PLOOP, RESET, and RESTORE	not listed
CYCLE, INSTRET, PTWALK (p. 91)	—	RDPMC	architecturally defined counter increment events and RESET	0
PTCR (p. 16)	root_page, LA57, PE	RDCR	WRCR and named architectural transitions	0
ASCR (p. 16)	ASID, AE	RDCR	WRCR and named architectural transitions	0
ECR (p. 16)	MAX_EDEPTH, NMI_P, V	RDCR	WRCR and named architectural transitions	0
UPC (p. 16)	—	RDCR	WRCR and named architectural transitions	0
USP (p. 16)	—	RDCR	WRCR and named architectural transitions	0
UCS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
UDS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
USS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
UCTL (p. 16)	V, STATUS, FLAGS	RDCR	WRCR and named architectural transitions	0
UINFO (p. 16)	—	RDCR	WRCR and named architectural transitions	0
EPC (p. 16)	—	RDCR	WRCR and named architectural transitions	0
ECS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
EDS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
SSS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
SSP (p. 16)	—	RDCR	WRCR and named architectural transitions	0
ISS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
ISP (p. 16)	—	RDCR	WRCR and named architectural transitions	0
FSS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
FSP (p. 16)	—	RDCR	WRCR and named architectural transitions	0
DSS (p. 16)	—	RDCR	WRCR and named architectural transitions	0
DSP (p. 16)	—	RDCR	WRCR and named architectural transitions	0
BOOTPC (p. 16)	—	RDCR	WRCR and named architectural transitions	platform supplied on cold reset; preserved by warm RESET

Table A-2 (continued)

State	Fields	Readers	Writers	Reset
BOOTCFG (p. 16)	—	RDCR	WRCR and named architectural transitions	platform supplied on cold reset; preserved by warm RESET
PMC (p. 16)	EN	RDCR	WRCR and named architectural transitions	0

A.3 Architectural Event Index

Table A-3. Architectural Event Index

Code	Event	Producer	Priority	Frame	Definition
EXC:00	DEBUG_TRACE	completion of a TF-selected trace unit	5	BASIC	definition (p. 70)
EXC:02	BREAKPOINT	BKPT	4	BASIC	definition (p. 70)
EXC:03	ILLEGAL_INSTRUCTION	decode or instruction-validity check; DIVMODS, DIVMODU, FSINCOSA, ILLEGAL	3	ERROR	definition (p. 70)
EXC:08	PRIVILEGE_FAULT	privilege check	3	BASIC	definition (p. 70)
EXC:09	PAGE_FAULT	segment, translation, memory-type, or atomic-alignment check	3	PAGE_FAULT	definition (p. 70)
EXC:0a	DIVIDE_ERROR	integer divide operation; DIVMODS, DIVMODU, DIVS, DIVU, MODS, MODU	3	ERROR	definition (p. 70)
EXC:0b	VECTOR_RANGE_ERROR	PLoop offset or vector lane-index validation; PLoop, VEXTRACT, VGATHER1, VINSERT, VSCATTER1	3	ERROR	definition (p. 70)
EXC:0c	ACCESS_FAULT	physical-address-width or MMIO access-rule check	3	PAGE_FAULT	definition (p. 70)
EXC:0d	INVALID_CONTROL_STATE	control-state validation; ERET, SYSCALL, WRCR, WRSTATUS	3	ERROR	definition (p. 70)
EXC:0e	FLOATING_POINT_FAULT	enabled floating-point exception condition; FSINCOSA	3	ERROR	definition (p. 70)
EXC:18	DOUBLE_FAULT	architectural event-delivery failure	1	AUXILIARY	definition (p. 70)
EXC:19	MACHINE_CHECK	processor or memory-system machine-check source	2	AUXILIARY	definition (p. 70)
EXC:1a	BUS_ERROR	synchronous acknowledged bus failure	3	AUXILIARY	definition (p. 70)
EXC:20	SYSTEM_CALL	SYSCALL	4	BASIC	definition (p. 70)
NMI:source	NMI	admitted non-maskable interrupt source	6	BASIC	definition (p. 70)
INTERRUPT:platform	INTERRUPT	admitted maskable interrupt source	7	BASIC	definition (p. 70)

A.4 CPUID Feature Index

Table A-4. CPUID Feature Index

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FP (p. 87)	00000001:0000:1/bit0	FABS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FBNDII	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FBNDIX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FBNDXI	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FBNDXX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCEIL	F0–F15, FFLAGS, FSTATUS, SAVE component FP

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FP (p. 87)	00000001:0000:1/bit0	FCLASS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCLR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCMP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCOPYSIGN	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCVT	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FCVTU	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FDIV	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FFLOOR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FGETEXP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FGETMAN	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FINT	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FINTRZ	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMAX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMIN	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMOD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMOV	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMOVCR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMOVcc	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FMUL	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FNEG	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FNMADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FNMSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FPOPP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FPUSHP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FREM	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FROUND	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FSCALE	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FSQRT	F0–F15, FFLAGS, FSTATUS, SAVE component FP

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FP (p. 87)	00000001:0000:1/bit0	FSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FTEST	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FTRUNC	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	FXCHG	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	RDFFLAGS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	RDFSTATUS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	WRFFLAGS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 87)	00000001:0000:1/bit0	WRFSTATUS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FACOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FASINA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FATANA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FATANHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FCOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FCOSHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FETOXA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FETOXM1A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FLOG10A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FLOG2A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FLOGNA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FLOGNP1A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FSINA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FSINCOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FSINHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FTANA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FTANHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FTENTOX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 87)	00000001:0000:1/bit1	FTWOTOXA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
VECTOR (p. 87)	00000001:0000:1/bit2	BPALL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	BPANY	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	BPNONE	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PAND	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PCOUNT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PFALSE	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PFIRST	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PHEAD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PLAST	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PLLOOP	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	PMOV	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PNOT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	POR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PPACKHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PPACKLO	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PPERM	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PSEL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PSLICE	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PSLIDEDN	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	PSLIDEUP	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PTAIL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PTRNHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PTRNLO	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PTRUE	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PUNPKHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PUNPKLO	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PUZIPHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PUZIPL0	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	PXOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PZIPHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	PZIPL0	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VABS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VADD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VAND	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCEIL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCLASS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCLR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VCLS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCLZ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCMPcc	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCOPYSIGN	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCTS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCTZ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTH	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTQ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTUD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTUH	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTUL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTUQ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VCVTUS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VDIV	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VDUP	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTRACT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTSL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTSQ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTSW	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTZL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTZQ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VEXTZW	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VFLOOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VGATHER1	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VINDEX	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VINSERT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VLCADD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VLCNT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMADD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMAX	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMAXS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMAXU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMIN	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VMINS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMINU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMOV	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMOVZ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMSUB	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMUL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMULHS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMULHSU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VMULHU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VNEG	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VNMADD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VNMSUB	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VNOT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VPERM	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VPOPCNT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDADD	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDAND	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMAX	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMAXS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMAXU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMIN	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMINS	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDMINU	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREDXOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VREVBYTE	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VR0L	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VR0R	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VROUND	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSAR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSCATTER1	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSHL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSHR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSlice	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSLIDEDN	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VSLIDEUP	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSQRT	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VSUB	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTESTNZ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTESTZ	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTRNHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTRNLO	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTRUNC	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTRUNCB	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
VECTOR (p. 87)	00000001:0000:1/bit2	VTRUNCL	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VTRUNCW	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VUZIPHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VUZIPL0	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VXOR	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VZIPHI	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR
VECTOR (p. 87)	00000001:0000:1/bit2	VZIPL0	V0, V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, V16, V17, V18, V19, V20, V21, V22, V23, V24, V25, V26, V27, V28, V29, V30, V31, P0, P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13, P14, P15, SAVE component VECTOR